

```

=== mobile/src/components/AnimatedCheckbox.tsx ===
1 import React from 'react';
2 import { Pressable, View, StyleSheet } from 'react-native';
3 import Svg, { Path } from 'react-native-svg';
4
5 /**
6  * 动画复选框组件属性
7  */
8 interface CheckboxProps {
9   /** 是否选中 */
10  checked: boolean;
11  /** 切换选中状态的回调 */
12  onToggle: () => void;
13  /** 复选框大小 */
14  size?: number;
15  /** 复选框颜色 */
16  color?: string;
17 }
18
19 /**
20  * 动画复选框组件
21  * 提供自定义样式的复选框，包含SVG对勾
22  */
23 const AnimatedCheckbox: React.FC<CheckboxProps> = ({
24   checked,
25   onToggle,
26   size = 24,
27   color = '#80FF00',
28 }) => {
29   return (
30     <Pressable onPress={onToggle} style={styles.container}>
31       <View style={[styles.box, { width: size * 1.5, height: size * 1.5, borderColor: color }]}>
32         { /* 选中时显示SVG对勾 */ }
33         {checked && (
34           <Svg
35             width={size * 1.5}
36             height={size * 1.5}
37             viewBox="0 0 72 97"
38             style={styles.checkmark}
39           >
40             <Path
41               d="M28.72 95.673c-6.37 6.44-11.034-4.445-15.746-8.048-4.947-3.783-8.859-10.482-10.847-16.446-6.68
42               strokeWidth="4px"
43               stroke={color}
44               fill="none"
45               strokeLinecap="round"
46               strokeLinejoin="round"
47             />
48           </Svg>
49         )}

```

```

50   </View>
51   </Pressable>
52 );
53 };
54
55 const styles = StyleSheet.create({
56   /** 容器样式 */
57   container: {
58     justifyContent: 'center',
59     alignItems: 'center',
60   },
61   /** 复选框边框样式 */
62   box: {
63     borderWidth: 2,
64     borderRadius: 4,
65     justifyContent: 'center',
66     alignItems: 'center',
67     overflow: 'hidden',
68   },
69   /** 对勾样式 */
70   checkmark: {
71     position: 'absolute',
72   },
73 });
74
75 export default AnimatedCheckbox;
76
77 === mobile/src/components/FloatingInputBar.tsx ===
78 import React, { useRef, useEffect } from 'react';
79 import {
80   View,
81   Text,
82   TextInput,
83   TouchableOpacity,
84   Animated,
85   Keyboard,
86   Platform,
87   KeyboardEvent,
88   StyleSheet,
89   InteractionManager,
90   KeyboardTypeOptions,
91 } from 'react-native';
92 import { Ionicons } from '@expo/vector-icons';
93
94 // =====
95 // FloatingInputBar
96 // 键盘上方浮动输入栏（极简版）
97 //
98 // 点击页面上的“伪输入框”后，浮层栏出现在键盘上方并自动聚焦。
99 // 输入内容实时同步到原始字段。点「完成」收起键盘。

```

```

99 // =====
100
101 export interface InputFieldConfig {
102   id: string;
103   label: string;
104   icon: string;
105   value: string;
106   onChangeText: (text: string) => void;
107   keyboardType?: KeyboardTypeOptions;
108   secureTextEntry?: boolean;
109   placeholder?: string;
110 }
111
112 interface FloatingInputBarProps {
113   fields: InputFieldConfig[];
114   activeFieldId: string | null;
115   onDismiss: () => void;
116   colors: {
117     backgroundDark: string;
118     background: string;
119     primary: string;
120     textPrimary: string;
121     textSecondary: string;
122     borderLight: string;
123     border: string;
124   };
125 }
126
127 const BAR_BOTTOM_SPACING = 8; // 输入栏与键盘之间的间距
128
129 export const FloatingInputBar: React.FC<FloatingInputBarProps> = ({
130   fields,
131   activeFieldId,
132   onDismiss,
133   colors,
134 }) => {
135   const inputRef = useRef<TextInput>(null);
136   const translateY = useRef(new Animated.Value(0)).current;
137   const opacity = useRef(new Animated.Value(0)).current;
138
139   const activeField = fields.find((f) => f.id === activeFieldId);
140
141   // ---- 字段切换或首次挂载时聚焦 ----
142   // key={activeFieldId} 确保每次切换都创建新的 TextInput 实例
143   // autoFocus 处理首次挂载, InteractionManager 确保动画完成后聚焦
144   useEffect(() => {
145     if (activeFieldId) {
146       InteractionManager.runAfterInteractions(() => {
147         if (inputRef.current) {
148           inputRef.current.focus();

```

```

149     }
150   });
151 }
152 }, [activeFieldId]);
153
154 // ---- 键盘动画 ----
155 useEffect(() => {
156   const showEvent =
157     Platform.OS === 'ios' ? 'keyboardWillShow' : 'keyboardDidShow';
158   const hideEvent =
159     Platform.OS === 'ios' ? 'keyboardWillHide' : 'keyboardDidHide';
160
161   const showSub = Keyboard.addListener(showEvent, (e: KeyboardEvent) => {
162     // translateY 为负 = 向上移动, 减去 BAR_BOTTOM_SPACING 留出间距
163     const targetY = -(e.endCoordinates.height + BAR_BOTTOM_SPACING);
164
165     Animated.parallel([
166       Animated.timing(translateY, {
167         toValue: targetY,
168         duration: e.duration || 280,
169         useNativeDriver: true,
170       }),
171       Animated.timing(opacity, {
172         toValue: 1,
173         duration: (e.duration || 280) * 0.5,
174         useNativeDriver: true,
175       }),
176     ]).start();
177   });
178
179   const hideSub = Keyboard.addListener(hideEvent, (e: KeyboardEvent) => {
180     Animated.parallel([
181       Animated.timing(translateY, {
182         toValue: 0,
183         duration: e.duration || 280,
184         useNativeDriver: true,
185       }),
186       Animated.timing(opacity, {
187         toValue: 0,
188         duration: (e.duration || 280) * 0.6,
189         useNativeDriver: true,
190       }),
191     ]).start();
192   });
193
194   return () => {
195     showSub.remove();
196     hideSub.remove();
197   };
198 }, [translateY, opacity]);

```

```

199
200 // ---- 收起 ----
201 const handleDone = () => {
202   onDismiss();
203 };
204
205 if (!activeField) return null;
206
207 return (
208   <Animated.View
209     style={[
210       styles.container,
211       {
212         backgroundColor: colors.backgroundDark,
213         borderTopColor: colors.borderLight,
214       },
215       {
216         transform: [{ translateY }],
217         opacity,
218       },
219     ]}
220   >
221     /* 输入行 */
222     <View
223       style={[
224         styles.inputRow,
225         {
226           backgroundColor: colors.background,
227           borderColor: colors.borderLight,
228         },
229       ]}
230     >
231       <Ionicons
232         name={activeField.icon as any}
233         size={18}
234         color={colors.primary}
235         style={styles.inputIcon}
236       />
237       <TextInput
238         key={activeFieldId}
239         ref={inputRef}
240         style={[styles.textInput, { color: colors.textPrimary }]}
241         value={activeField.value}
242         onChangeText={activeField.onChangeText}
243         placeholder={activeField.placeholder}
244         placeholderTextColor={colors.textSecondary}
245         keyboardType={activeField.keyboardType || 'default'}
246         secureTextEntry={activeField.secureTextEntry}
247         autoCapitalize="none"
248         autoCorrect={false}

```

```

249     autoFocus
250     returnKeyType="done"
251     onSubmitEditing={handleDone}
252   />
253   {activeField.value.length > 0 && (
254     <TouchableOpacity
255       onPress={() => activeField.onChangeText("")}
256       style={styles.clearButton}
257       hitSlop={{ top: 8, bottom: 8, left: 8, right: 8 }}
258     >
259       <Ionicons name="close-circle" size={18} color={colors.textSecondary} />
260     </TouchableOpacity>
261   )}
262   <View style={styles.doneDivider} />
263   <TouchableOpacity onPress={handleDone} style={styles.doneButton}>
264     <Text style={[styles.doneText, { color: colors.primary }]}>完成</Text>
265   </TouchableOpacity>
266 </View>
267 </Animated.View>
268 );
269 };
270
271 const styles = StyleSheet.create({
272   container: {
273     position: 'absolute',
274     bottom: 0,
275     left: 0,
276     right: 0,
277     borderTopWidth: StyleSheet.hairlineWidth,
278     paddingVertical: 10,
279     paddingHorizontal: 12,
280   },
281   inputRow: {
282     flexDirection: 'row',
283     alignItems: 'center',
284     borderRadius: 14,
285     borderWidth: 1,
286     paddingHorizontal: 14,
287     height: 48,
288   },
289   inputIcon: {
290     marginRight: 10,
291   },
292   textInput: {
293     flex: 1,
294     fontSize: 16,
295     paddingVertical: 0,
296   },
297   clearButton: {
298     padding: 4,

```

```

299   marginLeft: 4,
300 },
301 doneDivider: {
302   width: 1,
303   height: 20,
304   backgroundColor: 'rgba(128,128,128,0.25)',
305   marginHorizontal: 10,
306 },
307 doneButton: {
308   paddingVertical: 6,
309   paddingHorizontal: 4,
310   borderRadius: 8,
311 },
312 doneText: {
313   fontSize: 15,
314   fontWeight: '700',
315 },
316 });
317
318 === mobile/src/components/GameHelpModal.tsx ===
319 import React, { useMemo } from 'react';
320 import { View, Text, StyleSheet, Modal, TouchableOpacity, ScrollView } from 'react-native';
321 import { Ionicons } from '@expo/vector-icons';
322 import { useTheme } from '../contexts/ThemeContext';
323
324 interface GameHelpModalProps {
325   visible: boolean;
326   onClose: () => void;
327 }
328
329 const GameHelpModal: React.FC<GameHelpModalProps> = ({ visible, onClose }) => {
330   const { colors } = useTheme();
331
332   const styles = useMemo(() => StyleSheet.create({
333     container: {
334       flex: 1,
335       backgroundColor: colors.background,
336     },
337     header: {
338       flexDirection: 'row',
339       justifyContent: 'space-between',
340       alignItems: 'center',
341       padding: 16,
342       paddingTop: 48,
343       borderBottomWidth: 1,
344       borderBottomColor: colors.borderDark,
345     },
346     closeBtn: { padding: 8 },
347     title: {
348       color: colors.textPrimary,

```

```

348     fontSize: 20,
349     fontWeight: '700',
350   },
351   placeholder: { width: 44 },
352   content: { padding: 16 },
353   section: {
354     marginBottom: 24,
355     backgroundColor: colors.borderLight,
356     borderRadius: 12,
357     padding: 16,
358     borderWidth: 1,
359     borderColor: colors.border,
360   },
361   sectionTitle: {
362     color: colors.warning,
363     fontSize: 16,
364     fontWeight: '700',
365     marginBottom: 8,
366   },
367   sectionDesc: {
368     color: colors.textSecondary,
369     fontSize: 14,
370     lineHeight: 22,
371   },
372   rulesList: { gap: 6 },
373   ruleItem: {
374     color: colors.textSecondary,
375     fontSize: 14,
376     lineHeight: 22,
377   },
378  )), [colors]);
379
380 return (
381   <Modal
382     visible={visible}
383     animationType="slide"
384     onRequestClose={onClose}
385   >
386     <View style={styles.container}>
387       <View style={styles.header}>
388         <TouchableOpacity onPress={onClose} style={styles.closeBtn}>
389           <Ionicons name="close" size={28} color={colors.textSecondary} />
390         </TouchableOpacity>
391         <Text style={styles.title}>游戏说明</Text>
392         <View style={styles.placeholder} />
393       </View>
394
395       <ScrollView style={styles.content} showsVerticalScrollIndicator={false}>
396         <View style={styles.section}>
397           <Text style={styles.sectionTitle}> 游戏恢复体力 </Text>

```



```

398     <Text style={styles.sectionDesc}>
399         每天可游玩3次小游戏恢复体力，完成游戏后可获得体力和能量奖励。
400     </Text>
401 </View>
402
403 <View style={styles.section}>
404     <Text style={styles.sectionTitle}> 数独</Text>
405     <Text style={styles.sectionDesc}>
406         在9×9的网格中填入数字1-9，使每行、每列、每个3×3宫格内的数字都不重复。完成一局即可获得奖励！
407     </Text>
408 </View>
409
410 <View style={styles.section}>
411     <Text style={styles.sectionTitle}> 推箱子</Text>
412     <Text style={styles.sectionDesc}>
413         将所有箱子推到目标位置上即可过关。完成3关即可获得奖励！
414     </Text>
415 </View>
416
417 <View style={styles.section}>
418     <Text style={styles.sectionTitle}> 游戏规则</Text>
419     <View style={styles.rulesList}>
420         <Text style={styles.ruleItem}>• 数独：完成一局即可获得奖励</Text>
421         <Text style={styles.ruleItem}>• 推箱子：完成3关即可获得奖励</Text>
422         <Text style={styles.ruleItem}>• 每天可游玩3次，凌晨5点重置次数</Text>
423         <Text style={styles.ruleItem}>• 完成游戏后奖励会自动添加到你的账户</Text>
424         <Text style={styles.ruleItem}>• 游戏过程中可随时退出，不会消耗次数</Text>
425     </View>
426 </View>
427 </ScrollView>
428 </View>
429 </Modal>
430 );
431 };
432
433 export default GameHelpModal;
434
435 === mobile/src/components/GameModal.tsx ===
436 import React, { useState, useEffect, useMemo } from 'react';
437 import { View, Text, StyleSheet, TouchableOpacity, ScrollView, Alert, Modal } from 'react-native';
438 import { Ionicons } from '@expo/vector-icons';
439 import { useTheme } from '../contexts/ThemeContext';
440 import {
441     sokobanLevels,
442     getLevelByIndex,
443     calculateRewards,
444     mockSubmitGameResult,
445     type GameResult,
446     } from '../services/gameService';

```

```

447 // ===== 类型定义 =====
448
449 /**
450  * 游戏弹窗组件属性
451  */
452 interface GameModalProps {
453   /** 是否显示弹窗 */
454   visible: boolean;
455   /** 关闭弹窗回调 */
456   onClose: () => void;
457   /** 游戏完成回调, 返回奖励 */
458   onGameComplete: (rewards: { stamina: number; energy: number }) => void;
459   /** 怪兽类型, 影响奖励计算 */
460   monsterType?: string;
461 }
462
463 /**
464  * 当前显示屏幕类型
465  */
466 type Screen = 'welcome' | 'tutorial' | 'game';
467
468 /**
469  * 游戏类型
470  */
471 type GameType = 'sudoku' | 'sokoban';
472
473 // ===== 数独游戏组件 =====
474
475 /**
476  * 数独游戏组件
477  */
478 const SudokuGame = ({ onWin, colors }: { onWin: () => void; colors: any }) => {
479   /** 游戏面板, 0表示空 */
480   const [board, setBoard] = useState<number[][]>([]);
481   /** 答案面板 */
482   const [solution, setSolution] = useState<number[][]>([]);
483   /** 哪些位置是初始给定的 (不可修改) */
484   const [given, setGiven] = useState<boolean[][]>([]);
485   /** 当前选中的格子位置 */
486   const [selected, setSelected] = useState<{ row: number; col: number } | null>(null);
487
488   /**
489    * 检查数字在当前位置是否有效
490    */
491   const isValid = (grid: number[][], row: number, col: number, num: number): boolean => {
492     // 检查行
493     for (let i = 0; i < 9; i++) {
494       if (grid[row][i] === num || grid[i][col] === num) return false;
495     }
496     // 检查3x3宫格

```

```

497     const boxRow = Math.floor(row / 3) * 3;
498     const boxCol = Math.floor(col / 3) * 3;
499     for (let i = 0; i < 3; i++) {
500         for (let j = 0; j < 3; j++) {
501             if (grid[boxRow + i][boxCol + j] === num) return false;
502         }
503     }
504     return true;
505 };
506
507 /**
508  * 使用回溯法生成数独答案
509  */
510 const solveSudoku = (grid: number[][]): boolean => {
511     for (let row = 0; row < 9; row++) {
512         for (let col = 0; col < 9; col++) {
513             if (grid[row][col] === 0) {
514                 // 随机打乱数字1-9
515                 const nums = [1, 2, 3, 4, 5, 6, 7, 8, 9].sort(() => Math.random() - 0.5);
516                 for (const num of nums) {
517                     if (isValid(grid, row, col, num)) {
518                         grid[row][col] = num;
519                         if (solveSudoku(grid)) return true;
520                         grid[row][col] = 0;
521                     }
522                 }
523                 return false;
524             }
525         }
526     }
527     return true;
528 };
529
530 /**
531  * 初始化游戏
532  */
533 const initGame = () => {
534     // 生成答案
535     const newSolution = Array(9).fill(0).map(() => Array(9).fill(0));
536     solveSudoku(newSolution);
537
538     // 复制答案到面板，然后挖空一些格子
539     const newBoard = newSolution.map(row => [...row]);
540     const newGiven = Array(9).fill(0).map(() => Array(9).fill(false));
541
542     // 随机挖空35个格子
543     const cellsToRemove = 35;
544     let removed = 0;
545     while (removed < cellsToRemove) {
546         const row = Math.floor(Math.random() * 9);

```

```

547     const col = Math.floor(Math.random() * 9);
548     if (newBoard[row][col] !== 0) {
549         newBoard[row][col] = 0;
550         removed++;
551     }
552 }
553
554 // 标记哪些是给定的
555 for (let row = 0; row < 9; row++) {
556     for (let col = 0; col < 9; col++) {
557         newGiven[row][col] = newSolution[row][col] === newBoard[row][col] && newBoard[row][col] !== 0;
558     }
559 }
560
561 setSolution(newSolution);
562 setBoard(newBoard);
563 setGiven(newGiven);
564 setSelected(null);
565 };
566
567 // 组件挂载时初始化游戏
568 useEffect(() => {
569     initGame();
570 }, []);
571
572 /**
573  * 检查游戏是否胜利
574  */
575 const checkWin = (): boolean => {
576     for (let row = 0; row < 9; row++) {
577         for (let col = 0; col < 9; col++) {
578             if (board[row][col] !== solution[row][col]) return false;
579         }
580     }
581     return true;
582 };
583
584 /**
585  * 输入数字处理
586  */
587 const handleInput = (num: number) => {
588     if (!selected || given[selected.row][selected.col]) return;
589
590     const newBoard = board.map(row => [...row]);
591     newBoard[selected.row][selected.col] = num;
592     setBoard(newBoard);
593
594     // 检查胜利
595     if (checkWin()) {
596         setTimeout(() => onWin(), 300);

```

```

597   }
598 };
599
600 /**
601  * 清除数字
602  */
603 const handleClear = () => {
604   if (!selected || given[selected.row][selected.col]) return;
605
606   const newBoard = board.map(row => [...row]);
607   newBoard[selected.row][selected.col] = 0;
608   setBoard(newBoard);
609 };
610
611 /**
612  * 获取格子背景色
613  */
614 const getCellColor = (row: number, col: number): string => {
615   if (!selected) return colors.background;
616   if (selected.row === row && selected.col === col) return colors.borderDark;
617   if (selected.row === row || selected.col === col) return colors.borderLight;
618   const boxRow = Math.floor(selected.row / 3) * 3;
619   const boxCol = Math.floor(selected.col / 3) * 3;
620   if (row >= boxRow && row < boxRow + 3 && col >= boxCol && col < boxCol + 3) {
621     return colors.borderLight;
622   }
623   return colors.background;
624 };
625
626 return (
627   <View style={styles.gameContainer}>
628     /* 头部 */
629     <View style={styles.sudokuHeader}>
630       <Text style={styles.sudokuTitle}> 数独</Text>
631       <TouchableOpacity style={styles.newGameBtn} onPress={initGame}>
632         <Ionicons name="refresh" size={16} color={colors.primary} />
633         <Text style={styles.newGameText}> 新游戏</Text>
634       </TouchableOpacity>
635     </View>
636
637     /* 游戏面板 */
638     <View style={styles.sudokuGrid}>
639       {board.map((row, rowIndex) => (
640         <View key={rowIndex} style={styles.sudokuRow}>
641           {row.map((cell, colIndex) => (
642             <TouchableOpacity
643               key={colIndex}
644               style={[styles.sudokuCell, { backgroundColor: getCellColor(rowIndex, colIndex) }]}
645               onPress={() => !given[rowIndex][colIndex] && setSelected({ row: rowIndex, col: colIndex })}
646               disabled={given[rowIndex][colIndex]}

```

```

647         >
648         {cell !== 0 && (
649             <Text style=[
650                 styles.sudokuCellText,
651                 given[rowIndex][colIndex] ? styles.sudokuGiven : styles.sudokuInput
652             ]>
653             {cell}
654         </Text>
655     )}
656 </TouchableOpacity>
657 )))
658 </View>
659 )))
660 </View>
661
662 {/* 数字键盘 */}
663 <View style={styles.numberPad}>
664     {[1, 2, 3, 4, 5, 6, 7, 8, 9].map(num => (
665         <TouchableOpacity
666             key={num}
667             style={styles.numberBtn}
668             onPress={() => handleInput(num)}
669         >
670             <Text style={styles.numberBtnText}>{num}</Text>
671         </TouchableOpacity>
672     )]}
673     <TouchableOpacity style={styles.clearBtn} onPress={handleClear}>
674         <Text style={styles.clearBtnText}>清除</Text>
675     </TouchableOpacity>
676 </View>
677
678 </View>
679 );
680 };
681
682 // ===== 推箱子游戏组件 =====
683
684 /**
685  * 推箱子游戏组件
686  */
687 const SokobanGame = ({ onWin, monsterType, colors }: { onWin: (level: number) => void; monsterType?: string; c
688 /** 当前关卡索引 */
689 const [currentLevel, setCurrentLevel] = useState(0);
690 /** 玩家位置 */
691 const [playerPos, setPlayerPos] = useState({ x: 0, y: 0 });
692 /** 游戏地图 */
693 const [gameMap, setGameMap] = useState<string[][]>([]);
694
695 /**
696  * 初始化指定关卡

```

```

697  */
698  const initLevel = (levelIndex: number) => {
699    const level = getLevelByIndex(levelIndex);
700    const newMap = level.map.map(row => row.split(""));
701    let pPos = { x: 0, y: 0 };
702
703    // 找到玩家位置并将其从地图中移除
704    for (let y = 0; y < newMap.length; y++) {
705      for (let x = 0; x < newMap[y].length; x++) {
706        if (newMap[y][x] === '@') {
707          pPos = { x, y };
708          newMap[y][x] = ' ';
709        }
710      }
711    }
712
713    setCurrentLevel(levelIndex);
714    setGameMap(newMap);
715    setPlayerPos(pPos);
716  };
717
718  // 组件挂载时初始化第一关
719  useEffect(() => {
720    initLevel(0);
721  }, []);
722
723  /**
724   * 检查当前地图是否胜利
725   */
726  const checkWin = (map: string[][]): boolean => {
727    for (const row of map) {
728      if (row.includes("$") || row.includes('.')) return false;
729    }
730    return true;
731  };
732
733  /**
734   * 移动玩家
735   */
736  const move = (dx: number, dy: number) => {
737    // 重置当前关卡
738    if (dx === 0 && dy === 0) {
739      initLevel(currentLevel);
740      return;
741    }
742
743    const newX = playerPos.x + dx;
744    const newY = playerPos.y + dy;
745
746    // 检查边界

```

```

747 if (newY < 0 || newY >= gameMap.length || newX < 0 || newX >= gameMap[newY].length) return;
748
749 const target = gameMap[newY][newX];
750 if (target === '#') return; // 撞墙
751
752 const newMap = gameMap.map(row => [...row]);
753
754 // 推箱子
755 if (target === '$' || target === '*') {
756   const boxNewX = newX + dx;
757   const boxNewY = newY + dy;
758
759   // 检查箱子移动位置
760   if (boxNewY < 0 || boxNewY >= gameMap.length || boxNewX < 0 || boxNewX >= gameMap[boxNewY].length) retur
761
762   const boxTarget = gameMap[boxNewY][boxNewX];
763   if (boxTarget === '.' || boxTarget === '*') {
764     newMap[newY][newX] = target === '*' ? ' ': '.';
765     newMap[boxNewY][boxNewX] = boxTarget === '.' ? '*' : '$';
766   } else {
767     return;
768   }
769 }
770
771 setGameMap(newMap);
772 setPlayerPos({ x: newX, y: newY });
773
774 // 检查胜利
775 const currentLevelCopy = currentLevel;
776 if (checkWin(newMap)) {
777   setTimeout(() => {
778     if (currentLevelCopy < 2) {
779       initLevel(currentLevelCopy + 1);
780     } else {
781       onWin(currentLevelCopy + 1);
782     }
783   }, 500);
784 }
785 };
786
787 /**
788  * 获取格子内容和样式
789  */
790 const getCellContent = (char: string, x: number, y: number): { type: string; content: string } => {
791   if (playerPos.x === x && playerPos.y === y) {
792     return { type: 'player', content: '' };
793   }
794   switch (char) {
795     case '#': return { type: 'wall', content: '' };
796     case '$': return { type: 'box', content: '' };

```



```

797     case '*': return { type: 'box-on-target', content: '' };
798     case '.': return { type: 'target', content: '' };
799     default: return { type: 'floor', content: '' };
800 }
801 };
802
803 const level = getLevelByIndex(currentLevel);
804
805 return (
806   <View style={styles.gameContainer}>
807     /* 头部 */
808     <View style={styles.sokobanHeader}>
809       <Text style={styles.levelName}>{level.name} / {sokobanLevels.length}</Text>
810       <TouchableOpacity style={styles.newGameBtn} onPress={() => initLevel(0)}>
811         <Ionicons name="refresh" size={16} color={colors.primary} />
812         <Text style={styles.newGameText}>重新开始</Text>
813       </TouchableOpacity>
814     </View>
815
816     /* 游戏地图 */
817     <View style={styles.sokobanGrid}>
818       {gameMap.map((row, rowIndex) => (
819         <View key={rowIndex} style={styles.sokobanRow}>
820           {row.map((char, colIndex) => {
821             const { type, content } = getCellContent(char, colIndex, rowIndex);
822             const cellStyle = type === 'wall' ? styles.sokobanCellWall :
823               type === 'floor' ? styles.sokobanCellFloor :
824               type === 'target' ? styles.sokobanCellTarget :
825               type === 'player' ? styles.sokobanCellPlayer :
826               type === 'box' ? styles.sokobanCellBox :
827               styles.sokobanCellBoxOnTarget;
828             return (
829               <View key={colIndex} style={[styles.sokobanCell, cellStyle]}>
830                 <Text style={styles.sokobanCellText}>{content}</Text>
831               </View>
832             );
833           })}
834         </View>
835       ))}
836     </View>
837
838     /* 方向控制 */
839     <View style={styles.sokobanControls}>
840       <TouchableOpacity style={styles.sokobanBtn} onPress={() => move(0, -1)}>
841         <Text style={styles.sokobanBtnText}></Text>
842       </TouchableOpacity>
843       <View style={styles.sokobanBtnRow}>
844         <TouchableOpacity style={styles.sokobanBtn} onPress={() => move(-1, 0)}>
845           <Text style={styles.sokobanBtnText}></Text>
846         </TouchableOpacity>

```

```

847     <TouchableOpacity style={styles.sokobanBtn} onPress={() => move(0, 0)}>
848       <Text style={styles.sokobanBtnText}></Text>
849     </TouchableOpacity>
850     <TouchableOpacity style={styles.sokobanBtn} onPress={() => move(1, 0)}>
851       <Text style={styles.sokobanBtnText}></Text>
852     </TouchableOpacity>
853   </View>
854   <TouchableOpacity style={styles.sokobanBtn} onPress={() => move(0, 1)}>
855     <Text style={styles.sokobanBtnText}></Text>
856   </TouchableOpacity>
857 </View>
858
859 </View>
860 );
861 };
862
863 // ===== 主游戏弹窗组件 =====
864
865 /**
866  * 游戏弹窗主组件
867  * 包含游戏选择、教程和游戏界面
868  */
869 const GameModal = ({ visible, onClose, onGameComplete, monsterType }: GameModalProps) => {
870   const { colors } = useTheme();
871
872   // ===== 样式 =====
873
874   const styles = useMemo(() => StyleSheet.create({
875     container: {
876       flex: 1,
877       backgroundColor: colors.background,
878     },
879     header: {
880       flexDirection: 'row',
881       justifyContent: 'space-between',
882       alignItems: 'center',
883       padding: 16,
884       paddingTop: 40,
885       borderBottomWidth: 1,
886       borderBottomColor: 'colors.borderDark',
887     },
888     closeBtn: {
889       padding: 8,
890     },
891     title: {
892       fontSize: 20,
893       fontWeight: 'bold',
894       color: colors.textPrimary,
895     },
896     placeholder: {

```

```

897   width: 44,
898 },
899 welcomeContent: {
900   flex: 1,
901   padding: 16,
902 },
903 welcomeSection: {
904   textAlign: 'center',
905   marginBottom: 24,
906 },
907 welcomeTitle: {
908   fontSize: 28,
909   fontWeight: 'bold',
910   color: colors.primary,
911   marginBottom: 8,
912 },
913 welcomeSubtitle: {
914   fontSize: 14,
915   color: colors.textSecondary,
916 },
917 sectionTitle: {
918   fontSize: 16,
919   fontWeight: 'bold',
920   color: colors.warning,
921   marginBottom: 12,
922 },
923 rewardsSection: {
924   backgroundColor: 'rgba(255,215,0,0.08)',
925   borderRadius: 12,
926   padding: 16,
927   marginBottom: 16,
928 },
929 rewardsRow: {
930   flexDirection: 'row',
931   justifyContent: 'center',
932   gap: 40,
933 },
934 rewardBox: {
935   alignItems: 'center',
936 },
937 rewardIcon: {
938   fontSize: 32,
939   marginBottom: 4,
940 },
941 rewardAmount: {
942   fontSize: 20,
943   fontWeight: 'bold',
944   color: colors.warning,
945 },
946 rewardName: {

```

```

947   fontSize: 12,
948   color: colors.textSecondary,
949 },
950 rewardNote: {
951   fontSize: 12,
952   color: colors.textSecondary,
953   textAlign: 'center',
954   marginTop: 12,
955 },
956 gamesSection: {
957   marginBottom: 16,
958 },
959 gameCard: {
960   flexDirection: 'row',
961   alignItems: 'center',
962   backgroundColor: colors.surface,
963   borderRadius: 12,
964   padding: 16,
965   marginBottom: 12,
966   borderWidth: 1,
967   borderColor: 'colors.borderDark',
968 },
969 gameCardIcon: {
970   fontSize: 36,
971   marginRight: 12,
972 },
973 gameCardInfo: {
974   flex: 1,
975 },
976 gameCardTitle: {
977   fontSize: 16,
978   fontWeight: 'bold',
979   color: colors.textPrimary,
980   marginBottom: 4,
981 },
982 gameCardDesc: {
983   fontSize: 12,
984   color: colors.textSecondary,
985 },
986 gameCardArrow: {
987   fontSize: 20,
988   color: colors.primary,
989 },
990 tipsSection: {
991   backgroundColor: 'rgba(123,117,216,0.08)',
992   borderRadius: 12,
993   padding: 16,
994 },
995 tipsText: {
996   fontSize: 12,

```

```

997     color: colors.textSecondary,
998     lineHeight: 20,
999   },
1000   tipsList: {
1001     gap: 6,
1002   },
1003   tiplItem: {
1004     fontSize: 12,
1005     color: colors.textSecondary,
1006     lineHeight: 20,
1007   },
1008   tutorialContent: {
1009     flex: 1,
1010     padding: 16,
1011   },
1012   backBtn: {
1013     flexDirection: 'row',
1014     alignItems: 'center',
1015     gap: 4,
1016     marginBottom: 16,
1017     zIndex: 200,
1018     position: 'relative',
1019   },
1020   backBtnText: {
1021     color: colors.textSecondary,
1022     fontSize: 14,
1023   },
1024   tutorialHeader: {
1025     flexDirection: 'row',
1026     alignItems: 'center',
1027     gap: 12,
1028     marginBottom: 20,
1029   },
1030   tutorialIcon: {
1031     fontSize: 40,
1032   },
1033   tutorialTitle: {
1034     fontSize: 22,
1035     fontWeight: 'bold',
1036     color: colors.textPrimary,
1037   },
1038   tutorialSection: {
1039     marginBottom: 24,
1040     paddingBottom: 8,
1041   },
1042   tutorialSectionTitle: {
1043     fontSize: 16,
1044     fontWeight: 'bold',
1045     color: colors.warning,
1046     marginBottom: 8,

```

```

1047 },
1048 tutorialText: {
1049   fontSize: 14,
1050   color: colors.textSecondary,
1051   lineHeight: 22,
1052 },
1053 tutorialSteps: {
1054   gap: 6,
1055 },
1056 tutorialStep: {
1057   fontSize: 14,
1058   color: colors.textSecondary,
1059   lineHeight: 22,
1060 },
1061 startBtn: {
1062   backgroundColor: colors.primary,
1063   borderRadius: 12,
1064   padding: 16,
1065   alignItems: 'center',
1066   marginTop: 32,
1067   marginBottom: 32,
1068 },
1069 startBtnText: {
1070   fontSize: 16,
1071   fontWeight: 'bold',
1072   color: '#FFFFFF',
1073 },
1074 gameScreen: {
1075   flex: 1,
1076   padding: 16,
1077   paddingTop: 40,
1078 },
1079 gameContent: {
1080   flex: 1,
1081 },
1082 gameContainer: {
1083   flex: 1,
1084 },
1085 // 数独样式
1086 sudokuHeader: {
1087   flexDirection: 'row',
1088   justifyContent: 'space-between',
1089   alignItems: 'center',
1090   marginBottom: 16,
1091 },
1092 sudokuTitle: {
1093   fontSize: 18,
1094   fontWeight: 'bold',
1095   color: colors.textPrimary,
1096 },

```

```

1097 newGameBtn: {
1098   flexDirection: 'row',
1099   alignItems: 'center',
1100   gap: 4,
1101   padding: 8,
1102   backgroundColor: 'colors.borderLight',
1103   borderRadius: 8,
1104 },
1105 newGameText: {
1106   fontSize: 12,
1107   color: colors.primary,
1108 },
1109 sudokuGrid: {
1110   backgroundColor: colors.backgroundDark,
1111   borderRadius: 8,
1112   padding: 4,
1113   marginBottom: 16,
1114   borderWidth: 2,
1115   borderColor: colors.primary,
1116 },
1117 sudokuRow: {
1118   flexDirection: 'row',
1119 },
1120 sudokuCell: {
1121   flex: 1,
1122   aspectRatio: 1,
1123   alignItems: 'center',
1124   justifyContent: 'center',
1125   borderWidth: 1,
1126   borderColor: 'colors.borderDark',
1127 },
1128 sudokuCellText: {
1129   fontSize: 18,
1130   fontWeight: 'bold',
1131 },
1132 sudokuGiven: {
1133   color: colors.primary,
1134 },
1135 sudokuInput: {
1136   color: colors.warning,
1137 },
1138 sudokuSolution: {
1139   color: colors.success,
1140 },
1141 numberPad: {
1142   flexDirection: 'row',
1143   flexWrap: 'wrap',
1144   gap: 6,
1145   marginBottom: 16,
1146 },

```

```

1147 numberBtn: {
1148   width: '18%',
1149   aspectRatio: 1,
1150   backgroundColor: 'colors.borderDark',
1151   borderRadius: 8,
1152   alignItems: 'center',
1153   justifyContent: 'center',
1154 },
1155 numberBtnText: {
1156   fontSize: 18,
1157   fontWeight: 'bold',
1158   color: colors.primary,
1159 },
1160 clearBtn: {
1161   width: '18%',
1162   aspectRatio: 1,
1163   backgroundColor: 'rgba(255,107,107,0.2)',
1164   borderRadius: 8,
1165   alignItems: 'center',
1166   justifyContent: 'center',
1167 },
1168 clearBtnText: {
1169   fontSize: 12,
1170   fontWeight: 'bold',
1171   color: '#FF6B6B',
1172 },
1173 hintBtn: {
1174   backgroundColor: 'rgba(255,215,0,0.15)',
1175   borderRadius: 10,
1176   padding: 14,
1177   alignItems: 'center',
1178   borderWidth: 1,
1179   borderColor: 'rgba(255,215,0,0.4)',
1180 },
1181 hintBtnText: {
1182   fontSize: 14,
1183   fontWeight: 'bold',
1184   color: colors.warning,
1185 },
1186 // 推箱子样式
1187 sokobanHeader: {
1188   flexDirection: 'row',
1189   justifyContent: 'space-between',
1190   alignItems: 'center',
1191   marginBottom: 12,
1192 },
1193 levelName: {
1194   fontSize: 16,
1195   fontWeight: 'bold',
1196   color: colors.warning,

```



```

1197 },
1198 sokobanGrid: {
1199   backgroundColor: colors.backgroundDark,
1200   borderRadius: 8,
1201   padding: 8,
1202   marginBottom: 16,
1203   borderWidth: 2,
1204   borderColor: colors.orange,
1205 },
1206 sokobanRow: {
1207   flexDirection: 'row',
1208   justifyContent: 'center',
1209 },
1210 sokobanCell: {
1211   width: 40,
1212   height: 40,
1213   alignItems: 'center',
1214   justifyContent: 'center',
1215   borderRadius: 4,
1216   margin: 1,
1217 },
1218 sokobanCellWall: {
1219   backgroundColor: '#3A3A5C',
1220 },
1221 sokobanCellFloor: {
1222   backgroundColor: colors.background,
1223 },
1224 sokobanCellTarget: {
1225   backgroundColor: 'rgba(255,215,0,0.2)',
1226 },
1227 sokobanCellPlayer: {
1228   backgroundColor: 'colors.borderDark',
1229 },
1230 sokobanCellBox: {
1231   backgroundColor: 'rgba(255,125,0,0.4)',
1232 },
1233 sokobanCellBoxOnTarget: {
1234   backgroundColor: 'rgba(58,227,116,0.4)',
1235 },
1236 sokobanCellText: {
1237   fontSize: 20,
1238 },
1239 sokobanControls: {
1240   alignItems: 'center',
1241   gap: 4,
1242   marginBottom: 16,
1243 },
1244 sokobanBtnRow: {
1245   flexDirection: 'row',
1246   gap: 4,

```

```

1247 },
1248 sokobanBtn: {
1249   width: 55,
1250   height: 55,
1251   backgroundColor: 'colors.borderDark',
1252   borderRadius: 10,
1253   alignItems: 'center',
1254   justifyContent: 'center',
1255   borderWidth: 1,
1256   borderColor: 'colors.borderDark',
1257 },
1258 sokobanBtnText: {
1259   fontSize: 22,
1260 },
1261 answerContainer: {
1262   backgroundColor: 'rgba(255,215,0,0.08)',
1263   borderRadius: 10,
1264   padding: 12,
1265   marginBottom: 12,
1266   borderWidth: 1,
1267   borderColor: 'rgba(255,215,0,0.25)',
1268 },
1269 answerTitle: {
1270   fontSize: 13,
1271   fontWeight: 'bold',
1272   color: colors.warning,
1273   marginBottom: 8,
1274 },
1275 answerSteps: {
1276   flexDirection: 'row',
1277   flexWrap: 'wrap',
1278   gap: 8,
1279   marginBottom: 8,
1280 },
1281 stepPending: {
1282   fontSize: 14,
1283   color: colors.textSecondary,
1284 },
1285 stepDone: {
1286   fontSize: 14,
1287   color: colors.success,
1288 },
1289 answerButtons: {
1290   flexDirection: 'row',
1291   gap: 8,
1292 },
1293 smallBtn: {
1294   flex: 1,
1295   padding: 8,
1296   backgroundColor: 'colors.borderDark',

```

```

1297   borderRadius: 6,
1298   alignItems: 'center',
1299 },
1300 smallBtnText: {
1301   fontSize: 11,
1302   color: colors.primary,
1303   fontWeight: 'bold',
1304 },
1305 winOverlay: {
1306   position: 'absolute',
1307   top: 0,
1308   left: 0,
1309   right: 0,
1310   bottom: 0,
1311   backgroundColor: 'rgba(0,0,0,0.7)',
1312   alignItems: 'center',
1313   justifyContent: 'center',
1314   zIndex: 100,
1315   pointerEvents: 'none',
1316 },
1317 winText: {
1318   fontSize: 32,
1319   fontWeight: 'bold',
1320   color: colors.warning,
1321 },
1322 submittingText: {
1323   fontSize: 14,
1324   color: colors.textSecondary,
1325   marginTop: 16,
1326 },
1327 )), [colors]);
1328  /** 当前显示的屏幕 */
1329  const [currentScreen, setCurrentScreen] = useState<Screen>('welcome');
1330  /** 当前选中的游戏类型 */
1331  const [currentGame, setCurrentGame] = useState<GameType>('sudoku');
1332  /** 是否游戏胜利 */
1333  const [gameWon, setGameWon] = useState(false);
1334  /** 是否正在提交结果 */
1335  const [isSubmitting, setIsSubmitting] = useState(false);
1336
1337  /**
1338   * 处理游戏胜利
1339   */
1340  const handleWin = async (level?: number) => {
1341    console.log('[GameModal] 游戏胜利 - 关卡:', level);
1342    setGameWon(true);
1343    setIsSubmitting(true);
1344
1345    try {
1346      const result: GameResult = await mockSubmitGameResult(

```

```

1347     level || 1,
1348     true,
1349     monsterType
1350   );
1351
1352   if (result.success) {
1353     console.log('[GameModal] 游戏结果提交成功 - 奖励:', result.rewards);
1354     setTimeout(() => {
1355       setGameWon(false);
1356       setIsSubmitting(false);
1357       onGameComplete(result.rewards);
1358     }, 500);
1359   } else {
1360     throw new Error(result.message || '游戏提交失败');
1361   }
1362 } catch (error) {
1363   console.error('[GameModal] 游戏提交失败:', error);
1364   setIsSubmitting(false);
1365   setGameWon(false);
1366   Alert.alert('提交失败', '游戏结果提交失败, 请重试', [
1367     { text: '重试', onPress: () => handleWin(level) },
1368     { text: '取消', style: 'cancel' }
1369   ]);
1370 }
1371 };
1372
1373 /**
1374  * 显示游戏教程
1375  */
1376 const showTutorial = (gameType: GameType) => {
1377   setCurrentGame(gameType);
1378   setCurrentScreen('tutorial');
1379 };
1380
1381 /**
1382  * 开始游戏
1383  */
1384 const startGame = () => {
1385   setCurrentScreen('game');
1386   setGameWon(false);
1387 };
1388
1389 /**
1390  * 返回上一步
1391  */
1392 const goBack = () => {
1393   if (currentScreen === 'game') {
1394     Alert.alert('确认退出', '退出后当前游戏进度将丢失, 确定退出吗? ', [
1395       { text: '取消', style: 'cancel' },
1396       { text: '确定', onPress: () => {

```

```

1397     setCurrentScreen('tutorial');
1398     setGameWon(false);
1399   }}
1400   });
1401   } else {
1402     setCurrentScreen('welcome');
1403     setGameWon(false);
1404   }
1405   };
1406
1407   /**
1408    * 关闭弹窗
1409    */
1410   const handleClose = () => {
1411     if (currentScreen === 'game') {
1412       Alert.alert(' 确认退出', '退出后当前游戏进度将丢失， 确定退出吗? ', [
1413         { text: '取消', style: 'cancel' },
1414         { text: '确定', onPress: () => {
1415           setCurrentScreen('welcome');
1416           setGameWon(false);
1417           onClose();
1418         }}
1419       ]);
1420     } else {
1421       setCurrentScreen('welcome');
1422       setGameWon(false);
1423       onClose();
1424     }
1425   };
1426
1427   return (
1428     <Modal
1429       visible={visible}
1430       animationType="slide"
1431       onRequestClose={handleClose}
1432     >
1433     <View style={styles.container}>
1434       {/* 头部栏 */}
1435       <View style={styles.header}>
1436         <TouchableOpacity onPress={handleClose} style={styles.closeBtn}>
1437           <Ionicons name="close" size={28} color={colors.textSecondary} />
1438         </TouchableOpacity>
1439         <Text style={styles.title}>游戏恢复</Text>
1440         <View style={styles.placeholder} />
1441       </View>
1442
1443       {/* 欢迎页面 */}
1444       {currentScreen === 'welcome' && (
1445         <ScrollView style={styles.welcomeContent} showsVerticalScrollIndicator={false}>
1446         <View style={styles.welcomeSection}>

```

```

1447     <Text style={styles.welcomeTitle}> 游戏恢复</Text>
1448     <Text style={styles.welcomeSubtitle}>完成小游戏，恢复体力和能量！</Text>
1449   </View>
1450
1451   {/* 奖励说明 */}
1452   <View style={styles.rewardsSection}>
1453     <Text style={styles.sectionTitle}> 游戏奖励</Text>
1454     <View style={styles.rewardsRow}>
1455       <View style={styles.rewardBox}>
1456         <Text style={styles.rewardIcon}> </Text>
1457         <Text style={styles.rewardAmount}> +5</Text>
1458         <Text style={styles.rewardName}>体力</Text>
1459       </View>
1460       <View style={styles.rewardBox}>
1461         <Text style={styles.rewardIcon}> </Text>
1462         <Text style={styles.rewardAmount}> +3</Text>
1463         <Text style={styles.rewardName}>能量</Text>
1464       </View>
1465     </View>
1466     <Text style={styles.rewardNote}> 叛逆小怪可获得双倍奖励！</Text>
1467   </View>
1468
1469   {/* 游戏选择 */}
1470   <View style={styles.gamesSection}>
1471     <Text style={styles.sectionTitle}> 选择游戏</Text>
1472
1473     <TouchableOpacity
1474       style={styles.gameCard}
1475       onPress={() => showTutorial('sudoku')}
1476     >
1477       <View style={styles.gameCardIcon}> </View>
1478       <View style={styles.gameCardInfo}>
1479         <Text style={styles.gameCardTitle}>数独</Text>
1480         <Text style={styles.gameCardDesc}>
1481           填写9×9网格，使每行、每列、每个3×3宫格都包含1-9不重复的数字。完成一局即可获得奖励！
1482         </Text>
1483       </View>
1484       <Text style={styles.gameCardArrow}>→</Text>
1485     </TouchableOpacity>
1486
1487     <TouchableOpacity
1488       style={styles.gameCard}
1489       onPress={() => showTutorial('sokoban')}
1490     >
1491       <View style={styles.gameCardIcon}> </View>
1492       <View style={styles.gameCardInfo}>
1493         <Text style={styles.gameCardTitle}>推箱子</Text>
1494         <Text style={styles.gameCardDesc}>
1495           将所有箱子推到目标位置上即可过关。完成3关即可获得奖励！
1496         </Text>

```

```

1497     </View>
1498     <Text style={styles.gameCardArrow}>→</Text>
1499 </TouchableOpacity>
1500 </View>
1501
1502 {/* 游戏规则 */}
1503 <View style={styles.tipsSection}>
1504   <Text style={styles.sectionTitle}> 游戏规则</Text>
1505   <View style={styles.tipsList}>
1506     <Text style={styles.tipltem}>• 数独：完成一局即可获得奖励</Text>
1507     <Text style={styles.tipltem}>• 推箱子：完成3关即可获得奖励</Text>
1508     <Text style={styles.tipltem}>• 每天可游玩3次，凌晨5点重置次数</Text>
1509     <Text style={styles.tipltem}>• 完成游戏后奖励会自动添加到你的账户</Text>
1510     <Text style={styles.tipltem}>• 游戏过程中可随时退出，不会消耗次数</Text>
1511   </View>
1512 </View>
1513 </ScrollView>
1514 }}
1515
1516 {/* 教程页面 */}
1517 {currentScreen === 'tutorial' && (
1518   <ScrollView style={styles.tutorialContent} showsVerticalScrollIndicator={false}>
1519     <TouchableOpacity onPress={goBack} style={styles.backBtn}>
1520       <Ionicons name="arrow-back" size={20} color={colors.textSecondary} />
1521       <Text style={styles.backBtnText}>返回</Text>
1522     </TouchableOpacity>
1523
1524     {/* 数独教程 */}
1525     {currentGame === 'sudoku' && (
1526       <>
1527         <View style={styles.tutorialHeader}>
1528           <Text style={styles.tutorialIcon}></Text>
1529           <Text style={styles.tutorialTitle}>数独教程</Text>
1530         </View>
1531
1532         <View style={styles.tutorialSection}>
1533           <Text style={styles.tutorialSectionTitle}> 游戏目标</Text>
1534           <Text style={styles.tutorialText}>
1535             在9×9的网格中填入数字1-9，使每行、每列、每个3×3宫格内的数字都不重复。
1536           </Text>
1537         </View>
1538
1539         <View style={styles.tutorialSection}>
1540           <Text style={styles.tutorialSectionTitle}> 操作方式</Text>
1541           <View style={styles.tutorialSteps}>
1542             <Text style={styles.tutorialStep}>1. 点击空格选中它</Text>
1543             <Text style={styles.tutorialStep}>2. 点击下方数字填入</Text>
1544             <Text style={styles.tutorialStep}>3. 完成所有格子即可过关</Text>
1545           </View>
1546         </View>

```

```

1547
1548     <TouchableOpacity style={styles.startBtn} onPress={startGame}>
1549         <Text style={styles.startBtnText}>开始游戏</Text>
1550     </TouchableOpacity>
1551 </>
1552 }}
1553
1554  {/* 推箱子教程 */}
1555 {currentGame === 'sokoban' && (
1556     <>
1557         <View style={styles.tutorialHeader}>
1558             <Text style={styles.tutorialIcon}></Text>
1559             <Text style={styles.tutorialTitle}>推箱子教程</Text>
1560         </View>
1561
1562         <View style={styles.tutorialSection}>
1563             <Text style={styles.tutorialSectionTitle}> 游戏目标</Text>
1564             <Text style={styles.tutorialText}>
1565                 将所有箱子推到目标位置 () 上即可过关。箱子只能推，不能拉。完成3关即可获得奖励!
1566             </Text>
1567         </View>
1568
1569         <View style={styles.tutorialSection}>
1570             <Text style={styles.tutorialSectionTitle}> 操作方式</Text>
1571             <View style={styles.tutorialSteps}>
1572                 <Text style={styles.tutorialStep}>1. 点击方向键移动角色 () </Text>
1573                 <Text style={styles.tutorialStep}>2. 中间按钮重置当前关卡</Text>
1574                 <Text style={styles.tutorialStep}>3. 完成当前关卡自动进入下一关</Text>
1575                 <Text style={styles.tutorialStep}>4. 完成3关后自动获得奖励</Text>
1576             </View>
1577         </View>
1578
1579         <TouchableOpacity style={styles.startBtn} onPress={startGame}>
1580             <Text style={styles.startBtnText}>开始游戏</Text>
1581         </TouchableOpacity>
1582     </>
1583 }}
1584 </ScrollView>
1585 }}
1586
1587  {/* 游戏界面 */}
1588 {currentScreen === 'game' && (
1589     <View style={styles.gameScreen}>
1590         <TouchableOpacity onPress={goBack} style={styles.backBtn}>
1591             <Ionicons name="arrow-back" size={20} color={colors.textSecondary} />
1592             <Text style={styles.backBtnText}>返回</Text>
1593         </TouchableOpacity>
1594
1595          {/* 游戏内容容器 */}
1596         <View style={styles.gameContent}>

```



```

1597     {/* 胜利遮罩 */}
1598     {gameWon && (
1599         <View style={styles.winOverlay}>
1600             <Text style={styles.winText}> 恭喜过关! </Text>
1601             {isSubmitting && (
1602                 <Text style={styles.submittingText}>正在提交奖励...</Text>
1603             )}
1604         </View>
1605     )}
1606
1607     {/* 根据游戏类型显示对应游戏 */}
1608     {currentGame === 'sudoku' && <SudokuGame onWin={() => handleWin(1)} colors={colors} />}
1609     {currentGame === 'sokoban' && <SokobanGame onWin={handleWin} monsterType={monsterType} colors={c
1610     </View>
1611 </View>
1612 }}
1613 </View>
1614 </Modal>
1615 );
1616 };
1617
1618
1619 export default GameModal;
1620
1621
=== mobile/src/components/HelpModal.tsx ===
1621 import React, { useState, useMemo } from 'react';
1622 import { View, Text, StyleSheet, ScrollView, TouchableOpacity, Alert, TextInput } from 'react-native';
1623 import { Ionicons } from '@expo/vector-icons';
1624 import { useTheme } from '../contexts/ThemeContext';
1625
1626 /**
1627  * 帮助与反馈弹窗组件属性
1628  */
1629 interface HelpModalProps {
1630     /** 是否显示弹窗 */
1631     visible: boolean;
1632     /** 关闭弹窗回调 */
1633     onClose: () => void;
1634 }
1635
1636 /**
1637  * 反馈问题示例数据
1638  */
1639 const feedbackExamples = [
1640     { id: 1, title: '体力异常扣除', description: '进入游戏时体力未正确扣除, 或游戏结束后奖励未正确添加' },
1641     { id: 2, title: '学习资源不存在', description: '跳转后的学习资源页面无法打开或内容为空' },
1642     { id: 3, title: '游戏无法正常进行', description: '推箱子/数独游戏无法正常开始、关卡无法切换或无法提交' },
1643     { id: 4, title: '怪兽数据异常', description: '怪兽等级、经验、体力值等数据显示异常或无法保存' },
1644     { id: 5, title: '任务系统问题', description: '任务无法添加、完成或删除, 进度无法保存' },
1645     { id: 6, title: '专注计时问题', description: '专注计时器无法启动、暂停或计时不准确' },

```

```

1646 ];
1647
1648 /**
1649  * 帮助与反馈弹窗组件
1650  * 包含使用说明、问题反馈示例和联系信息
1651  */
1652 const HelpModal = ({ visible, onClose }: HelpModalProps) => {
1653   const { colors } = useTheme();
1654   /** 当前选中的问题示例 */
1655   const [selectedExample, setSelectedExample] = useState<number | null>(null);
1656   /** 输入的反馈内容 */
1657   const [feedbackText, setFeedbackText] = useState("");
1658
1659   /**
1660    * 提交反馈处理
1661    */
1662   const handleSubmitFeedback = () => {
1663     // 验证反馈内容不为空
1664     if (!feedbackText.trim()) {
1665       Alert.alert('提示', '请输入问题描述');
1666       return;
1667     }
1668
1669     // 显示提交成功提示 (模拟)
1670     Alert.alert(
1671       '提交成功',
1672       '感谢您的反馈, 我们会尽快处理! ',
1673       [{ text: '确定', onPress: () => {
1674         setFeedbackText("");
1675         setSelectedExample(null);
1676       }}]
1677     );
1678   };
1679
1680   const styles = useMemo(() => StyleSheet.create({
1681     modalContainer: {
1682       position: 'absolute',
1683       top: 0,
1684       left: 0,
1685       right: 0,
1686       bottom: 0,
1687       backgroundColor: 'rgba(0, 0, 0, 0.5)',
1688       zIndex: 1000,
1689       alignItems: 'center',
1690       justifyContent: 'center',
1691     },
1692     modalVisible: { opacity: 1 },
1693     modalHidden: { opacity: 0, pointerEvents: 'none' },
1694     modalContent: {
1695       width: '90%',

```

```

1696     maxHeight: '85%',
1697     backgroundColor: colors.background,
1698     borderRadius: 24,
1699     overflow: 'hidden',
1700 },
1701 header: {
1702     flexDirection: 'row',
1703     justifyContent: 'space-between',
1704     alignItems: 'center',
1705     padding: 16,
1706     paddingTop: 32,
1707     borderBottomWidth: 1,
1708     borderBottomColor: colors.borderDark,
1709 },
1710 closeBtn: { padding: 8 },
1711 title: {
1712     color: colors.textPrimary,
1713     fontSize: 20,
1714     fontWeight: '700',
1715     fontFamily: 'Courier',
1716 },
1717 placeholder: { width: 44 },
1718 content: {
1719     paddingHorizontal: 20,
1720     paddingTop: 8,
1721 },
1722 section: { marginBottom: 28 },
1723 sectionTitle: {
1724     color: colors.primary,
1725     fontSize: 16,
1726     fontWeight: '700',
1727     marginBottom: 14,
1728 },
1729 sectionDivider: {
1730     height: 1,
1731     backgroundColor: colors.borderDark,
1732     marginBottom: 14,
1733 },
1734 sectionDesc: {
1735     color: colors.textSecondary,
1736     fontSize: 13,
1737     marginBottom: 14,
1738     lineHeight: 20,
1739 },
1740 helpItem: {
1741     backgroundColor: colors.borderLight,
1742     borderRadius: 12,
1743     padding: 14,
1744     marginBottom: 10,
1745 },

```

```

1746   helpTitle: {
1747     color: colors.textPrimary,
1748     fontSize: 14,
1749     fontWeight: '600',
1750     marginBottom: 6,
1751   },
1752   helpText: {
1753     color: colors.textSecondary,
1754     fontSize: 13,
1755     lineHeight: 22,
1756   },
1757   examplesList: { marginBottom: 18 },
1758   exampleItem: {
1759     flexDirection: 'row',
1760     alignItems: 'flex-start',
1761     padding: 14,
1762     backgroundColor: colors.borderLight,
1763     borderRadius: 12,
1764     marginBottom: 8,
1765     borderWidth: 1,
1766     borderColor: 'transparent',
1767   },
1768   exampleItemSelected: {
1769     backgroundColor: colors.borderDark,
1770     borderColor: colors.borderDark,
1771   },
1772   exampleRadio: {
1773     width: 20,
1774     height: 20,
1775     borderRadius: 10,
1776     borderWidth: 2,
1777     borderColor: colors.textTertiary,
1778     alignItems: 'center',
1779     justifyContent: 'center',
1780     marginTop: 2,
1781     marginRight: 12,
1782   },
1783   exampleRadioInner: {
1784     width: 10,
1785     height: 10,
1786     borderRadius: 5,
1787     backgroundColor: colors.primary,
1788   },
1789   exampleContent: { flex: 1 },
1790   exampleTitle: {
1791     color: colors.textPrimary,
1792     fontSize: 14,
1793     fontWeight: '600',
1794     marginBottom: 4,
1795   },

```

```

1796 exampleDesc: {
1797   color: colors.textSecondary,
1798   fontSize: 12,
1799   lineHeight: 20,
1800 },
1801 feedbackInputContainer: { marginBottom: 18 },
1802 feedbackLabel: {
1803   color: colors.textSecondary,
1804   fontSize: 13,
1805   marginBottom: 10,
1806 },
1807 feedbackInput: {
1808   backgroundColor: colors.borderLight,
1809   borderRadius: 12,
1810   padding: 14,
1811   color: colors.textPrimary,
1812   fontSize: 14,
1813   minHeight: 100,
1814   textAlignVertical: 'top',
1815   lineHeight: 22,
1816 },
1817 submitBtn: {
1818   backgroundColor: colors.primary,
1819   borderRadius: 12,
1820   padding: 14,
1821   alignItems: 'center',
1822 },
1823 submitBtnText: {
1824   color: 'FFFFFF',
1825   fontSize: 15,
1826   fontWeight: '600',
1827 },
1828 contactText: {
1829   color: colors.textSecondary,
1830   fontSize: 13,
1831   lineHeight: 22,
1832   marginBottom: 14,
1833 },
1834 contactCard: {
1835   backgroundColor: colors.borderLight,
1836   borderRadius: 12,
1837   padding: 14,
1838 },
1839 contactItem: {
1840   color: colors.textPrimary,
1841   fontSize: 13,
1842   lineHeight: 22,
1843   marginBottom: 8,
1844 },
1845 contactItemLast: { marginBottom: 0 },

```

```

1846    )), [colors]);
1847
1848     return (
1849       <View style={{[styles.modalContainer, visible ? styles.modalVisible : styles.modalHidden]}}>
1850         <View style={styles.modalContent}>
1851           {/* 头部 */}
1852           <View style={styles.header}>
1853             <TouchableOpacity onPress={onClose} style={styles.closeBtn}>
1854               <Ionicons name="close" size={28} color={colors.textSecondary} />
1855             </TouchableOpacity>
1856             <Text style={styles.title}>帮助与反馈</Text>
1857             <View style={styles.placeholder} />
1858           </View>
1859
1860           {/* 滚动内容区域 */}
1861           <ScrollView style={styles.content} contentContainerStyle={{ paddingBottom: 40 }} showsVerticalScrollIndicator={false}>
1862             {/* 使用帮助 */}
1863             <View style={styles.section}>
1864               <Text style={styles.sectionTitle}>使用帮助</Text>
1865               <View style={styles.sectionDivider} />
1866               <View style={styles.helpItem}>
1867                 <Text style={styles.helpTitle}>体力系统</Text>
1868                 <Text style={styles.helpText}>
1869                   专注学习会消耗体力，完成小游戏可恢复体力。游戏说明请在怪兽页面查看。
1870                 </Text>
1871               </View>
1872               <View style={styles.helpItem}>
1873                 <Text style={styles.helpTitle}>学习资源</Text>
1874                 <Text style={styles.helpText}>
1875                   在技能树页面跳转学习资源时会消耗10点体力，请确保体力充足。
1876                 </Text>
1877               </View>
1878               <View style={styles.helpItem}>
1879                 <Text style={styles.helpTitle}>每日重置</Text>
1880                 <Text style={styles.helpText}>
1881                   每天凌晨5点重置游戏次数和每日数据。
1882                 </Text>
1883               </View>
1884             </View>
1885
1886             {/* 问题反馈 */}
1887             <View style={styles.section}>
1888               <Text style={styles.sectionTitle}>问题反馈</Text>
1889               <View style={styles.sectionDivider} />
1890               <Text style={styles.sectionDesc}>请选择您遇到的问题类型，或直接描述问题：</Text>
1891
1892               {/* 问题示例列表 */}
1893               <View style={styles.examplesList}>
1894                 {feedbackExamples.map((example) => (
1895                   <TouchableOpacity

```

```

1896         key={example.id}
1897         style={{
1898             styles.exampleItem,
1899             selectedExample === example.id && styles.exampleItemSelected
1900         }}
1901         onPress={() => {
1902             setSelectedExample(selectedExample === example.id ? null : example.id);
1903             if (selectedExample !== example.id) {
1904                 setFeedbackText(example.description);
1905             }
1906         }}
1907     >
1908     <View style={styles.exampleRadio}>
1909         {selectedExample === example.id && (
1910             <View style={styles.exampleRadioInner} />
1911         )}
1912     </View>
1913     <View style={styles.exampleContent}>
1914         <Text style={styles.exampleTitle}>{example.title}</Text>
1915         <Text style={styles.exampleDesc}>{example.description}</Text>
1916     </View>
1917 </TouchableOpacity>
1918 )))
1919 </View>
1920
1921 {/* 反馈输入框 */}
1922 <View style={styles.feedbackInputContainer}>
1923     <Text style={styles.feedbackLabel}>问题描述</Text>
1924     <TextInput
1925         style={styles.feedbackInput}
1926         placeholder="请详细描述您遇到的问题..."
1927         placeholderTextColor={colors.textTertiary}
1928         value={feedbackText}
1929         onChangeText={setFeedbackText}
1930         multiline
1931         numberOfLines={4}
1932     />
1933 </View>
1934
1935 {/* 提交按钮 */}
1936 <TouchableOpacity style={styles.submitBtn} onPress={handleSubmitFeedback}>
1937     <Text style={styles.submitBtnText}>提交反馈</Text>
1938 </TouchableOpacity>
1939 </View>
1940
1941 {/* 联系我们 */}
1942 <View style={styles.section}>
1943     <Text style={styles.sectionTitle}>联系我们</Text>
1944     <View style={styles.sectionDivider} />
1945     <Text style={styles.contactText}>

```

```

1946         如果您在使用过程中有任何问题或建议，欢迎随时联系我们：
1947     </Text>
1948     <View style={styles.contactCard}>
1949         <Text style={styles.contactItem}> 邮箱：support@learnflow.com</Text>
1950         <Text style={styles.contactItem, styles.contactItemLast}> 客服：工作日 9:00-18:00</Text>
1951     </View>
1952 </View>
1953 </ScrollView>
1954 </View>
1955 </View>
1956 );
1957 };
1958
1959 export default HelpModal;
1960
1961 === mobile/src/components/MiniGames.tsx ===
1962 import React, { useState, useEffect, useMemo } from 'react';
1963 import { View, Text, StyleSheet, TouchableOpacity, ScrollView } from 'react-native';
1964 import { Ionicons } from '@expo/vector-icons';
1965 import { useTheme } from '../contexts/ThemeContext';
1966
1967 type GameType = 'sudoku';
1968 type Screen = 'welcome' | 'tutorial' | 'game';
1969
1970 interface MiniGamesProps {
1971   onGameComplete: (rewards: { stamina: number; energy: number }) => void;
1972   onClose: () => void;
1973 }
1974
1975 // 数独部分
1976 const MiniGames = ({ onGameComplete, onClose }: MiniGamesProps) => {
1977   const { colors } = useTheme();
1978   const [currentScreen, setCurrentScreen] = useState<Screen>('welcome');
1979   const [currentGame, setCurrentGame] = useState<GameType>('sudoku');
1980   const [gameWon, setGameWon] = useState(false);
1981
1982   const handleWin = () => {
1983     console.log('[MiniGames] 游戏胜利');
1984     setGameWon(true);
1985   };
1986
1987   const handleCollectReward = () => {
1988     console.log('[MiniGames] 领取奖励');
1989     onGameComplete({ stamina: 5, energy: 3 });
1990   };
1991
1992   const showTutorial = (gameType: GameType) => {
1993     console.log('[MiniGames] 显示教程:', gameType);
1994     setCurrentGame(gameType);
1995     setCurrentScreen('tutorial');
1996   };

```



```

1995   };
1996
1997   const startGame = () => {
1998     console.log('[MiniGames] 开始游戏:', currentGame);
1999     setCurrentScreen('game');
2000     setGameWon(false);
2001   };
2002
2003   const goBack = () => {
2004     console.log('[MiniGames] 返回, 当前页面:', currentScreen);
2005     if (currentScreen === 'game') {
2006       setCurrentScreen('tutorial');
2007     } else if (currentScreen === 'tutorial') {
2008       setCurrentScreen('welcome');
2009     } else {
2010       setCurrentScreen('welcome');
2011     }
2012     setGameWon(false);
2013   };
2014
2015   const styles = useMemo(() => StyleSheet.create({
2016     container: {
2017       flex: 1,
2018       backgroundColor: colors.background,
2019     },
2020     header: {
2021       flexDirection: 'row',
2022       alignItems: 'center',
2023       justifyContent: 'space-between',
2024       paddingHorizontal: 20,
2025       paddingVertical: 16,
2026       backgroundColor: colors.card,
2027       borderBottomWidth: 1,
2028       borderBottomColor: colors.borderDark,
2029     },
2030     closeBtn: {
2031       alignSelf: 'flex-end',
2032       padding: 4,
2033       marginBottom: 8,
2034     },
2035     title: {
2036       color: colors.textPrimary,
2037       fontSize: 18,
2038       fontWeight: '700',
2039       fontFamily: 'Courier',
2040     },
2041     placeholder: {
2042       width: 36,
2043     },
2044     gameSelector: {

```

```

2045   flexDirection: 'row',
2046   padding: 16,
2047   gap: 12,
2048 },
2049 gameTab: {
2050   flex: 1,
2051   paddingVertical: 12,
2052   alignItems: 'center',
2053   borderRadius: 12,
2054   backgroundColor: colors.borderDark,
2055   borderWidth: 1,
2056   borderColor: colors.borderDark,
2057 },
2058 gameTabActive: {
2059   backgroundColor: 'rgba(123,117,216,0.3)',
2060   borderColor: colors.primary,
2061 },
2062 gameTabText: {
2063   color: colors.textSecondary,
2064   fontSize: 16,
2065   fontWeight: '600',
2066   fontFamily: 'Courier',
2067 },
2068 gameTabTextActive: {
2069   color: colors.primary,
2070 },
2071 content: {
2072   flex: 1,
2073 },
2074 gameContainer: {
2075   padding: 20,
2076   alignItems: 'center',
2077 },
2078 newGameBtn: {
2079   flexDirection: 'row',
2080   alignItems: 'center',
2081   gap: 6,
2082   paddingHorizontal: 16,
2083   paddingVertical: 8,
2084   borderRadius: 12,
2085   backgroundColor: colors.borderDark,
2086   marginBottom: 20,
2087 },
2088 sudokuHeader: {
2089   flexDirection: 'row',
2090   justifyContent: 'space-between',
2091   alignItems: 'center',
2092 },
2093 solutionBtnActive: {
2094   backgroundColor: 'rgba(255,107,107,0.1)',

```

```

2095 },
2096 newGameText: {
2097   color: colors.primary,
2098   fontSize: 14,
2099   fontWeight: '600',
2100   fontFamily: 'Courier',
2101 },
2102 // 数独样式
2103 sudokuGrid: {
2104   width: '100%',
2105   aspectRatio: 1,
2106   flexDirection: 'row',
2107   flexWrap: 'wrap',
2108   backgroundColor: colors.card,
2109   borderWidth: 2,
2110   borderColor: colors.primary,
2111   borderRadius: 12,
2112   overflow: 'hidden',
2113 },
2114 sudokuCell: {
2115   width: '11.111%',
2116   aspectRatio: 1,
2117   alignItems: 'center',
2118   justifyContent: 'center',
2119   backgroundColor: colors.background,
2120   borderWidth: 0.5,
2121   borderColor: 'rgba(123,117,216,0.3)',
2122 },
2123 sudokuCellSelected: {
2124   backgroundColor: colors.borderDark,
2125 },
2126 sudokuCellGiven: {
2127   backgroundColor: colors.borderDark,
2128 },
2129 sudokuCellText: {
2130   fontSize: 18,
2131   fontWeight: '700',
2132   fontFamily: 'Courier',
2133 },
2134 sudokuCellTextGiven: {
2135   color: colors.primary,
2136 },
2137 sudokuCellTextUser: {
2138   color: colors.warning,
2139 },
2140 sudokuCellTextSolution: {
2141   color: colors.success,
2142   opacity: 0.7,
2143 },
2144 numberPad: {

```

```

2145   flexDirection: 'row',
2146   flexWrap: 'wrap',
2147   gap: 8,
2148   marginTop: 20,
2149   justifyContent: 'center',
2150 },
2151 numberBtn: {
2152   width: 48,
2153   height: 48,
2154   alignItems: 'center',
2155   justifyContent: 'center',
2156   backgroundColor: colors.borderDark,
2157   borderRadius: 12,
2158   borderWidth: 1,
2159   borderColor: 'rgba(123,117,216,0.3)',
2160 },
2161 numberBtnText: {
2162   color: colors.primary,
2163   fontSize: 20,
2164   fontWeight: '700',
2165   fontFamily: 'Courier',
2166 },
2167 numberBtnTextDisabled: {
2168   color: '#555',
2169 },
2170 numberPadDisabled: {
2171   opacity: 0.4,
2172 },
2173 clearBtn: {
2174   backgroundColor: 'rgba(255,107,107,0.15)',
2175   borderColor: 'rgba(255,107,107,0.3)',
2176 },
2177 // 推箱子样式
2178 sokobanHeader: {
2179   flexDirection: 'row',
2180   alignItems: 'center',
2181   gap: 16,
2182   marginBottom: 20,
2183 },
2184 levelName: {
2185   color: colors.warning,
2186   fontSize: 16,
2187   fontWeight: '700',
2188   fontFamily: 'Courier',
2189 },
2190 sokobanGrid: {
2191   backgroundColor: colors.card,
2192   borderWidth: 2,
2193   borderColor: '#FF7D00',
2194   borderRadius: 12,

```

```

2195 padding: 8,
2196 marginBottom: 24,
2197 },
2198 sokobanRow: {
2199   flexDirection: 'row',
2200 },
2201 sokobanCell: {
2202   width: 36,
2203   height: 36,
2204   alignItems: 'center',
2205   justifyContent: 'center',
2206 },
2207 sokobanCellWall: {
2208   backgroundColor: '#3A3A5C',
2209 },
2210 sokobanCellFloor: {
2211   backgroundColor: colors.background,
2212 },
2213 sokobanCellTarget: {
2214   backgroundColor: 'rgba(255,215,0,0.2)',
2215 },
2216 sokobanCellPlayer: {
2217   backgroundColor: 'rgba(123,117,216,0.3)',
2218 },
2219 sokobanCellBox: {
2220   backgroundColor: 'rgba(255,125,0,0.4)',
2221 },
2222 sokobanCellBoxOnTarget: {
2223   backgroundColor: 'rgba(58,227,116,0.4)',
2224 },
2225 sokobanCellText: {
2226   fontSize: 20,
2227 },
2228 sokobanControls: {
2229   alignItems: 'center',
2230   gap: 8,
2231 },
2232 controlRow: {
2233   flexDirection: 'row',
2234   gap: 8,
2235 },
2236 controlBtn: {
2237   width: 60,
2238   height: 60,
2239   alignItems: 'center',
2240   justifyContent: 'center',
2241   backgroundColor: colors.borderDark,
2242   borderWidth: 1,
2243   borderColor: 'rgba(123,117,216,0.3)',
2244   borderRadius: 12,

```

```

2245 },
2246 resetBtn: {
2247   backgroundColor: 'rgba(255,215,0,0.15)',
2248   borderColor: 'rgba(255,215,0,0.3)',
2249 },
2250 // 胜利界面
2251 winContainer: {
2252   flex: 1,
2253   alignItems: 'center',
2254   justifyContent: 'center',
2255   paddingHorizontal: 40,
2256 },
2257 winTitle: {
2258   fontSize: 32,
2259   fontWeight: '800',
2260   color: colors.warning,
2261   marginBottom: 12,
2262   fontFamily: 'Courier',
2263 },
2264 winText: {
2265   fontSize: 18,
2266   color: colors.textPrimary,
2267   marginBottom: 32,
2268   fontFamily: 'Courier',
2269 },
2270 rewardsContainer: {
2271   flexDirection: 'row',
2272   gap: 32,
2273   marginBottom: 32,
2274 },
2275 rewardItem: {
2276   alignItems: 'center',
2277 },
2278 rewardValue: {
2279   fontSize: 28,
2280   fontWeight: '800',
2281   color: colors.warning,
2282   fontFamily: 'Courier',
2283 },
2284 rewardLabel: {
2285   fontSize: 14,
2286   color: colors.textSecondary,
2287   fontFamily: 'Courier',
2288 },
2289 collectBtn: {
2290   paddingHorizontal: 32,
2291   paddingVertical: 14,
2292   backgroundColor: colors.primary,
2293   borderRadius: 16,
2294 },

```

```

2295 collectBtnText: {
2296   color: colors.textPrimary,
2297   fontSize: 16,
2298   fontWeight: '700',
2299   fontFamily: 'Courier',
2300 },
2301 // 欢迎页面样式
2302 welcomeContent: {
2303   flex: 1,
2304   padding: 20,
2305   paddingBottom: 40,
2306 },
2307 welcomeSection: {
2308   alignItems: 'center',
2309   marginBottom: 32,
2310   marginTop: 20,
2311 },
2312 welcomeBackBtn: {
2313   flexDirection: 'row',
2314   alignItems: 'center',
2315   gap: 6,
2316   paddingVertical: 12,
2317   marginBottom: 8,
2318 },
2319 welcomeTitle: {
2320   fontSize: 28,
2321   fontWeight: '800',
2322   color: colors.warning,
2323   marginBottom: 8,
2324   fontFamily: 'Courier',
2325 },
2326 welcomeSubtitle: {
2327   fontSize: 16,
2328   color: colors.textSecondary,
2329   fontFamily: 'Courier',
2330 },
2331 rewardsSection: {
2332   backgroundColor: colors.surface,
2333   borderRadius: 16,
2334   padding: 20,
2335   marginBottom: 20,
2336   borderWidth: 1,
2337   borderColor: colors.borderDark,
2338 },
2339 rewardsRow: {
2340   flexDirection: 'row',
2341   justifyContent: 'center',
2342   gap: 40,
2343   marginBottom: 12,
2344 },

```

```

2345   rewardBox: {
2346     alignItems: 'center',
2347   },
2348   rewardIcon: {
2349     fontSize: 32,
2350     marginBottom: 8,
2351   },
2352   rewardAmount: {
2353     fontSize: 24,
2354     fontWeight: '800',
2355     color: colors.warning,
2356     fontFamily: 'Courier',
2357   },
2358   rewardName: {
2359     fontSize: 14,
2360     color: colors.textSecondary,
2361     fontFamily: 'Courier',
2362   },
2363   rewardNote: {
2364     fontSize: 12,
2365     color: colors.primary,
2366     fontFamily: 'Courier',
2367     textAlign: 'center',
2368   },
2369   gamesSection: {
2370     marginBottom: 20,
2371   },
2372   sectionTitle: {
2373     fontSize: 16,
2374     fontWeight: '700',
2375     color: colors.textPrimary,
2376     marginBottom: 12,
2377     fontFamily: 'Courier',
2378   },
2379   gameCard: {
2380     backgroundColor: colors.surface,
2381     borderRadius: 16,
2382     padding: 16,
2383     marginBottom: 12,
2384     flexDirection: 'row',
2385     gap: 16,
2386     borderWidth: 1,
2387     borderColor: 'rgba(123,117,216,0.15)',
2388   },
2389   gameCardIcon: {
2390     width: 50,
2391     height: 50,
2392     alignItems: 'center',
2393     justifyContent: 'center',
2394     backgroundColor: colors.borderDark,

```



```

2395     borderRadius: 12,
2396   },
2397   gameCardIconText: {
2398     fontSize: 28,
2399   },
2400   gameCardInfo: {
2401     flex: 1,
2402   },
2403   gameCardTitle: {
2404     fontSize: 18,
2405     fontWeight: '700',
2406     color: colors.textPrimary,
2407     marginBottom: 6,
2408     fontFamily: 'Courier',
2409   },
2410   gameCardDesc: {
2411     fontSize: 13,
2412     color: colors.textSecondary,
2413     marginBottom: 8,
2414     fontFamily: 'Courier',
2415   },
2416   gameCardRules: {
2417     fontSize: 12,
2418     color: colors.textTertiary,
2419     fontFamily: 'Courier',
2420     lineHeight: 18,
2421   },
2422   tipsSection: {
2423     backgroundColor: 'rgba(255,215,0,0.1)',
2424     borderRadius: 16,
2425     padding: 16,
2426     marginBottom: 20,
2427   },
2428   tipsText: {
2429     fontSize: 13,
2430     color: colors.textPrimary,
2431     fontFamily: 'Courier',
2432     lineHeight: 20,
2433   },
2434   backBtn: {
2435     flexDirection: 'row',
2436     alignItems: 'center',
2437     gap: 6,
2438     marginTop: 40,
2439     paddingHorizontal: 20,
2440     paddingVertical: 12,
2441   },
2442   backBtnText: {
2443     color: colors.textSecondary,
2444     fontSize: 14,

```

```

2445     fontFamily: 'Courier',
2446   },
2447   gameCardArrow: {
2448     fontSize: 20,
2449     color: colors.textSecondary,
2450     alignSelf: 'center',
2451   },
2452   // 教程页面样式
2453   tutorialContent: {
2454     flex: 1,
2455     padding: 20,
2456     paddingBottom: 60,
2457   },
2458   tutorialHeader: {
2459     alignItems: 'center',
2460     marginBottom: 24,
2461   },
2462   tutorialIcon: {
2463     fontSize: 48,
2464     marginBottom: 8,
2465   },
2466   tutorialTitle: {
2467     fontSize: 24,
2468     fontWeight: '800',
2469     color: colors.warning,
2470     fontFamily: 'Courier',
2471   },
2472   tutorialSection: {
2473     backgroundColor: colors.surface,
2474     borderRadius: 16,
2475     padding: 16,
2476     marginBottom: 16,
2477     borderWidth: 1,
2478     borderColor: 'rgba(123,117,216,0.15)',
2479   },
2480   tutorialSectionTitle: {
2481     fontSize: 16,
2482     fontWeight: '700',
2483     color: colors.textPrimary,
2484     marginBottom: 12,
2485     fontFamily: 'Courier',
2486   },
2487   tutorialText: {
2488     fontSize: 14,
2489     color: colors.textSecondary,
2490     fontFamily: 'Courier',
2491     lineHeight: 20,
2492   },
2493   exampleGrid: {
2494     marginTop: 12,

```

```

2495 },
2496 exampleTitle: {
2497   fontSize: 13,
2498   color: colors.primary,
2499   fontFamily: 'Courier',
2500   marginBottom: 8,
2501 },
2502 smallGrid: {
2503   backgroundColor: colors.card,
2504   borderRadius: 8,
2505   padding: 4,
2506 },
2507 smallRow: {
2508   flexDirection: 'row',
2509 },
2510 smallCell: {
2511   width: 24,
2512   height: 24,
2513   alignItems: 'center',
2514   justifyContent: 'center',
2515   borderWidth: 0.5,
2516   borderColor: 'rgba(123,117,216,0.3)',
2517 },
2518 smallCellText: {
2519   fontSize: 12,
2520   color: colors.primary,
2521   fontFamily: 'Courier',
2522 },
2523 stepList: {
2524   gap: 12,
2525 },
2526 stepItem: {
2527   flexDirection: 'row',
2528   gap: 12,
2529   alignItems: 'center',
2530 },
2531 stepNumber: {
2532   width: 28,
2533   height: 28,
2534   borderRadius: 14,
2535   backgroundColor: colors.borderDark,
2536   color: colors.primary,
2537   fontSize: 14,
2538   fontWeight: '700',
2539   fontFamily: 'Courier',
2540   textAlign: 'center',
2541   textAlignVertical: 'center',
2542 },
2543 stepText: {
2544   fontSize: 14,

```

```

2545   color: colors.textPrimary,
2546   fontFamily: 'Courier',
2547 },
2548 legend: {
2549   gap: 8,
2550 },
2551 legendItem: {
2552   flexDirection: 'row',
2553   gap: 10,
2554   alignItems: 'center',
2555 },
2556 legendIcon: {
2557   fontSize: 20,
2558 },
2559 legendText: {
2560   fontSize: 13,
2561   color: colors.textSecondary,
2562   fontFamily: 'Courier',
2563 },
2564 controlsExample: {
2565   alignItems: 'center',
2566   marginTop: 8,
2567 },
2568 controlGrid: {
2569   flexDirection: 'row',
2570   flexWrap: 'wrap',
2571   width: 170,
2572   gap: 8,
2573 },
2574 controlExampleBtn: {
2575   width: 50,
2576   height: 50,
2577   borderRadius: 12,
2578   backgroundColor: colors.borderDark,
2579   alignItems: 'center',
2580   justifyContent: 'center',
2581 },
2582 controlExampleBtnCenter: {
2583   width: 50,
2584   height: 50,
2585   borderRadius: 12,
2586   backgroundColor: 'rgba(255,215,0,0.2)',
2587   alignItems: 'center',
2588   justifyContent: 'center',
2589 },
2590 startBtn: {
2591   marginTop: 8,
2592   marginBottom: 40,
2593   paddingVertical: 14,
2594   borderRadius: 16,

```

```

2595     backgroundColor: colors.primary,
2596   },
2597   startBtnText: {
2598     color: colors.textPrimary,
2599     fontSize: 16,
2600     fontWeight: '700',
2601     fontFamily: 'Courier',
2602     textAlign: 'center',
2603   },
2604   }), [colors]);
2605
2606   const SudokuGame = ({ onWin }: { onWin: () => void }) => {
2607     const [board, setBoard] = useState<number[][]>([]);
2608     const [solution, setSolution] = useState<number[][]>([]);
2609     const [given, setGiven] = useState<boolean[][]>([]);
2610     const [selected, setSelected] = useState<{ row: number; col: number } | null>(null);
2611     const [showSolution, setShowSolution] = useState(false);
2612
2613     // 数独生成逻辑
2614     const isValid = (grid: number[][], row: number, col: number, num: number) => {
2615       for (let i = 0; i < 9; i++) {
2616         if (grid[row][i] === num || grid[i][col] === num) return false;
2617         const boxRow = Math.floor(row / 3) * 3 + Math.floor(i / 3);
2618         const boxCol = Math.floor(col / 3) * 3 + (i % 3);
2619         if (grid[boxRow][boxCol] === num) return false;
2620       }
2621       return true;
2622     };
2623
2624     const solveSudoku = (grid: number[][]) => {
2625       for (let row = 0; row < 9; row++) {
2626         for (let col = 0; col < 9; col++) {
2627           if (grid[row][col] === 0) {
2628             const nums = [1, 2, 3, 4, 5, 6, 7, 8, 9].sort(() => Math.random() - 0.5);
2629             for (let num of nums) {
2630               if (isValid(grid, row, col, num)) {
2631                 grid[row][col] = num;
2632                 if (solveSudoku(grid)) return true;
2633                 grid[row][col] = 0;
2634               }
2635             }
2636             return false;
2637           }
2638         }
2639       }
2640       return true;
2641     };
2642
2643     const initGame = () => {
2644       console.log('[MiniGames] 初始化数独游戏');

```

```

2645   const newSolution = Array(9).fill(0).map(() => Array(9).fill(0));
2646   solveSudoku(newSolution);
2647
2648   const newBoard = newSolution.map(row => [...row]);
2649   const newGiven = Array(9).fill(0).map(() => Array(9).fill(false));
2650
2651   const cellsToRemove = 40 + Math.floor(Math.random() * 6);
2652   let removed = 0;
2653   while (removed < cellsToRemove) {
2654     const row = Math.floor(Math.random() * 9);
2655     const col = Math.floor(Math.random() * 9);
2656     if (newBoard[row][col] !== 0) {
2657       newBoard[row][col] = 0;
2658       removed++;
2659     } else {
2660       newGiven[row][col] = true;
2661     }
2662   }
2663
2664   // 标记初始给定的数字
2665   for (let row = 0; row < 9; row++) {
2666     for (let col = 0; col < 9; col++) {
2667       newGiven[row][col] = newSolution[row][col] === newBoard[row][col] && newBoard[row][col] !== 0;
2668     }
2669   }
2670
2671   setSolution(newSolution);
2672   setBoard(newBoard);
2673   setGiven(newGiven);
2674   setSelected(null);
2675   setShowSolution(false);
2676   };
2677
2678   useEffect(() => {
2679     initGame();
2680   }, []);
2681
2682   const checkWin = () => {
2683     for (let row = 0; row < 9; row++) {
2684       for (let col = 0; col < 9; col++) {
2685         if (board[row][col] !== solution[row][col]) return false;
2686       }
2687     }
2688     return true;
2689   };
2690
2691   const inputNumber = (num: number) => {
2692     if (!selected || given[selected.row][selected.col]) return;
2693
2694     const newBoard = board.map(row => [...row]);

```

```

2695     newBoard[selected.row][selected.col] = num;
2696     setBoard(newBoard);
2697
2698     // 检查是否完成
2699     if (checkWin()) {
2700         setTimeout(() => onWin(), 300);
2701     }
2702 };
2703
2704 const clearCell = () => {
2705     if (!selected || given[selected.row][selected.col]) return;
2706
2707     const newBoard = board.map(row => [...row]);
2708     newBoard[selected.row][selected.col] = 0;
2709     setBoard(newBoard);
2710 };
2711
2712 return (
2713     <View style={styles.gameContainer}>
2714         <View style={styles.sudokuHeader}>
2715             <TouchableOpacity style={styles.newGameBtn} onPress={initGame}>
2716                 <Icons name="refresh" size={16} color={colors.primary} />
2717                 <Text style={styles.newGameText}>新游戏</Text>
2718             </TouchableOpacity>
2719             <TouchableOpacity
2720                 style={[styles.newGameBtn, showSolution && styles.solutionBtnActive]}
2721                 onPress={() => setShowSolution(!showSolution)}
2722             >
2723                 <Icons name={showSolution ? 'eye-off' : 'eye'} size={16} color={showSolution ? '#FF6B6B' : '#5D9B
2724                 <Text style={[styles.newGameText, showSolution && { color: '#FF6B6B' }]}>
2725                     {showSolution ? '隐藏答案' : '显示答案'}
2726                 </Text>
2727             </TouchableOpacity>
2728         </View>
2729
2730         <View style={styles.sudokuGrid}>
2731             {board.map((row, rowIndex) => (
2732                 <React.Fragment key={rowIndex}>
2733                     {row.map((cell, colIndex) => (
2734                         <TouchableOpacity
2735                             key={`${rowIndex}-${colIndex}`}
2736                             style={[
2737                                 styles.sudokuCell,
2738                                 selected?.row === rowIndex && selected?.col === colIndex && styles.sudokuCellSelected,
2739                                 given[rowIndex][colIndex] && styles.sudokuCellGiven,
2740                                 (colIndex === 2 || colIndex === 5) && { borderRightWidth: 2, borderRightColor: '#5D9BFA' },
2741                                 (rowIndex === 2 || rowIndex === 5) && { borderBottomWidth: 2, borderBottomColor: colors.prim
2742                             ]}
2743                             onPress={() => setSelected({ row: rowIndex, col: colIndex })}
2744                         >

```

```

2745     {cell !== 0 ? (
2746       <Text style={[
2747         styles.sudokuCellText,
2748         given[rowIndex][colIndex] ? styles.sudokuCellTextGiven : styles.sudokuCellTextUser
2749       ]}>
2750         {cell}
2751       </Text>
2752     ) : showSolution && solution[rowIndex]?.[colIndex] !== 0 ? (
2753       <Text style={[styles.sudokuCellText, styles.sudokuCellTextSolution]}>
2754         {solution[rowIndex][colIndex]}
2755       </Text>
2756     ) : null}
2757   </TouchableOpacity>
2758 ))}
2759 </React.Fragment>
2760 )))
2761 </View>
2762
2763 <View style={[styles.numberPad, showSolution && styles.numberPadDisabled]}>
2764   {[1, 2, 3, 4, 5, 6, 7, 8, 9].map(num => (
2765     <TouchableOpacity key={num} style={styles.numberBtn} onPress={() => inputNumber(num)} disabled={show
2766       <Text style={[styles.numberBtnText, showSolution && styles.numberBtnTextDisabled]}>{num}</Text>
2767     </TouchableOpacity>
2768   ))}
2769   <TouchableOpacity style={[styles.numberBtn, styles.clearBtn]} onPress={clearCell} disabled={showSoluti
2770     <Ionicons name="backspace-outline" size={18} color={showSolution ? '#555' : '#FF6B6B'} />
2771   </TouchableOpacity>
2772 </View>
2773 </View>
2774 );
2775
2776 };
2777
2778 return (
2779   <View style={styles.container}>
2780     {currentScreen === 'welcome' && (
2781       <ScrollView style={styles.welcomeContent} showsVerticalScrollIndicator={false}>
2782         <TouchableOpacity onPress={onClose} style={styles.welcomeBackBtn}>
2783           <Ionicons name="arrow-back" size={20} color={colors.textSecondary} />
2784           <Text style={styles.backBtnText}>返回</Text>
2785         </TouchableOpacity>
2786
2787         <View style={styles.welcomeSection}>
2788           <Text style={styles.welcomeTitle}> 游戏恢复</Text>
2789           <Text style={styles.welcomeSubtitle}>完成小游戏, 恢复体力和能量! </Text>
2790         </View>
2791
2792         <View style={styles.rewardsSection}>
2793           <Text style={styles.sectionTitle}> 游戏奖励</Text>
2794           <View style={styles.rewardsRow}>

```



```

2795     <View style={styles.rewardBox}>
2796       <Text style={styles.rewardIcon}> </Text>
2797       <Text style={styles.rewardAmount}> + 5 </Text>
2798       <Text style={styles.rewardName}> 体力 </Text>
2799     </View>
2800     <View style={styles.rewardBox}>
2801       <Text style={styles.rewardIcon}> </Text>
2802       <Text style={styles.rewardAmount}> + 3 </Text>
2803       <Text style={styles.rewardName}> 能量 </Text>
2804     </View>
2805   </View>
2806   <Text style={styles.rewardNote}> 叛逆小怪可获得双倍奖励! </Text>
2807 </View>
2808
2809 <View style={styles.gamesSection}>
2810   <Text style={styles.sectionTitle}> 选择游戏 </Text>
2811
2812   <TouchableOpacity
2813     style={styles.gameCard}
2814     onPress={() => showTutorial('sudoku')}
2815   >
2816     <View style={styles.gameCardIcon}>
2817       <Text style={styles.gameCardIconText}> </Text>
2818     </View>
2819     <View style={styles.gameCardInfo}>
2820       <Text style={styles.gameCardTitle}> 数独 </Text>
2821       <Text style={styles.gameCardDesc}>
2822         填写9×9网格, 使每行、每列、每个3×3官格都包含1-9不重复的数字
2823       </Text>
2824       <Text style={styles.gameCardRules}>
2825         • 点击空格选中, 然后点击数字填入
2826         {'\n'}• 完成所有格子即可过关
2827       </Text>
2828     </View>
2829     <Text style={styles.gameCardArrow}> → </Text>
2830   </TouchableOpacity>
2831 </View>
2832
2833 <View style={styles.tipsSection}>
2834   <Text style={styles.sectionTitle}> 小贴士 </Text>
2835   <Text style={styles.tipsText}>
2836     • 每天可游玩3次, 凌晨5点重置次数
2837     {'\n'}• 完成游戏后奖励会自动添加到你的账户
2838     {'\n'}• 游戏过程中可随时退出, 不会消耗次数
2839   </Text>
2840 </View>
2841 </ScrollView>
2842 ))
2843
2844 {currentScreen === 'tutorial' && (

```

```

2845 <ScrollView style={styles.tutorialContent} showsVerticalScrollIndicator={false}>
2846   <TouchableOpacity onPress={goBack} style={styles.backBtn}>
2847     <Ionicons name="arrow-back" size={20} color={colors.textSecondary} />
2848     <Text style={styles.backBtnText}>返回</Text>
2849   </TouchableOpacity>
2850
2851   {currentGame === 'sudoku' && (
2852     <>
2853       <View style={styles.tutorialHeader}>
2854         <Text style={styles.tutorialIcon}></Text>
2855         <Text style={styles.tutorialTitle}>数独教程</Text>
2856       </View>
2857
2858       <View style={styles.tutorialSection}>
2859         <Text style={styles.tutorialSectionTitle}> 游戏目标</Text>
2860         <Text style={styles.tutorialText}>
2861           在9×9的网格中填入数字1-9，使每行、每列、每个3×3宫格内的数字都不重复。
2862         </Text>
2863         <View style={styles.exampleGrid}>
2864           <Text style={styles.exampleTitle}>示例： </Text>
2865           <View style={styles.smallGrid}>
2866             {[
2867               [5, 3, 0, 0, 7, 0, 0, 0, 0],
2868               [6, 0, 0, 1, 9, 5, 0, 0, 0],
2869               [0, 9, 8, 0, 0, 0, 0, 6, 0],
2870             ].map((row, ri) => (
2871               <View key={ri} style={styles.smallRow}>
2872                 {row.map((cell, ci) => (
2873                   <View
2874                     key={ci}
2875                     style={[styles.smallCell, ci === 2 && { borderRightWidth: 2, borderRightColor: '#5
2876                   >
2877                     {cell !== 0 && <Text style={styles.smallCellText}>{cell}</Text>}
2878                   </View>
2879                 )))}
2880               </View>
2881             )]}
2882           </View>
2883         </View>
2884       </View>
2885
2886       <View style={styles.tutorialSection}>
2887         <Text style={styles.tutorialSectionTitle}> 玩法步骤</Text>
2888         <View style={styles.stepList}>
2889           <View style={styles.stepItem}>
2890             <Text style={styles.stepNumber}>1</Text>
2891             <Text style={styles.stepText}>点击空格选中它</Text>
2892           </View>
2893           <View style={styles.stepItem}>
2894             <Text style={styles.stepNumber}>2</Text>

```

```

2895         <Text style={styles.stepText}>点击底部数字键盘填入数字</Text>
2896     </View>
2897     <View style={styles.stepItem}>
2898         <Text style={styles.stepNumber}>3</Text>
2899         <Text style={styles.stepText}>如果填入错误会显示红色提示</Text>
2900     </View>
2901     <View style={styles.stepItem}>
2902         <Text style={styles.stepNumber}>4</Text>
2903         <Text style={styles.stepText}>填满所有格子即可过关! </Text>
2904     </View>
2905 </View>
2906 </View>
2907
2908 <View style={styles.tutorialSection}>
2909     <Text style={styles.tutorialSectionTitle}> 技巧提示</Text>
2910     <Text style={styles.tutorialText}>
2911         • 先从数字多的行、列或宫格开始
2912         {\n}• 每个数字在每行、每列、每个宫格只能出现一次
2913         {\n}• 如果卡住了, 可以点击"新游戏"重新开始
2914     </Text>
2915 </View>
2916
2917 <TouchableOpacity style={styles.startBtn} onPress={startGame}>
2918     <Text style={styles.startBtnText}>开始游戏 </Text>
2919 </TouchableOpacity>
2920 </>
2921 }}
2922 </ScrollView>
2923 }}
2924
2925 {currentScreen === 'game' && (
2926     !gameWon ? (
2927         <>
2928             <TouchableOpacity onPress={goBack} style={styles.backBtn}>
2929                 <Ionicons name="arrow-back" size={20} color={colors.textSecondary} />
2930                 <Text style={styles.backBtnText}>返回教程</Text>
2931             </TouchableOpacity>
2932
2933             <ScrollView style={styles.content} showsVerticalScrollIndicator={false}>
2934                 <SudokuGame onWin={handleWin} />
2935             </ScrollView>
2936         </>
2937     ) : (
2938         <View style={styles.winContainer}>
2939             <Text style={styles.winTitle}> 胜利! </Text>
2940             <Text style={styles.winText}>恭喜你完成了游戏! </Text>
2941             <View style={styles.rewardsContainer}>
2942                 <View style={styles.rewardItem}>
2943                     <Text style={styles.rewardValue}>+5</Text>
2944                     <Text style={styles.rewardLabel}>体力</Text>

```

```

2945     </View>
2946     <View style={styles.rewardItem}>
2947         <Text style={styles.rewardValue}> +3 </Text>
2948         <Text style={styles.rewardLabel}>能量</Text>
2949     </View>
2950 </View>
2951 <TouchableOpacity style={styles.collectBtn} onPress={handleCollectReward}>
2952     <Text style={styles.collectBtnText}>太棒了! 领取奖励</Text>
2953 </TouchableOpacity>
2954 </View>
2955 )
2956 })
2957 </View>
2958 );
2959
2960
2961 };
2962
2963 export default MiniGames;
2964
2965 === mobile/src/components/MonsterIcon.tsx ===
2966 import React from 'react';
2967 import Svg, {
2968     Circle,
2969     Ellipse,
2970     Path,
2971     G,
2972 } from 'react-native-svg';
2973
2974 type MonsterType = 'lively' | 'calm' | 'rebel';
2975
2976 interface MonsterIconProps {
2977     type: MonsterType;
2978     size: number;
2979 }
2980
2981 // =====
2982 // 活力小怪 — 宽猫耳，收窄圆身，圆手
2983 // =====
2984 const LivelyMonster = () => (
2985     <G>
2986         {/* 腿部 */}
2987         <Ellipse cx="34" cy="86" rx="7" ry="6" fill="#E8A090" />
2988         <Ellipse cx="66" cy="86" rx="7" ry="6" fill="#E8A090" />
2989
2990         {/* 左臂 + 圆手 */}
2991         <Path
2992             d="M24 54 Q12 52 8 60 Q6 66 14 66 Q22 66 26 60"
2993             fill="#F5C6B8"
2994         />

```

```

2994  {/* 右臂 + 圆手 */}
2995  <Path
2996      d="M76 54 Q88 52 92 60 Q94 66 86 66 Q78 66 74 60"
2997      fill="#F5C6B8"
2998  />
2999
3000  {/* 左猫耳 — 居中, 头顶 */}
3001  <Path
3002      d="M34 26 Q28 10 26 2 Q34 10 38 22 L48 22 Z"
3003      fill="#F5C6B8"
3004  />
3005  {/* 左猫耳 — 内侧 */}
3006  <Path
3007      d="M36 24 Q32 12 31 6 Q36 12 39 22 L44 22 Z"
3008      fill="#F0AAA0"
3009  />
3010  {/* 右猫耳 — 居中, 头顶 */}
3011  <Path
3012      d="M66 26 Q72 10 74 2 Q66 10 62 22 L52 22 Z"
3013      fill="#F5C6B8"
3014  />
3015  {/* 右猫耳 — 内侧 */}
3016  <Path
3017      d="M64 24 Q68 12 69 6 Q64 12 61 22 L56 22 Z"
3018      fill="#F0AAA0"
3019  />
3020
3021  {/* 身体 — 收窄 */}
3022  <Path
3023      d="M30 24 C30 24 18 42 18 58 C18 72 28 82 50 82 C72 82 82 72 82 58 C82 42 70 24 70 24 C62 20 38 20 30 24
3024      fill="#F5C6B8"
3025  />
3026
3027  {/* 左眼白 */}
3028  <Circle cx="37" cy="49" r="9.5" fill="#FFFFFF" />
3029  {/* 左瞳孔 */}
3030  <Circle cx="39.5" cy="49.5" r="5.5" fill="#2D1B0E" />
3031  {/* 左大高光 */}
3032  <Circle cx="42" cy="46.5" r="2.8" fill="#FFFFFF" />
3033  {/* 左小高光 */}
3034  <Circle cx="37.5" cy="51.5" r="1.3" fill="#FFFFFF" />
3035
3036  {/* 右眼白 */}
3037  <Circle cx="63" cy="49" r="9.5" fill="#FFFFFF" />
3038  {/* 右瞳孔 */}
3039  <Circle cx="65.5" cy="49.5" r="5.5" fill="#2D1B0E" />
3040  {/* 右大高光 */}
3041  <Circle cx="68" cy="46.5" r="2.8" fill="#FFFFFF" />
3042  {/* 右小高光 */}
3043  <Circle cx="63.5" cy="51.5" r="1.3" fill="#FFFFFF" />

```

```

3044
3045  /* 左腮红 */
3046  <Ellipse cx="25" cy="60" rx="7" ry="4.5" fill="#F0A090" opacity="0.45" />
3047  /* 右腮红 */
3048  <Ellipse cx="75" cy="60" rx="7" ry="4.5" fill="#F0A090" opacity="0.45" />
3049
3050  /* 嘴巴 — W形微笑 */
3051  <Path
3052    d="M41 62 Q45 67 50 62 Q55 67 59 62"
3053    fill="none"
3054    stroke="#2D1B0E"
3055    strokeWidth="2.2"
3056    strokeLinecap="round"
3057    strokeLinejoin="round"
3058  />
3059 </G>
3060 );
3061
3062 // =====
3063 // 沉稳小怪 — 天线+侧耳，平头身体
3064 // =====
3065 const CalmMonster = () => (
3066   <G>
3067     /* 腿部 */
3068     <Ellipse cx="34" cy="84" rx="8" ry="9" fill="#E87840" />
3069     <Ellipse cx="66" cy="84" rx="8" ry="9" fill="#E87840" />
3070
3071     /* 左臂 */
3072     <Path
3073       d="M20 54 Q10 52 8 60 Q8 66 16 66 Q22 66 24 60"
3074       fill="#FF9B5E"
3075     />
3076     /* 右臂 */
3077     <Path
3078       d="M80 54 Q90 52 92 60 Q92 66 84 66 Q78 66 76 60"
3079       fill="#FF9B5E"
3080     />
3081
3082     /* 左天线杆 */
3083     <Path d="M38 26 L34 8 L42 8 Z" fill="#FF9B5E" />
3084     /* 左天线球 */
3085     <Circle cx="38" cy="6" r="5" fill="#FF7A36" />
3086     /* 右天线杆 */
3087     <Path d="M62 26 L58 8 L66 8 Z" fill="#FF9B5E" />
3088     /* 右天线球 */
3089     <Circle cx="62" cy="6" r="5" fill="#FF7A36" />
3090
3091     /* 左小圆耳 */
3092     <Circle cx="16" cy="46" r="9" fill="#FF9B5E" />
3093     /* 右小圆耳 */

```

```

3094   <Circle cx="84" cy="46" r="9" fill="#FF9B5E" />
3095
3096   {/* 身体 — 平头 */}
3097   <Path
3098     d="M28 26 C28 26 22 40 20 54 C18 68 26 80 50 80 C74 80 82 68 80 54 C78 40 72 26 72 26 Z"
3099     fill="#FF9B5E"
3100   />
3101
3102   {/* 左眼白 */}
3103   <Circle cx="36" cy="48" r="9" fill="#FFFFFF" />
3104   {/* 左瞳孔 */}
3105   <Circle cx="39" cy="48.5" r="5.5" fill="#1A1A2E" />
3106   {/* 左大高光 */}
3107   <Circle cx="41.5" cy="45" r="2.8" fill="#FFFFFF" />
3108   {/* 左小高光 */}
3109   <Circle cx="36.5" cy="51" r="1.3" fill="#FFFFFF" />
3110
3111   {/* 右眼白 */}
3112   <Circle cx="64" cy="48" r="9" fill="#FFFFFF" />
3113   {/* 右瞳孔 */}
3114   <Circle cx="67" cy="48.5" r="5.5" fill="#1A1A2E" />
3115   {/* 右大高光 */}
3116   <Circle cx="69.5" cy="45" r="2.8" fill="#FFFFFF" />
3117   {/* 右小高光 */}
3118   <Circle cx="64.5" cy="51" r="1.3" fill="#FFFFFF" />
3119
3120   {/* 左腮红 */}
3121   <Ellipse cx="24" cy="58" rx="6" ry="4" fill="#FF7A36" opacity="0.35" />
3122   {/* 右腮红 */}
3123   <Ellipse cx="76" cy="58" rx="6" ry="4" fill="#FF7A36" opacity="0.35" />
3124
3125   {/* 嘴巴 — 简洁短弧 */}
3126   <Path
3127     d="M44 62 Q50 67 56 62"
3128     fill="none"
3129     stroke="#1A1A2E"
3130     strokeWidth="2.2"
3131     strokeLinecap="round"
3132   />
3133 </G>
3134 );
3135
3136 // =====
3137 // 叛逆小怪 — 宽弧恶魔角，收窄圆身，歪嘴+虎牙，触手
3138 // =====
3139 const RebelMonster = () => (
3140   <G>
3141     {/* 腿部 */}
3142     <Ellipse cx="34" cy="86" rx="7" ry="7" fill="#9B85D6" />
3143     <Ellipse cx="66" cy="86" rx="7" ry="7" fill="#9B85D6" />

```

```

3144
3145  /* 左触手 — 弯曲伸出 */
3146  <Path
3147    d="M24 52 Q10 44 8 54 Q6 64 20 66 Q26 66 26 58"
3148    fill="#B8A4E6"
3149  />
3150  /* 右触手 — 弯曲伸出 */
3151  <Path
3152    d="M76 52 Q90 44 92 54 Q94 64 80 66 Q74 66 74 58"
3153    fill="#B8A4E6"
3154  />
3155
3156  /* 左恶魔角 — 瘦三角形 */
3157  <Path
3158    d="M34 24 L24 4 L40 22 Z"
3159    fill="#A088D6"
3160  />
3161  /* 右恶魔角 — 瘦三角形 */
3162  <Path
3163    d="M66 24 L76 4 L60 22 Z"
3164    fill="#A088D6"
3165  />
3166
3167  /* 身体 — 收窄 */
3168  <Path
3169    d="M28 24 C28 24 16 40 16 58 C16 74 28 84 50 84 C72 84 84 74 84 58 C84 40 72 24 72 24 C64 18 36 18 28 24
3170    fill="#B8A4E6"
3171  />
3172
3173  /* 左眼白 */
3174  <Circle cx="36" cy="48" r="9.5" fill="#FFFFFF" />
3175  /* 左瞳孔 */
3176  <Circle cx="39" cy="48.5" r="5.5" fill="#1A0A2E" />
3177  /* 左大高光 */
3178  <Circle cx="41.5" cy="45" r="2.8" fill="#FFFFFF" />
3179  /* 左小高光 */
3180  <Circle cx="36.5" cy="51" r="1.3" fill="#FFFFFF" />
3181
3182  /* 右眼白 */
3183  <Circle cx="64" cy="48" r="9.5" fill="#FFFFFF" />
3184  /* 右瞳孔 */
3185  <Circle cx="67" cy="48.5" r="5.5" fill="#1A0A2E" />
3186  /* 右大高光 */
3187  <Circle cx="69.5" cy="45" r="2.8" fill="#FFFFFF" />
3188  /* 右小高光 */
3189  <Circle cx="64.5" cy="51" r="1.3" fill="#FFFFFF" />
3190
3191  /* 左腮红 */
3192  <Ellipse cx="23" cy="58" rx="6.5" ry="4" fill="#D090C0" opacity="0.4" />
3193  /* 右腮红 */

```



```

3194   <Ellipse cx="77" cy="58" rx="6.5" ry="4" fill="#D090C0" opacity="0.4" />
3195
3196   {/* 歪嘴 — 左侧高右侧低 + 虎牙 */}
3197   <Path
3198     d="M42 58 Q46 62 50 60 Q54 66 60 62"
3199     fill="none"
3200     stroke="#1A0A2E"
3201     strokeWidth="2.2"
3202     strokeLinecap="round"
3203   />
3204   {/* 虎牙 — 右侧歪嘴露出 */}
3205   <Path
3206     d="M55 60 L56 68 L59 60 Z"
3207     fill="FFFFFF"
3208     stroke="#1A0A2E"
3209     strokeWidth="1"
3210     strokeLinejoin="round"
3211   />
3212 </G>
3213 );
3214
3215 const MonsterIcon = ({ type, size }: MonsterIconProps) => {
3216   const body = () => {
3217     switch (type) {
3218       case 'lively':
3219         return <CalmMonster />;
3220       case 'calm':
3221         return <LivelyMonster />;
3222       case 'rebel':
3223         return <RebelMonster />;
3224       default:
3225         return <CalmMonster />;
3226     }
3227   }();
3228
3229   return (
3230     <Svg width={size} height={size} viewBox="0 0 100 100">
3231       {body}
3232     </Svg>
3233   );
3234 };
3235
3236 export default MonsterIcon;
3237
3238 === mobile/src/components/SubscriptionModal.tsx ===
3238 import React, { useState, useMemo } from 'react';
3239 import {
3240   View, Text, StyleSheet, TouchableOpacity, Modal, ScrollView,
3241   Dimensions, TextInput, Alert,
3242 } from 'react-native';

```

```

3243 import { SafeAreaView } from 'react-native-safe-area-context';
3244 import { Ionicons } from '@expo/vector-icons';
3245 import { useTheme } from '../contexts/ThemeContext';
3246 import {
3247   PRICING_PLANS, PRO_FEATURES,
3248   type PlanId,
3249 } from '../utils/pricing';
3250 import { proService } from '../services/api';
3251 import { getCurrentUser } from '../utils/auth';
3252 import AsyncStorage from '@react-native-async-storage/async-storage';
3253 import { SUBSCRIPTION_STORAGE_KEY } from '../utils/pricing';
3254
3255 const { width: SCREEN_WIDTH } = Dimensions.get('window');
3256
3257 interface SubscriptionModalProps {
3258   visible: boolean;
3259   onClose: () => void;
3260   onProActivated?: () => void;
3261 }
3262
3263 const SubscriptionModal: React.FC<SubscriptionModalProps> = ({ visible, onClose, onProActivated }) => {
3264   const { colors, isDark } = useTheme();
3265   const [selectedPlan, setSelectedPlan] = useState<PlanId>('monthly');
3266   const [showActivate, setShowActivate] = useState(false);
3267   const [activateCode, setActivateCode] = useState('');
3268   const [activating, setActivating] = useState(false);
3269
3270   const selectedPlanData = PRICING_PLANS.find(p => p.id === selectedPlan)!;
3271
3272   const handleSubscribe = () => {
3273     setShowActivate(true);
3274   };
3275
3276   const handleActivate = async () => {
3277     const code = activateCode.trim().toUpperCase();
3278     if (!code) {
3279       Alert.alert('提示', '请输入激活码');
3280       return;
3281     }
3282
3283     setActivating(true);
3284     try {
3285       const user = await getCurrentUser();
3286       if (!user?.id) {
3287         Alert.alert('提示', '请先登录');
3288         return;
3289       }
3290
3291       const result = await proService.activate(code, user.id);
3292

```

```

3293 // 存入本地缓存
3294 await AsyncStorage.setItem(SUBSCRIPTION_STORAGE_KEY, JSON.stringify({
3295   isPro: true,
3296   planId: result.planId,
3297   expiresAt: result.expiresAt,
3298   activatedAt: new Date().toISOString(),
3299 }));
3300
3301 Alert.alert('激活成功', '恭喜! Pro 会员已激活${result.expiresAt ? '' : ' (永久) '}, [
3302   {
3303     text: '太好了',
3304     onPress: () => {
3305       setShowActivate(false);
3306       setActivateCode("");
3307       onClose();
3308       onProActivated?.();
3309     },
3310   },
3311   []
3312 ];
3313 } catch (error: any) {
3314   Alert.alert('激活失败', error?.message || '激活码无效, 请检查后重试');
3315 } finally {
3316   setActivating(false);
3317 }
3318
3319 const cardWidth = useMemo(() => {
3320   const padding = 32;
3321   const gaps = 20;
3322   return Math.floor((SCREEN_WIDTH - padding - gaps) / 3);
3323 }, []);
3324
3325 const s = useMemo(() => StyleSheet.create({
3326   overlay: {
3327     flex: 1,
3328     backgroundColor: 'rgba(0,0,0,0.92)',
3329     justifyContent: 'flex-end',
3330   },
3331   container: {
3332     width: '100%',
3333     flex: 1,
3334     backgroundColor: colors.background,
3335     borderTopLeftRadius: 24,
3336     borderTopRightRadius: 24,
3337   },
3338   handleBar: {
3339     width: 36,
3340     height: 4,
3341     borderRadius: 2,
3342     backgroundColor: isDark ? '#334155' : '#D1D5DB',

```

```

3343     alignSelf: 'center',
3344     marginTop: 12,
3345     marginBottom: 8,
3346   },
3347   header: {
3348     paddingHorizontal: 24,
3349     paddingTop: 16,
3350     paddingBottom: 20,
3351     alignItems: 'center',
3352   },
3353   title: {
3354     fontSize: 22,
3355     fontWeight: '700' as const,
3356     color: colors.textPrimary,
3357     marginBottom: 4,
3358   },
3359   subtitle: {
3360     fontSize: 14,
3361     color: colors.textSecondary,
3362   },
3363   proBadgeInline: {
3364     flexDirection: 'row',
3365     alignItems: 'center',
3366     backgroundColor: colors.proBg,
3367     borderWidth: 1,
3368     borderColor: colors.proBorder,
3369     borderRadius: 20,
3370     paddingHorizontal: 10,
3371     paddingVertical: 4,
3372     marginTop: 12,
3373     gap: 6,
3374   },
3375   proBadgeText: {
3376     fontSize: 13,
3377     fontWeight: '600' as const,
3378     color: colors.pro,
3379   },
3380   // 定价卡片
3381   plansContainer: {
3382     flexDirection: 'row',
3383     justifyContent: 'center',
3384     gap: 10,
3385     marginBottom: 20,
3386     paddingHorizontal: 16,
3387   },
3388   planCard: {
3389     width: cardWidth,
3390     borderRadius: 14,
3391     padding: 14,
3392     alignItems: 'center',

```

```

3393     borderWidth: 1.5,
3394     height: 150,
3395     justifyContent: 'center',
3396   },
3397   planSelected: { borderColor: colors.pro, backgroundColor: colors.proBg },
3398   planUnselected: { borderColor: colors.hairline, backgroundColor: colors.surface },
3399   planBadge: {
3400     position: 'absolute',
3401     top: -1,
3402     right: -1,
3403     backgroundColor: colors.pro,
3404     paddingHorizontal: 8,
3405     paddingVertical: 2,
3406     borderTopRightRadius: 14,
3407     borderBottomLeftRadius: 10,
3408   },
3409   planBadgeText: {
3410     fontSize: 9,
3411     fontWeight: '700' as const,
3412     color: '#FFFFFF',
3413   },
3414   planLabel: {
3415     fontSize: 13,
3416     fontWeight: '600' as const,
3417     color: colors.textSecondary,
3418     marginBottom: 6,
3419   },
3420   planPriceRow: {
3421     flexDirection: 'row',
3422     alignItems: 'baseline',
3423     gap: 2,
3424   },
3425   planPriceSymbol: {
3426     fontSize: 16,
3427     fontWeight: '700' as const,
3428     color: colors.pro,
3429   },
3430   planPrice: {
3431     fontSize: 28,
3432     fontWeight: '800' as const,
3433     color: colors.textPrimary,
3434   },
3435   planOriginalPrice: {
3436     fontSize: 12,
3437     color: colors.stone,
3438     textDecorationLine: 'line-through',
3439     marginLeft: 4,
3440   },
3441   planUnit: {
3442     fontSize: 11,

```

```

3443     color: colors.muted,
3444     marginTop: 2,
3445   },
3446   // 激活码输入区域
3447   activateSection: {
3448     paddingHorizontal: 24,
3449     paddingVertical: 20,
3450     alignItems: 'center',
3451   },
3452   activateHint: {
3453     fontSize: 14,
3454     color: colors.textSecondary,
3455     textAlign: 'center',
3456     marginBottom: 16,
3457     lineHeight: 20,
3458   },
3459   activateHintBold: {
3460     fontWeight: '600',
3461     color: colors.primary,
3462   },
3463   codeInputWrap: {
3464     flexDirection: 'row',
3465     alignItems: 'center',
3466     gap: 10,
3467     width: '100%',
3468     marginBottom: 20,
3469   },
3470   codeInput: {
3471     flex: 1,
3472     height: 50,
3473     borderRadius: 12,
3474     borderWidth: 1.5,
3475     borderColor: colors.proBorder,
3476     backgroundColor: colors.proBg,
3477     paddingHorizontal: 16,
3478     fontSize: 18,
3479     fontWeight: '700',
3480     color: colors.pro,
3481     textAlign: 'center',
3482     letterSpacing: 3,
3483   },
3484   // 对比表
3485   compareHeader: {
3486     flexDirection: 'row',
3487     paddingHorizontal: 20,
3488     paddingVertical: 10,
3489     borderBottomWidth: 1,
3490     borderColor: colors.hairline,
3491     backgroundColor: colors.background,
3492   },

```

```

3493   compareHeaderCell: {
3494     flex: 1,
3495     fontSize: 13,
3496     fontWeight: '600' as const,
3497     color: colors.textSecondary,
3498     textAlign: 'center',
3499   },
3500   compareRow: {
3501     flexDirection: 'row',
3502     paddingHorizontal: 20,
3503     paddingVertical: 12,
3504     alignItems: 'center',
3505   },
3506   compareCell: { flex: 1, textAlign: 'center' },
3507   // 底部
3508   bottomArea: {
3509     paddingHorizontal: 24,
3510     paddingTop: 16,
3511     paddingBottom: 32,
3512     gap: 10,
3513   },
3514   subscribeBtn: {
3515     height: 50,
3516     borderRadius: 25,
3517     alignItems: 'center',
3518     justifyContent: 'center',
3519     flexDirection: 'row',
3520     gap: 8,
3521   },
3522   subscribeBtnText: {
3523     fontSize: 16,
3524     fontWeight: '700' as const,
3525     color: 'FFFFFF',
3526   },
3527   restoreBtn: {
3528     height: 40,
3529     borderRadius: 12,
3530     alignItems: 'center',
3531     justifyContent: 'center',
3532     borderWidth: 1,
3533     borderColor: colors.hairline,
3534   },
3535   restoreBtnText: {
3536     fontSize: 14,
3537     color: colors.textSecondary,
3538   },
3539   closeBtn: {
3540     position: 'absolute',
3541     top: 16,
3542     right: 16,

```

```

3543     width: 32,
3544     height: 32,
3545     borderRadius: 16,
3546     backgroundColor: colors.surfaceLight,
3547     alignItems: 'center',
3548     justifyContent: 'center',
3549     zIndex: 10,
3550   },
3551   // 支付页头部
3552   activateHeader: {
3553     flexDirection: 'row',
3554     alignItems: 'center',
3555     paddingHorizontal: 24,
3556     paddingTop: 16,
3557     paddingBottom: 20,
3558     gap: 12,
3559   },
3560   backBtn: {
3561     width: 32,
3562     height: 32,
3563     borderRadius: 16,
3564     backgroundColor: colors.surfaceLight,
3565     alignItems: 'center',
3566     justifyContent: 'center',
3567   },
3568   activateTitle: {
3569     fontSize: 18,
3570     fontWeight: '600' as const,
3571     color: colors.textPrimary,
3572   },
3573   // 价格摘要
3574   summaryCard: {
3575     marginHorizontal: 24,
3576     marginVertical: 16,
3577     borderRadius: 14,
3578     padding: 16,
3579     backgroundColor: colors.surface,
3580     borderWidth: 1,
3581     borderColor: colors.hairline,
3582   },
3583   summaryRow: {
3584     flexDirection: 'row',
3585     justifyContent: 'space-between',
3586     alignItems: 'center',
3587   },
3588   summaryLabel: { fontSize: 14, color: colors.textSecondary },
3589   summaryValue: { fontSize: 18, fontWeight: '700' as const, color: colors.textPrimary },
3590   summaryContact: {
3591     fontSize: 13,
3592     color: colors.primary,

```



```

3593     textAlign: 'center',
3594     marginTop: 12,
3595     lineHeight: 20,
3596   },
3597  )), [colors, isDark, cardWidth]);
3598
3599   // ---- 激活码输入弹窗 ----
3600   if (showActivate) {
3601     return (
3602       <Modal visible={visible} transparent animationType="slide" onRequestClose={onClose}>
3603         <View style={s.overlay}>
3604           <View style={s.container}>
3605             <SafeAreaView style={{ flex: 1 }} edges={['bottom']}>
3606               <View style={s.activateHeader}>
3607                 <TouchableOpacity style={s.backBtn} onPress={() => setShowActivate(false)}>
3608                   <Ionicons name="chevron-back" size={20} color={colors.textPrimary} />
3609                 </TouchableOpacity>
3610                 <Text style={s.activateTitle}>激活 Pro 会员</Text>
3611               </View>
3612
3613               <View style={s.summaryCard}>
3614                 <View style={[s.summaryRow, { marginBottom: 10 }]}>
3615                   <Text style={s.summaryLabel}>已选套餐</Text>
3616                   <Text style={s.summaryValue}>{selectedPlanData.label} · ¥{selectedPlanData.price}</Text>
3617                 </View>
3618                 <Text style={s.summaryContact}>
3619                   请通过以下方式联系获取激活码: {'\n'}
3620                   QQ: 971117427 / 微信: lw971117427
3621                   QQ: 2067567633 / 微信: wzy936490679
3622                 </Text>
3623               </View>
3624
3625               <View style={s.activateSection}>
3626
3627                 <View style={s.codeInputWrap}>
3628                   <TextInput
3629                     style={s.codeInput}
3630                     value={activateCode}
3631                     onChangeText={setActivateCode}
3632                     placeholder="输入激活码"
3633                     placeholderTextColor={colors.textTertiary}
3634                     autoCapitalize="characters"
3635                     maxLength={16}
3636                   />
3637                 </View>
3638
3639                 <TouchableOpacity
3640                   style={[s.subscribeBtn, { backgroundColor: colors.pro, width: '100%' }, (!activateCode.trim()
3641                     onPress={handleActivate}
3642                     disabled={!activateCode.trim() || activating}

```

```

3643         activeOpacity={0.85}
3644     >
3645         <Ionicons name="key" size={18} color="#FFF" />
3646         <Text style={s.subscribeBtnText}>
3647             {activating ? '激活中...' : '激活 Pro'}
3648         </Text>
3649     </TouchableOpacity>
3650 </View>
3651 </SafeAreaView>
3652 </View>
3653 </View>
3654 </Modal>
3655 );
3656 }
3657
3658 // ---- 主弹窗：套餐选择 ----
3659 return (
3660 <Modal visible={visible} transparent animationType="slide" onRequestClose={onClose}>
3661     <View style={s.overlay}>
3662         <View style={s.container}>
3663             <SafeAreaView style={{ flex: 1 }} edges={['bottom']}>
3664                 <TouchableOpacity style={s.closeBtn} onPress={onClose} activeOpacity={0.7}>
3665                     <Ionicons name="close" size={18} color={colors.textSecondary} />
3666                 </TouchableOpacity>
3667
3668                 <View style={s.handleBar} />
3669
3670                 <View style={s.header}>
3671                     <Text style={s.title}>解锁 LearnFlow Pro</Text>
3672                     <Text style={s.subtitle}>释放全部学习潜力</Text>
3673                     <View style={s.proBadgeInline}>
3674                         <Ionicons name="star" size={14} color={colors.pro} />
3675                         <Text style={s.proBadgeText}>Pro 会员专属权益</Text>
3676                     </View>
3677                 </View>
3678
3679                 {/* 定价方案 */}
3680                 <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={s.plansContainer}
3681                     {PRICING_PLANS.map(plan => {
3682                         const isSelected = selectedPlan === plan.id;
3683                         return (
3684                             <TouchableOpacity
3685                                 key={plan.id}
3686                                 style={{[s.planCard, isSelected ? s.planSelected : s.planUnselected]}}
3687                                 onPress={() => setSelectedPlan(plan.id)}
3688                                 activeOpacity={0.85}
3689                             >
3690                                 {(plan.badge || plan.isPopular) && (
3691                                     <View style={s.planBadge}>
3692                                         <Text style={s.planBadgeText}>{plan.badge || '推荐'}</Text>

```

```

3693     </View>
3694   })
3695   <Text style={s.planLabel}>{plan.label}</Text>
3696   {plan.renewPrice ? (
3697     <>
3698       <View style={s.planPriceRow}>
3699         <Text style={s.planPriceSymbol}>¥</Text>
3700         <Text style={s.planPrice}>{plan.price}</Text>
3701       </View>
3702       <Text style={[s.planUnit, { fontSize: 10 }]}> 续 ¥{plan.renewPrice}/月</Text>
3703     </>
3704   ) : (
3705     <>
3706       <View style={s.planPriceRow}>
3707         <Text style={s.planPriceSymbol}>¥</Text>
3708         <Text style={s.planPrice}>{plan.price}</Text>
3709       </View>
3710       <View style={{ flexDirection: 'row', alignItems: 'baseline', gap: 2 }}>
3711         {plan.originalPrice && (
3712           <Text style={s.planOriginalPrice}>¥{plan.originalPrice}</Text>
3713         )}
3714         <Text style={s.planUnit}>/ {plan.unit}</Text>
3715       </View>
3716     </>
3717   )}
3718 </TouchableOpacity>
3719 );
3720 )))
3721 </ScrollView>
3722
3723 /* 权益对比 */
3724 <View style={s.compareHeader}>
3725   <Text style={[s.compareHeaderCell, { flex: 1.2 }]}>功能</Text>
3726   <Text style={s.compareHeaderCell}>免费版</Text>
3727   <View style={[s.compareHeaderCell, { flexDirection: 'row', alignItems: 'center', justifyContent: '
3728     <Icons name="star" size={13} color={colors.pro} />
3729     <Text style={{ color: colors.pro, fontSize: 13, fontWeight: '600' as const }}>Pro 版</Text>
3730   </View>
3731 </View>
3732 <ScrollView showsVerticalScrollIndicator={false}>
3733   {PRO_FEATURES.map((feat, i) => (
3734     <View key={i} style={[s.compareRow, i < PRO_FEATURES.length - 1 && { borderBottomWidth: StyleShe
3735     <View style={[s.compareCell, { flex: 1.2, flexDirection: 'row', alignItems: 'center', gap: 6 }
3736     <Icons name={feat.icon as any} size={14} color={colors.pro} />
3737     <Text style={{ fontSize: 13, color: colors.textPrimary }}>{feat.title}</Text>
3738   </View>
3739   <Text style={[s.compareCell, { color: colors.stone, fontSize: 12 }]}>{feat.freeLimit}</Text>
3740   <Text style={[s.compareCell, { color: colors.pro, fontSize: 12, fontWeight: '600' as const }]}
3741 </View>
3742 )))

```

```

3743     </ScrollView>
3744
3745     {/* 底部操作 */}
3746     <View style={s.bottomArea}>
3747         <TouchableOpacity
3748             style={[s.subscribeBtn, { backgroundColor: colors.pro }]}
3749             onPress={handleSubscribe}
3750             activeOpacity={0.85}
3751             >
3752             <Icons name="key" size={18} color="#FFF" />
3753             <Text style={s.subscribeBtnText}>
3754                 输入激活码 · ¥(selectedPlanData.price)/(selectedPlanData.unit)
3755             </Text>
3756         </TouchableOpacity>
3757         <TouchableOpacity style={s.restoreBtn} onPress={onClose} activeOpacity={0.7}>
3758             <Text style={s.restoreBtnText}>暂不升级，先用免费版</Text>
3759         </TouchableOpacity>
3760     </View>
3761 </SafeAreaView>
3762 </View>
3763 </View>
3764 </Modal>
3765 );
3766 };
3767
3768 export default SubscriptionModal;
3769
3770 === mobile/src/components/index.ts ===
3771 export * from './ui';
3772 export * from './skill-tree';
3773 export { default as AnimatedCheckbox } from './AnimatedCheckbox';
3774 export { default as GameHelpModal } from './GameHelpModal';
3775 export { default as GameModal } from './GameModal';
3776 export { default as HelpModal } from './HelpModal';
3777 export { default as MonsterIcon } from './MonsterIcon';
3778 export { default as MiniGames } from './MiniGames';
3779
3780 === mobile/src/components/skill-tree/SkillTreeNode.tsx ===
3781 import React from 'react';
3782 import { View, Text, TouchableOpacity, StyleSheet, Linking } from 'react-native';
3783 import { SkillNode as SkillNodeType } from '../types/skill';
3784 import { COLORS, SPACING } from '../utils/constants';
3785
3786 /**
3787  * 技能树节点组件属性
3788  */
3789 interface SkillTreeNodeProps {
3790     /** 节点数据 */
3791     node: SkillNodeType;
3792     /** 缩进层级 */
3793     level: number;

```

```

3791  /** 是否展开显示子节点 */
3792  isExpanded: boolean;
3793  /** 切换展开状态回调 */
3794  onToggle: (nodeId: string) => void;
3795  /** 点击学习资源链接回调 */
3796  onLinkPress: (url: string) => void;
3797  }
3798
3799  /**
3800   * 技能树节点组件
3801   * 递归渲染技能树中的节点，支持展开/折叠子节点
3802   */
3803  const SkillTreeNode: React.FC<SkillTreeNodeProps> = ({
3804    node,
3805    level,
3806    isExpanded,
3807    onToggle,
3808    onLinkPress,
3809  }) => {
3810    const hasChildren = node.children && node.children.length > 0;
3811    const marginLeft = level * SPACING.MEDIUM;
3812
3813    /**
3814     * 切换节点展开状态
3815     */
3816    const handleToggle = () => {
3817      if (hasChildren) {
3818        onToggle(node.id);
3819      }
3820    };
3821
3822    /**
3823     * 处理学习资源链接点击
3824     */
3825    const handleLinkPress = (url: string) => {
3826      onLinkPress(url);
3827    };
3828
3829    return (
3830      <View style={[styles.container, { marginLeft }]}>
3831        /* 节点头部 */
3832        <TouchableOpacity
3833          style={[styles.nodeHeader, hasChildren && styles.nodeHeaderInteractive]}
3834          onPress={handleToggle}
3835          disabled={!hasChildren}
3836        >
3837          <Text style={styles.nodeName} numberOfLines={1}>
3838            {node.name}
3839          </Text>
3840          {hasChildren && (

```

```

3841     <Text style={styles.expandIcon}>
3842         {isExpanded ? '▼': ''}
3843     </Text>
3844 })
3845 </TouchableOpacity>
3846
3847 /* 节点描述 */
3848 {node.description && (
3849     <Text style={styles.nodeDescription}>
3850         {node.description}
3851     </Text>
3852 )}
3853
3854 /* 学习资源链接 */
3855 {node.links && node.links.length > 0 && (
3856     <View style={styles.linksContainer}>
3857         {node.links.map((link, index) => (
3858             <TouchableOpacity
3859                 key={index}
3860                 style={styles.linkButton}
3861                 onPress={() => handleLinkPress(link.url)}
3862             >
3863                 <Text style={styles.linkText} numberOfLines={1}>
3864                     {link.title || '学习资源'}
3865                 </Text>
3866                 <Text style={styles.linkIcon}>→</Text>
3867             </TouchableOpacity>
3868         ))}
3869     </View>
3870 )}
3871
3872 /* 子节点 - 递归渲染 */
3873 {isExpanded && hasChildren && (
3874     <View style={styles.childrenContainer}>
3875         {node.children!.map((child) => (
3876             <SkillTreeNode
3877                 key={child.id}
3878                 node={child}
3879                 level={level + 1}
3880                 isExpanded={isExpanded}
3881                 onToggle={onToggle}
3882                 onLinkPress={onLinkPress}
3883             />
3884         ))}
3885     </View>
3886 )}
3887 </View>
3888 );
3889 };
3890

```

```

3891 const styles = StyleSheet.create({
3892   /** 容器样式 */
3893   container: {
3894     marginBottom: SPACING.SMALL,
3895   },
3896   /** 节点头部样式 */
3897   nodeHeader: {
3898     flexDirection: 'row',
3899     alignItems: 'center',
3900     padding: SPACING.MEDIUM,
3901     backgroundColor: COLORS.WHITE,
3902     borderRadius: 8,
3903     borderWidth: 1,
3904     borderColor: COLORS.BORDER,
3905   },
3906   /** 可交互节点头部样式（有子节点时） */
3907   nodeHeaderInteractive: {
3908     // 可在此添加可交互状态的样式
3909   },
3910   /** 节点名称样式 */
3911   nodeName: {
3912     fontSize: 16,
3913     fontWeight: '600',
3914     color: COLORS.TEXT_PRIMARY,
3915     flex: 1,
3916   },
3917   /** 展开/折叠图标样式 */
3918   expandIcon: {
3919     fontSize: 12,
3920     color: COLORS.TEXT_SECONDARY,
3921     marginLeft: SPACING.SMALL,
3922   },
3923   /** 节点描述样式 */
3924   nodeDescription: {
3925     fontSize: 14,
3926     color: COLORS.TEXT_SECONDARY,
3927     marginTop: SPACING.SMALL,
3928     marginLeft: SPACING.MEDIUM,
3929     lineHeight: 20,
3930   },
3931   /** 学习资源链接容器样式 */
3932   linksContainer: {
3933     marginTop: SPACING.SMALL,
3934   },
3935   /** 学习资源链接按钮样式 */
3936   linkButton: {
3937     flexDirection: 'row',
3938     justifyContent: 'space-between',
3939     alignItems: 'center',
3940     padding: SPACING.MEDIUM,

```

```

3941   backgroundColor: '#e8f4fd',
3942   borderRadius: 6,
3943   marginTop: SPACING.SMALL,
3944 },
3945 /** 学习资源链接文字样式 */
3946 linkText: {
3947   fontSize: 14,
3948   color: COLORS.PRIMARY,
3949   flex: 1,
3950 },
3951 /** 学习资源链接箭头样式 */
3952 linkIcon: {
3953   fontSize: 14,
3954   color: COLORS.PRIMARY,
3955   marginLeft: SPACING.SMALL,
3956 },
3957 /** 子节点容器样式 */
3958 childrenContainer: {
3959   marginTop: SPACING.SMALL,
3960 },
3961 });
3962
3963 export default SkillTreeNode;
3964
3965 === mobile/src/components/skill-tree/SkillTreeView.tsx ===
3966 import React, { useMemo } from 'react';
3967 import { View, Text, StyleSheet, TouchableOpacity, Linking } from 'react-native';
3968 import { Ionicons } from '@expo/vector-icons';
3969 import { SkillTree, PlatformType } from '../types/skill';
3970 import { PLATFORM_NAME_MAP } from '../data/skillTrees';
3971 import { useTheme } from '../contexts/ThemeContext';
3972
3973 interface SkillTreeViewProps {
3974   skillTree: SkillTree;
3975 }
3976
3977
3978
3979 const SkillTreeView: React.FC<SkillTreeViewProps> = ({ skillTree }) => {
3980   const { colors } = useTheme();
3981
3982   const getPlatformIcon = (platform: PlatformType) => {
3983     switch (platform) {
3984       case 'bilibili':
3985         return { icon: 'play-circle', color: '#FB7299' };
3986       case 'xiaohongshu':
3987         return { icon: 'book', color: '#FF2442' };
3988       case 'mooc':
3989         return { icon: 'school', color: colors.primary };

```



```

3990     default:
3991         return { icon: 'link', color: '#888' };
3992     }
3993 };
3994
3995 const getStageColor = (stage: string) => {
3996     switch (stage) {
3997         case 'beginner':
3998             return { bg: colors.borderLight, border: colors.borderDark, text: colors.primary };
3999         case 'intermediate':
4000             return { bg: 'rgba(72,209,176,0.15)', border: 'rgba(72,209,176,0.3)', text: '#48D1B0' };
4001         case 'advanced':
4002             return { bg: 'rgba(255,152,0,0.15)', border: 'rgba(255,152,0,0.3)', text: '#FF9800' };
4003         default:
4004             return { bg: colors.borderLight, border: colors.border, text: colors.textPrimary };
4005     }
4006 };
4007
4008 const StageNode: React.FC<{ name: string; duration: number; stageColor: { bg: string; border: string; text: st
4009     <View style={[styles.stageNode, { backgroundColor: stageColor.bg, borderColor: stageColor.border }]}>
4010         <Text style={[styles.stageName, { color: stageColor.text }]}>{name}</Text>
4011         <Text style={styles.stageDuration}>{duration}小时</Text>
4012     </View>
4013 );
4014
4015 const SkillNode: React.FC<{
4016     name: string;
4017     platform: PlatformType;
4018     duration: number;
4019     url: string;
4020     index: number;
4021     stageColor: { bg: string; border: string; text: string };
4022 }> = ({ name, platform, duration, url, index, stageColor }) => {
4023     const platformInfo = getPlatformIcon(platform);
4024
4025     const handlePress = async () => {
4026         try {
4027             await Linking.openURL(url);
4028         } catch (error) {
4029             console.error('Failed to open URL:', error);
4030         }
4031     };
4032
4033     return (
4034         <TouchableOpacity
4035             style={[styles.skillNode, { borderColor: stageColor.border }]}
4036             onPress={handlePress}
4037             activeOpacity={0.7}
4038         >
4039         <View style={styles.skillNodeHeader}>

```

```

4040     <View style={{styles.nodeBadge, { backgroundColor: colors.borderDark }}}>
4041       <Text style={{styles.nodeBadgeText, { color: colors.primary }}}>{index + 1}</Text>
4042     </View>
4043     <Ionicons name={platformInfo.icon as any} size={14} color={platformInfo.color} />
4044   </View>
4045   <Text style={{styles.skillNodeName, { color: colors.textPrimary }}}>{name}</Text>
4046   <View style={styles.skillNodeFooter}>
4047     <Text style={{styles.platformText, { color: platformInfo.color }}}>
4048       {PLATFORM_NAME_MAP[platform]}
4049     </Text>
4050     <Text style={styles.durationText}>{duration}h</Text>
4051   </View>
4052   <View style={styles.arrowIcon}>
4053     <Ionicons name="arrow-forward" size={14} color={colors.primary} />
4054   </View>
4055 </TouchableOpacity>
4056 );
4057 };
4058
4059
4060 const styles = useMemo(() => StyleSheet.create({
4061   container: {
4062     flex: 1,
4063     backgroundColor: colors.background,
4064     padding: 16,
4065   },
4066   header: {
4067     marginBottom: 24,
4068     alignItems: 'center',
4069   },
4070   title: {
4071     fontSize: 24,
4072     fontWeight: '800',
4073     color: colors.textPrimary,
4074     marginBottom: 8,
4075   },
4076   description: {
4077     fontSize: 14,
4078     color: colors.textSecondary,
4079     textAlign: 'center',
4080     marginBottom: 12,
4081   },
4082   totalDurationBadge: {
4083     flexDirection: 'row',
4084     alignItems: 'center',
4085     gap: 6,
4086     paddingHorizontal: 16,
4087     paddingVertical: 8,
4088     borderRadius: 20,
4089     backgroundColor: colors.borderLight,

```

```

4090 },
4091 totalDurationText: {
4092   fontSize: 12,
4093   color: colors.primary,
4094   fontWeight: '600',
4095 },
4096 treeContainer: {
4097   flex: 1,
4098   alignItems: 'center',
4099 },
4100 centerNode: {
4101   width: 100,
4102   height: 100,
4103   borderRadius: 50,
4104   backgroundColor: colors.primary,
4105   shadowColor: colors.primary,
4106   shadowOffset: { width: 0, height: 8 },
4107   shadowOpacity: 0.4,
4108   shadowRadius: 20,
4109   alignItems: 'center',
4110   justifyContent: 'center',
4111   marginBottom: 16,
4112   borderWidth: 3,
4113   borderColor: colors.borderDark,
4114 },
4115 centerNodeInner: {
4116   alignItems: 'center',
4117   gap: 6,
4118 },
4119 centerNodeText: {
4120   fontSize: 12,
4121   color: 'FFFFFF',
4122   fontWeight: '600',
4123 },
4124 stagesContainer: {
4125   flexDirection: 'row',
4126   gap: 12,
4127   width: '100%',
4128 },
4129 stageColumn: {
4130   flex: 1,
4131   alignItems: 'center',
4132 },
4133 middleStage: {
4134   marginTop: 20,
4135 },
4136 stageConnector: {
4137   height: 24,
4138   alignItems: 'center',
4139   marginBottom: 8,

```

```

4140 },
4141 connectorLine: {
4142   width: 3,
4143   height: '100%',
4144   borderRadius: 2,
4145 },
4146 stageNode: {
4147   paddingHorizontal: 12,
4148   paddingVertical: 8,
4149   borderRadius: 12,
4150   borderWidth: 1,
4151   alignItems: 'center',
4152   marginBottom: 12,
4153 },
4154 stageName: {
4155   fontSize: 12,
4156   fontWeight: '700',
4157   marginBottom: 2,
4158 },
4159 stageDuration: {
4160   fontSize: 10,
4161   color: '#888',
4162 },
4163 skillNodesList: {
4164   gap: 8,
4165   width: '100%',
4166 },
4167 skillNode: {
4168   backgroundColor: 'rgba(255,255,255,0.04)',
4169   borderWidth: 1,
4170   borderRadius: 12,
4171   padding: 12,
4172   position: 'relative',
4173 },
4174 skillNodeHeader: {
4175   flexDirection: 'row',
4176   alignItems: 'center',
4177   gap: 8,
4178   marginBottom: 6,
4179 },
4180 nodeBadge: {
4181   width: 20,
4182   height: 20,
4183   borderRadius: 6,
4184   alignItems: 'center',
4185   justifyContent: 'center',
4186 },
4187 nodeBadgeText: {
4188   fontSize: 10,
4189   fontWeight: '700',

```

```

4190   },
4191   skillNodeName: {
4192     fontSize: 13,
4193     fontWeight: '600',
4194     marginBottom: 8,
4195     lineHeight: 16,
4196   },
4197   skillNodeFooter: {
4198     flexDirection: 'row',
4199     alignItems: 'center',
4200     justifyContent: 'space-between',
4201   },
4202   platformText: {
4203     fontSize: 10,
4204     fontWeight: '500',
4205   },
4206   durationText: {
4207     fontSize: 10,
4208     color: '#888',
4209     fontWeight: '500',
4210   },
4211   arrowIcon: {
4212     position: 'absolute',
4213     right: 12,
4214     bottom: 12,
4215   },
4216   }), [colors]);
4217
4218   return (
4219     <View style={styles.container}>
4220       <View style={styles.header}>
4221         <Text style={styles.title}>{skillTree.title}</Text>
4222         <Text style={styles.description}>{skillTree.description}</Text>
4223         <View style={styles.totalDurationBadge}>
4224           <Ionicons name="time" size={14} color={colors.primary} />
4225           <Text style={styles.totalDurationText}>总时长 {skillTree.totalDuration}小时</Text>
4226         </View>
4227       </View>
4228
4229       <View style={styles.treeContainer}>
4230         <View style={styles.centerNode}>
4231           <View style={styles.centerNodeInner}>
4232             <Ionicons name="locate" size={28} color="#FFFFFF" />
4233             <Text style={styles.centerNodeText}>学习领域</Text>
4234           </View>
4235         </View>
4236
4237         <View style={styles.stagesContainer}>
4238           {skillTree.stages.map((stage, stageIndex) => {
4239             const stageColor = getStageColor(stage.id);

```

```

4240     return (
4241       <View key={stage.id} style={{styles.stageColumn, stageIndex === 1 && styles.middleStage}}>
4242         <View style={styles.stageConnector}>
4243           <View style={{styles.connectorLine, { backgroundColor: stageColor.text }}} />
4244         </View>
4245
4246         <StageNode name={stage.name} duration={stage.duration} stageColor={stageColor} />
4247
4248         <View style={styles.skillNodesList}>
4249           {stage.nodes.map((node, nodeIndex) => (
4250             <SkillNode
4251               key={node.id}
4252               name={node.name}
4253               platform={node.platform}
4254               duration={node.duration}
4255               url={node.url}
4256               index={nodeIndex}
4257               stageColor={stageColor}
4258             />
4259           ))}
4260         </View>
4261       </View>
4262     );
4263   }}}
4264 </View>
4265 </View>
4266 </View>
4267 );
4268 };
4269
4270 export default SkillTreeView;
4271
4272 === mobile/src/components/skill-tree/index.ts ===
4273 export { default as SkillTreeNode } from './SkillTreeNode';
4274 export { default as SkillTreeView } from './SkillTreeView';
4275
4276 === mobile/src/components/ui/Button.tsx ===
4277 import React from 'react';
4278 import { TouchableOpacity, Text, StyleSheet } from 'react-native';
4279 import { COLORS, SPACING } from '../utils/constants';
4280
4281 interface ButtonProps {
4282   title: string;
4283   onPress: () => void;
4284   disabled?: boolean;
4285   variant?: 'primary' | 'secondary' | 'outline';
4286   size?: 'small' | 'medium' | 'large';
4287   fullWidth?: boolean;
4288 }
4289
4290 const Button: React.FC<ButtonProps> = ({
4291   title,

```

```

4288   onPress,
4289   disabled = false,
4290   variant = 'primary',
4291   size = 'medium',
4292   fullWidth = true,
4293 }) => {
4294   const getButtonStyle = () => {
4295     const baseStyle = [
4296       styles.button,
4297       styles[`button_${variant}`],
4298       styles[`button_${size}`],
4299       fullWidth && styles.button_fullWidth,
4300       disabled && styles.button_disabled,
4301     ];
4302
4303     return baseStyle.filter(Boolean);
4304   };
4305
4306   const getTextStyle = () => {
4307     return [
4308       styles.text,
4309       styles[`text_${variant}`],
4310       styles[`text_${size}`],
4311       disabled && styles.text_disabled,
4312     ];
4313   };
4314
4315   return (
4316     <TouchableOpacity
4317       style={getButtonStyle()}
4318       onPress={onPress}
4319       disabled={disabled}
4320     >
4321       <Text style={getTextStyle()}>
4322         {title}
4323       </Text>
4324     </TouchableOpacity>
4325   );
4326 };
4327
4328 const styles = StyleSheet.create({
4329   button: {
4330     borderRadius: 8,
4331     justifyContent: 'center',
4332     alignItems: 'center',
4333   },
4334   button_primary: {
4335     backgroundColor: COLORS.PRIMARY,
4336   },
4337   button_secondary: {

```

```

4338   backgroundColor: COLORS.SUCCESS,
4339 },
4340 button_outline: {
4341   backgroundColor: 'transparent',
4342   borderWidth: 1,
4343   borderColor: COLORS.PRIMARY,
4344 },
4345 button_small: {
4346   height: 40,
4347   paddingHorizontal: SPACING.MEDIUM,
4348 },
4349 button_medium: {
4350   height: 50,
4351   paddingHorizontal: SPACING.MEDIUM,
4352 },
4353 button_large: {
4354   height: 60,
4355   paddingHorizontal: SPACING.LARGE,
4356 },
4357 button_fullWidth: {
4358   width: '100%',
4359 },
4360 button_disabled: {
4361   backgroundColor: COLORS.BORDER,
4362 },
4363 text: {
4364   fontWeight: '600',
4365 },
4366 text_primary: {
4367   color: COLORS.WHITE,
4368 },
4369 text_secondary: {
4370   color: COLORS.WHITE,
4371 },
4372 text_outline: {
4373   color: COLORS.PRIMARY,
4374 },
4375 text_small: {
4376   fontSize: 14,
4377 },
4378 text_medium: {
4379   fontSize: 16,
4380 },
4381 text_large: {
4382   fontSize: 18,
4383 },
4384 text_disabled: {
4385   color: COLORS.TEXT_SECONDARY,
4386 },
4387 });

```



```

4388
4389 export default Button;
    === mobile/src/components/ui/CuteButton.tsx ===
4390 import React from 'react';
4391 import { TouchableOpacity, Text, StyleSheet } from 'react-native';
4392 import { CUTE_COLORS, SPACING, BORDER_RADIUS, SHADOWS } from '../utils/constants';
4393
4394 interface CuteButtonProps {
4395   title: string;
4396   onPress: () => void;
4397   disabled?: boolean;
4398   variant?: 'primary' | 'secondary' | 'success' | 'warning' | 'danger';
4399   size?: 'small' | 'medium' | 'large';
4400   fullWidth?: boolean;
4401 }
4402
4403 const CuteButton: React.FC<CuteButtonProps> = ({
4404   title,
4405   onPress,
4406   disabled = false,
4407   variant = 'primary',
4408   size = 'medium',
4409   fullWidth = false,
4410 }) => {
4411   const getButtonColor = () => {
4412     switch (variant) {
4413       case 'primary':
4414         return CUTE_COLORS.PINK;
4415       case 'secondary':
4416         return CUTE_COLORS.SKY_BLUE;
4417       case 'success':
4418         return CUTE_COLORS.MINT;
4419       case 'warning':
4420         return CUTE_COLORS.BUTTER_YELLOW;
4421       case 'danger':
4422         return CUTE_COLORS.CORAL;
4423       default:
4424         return CUTE_COLORS.PINK;
4425     }
4426   };
4427
4428   const getButtonSize = () => {
4429     switch (size) {
4430       case 'small':
4431         return { height: 40, paddingHorizontal: SPACING.MEDIUM };
4432       case 'medium':
4433         return { height: 52, paddingHorizontal: SPACING.LARGE };
4434       case 'large':
4435         return { height: 64, paddingHorizontal: SPACING.XLARGE };
4436       default:

```

```

4437     return { height: 52, paddingHorizontal: SPACING.LARGE };
4438   }
4439 };
4440
4441 const getTextSize = () => {
4442   switch (size) {
4443     case 'small':
4444       return 14;
4445     case 'medium':
4446       return 16;
4447     case 'large':
4448       return 18;
4449     default:
4450       return 16;
4451   }
4452 };
4453
4454 return (
4455   <TouchableOpacity
4456     style={[
4457       styles.button,
4458       {
4459         backgroundColor: disabled ? CUTE_COLORS.GRAY : getButtonColor(),
4460         ...getButtonSize(),
4461         width: fullWidth ? '100%' : undefined,
4462       },
4463     ]}
4464     onPress={onPress}
4465     disabled={disabled}
4466     activeOpacity={0.8}
4467   >
4468     <Text
4469       style={[
4470         styles.text,
4471         {
4472           fontSize: getTextSize(),
4473           color: disabled ? CUTE_COLORS.DARK_GRAY : CUTE_COLORS.WHITE,
4474         },
4475       ]}
4476     >
4477       {title}
4478     </Text>
4479   </TouchableOpacity>
4480 );
4481 };
4482
4483 const styles = StyleSheet.create({
4484   button: {
4485     justifyContent: 'center',
4486     alignItems: 'center',

```

```

4487   borderRadius: BORDER_RADIUS.MEDIUM,
4488   ...SHADOWS.SOFT,
4489 },
4490 text: {
4491   fontWeight: '700',
4492   letterSpacing: 0.5,
4493 },
4494 });
4495
4496 export default CuteButton;
4497
=== mobile/src/components/ui/CuteCard.tsx ===
4498 import React from 'react';
4499 import { View, StyleSheet, ViewStyle } from 'react-native';
4500 import { CUTE_COLORS, SPACING, BORDER_RADIUS, SHADOWS } from '../utils/constants';
4501
4502 interface CuteCardProps {
4503   children: React.ReactNode;
4504   style?: ViewStyle;
4505   backgroundColor?: string;
4506   padding?: number;
4507 }
4508
4509 const CuteCard: React.FC<CuteCardProps> = ({
4510   children,
4511   style,
4512   backgroundColor = CUTE_COLORS.WHITE,
4513   padding = SPACING.LARGE,
4514 }) => {
4515   return (
4516     <View
4517       style={{
4518         styles.card,
4519         {
4520           backgroundColor,
4521           padding,
4522         },
4523         style,
4524       }}
4525     >
4526       {children}
4527     </View>
4528   );
4529 };
4530
4531 const styles = StyleSheet.create({
4532   card: {
4533     borderRadius: BORDER_RADIUS.LARGE,
4534     ...SHADOWS.SOFT,
4535   },

```

```

4536 });
4537
4538 export default CuteCard;
4539
4540 === mobile/src/components/ui/CuteInput.tsx ===
4541 import React from 'react';
4542 import { TextInput, StyleSheet, ViewStyle, TextStyle } from 'react-native';
4543 import { CUTE_COLORS, SPACING, BORDER_RADIUS } from '../utils/constants';
4544
4545 interface CuteInputProps {
4546   value: string;
4547   onChangeText: (text: string) => void;
4548   placeholder?: string;
4549   secureTextEntry?: boolean;
4550   multiline?: boolean;
4551   style?: ViewStyle;
4552   inputStyle?: TextStyle;
4553 }
4554
4555 const CuteInput: React.FC<CuteInputProps> = ({
4556   value,
4557   onChangeText,
4558   placeholder,
4559   secureTextEntry = false,
4560   multiline = false,
4561   style,
4562   inputStyle,
4563 }) => {
4564   return (
4565     <TextInput
4566       style={[
4567         styles.input,
4568         style,
4569         inputStyle,
4570         multiline ? styles.multiline : null,
4571       ]}
4572       value={value}
4573       onChangeText={onChangeText}
4574       placeholder={placeholder}
4575       placeholderTextColor={CUTE_COLORS.GRAY}
4576       secureTextEntry={secureTextEntry}
4577       multiline={multiline}
4578       selectionColor={CUTE_COLORS.PINK}
4579     />
4580   );
4581
4582 const styles = StyleSheet.create({
4583   input: {
4584     height: 52,

```

```

4585   borderWidth: 2,
4586   borderColor: CUTE_COLORS.LIGHT_PINK,
4587   backgroundColor: CUTE_COLORS.WHITE,
4588   paddingHorizontal: SPACING.MEDIUM,
4589   fontSize: 16,
4590   color: CUTE_COLORS.DARK_GRAY,
4591   borderRadius: BORDER_RADIUS.MEDIUM,
4592 },
4593 multiline: {
4594   height: 120,
4595   paddingVertical: SPACING.MEDIUM,
4596   textAlignVertical: 'top',
4597 },
4598 });
4599
4600 export default CuteInput;
4601
4602 === mobile/src/components/ui/Input.tsx ===
4603 import React from 'react';
4604 import { TextInput, StyleSheet, Text, View } from 'react-native';
4605 import { COLORS, SPACING, PIXEL_COLORS } from '../utils/constants';
4606
4607 interface InputProps {
4608   value: string;
4609   onChangeText: (text: string) => void;
4610   placeholder?: string;
4611   label?: string;
4612   secureTextEntry?: boolean;
4613   disabled?: boolean;
4614   error?: string;
4615   onSubmitEditing?: () => void;
4616 }
4617
4618 const Input: React.FC<InputProps> = ({
4619   value,
4620   onChangeText,
4621   placeholder,
4622   label,
4623   secureTextEntry = false,
4624   disabled = false,
4625   error,
4626   onSubmitEditing,
4627 }) => {
4628   return (
4629     <View style={styles.container}>
4630       {label && <Text style={styles.label}>{label}</Text>}
4631       <TextInput
4632         style={[
4633           styles.input,
4634           disabled && styles.input_disabled,

```

```

4634     error && styles.input_error,
4635   })
4636   value={value}
4637   onChangeText={onChangeText}
4638   placeholder={placeholder}
4639   placeholderTextColor={PIXEL_COLORS.PIXEL_LIGHT_GRAY}
4640   secureTextEntry={secureTextEntry}
4641   editable={!disabled}
4642   onSubmitEditing={onSubmitEditing}
4643   />
4644   {error && <Text style={styles.errorText}>{error}</Text>}
4645 </View>
4646 );
4647 };
4648
4649 const styles = StyleSheet.create({
4650   container: {
4651     marginBottom: SPACING.MEDIUM,
4652   },
4653   label: {
4654     fontSize: 16,
4655     fontWeight: '600',
4656     color: PIXEL_COLORS.TEXT_PRIMARY,
4657     marginBottom: SPACING.SMALL,
4658   },
4659   input: {
4660     width: '100%',
4661     height: 50,
4662     borderWidth: 1,
4663     borderColor: PIXEL_COLORS.PIXEL_GRAY,
4664     borderRadius: 8,
4665     paddingHorizontal: SPACING.MEDIUM,
4666     backgroundColor: PIXEL_COLORS.BACKGROUND_LIGHT,
4667     fontSize: 16,
4668     color: PIXEL_COLORS.TEXT_PRIMARY,
4669   },
4670   input_disabled: {
4671     backgroundColor: PIXEL_COLORS.PIXEL_GRAY,
4672     color: PIXEL_COLORS.TEXT_SECONDARY,
4673   },
4674   input_error: {
4675     borderColor: PIXEL_COLORS.ERROR,
4676   },
4677   errorText: {
4678     fontSize: 14,
4679     color: PIXEL_COLORS.ERROR,
4680     marginTop: SPACING.SMALL,
4681   },
4682 });
4683

```

```

4684 export default Input;
    === mobile/src/components/ui/Loading.tsx ===
4685 import React from 'react';
4686 import { View, Text, StyleSheet, ActivityIndicator } from 'react-native';
4687 import { COLORS } from '../utils/constants';
4688
4689 interface LoadingProps {
4690   visible: boolean;
4691   message?: string;
4692   type?: 'overlay' | 'inline';
4693 }
4694
4695 const Loading: React.FC<LoadingProps> = ({
4696   visible,
4697   message = '处理中...',
4698   type = 'overlay',
4699 }) => {
4700   if (!visible) return null;
4701
4702   if (type === 'overlay') {
4703     return (
4704       <View style={styles.overlay}>
4705         <View style={styles.loadingContainer}>
4706           <ActivityIndicator size="large" color={COLORS.PRIMARY} />
4707           <Text style={styles.loadingText}>{message}</Text>
4708         </View>
4709       </View>
4710     );
4711   }
4712
4713   return (
4714     <View style={styles.inlineContainer}>
4715       <ActivityIndicator size="small" color={COLORS.PRIMARY} />
4716       <Text style={styles.inlineText}>{message}</Text>
4717     </View>
4718   );
4719 };
4720
4721 const styles = StyleSheet.create({
4722   overlay: {
4723     position: 'absolute',
4724     top: 0,
4725     left: 0,
4726     right: 0,
4727     bottom: 0,
4728     backgroundColor: 'rgba(0,0,0,0.3)',
4729     justifyContent: 'center',
4730     alignItems: 'center',
4731     zIndex: 999,
4732   },

```

```

4733   loadingContainer: {
4734     backgroundColor: COLORS.WHITE,
4735     padding: 20,
4736     borderRadius: 8,
4737     alignItems: 'center',
4738     minWidth: 120,
4739   },
4740   loadingText: {
4741     marginTop: 10,
4742     fontSize: 16,
4743     color: COLORS.TEXT_PRIMARY,
4744     fontWeight: '600',
4745   },
4746   inlineContainer: {
4747     flexDirection: 'row',
4748     alignItems: 'center',
4749     padding: 10,
4750   },
4751   inlineText: {
4752     marginLeft: 10,
4753     fontSize: 14,
4754     color: COLORS.TEXT_SECONDARY,
4755   },
4756 });
4757
4758 export default Loading;
    === mobile/src/components/ui/PixelButton.tsx ===
4759 import React from 'react';
4760 import {
4761   TouchableOpacity,
4762   Text,
4763   StyleSheet,
4764   TextStyle,
4765   ViewStyle,
4766   ActivityIndicator,
4767 } from 'react-native';
4768 import { useTheme } from '../../contexts/ThemeContext';
4769 import { Ionicons } from '@expo/vector-icons';
4770
4771 interface PixelButtonProps {
4772   title: string;
4773   onPress: () => void;
4774   variant?: 'primary' | 'secondary' | 'outline';
4775   size?: 'small' | 'medium' | 'large';
4776   icon?: keyof typeof Ionicons.glyphMap;
4777   loading?: boolean;
4778   disabled?: boolean;
4779   style?: ViewStyle;
4780 }
4781

```



```

4782 const PixelButton: React.FC<PixelButtonProps> = ({
4783   title,
4784   onPress,
4785   variant = 'primary',
4786   size = 'medium',
4787   icon,
4788   loading = false,
4789   disabled = false,
4790   style,
4791 }) => {
4792   const { colors } = useTheme();
4793
4794   const getButtonColors = () => {
4795     switch (variant) {
4796       case 'secondary':
4797         return {
4798           bg: colors.surface,
4799           border: colors.hairline,
4800           text: colors.textPrimary,
4801           iconColor: colors.textPrimary,
4802         };
4803       case 'outline':
4804         return {
4805           bg: 'transparent',
4806           border: colors.primary,
4807           text: colors.primary,
4808           iconColor: colors.primary,
4809         };
4810       default:
4811         return {
4812           bg: colors.primary,
4813           border: colors.primary,
4814           text: colors.onPrimary,
4815           iconColor: colors.onPrimary,
4816         };
4817     }
4818   };
4819
4820   const buttonColors = getButtonColors();
4821   const isDisabled = disabled || loading;
4822
4823   return (
4824     <TouchableOpacity
4825       style={[
4826         styles.button,
4827         styles[size],
4828       ]
4829       {
4830         backgroundColor: isDisabled ? colors.border : buttonColors.bg,
4831         borderColor: buttonColors.border,

```

```

4832     shadowOffset: { width: 0, height: 2 },
4833     shadowOpacity: 0.08,
4834     shadowRadius: 8,
4835     elevation: variant === 'primary' ? 3 : 1,
4836     opacity: isDisabled ? 0.5 : 1,
4837   },
4838   style,
4839 ]}
4840 onPress={onPress}
4841 disabled={isDisabled}
4842 activeOpacity={0.7}
4843 >
4844 {loading ? (
4845   <ActivityIndicator size="small" color={buttonColors.text} />
4846 ) : (
4847   <>
4848     {icon && (
4849       <Ionicons
4850         name={icon}
4851         size={size === 'small' ? 16 : size === 'large' ? 22 : 18}
4852         color={buttonColors.iconColor}
4853         style={styles.icon}
4854       />
4855     )}
4856     <Text
4857       style={[
4858         styles.text,
4859         styles[`_${size}Text` as keyof typeof styles] as TextStyle,
4860         { color: buttonColors.text },
4861       ]}
4862     >
4863       {title}
4864     </Text>
4865   </>
4866 )}
4867 </TouchableOpacity>
4868 );
4869 };
4870
4871 const styles = StyleSheet.create({
4872   button: {
4873     flexDirection: 'row',
4874     alignItems: 'center',
4875     justifyContent: 'center',
4876     borderRadius: 14,
4877     borderWidth: 1,
4878   },
4879   small: {
4880     paddingVertical: 8,
4881     paddingHorizontal: 16,

```

```

4882   },
4883   medium: {
4884     paddingVertical: 12,
4885     paddingHorizontal: 24,
4886   },
4887   large: {
4888     paddingVertical: 16,
4889     paddingHorizontal: 32,
4890   },
4891   text: {
4892     fontWeight: '700',
4893     fontFamily: 'Courier',
4894   },
4895   smallText: {
4896     fontSize: 13,
4897   },
4898   mediumText: {
4899     fontSize: 15,
4900   },
4901   largeText: {
4902     fontSize: 17,
4903   },
4904   icon: {
4905     marginRight: 6,
4906   },
4907 });
4908
4909 export default PixelButton;
4910
4911 === mobile/src/components/ui/PixelCard.tsx ===
4912 import React from 'react';
4913 import { View, StyleSheet, ViewStyle } from 'react-native';
4914 import { useTheme } from '../../contexts/ThemeContext';
4915
4916 interface PixelCardProps {
4917   children: React.ReactNode;
4918   style?: ViewStyle;
4919   variant?: 'default' | 'highlight' | 'dark';
4920 }
4921
4922 const PixelCard: React.FC<PixelCardProps> = ({
4923   children,
4924   style,
4925   variant = 'default',
4926 }) => {
4927   const { colors } = useTheme();
4928
4929   const getCardColors = () => {
4930     switch (variant) {

```

```

4931     return {
4932       bg: colors.cardAi,
4933       border: colors.primary + '30',
4934     };
4935     case 'dark':
4936       return {
4937         bg: colors.surface,
4938         border: colors.hairline,
4939       };
4940     default:
4941       return {
4942         bg: colors.surface,
4943         border: colors.hairline,
4944       };
4945   }
4946 };
4947
4948 const cardColors = getCardColors();
4949
4950 return (
4951   <View
4952     style={[
4953       styles.card,
4954       {
4955         backgroundColor: cardColors.bg,
4956         borderColor: cardColors.border,
4957         shadowColor: colors.shadow,
4958         shadowOffset: { width: 0, height: 1 },
4959         shadowOpacity: 0.04,
4960         shadowRadius: 3,
4961         elevation: 2,
4962       },
4963       style,
4964     ]}
4965   >
4966     {children}
4967   </View>
4968 );
4969 };
4970
4971 const styles = StyleSheet.create({
4972   card: {
4973     borderRadius: 16,
4974     borderWidth: 1,
4975     padding: 16,
4976     overflow: 'hidden',
4977   },
4978 });
4979
4980 export default PixelCard;

```

```

4981
    === mobile/src/components/ui/PixelInput.tsx ===
4982 import React from 'react';
4983 import { View, TextInput, Text, StyleSheet, TextStyle } from 'react-native';
4984 import { useTheme } from '../../contexts/ThemeContext';
4985
4986 interface PixelInputProps {
4987   label?: string;
4988   value: string;
4989   onChangeText: (text: string) => void;
4990   placeholder?: string;
4991   secureTextEntry?: boolean;
4992   keyboardType?: 'default' | 'email-address' | 'numeric' | 'phone-pad';
4993   autoCapitalize?: 'none' | 'sentences' | 'words' | 'characters';
4994   multiline?: boolean;
4995   numberOfLines?: number;
4996   error?: string;
4997   style?: TextStyle;
4998 }
4999
5000 const PixelInput: React.FC<PixelInputProps> = ({
5001   label,
5002   value,
5003   onChangeText,
5004   placeholder = '',
5005   secureTextEntry = false,
5006   keyboardType = 'default',
5007   autoCapitalize = 'none',
5008   multiline = false,
5009   numberOfLines = 1,
5010   error,
5011   style,
5012 }) => {
5013   const { colors } = useTheme();
5014
5015   return (
5016     <View style={styles.container}>
5017       {label && (
5018         <Text style={[styles.label, { color: colors.textPrimary }]}>
5019           {label}
5020         </Text>
5021       )}
5022       <TextInput
5023         style={[
5024           styles.input,
5025           {
5026             backgroundColor: colors.inputBg,
5027             borderColor: error ? colors.error : colors.inputBorder,
5028             color: colors.textPrimary,
5029           },

```

```

5030     multiline && styles.multiline,
5031     style,
5032   })
5033   value={value}
5034   onChangeText={onChangeText}
5035   placeholder={placeholder}
5036   placeholderTextColor={colors.textTertiary}
5037   secureTextEntry={secureTextEntry}
5038   keyboardType={keyboardType}
5039   autoCapitalize={autoCapitalize}
5040   multiline={multiline}
5041   numberOfLines={numberOfLines}
5042   />
5043   {error && (
5044     <Text style={{styles.errorText, { color: colors.error }}}>
5045       {error}
5046     </Text>
5047   )}
5048 </View>
5049 );
5050 };
5051
5052 const styles = StyleSheet.create({
5053   container: {
5054     marginBottom: 16,
5055   },
5056   label: {
5057     fontSize: 14,
5058     fontWeight: '600',
5059     fontFamily: 'Courier',
5060     marginBottom: 8,
5061   },
5062   input: {
5063     borderWidth: 1,
5064     borderRadius: 10,
5065     paddingHorizontal: 16,
5066     paddingVertical: 14,
5067     fontSize: 16,
5068     fontFamily: 'Courier',
5069   },
5070   multiline: {
5071     minHeight: 100,
5072     textAlignVertical: 'top',
5073   },
5074   errorText: {
5075     fontSize: 12,
5076     marginTop: 6,
5077     fontFamily: 'Courier',
5078   },
5079 });

```

```

5080
5081 export default PixelInput;
5082
    === mobile/src/components/ui/PixelLoading.tsx ===
5083 import React, { useRef, useEffect } from 'react';
5084 import { View, Text, StyleSheet, Animated, Easing } from 'react-native';
5085 import { PIXEL_COLORS, SPACING } from '../utils/constants';
5086
5087 interface PixelLoadingProps {
5088   text?: string;
5089   size?: 'small' | 'medium' | 'large';
5090   color?: string;
5091 }
5092
5093 const PixelLoading: React.FC<PixelLoadingProps> = ({
5094   text = '加载中...',
5095   size = 'medium',
5096   color = PIXEL_COLORS.PIXEL_CYAN,
5097 }) => {
5098   const dots = [useRef(new Animated.Value(0)).current, useRef(new Animated.Value(0)).current, useRef(new Anima
5099
5100   useEffect(() => {
5101     const animations = dots.map((dot, index) =>
5102       Animated.loop(
5103         Animated.sequence([
5104           Animated.delay(index * 200),
5105           Animated.timing(dot, {
5106             toValue: 1,
5107             duration: 600,
5108             easing: Easing.inOut(Easing.quad),
5109             useNativeDriver: true,
5110           }),
5111           Animated.timing(dot, {
5112             toValue: 0,
5113             duration: 600,
5114             easing: Easing.inOut(Easing.quad),
5115             useNativeDriver: true,
5116           }),
5117         ])
5118       )
5119     );
5120
5121     animations.forEach(anim => anim.start());
5122
5123     return () => {
5124       animations.forEach(anim => anim.stop());
5125     };
5126   }, []);
5127
5128   const getDotSize = () => {

```

```

5129     switch (size) {
5130         case 'small':
5131             return 12;
5132         case 'large':
5133             return 20;
5134         default:
5135             return 16;
5136     }
5137 };
5138
5139 const dotSize = getDotSize();
5140
5141 return (
5142     <View style={styles.container}>
5143     <View style={styles.dotsContainer}>
5144         {dots.map((dot, index) => (
5145             <Animated.View
5146                 key={index}
5147                 style={[
5148                     styles.dot,
5149                     {
5150                         width: dotSize,
5151                         height: dotSize,
5152                         backgroundColor: color,
5153                         transform: [
5154                             {
5155                                 scale: dot.interpolate({
5156                                     inputRange: [0, 1],
5157                                     outputRange: [1, 1.3],
5158                                 }),
5159                             },
5160                         ],
5161                         opacity: dot.interpolate({
5162                             inputRange: [0, 1],
5163                             outputRange: [0.5, 1],
5164                         }),
5165                     },
5166                 ]}
5167             />
5168         ))}
5169     </View>
5170     {text ? <Text style={[styles.text, { color }]}>{text}</Text> : null}
5171 </View>
5172 );
5173 };
5174
5175 const styles = StyleSheet.create({
5176     container: {
5177         alignItems: 'center',
5178         justifyContent: 'center',

```



```

5179 padding: SPACING.LARGE,
5180 },
5181 dotsContainer: {
5182   flexDirection: 'row',
5183   alignItems: 'center',
5184   justifyContent: 'center',
5185   gap: 8,
5186 },
5187 dot: {
5188   marginHorizontal: 4,
5189 },
5190 text: {
5191   marginTop: SPACING.MEDIUM,
5192   fontSize: 14,
5193   fontWeight: '600',
5194   letterSpacing: 1,
5195 },
5196 });
5197
5198 export default PixelLoading;
5199
=== mobile/src/components/ui/index.ts ===
5200 export { default as Button } from './Button';
5201 export { default as Input } from './Input';
5202 export { default as Loading } from './Loading';
5203 export { default as CuteButton } from './CuteButton';
5204 export { default as CuteCard } from './CuteCard';
5205 export { default as CuteInput } from './CuteInput';
5206 export { default as PixelButton } from './PixelButton';
5207 export { default as PixelCard } from './PixelCard';
5208 export { default as PixelInput } from './PixelInput';
5209 export { default as PixelLoading } from './PixelLoading';
=== mobile/src/contexts/ThemeContext.tsx ===
5210 import React, { createContext, useContext, useState, useEffect, useCallback } from 'react';
5211 import AsyncStorage from '@react-native-async-storage/async-storage';
5212
5213 export interface ThemeColors {
5214   // 页面基础
5215   background: string;
5216   backgroundDark: string;
5217   canvas: string;
5218   surface: string;
5219   surfaceLight: string;
5220
5221   // 文字
5222   textPrimary: string;
5223   textSecondary: string;
5224   textTertiary: string;
5225   textInverse: string;
5226

```

```

5227 // 分类卡片背景
5228 cardAi: string;
5229 cardLife: string;
5230 cardLang: string;
5231 cardAdd: string;
5232 card: string; // 兼容旧接口，通用卡片
5233
5234 // 边框
5235 border: string;
5236 borderLight: string;
5237 borderDark: string;
5238 hairline: string;
5239 cardBorder: string;
5240
5241 // 进度条
5242 progressFill: string;
5243 progressFillAi: string;
5244 progressFillLife: string;
5245 progressFillLang: string;
5246 progressTrack: string;
5247
5248 // 徽章
5249 badgeBg: string;
5250 badgeText: string;
5251 badgeBorder: string;
5252 badgeBorderAi: string;
5253 badgeBorderLife: string;
5254 badgeBorderLang: string;
5255
5256 // 提示
5257 tipBg: string;
5258
5259 // 强调色
5260 primary: string;
5261 onPrimary: string;
5262 accentGreen: string;
5263 accentOrange: string;
5264 brandBlue: string;
5265 brandPurple: string;
5266 brandCoral: string;
5267 successText: string;
5268 error: string;
5269 warning: string;
5270
5271 // 装饰
5272 decorCircle: string;
5273
5274 // Pro / 付费模式
5275 pro: string;
5276 proBg: string;

```

```
5277 proBorder: string;
5278 proGradientStart: string;
5279 proGradientEnd: string;
5280
5281 // 中性色
5282 ink: string;
5283 charcoal: string;
5284 muted: string;
5285 steel: string;
5286 stone: string;
5287
5288 // UI 组件
5289 inputBg: string;
5290 inputBorder: string;
5291 tabBar: string;
5292 tabBarBorder: string;
5293 headerBg: string;
5294 modalOverlay: string;
5295 shadow: string;
5296
5297 // 技能树节点
5298 nodePending: string;
5299 nodeDoing: string;
5300 nodeDone: string;
5301
5302 // 霓虹 (保留兼容)
5303 neon: string;
5304 glow: string;
5305 highlight: string;
5306 glitchPrimary: string;
5307 glitchSecondary: string;
5308
5309 // 旧接口兼容
5310 success: string;
5311 orange: string;
5312 purple: string;
5313 pink: string;
5314 surfaceSoft: string;
5315 primarySoft: string;
5316 onDark: string;
5317 footerBg: string;
5318 hairlineSoft: string;
5319 successBg: string;
5320 brandCyan: string;
5321 brandBlueMid: string;
5322 brandBlueDeep: string;
5323 brandBlue700: string;
5324 brandBlue200: string;
5325 brandMagenta: string;
5326 inkStrong: string;
```

```

5327 }
5328
5329 // ===== 深色模式 — 暖碳黑 · 陶土金主色 =====
5330 export const darkTheme: ThemeColors = {
5331   // 页面基础
5332   background: '#171717',
5333   backgroundDark: '#0F0F0F',
5334   canvas: '#1E1E1E',
5335   surface: '#292929',
5336   surfaceLight: '#363636',
5337
5338   // 文字（暖调，在碳灰底上清晰柔和）
5339   textPrimary: '#EAE0D8',
5340   textSecondary: '#9A8E84',
5341   textTertiary: '#6B6158',
5342   textInverse: 'FFFFFF',
5343
5344   // 分类卡片背景
5345   cardAi: '#2A2420',    // 暗暖褐 - 专业技能
5346   cardLife: '#202A28',  // 暗灰绿 - 生活技能
5347   cardLang: '#2B2422',  // 暗褐 - 语言学习
5348   cardAdd: '#222222',   // 添加模块
5349   card: '#292929',      // 通用卡片
5350
5351   // 边框
5352   border: '#3A3A3A',
5353   borderLight: '#2E2E2E',
5354   borderDark: '#484848',
5355   hairline: '#2E2E2E',
5356   cardBorder: '#3A3A3A',
5357
5358   // 进度条
5359   progressFill: '#7AB07A',
5360   progressFillAi: '#D4A574', // 陶土金 - 专业技能
5361   progressFillLife: '#7AB07A', // 柔绿 - 生活技能
5362   progressFillLang: '#B892C8', // 淡紫 - 语言学习
5363   progressTrack: '#3A3A3A',
5364
5365   // 徽章
5366   badgeBg: '#292929',
5367   badgeText: '#B0A498',
5368   badgeBorder: '#484848',
5369   badgeBorderAi: '#D4A574',
5370   badgeBorderLife: '#7AB07A',
5371   badgeBorderLang: '#B892C8',
5372
5373   // 提示
5374   tipBg: '#292929',
5375
5376   // 强调色

```

```
5377 primary: '#D4A574',
5378 onPrimary: '#1A1510',
5379 accentGreen: '#7AB0A0',
5380 accentOrange: '#C89070',
5381 brandBlue: '#6B9AAA',
5382 brandPurple: '#B892C8',
5383 brandCoral: '#D4A574',
5384 successText: '#7AB07A',
5385 error: '#C86A6A',
5386 warning: '#C89070',
5387
5388 // 装饰
5389 decorCircle: '#363636',
5390
5391 // Pro / 付费模式
5392 pro: '#D4A574',
5393 proBg: 'rgba(212,165,116,0.10)',
5394 proBorder: 'rgba(212,165,116,0.25)',
5395 proGradientStart: '#D4A574',
5396 proGradientEnd: '#C89070',
5397
5398 // 中性色
5399 ink: '#EAE0D8',
5400 charcoal: '#9A8E84',
5401 muted: '#6B6158',
5402 steel: '#5A5048',
5403 stone: '#6B6158',
5404
5405 // UI 组件
5406 inputBg: '#292929',
5407 inputBorder: '#3A3A3A',
5408 tabBar: 'rgba(23,23,23,0.95)',
5409 tabBarBorder: '#2E2E2E',
5410 headerBg: '#171717',
5411 modalOverlay: 'rgba(0,0,0,0.75)',
5412 shadow: '#000000',
5413
5414 // 技能树节点
5415 nodePending: '#292929',
5416 nodeDoing: '#D4A574',
5417 nodeDone: '#7AB07A',
5418
5419 // 霓虹 (保留兼容)
5420 neon: '#7AB0A0',
5421 glow: '#7AB07A',
5422 highlight: '#D4A574',
5423 glitchPrimary: '#7AB0A0',
5424 glitchSecondary: '#B892C8',
5425
5426 // 旧接口兼容
```

```

5427 success: '#7AB07A',
5428 orange: '#C89070',
5429 purple: '#B892C8',
5430 pink: '#C8A0B0',
5431 surfaceSoft: '#222222',
5432 primarySoft: '#363636',
5433 onDark: 'FFFFFF',
5434 footerBg: '#171717',
5435 hairlineSoft: '#2A2A2A',
5436 successBg: 'rgba(122,176,122,0.12)',
5437 brandCyan: '#7AB0A0',
5438 brandBlueMid: '#6B9AAA',
5439 brandBlueDeep: '#5A8A9A',
5440 brandBlue700: '#4A7A8A',
5441 brandBlue200: '#C0D8E0',
5442 brandMagenta: '#C8A0B0',
5443 inkStrong: '#F5F0EA',
5444 };
5445
5446 // ===== 浅色模式 — 陶土色 · 温暖米白底 =====
5447 export const lightTheme: ThemeColors = {
5448   // 页面基础
5449   background: '#F5EDE0',
5450   backgroundDark: '#E8E0D4',
5451   canvas: 'FFFFFF',
5452   surface: 'FFFFFF',
5453   surfaceLight: '#F8F2EB',
5454
5455   // 文字（深褐，在暖米白底上清晰）
5456   textPrimary: '#3D3229',
5457   textSecondary: '#8B7D72',
5458   textTertiary: '#B0A498',
5459   textInverse: 'FFFFFF',
5460
5461   // 分类卡片背景
5462   cardAi: '#EBE5DC',    // 暖灰褐 - 专业技能
5463   cardLife: '#E2EBE3',  // 淡灰绿 - 生活技能
5464   cardLang: '#EDE3D8',  // 暖杏色 - 语言学习
5465   cardAdd: '#F8F2EB',   // 添加模块
5466   card: 'FFFFFF',       // 通用卡片
5467
5468   // 边框
5469   border: '#DED3C8',
5470   borderLight: '#EAE0D6',
5471   borderDark: '#CDC0B2',
5472   hairline: '#EAE0D6',
5473   cardBorder: '#DED3C8',
5474
5475   // 进度条
5476   progressFill: '#7BA67B',

```

```

5477 progressFillAi: '#C77D5A', // 陶土橙 - 专业技能
5478 progressFillLife: '#7BA67B', // 柔绿 - 生活技能
5479 progressFillLang: '#A08B8B', // 灰紫 - 语言学习
5480 progressTrack: '#E0D6CA',
5481
5482 // 徽章
5483 badgeBg: 'FFFFFF',
5484 badgeText: '#5A4A3E',
5485 badgeBorder: '#CDC0B2',
5486 badgeBorderAi: '#C77D5A',
5487 badgeBorderLife: '#7BA67B',
5488 badgeBorderLang: '#A08B8B',
5489
5490 // 提示
5491 tipBg: 'FFFFFF',
5492
5493 // 强调色
5494 primary: '#C77D5A',
5495 onPrimary: 'FFFFFF',
5496 accentGreen: '#5A8F7C',
5497 accentOrange: '#D4A574',
5498 brandBlue: '#7A9EB5',
5499 brandPurple: '#A08B8B',
5500 brandCoral: '#C77D5A',
5501 successText: '#5A8F7C',
5502 error: '#D45656',
5503 warning: '#D4A574',
5504
5505 // 装饰
5506 decorCircle: '#CDC0B2',
5507
5508 // Pro / 付费模式
5509 pro: '#D97706',
5510 proBg: 'rgba(217,119,6,0.08)',
5511 proBorder: 'rgba(217,119,6,0.25)',
5512 proGradientStart: '#D97706',
5513 proGradientEnd: '#B45309',
5514
5515 // 中性色
5516 ink: '#3D3229',
5517 charcoal: '#5A4A3E',
5518 muted: '#B0A498',
5519 steel: '#8B7D72',
5520 stone: '#B0A498',
5521
5522 // UI 组件
5523 inputBg: 'FFFFFF',
5524 inputBorder: '#DED3C8',
5525 tabBar: 'rgba(245,237,224,0.92)',
5526 tabBarBorder: '#EAE0D6',

```

```

5527 headerBg: '#F5EDE0',
5528 modalOverlay: 'rgba(0,0,0,0.25)',
5529 shadow: 'rgba(61,50,41,0.08)',
5530
5531 // 技能树节点
5532 nodePending: '#F2EDE6',
5533 nodeDoing: '#C77D5A',
5534 nodeDone: '#7BA67B',
5535
5536 // 霓虹 (保留兼容)
5537 neon: '#C77D5A',
5538 glow: '#5A8F7C',
5539 highlight: '#D4A574',
5540 glitchPrimary: '#5A8F7C',
5541 glitchSecondary: '#A08B8B',
5542
5543 // 旧接口兼容
5544 success: '#7BA67B',
5545 orange: '#D4A574',
5546 purple: '#A08B8B',
5547 pink: '#C89090',
5548 surfaceSoft: '#F2EDE6',
5549 primarySoft: '#F5EDE0',
5550 onDark: 'FFFFFF',
5551 footerBg: '#3D3229',
5552 hairlineSoft: 'EEE5DC',
5553 successBg: '#E8FFEA',
5554 brandCyan: '#7A9EB5',
5555 brandBlueMid: '#7A9EB5',
5556 brandBlueDeep: '#6A8EA5',
5557 brandBlue700: '#5A7E95',
5558 brandBlue200: '#D6E4ED',
5559 brandMagenta: '#C89090',
5560 inkStrong: '#1A1510',
5561 };
5562
5563 interface ThemeContextType {
5564   isDark: boolean;
5565   colors: ThemeColors;
5566   toggleTheme: () => void;
5567 }
5568
5569 const ThemeContext = createContext<ThemeContextType>({
5570   isDark: true,
5571   colors: darkTheme,
5572   toggleTheme: () => {},
5573 });
5574
5575 const THEME_STORAGE_KEY = 'app_theme';
5576

```



```

5577 export const ThemeProvider: React.FC<{ children: React.ReactNode }> = ({ children }) => {
5578   const [isDark, setIsDark] = useState(true);
5579
5580   useEffect(() => {
5581     const loadTheme = async () => {
5582       try {
5583         const stored = await AsyncStorage.getItem(THEME_STORAGE_KEY);
5584         if (stored !== null) {
5585           setIsDark(stored === 'dark');
5586         }
5587       } catch (e) {
5588         console.error('[Theme] 加载主题失败:', e);
5589       }
5590     };
5591     loadTheme();
5592   }, []);
5593
5594   const toggleTheme = useCallback(async () => {
5595     const next = !isDark;
5596     setIsDark(next);
5597     try {
5598       await AsyncStorage.setItem(THEME_STORAGE_KEY, next ? 'dark' : 'light');
5599     } catch (e) {
5600       console.error('[Theme] 保存主题失败:', e);
5601     }
5602   }, [isDark]);
5603
5604   const colors = isDark ? darkTheme : lightTheme;
5605
5606   return (
5607     <ThemeContext.Provider value={{ isDark, colors, toggleTheme }}>
5608       {children}
5609     </ThemeContext.Provider>
5610   );
5611 };
5612
5613 export const useTheme = () => useContext(ThemeContext);
5614
5615 export default ThemeContext;
5616
5617 === mobile/src/data/index.ts ===
5618 export * from './mockData';
5619 export * from './skillTrees';
5620
5621 === mobile/src/data/mockData.ts ===
5622 // 临时注释以避免类型错误
5623 // 这些类型定义将在后续补充
5624
5625 export const mockUser = {} as any;
5626 export const mockSkillTrees = [] as any[];

```

```

5625 export const studyStats = [] as any[];
5626 export const skillRadarData = [] as any[];
5627 export const monsters = [] as any[];
5628
    === mobile/src/data/skillTrees.ts ===
5629 import { SkillTree, StageType } from '../types/skill';
5630
5631 const PLATFORM_MAP = {
5632   'B站': 'bilibili',
5633   '小红书': 'xiaohongshu',
5634   '中国大学MOOC': 'mooc',
5635 } as const;
5636
5637 const STAGE_MAP: Record<StageType, string> = {
5638   beginner: '初级阶段',
5639   intermediate: '中级阶段',
5640   advanced: '高级阶段',
5641 };
5642
5643 export const aiProductManagerTree: SkillTree = {
5644   id: 'ai-product-manager',
5645   domain: 'ai-product-manager',
5646   title: 'AI产品经理',
5647   description: '掌握AI产品设计与落地的核心能力',
5648   totalDuration: 60,
5649   learningMethod: '1. 先观看MOOC课程建立基础认知框架；2. 结合B站视频深入理解大模型原理；3. 通过小红书案例实践产品思考；4. 每周完成一个Prompt工程练习',
5650   learningGoal: '成为具备AI产品设计能力的产品经理，能够独立完成AI产品需求分析、PRD撰写、项目推进，熟悉AI技术边界与商业落地路径',
5651   frameworkExplanation: '采用"认知-能力-实战"三层递进结构：初级阶段建立AI技术认知基础，中级阶段培养产品核心能力，高级阶段聚焦实战落地与商业合规，符合产品经理成长的自',
5652   stages: [
5653     {
5654       id: 'beginner',
5655       name: '基础认知打底',
5656       duration: 20,
5657       nodes: [
5658         {
5659           id: 'ai-basic-concepts',
5660           name: '人工智能基础概念通识',
5661           description: '系统学习AI基础概念与发展历程',
5662           stage: 'beginner',
5663           platform: PLATFORM_MAP['中国大学MOOC'],
5664           url: 'https://www.icourse163.org/course/HIT-1001997005',
5665           duration: 7,
5666         },
5667         {
5668           id: 'llm-fundamentals',
5669           name: '大模型LLM基础原理入门',
5670           description: '理解大语言模型的核心原理',
5671           stage: 'beginner',
5672           platform: PLATFORM_MAP['B站'],
5673           url: 'https://www.bilibili.com/video/BV1pu411o7BE',

```

```

5674     duration: 7,
5675 },
5676 {
5677     id: 'ai-terminology',
5678     name: 'AI行业名词/API/微调通俗科普',
5679     description: '掌握AI领域核心术语与API应用',
5680     stage: 'beginner',
5681     platform: PLATFORM_MAP['B站'],
5682     url: 'https://www.bilibili.com/video/BV1m84y1w7aG',
5683     duration: 6,
5684 },
5685 ],
5686 },
5687 {
5688     id: 'intermediate',
5689     name: '产品核心能力',
5690     duration: 20,
5691     nodes: [
5692     {
5693         id: 'product-analysis',
5694         name: '产品需求分析&用户思维',
5695         description: '培养产品思维与用户需求分析能力',
5696         stage: 'intermediate',
5697         platform: PLATFORM_MAP['B站'],
5698         url: 'https://www.bilibili.com/video/BV1sb411i7aG',
5699         duration: 7,
5700     },
5701     {
5702         id: 'prompt-engineering',
5703         name: 'Prompt提示工程实战',
5704         description: '掌握提示词设计与优化技巧',
5705         stage: 'intermediate',
5706         platform: PLATFORM_MAP['中国大学MOOC'],
5707         url: 'https://www.icourse163.org/course/BUPT-1002564002',
5708         duration: 7,
5709     },
5710     {
5711         id: 'rag-explained',
5712         name: 'RAG检索增强通俗拆解',
5713         description: '理解RAG技术原理与应用场景',
5714         stage: 'intermediate',
5715         platform: PLATFORM_MAP['B站'],
5716         url: 'https://www.bilibili.com/video/BV1cW4y1u7fH',
5717         duration: 6,
5718     },
5719     ],
5720 },
5721 {
5722     id: 'advanced',
5723     name: '实战落地+商业合规',

```

```

5724     duration: 20,
5725     nodes: [
5726       {
5727         id: 'ai-prd-document',
5728         name: 'AI产品PRD文档&项目协作',
5729         description: '学习AI产品文档撰写与项目管理',
5730         stage: 'advanced',
5731         platform: PLATFORM_MAP['B站'],
5732         url: 'https://www.bilibili.com/video/BV1jt411o7cQ',
5733         duration: 7,
5734       },
5735       {
5736         id: 'ai-commercialization',
5737         name: 'AI商业变现&行业落地案例',
5738         description: '探索AI商业化路径与成功案例',
5739         stage: 'advanced',
5740         platform: PLATFORM_MAP['B站'],
5741         url: 'https://www.bilibili.com/video/BV1kW411o7a9',
5742         duration: 7,
5743       },
5744       {
5745         id: 'ai-ethics-compliance',
5746         name: 'AI隐私合规&伦理规范',
5747         description: '了解AI伦理与数据隐私合规要求',
5748         stage: 'advanced',
5749         platform: PLATFORM_MAP['中国大学MOOC'],
5750         url: 'https://www.icourse163.org/course/SUFE-1003056001',
5751         duration: 6,
5752       },
5753     ],
5754   },
5755 ],
5756 };
5757
5758 export const personalFinanceTree: SkillTree = {
5759   id: 'personal-finance',
5760   domain: 'personal-finance',
5761   title: '理财进阶',
5762   description: '从零建立理财体系: 记账→安全配置→基金定投',
5763   totalDuration: 21,
5764   learningMethod: '1. MOOC《个人理财》1-2章打基础; 2. 开随手记App同步记账; 3. 支付宝买100元货币基金体验; 4. 读完《小狗钱钱》+《穷查理宝典》部分章节',
5765   learningGoal: '能独立做家庭财务盘点、完成保险+基金配置方案、开始指数基金定投, 形成长期理财习惯',
5766   frameworkExplanation: '一阶觉醒: 建立复利思维+开始记账 → 二阶锻造: 理解保险配置+货币基金 → 三阶出师: 指数定投实操+完成资产配置计划书',
5767
5768   stages: [
5769     {
5770       id: 'beginner',
5771       name: '理财思维启蒙',
5772       duration: 20,
5773       nodes: [

```

```

5774 {
5775     id: 'finance-basics',
5776     name: '理财底层逻辑+复利思维',
5777     description: '建立正确的理财观念与复利认知',
5778     stage: 'beginner',
5779     platform: PLATFORM_MAP['B站'],
5780     url: 'https://www.bilibili.com/video/BV1eK411578Q',
5781     duration: 7,
5782 },
5783 {
5784     id: 'budget-planning',
5785     name: '个人财务记账+收支规划',
5786     description: '掌握个人财务管理基础方法',
5787     stage: 'beginner',
5788     platform: PLATFORM_MAP['中国大学MOOC'],
5789     url: 'https://www.icourse163.org/course/NCWU-1002705001',
5790     duration: 7,
5791 },
5792 {
5793     id: 'financial-tools',
5794     name: '基础金融工具科普',
5795     description: '了解常见金融产品与工具',
5796     stage: 'beginner',
5797     platform: PLATFORM_MAP['B站'],
5798     url: 'https://www.bilibili.com/video/BV1sb411o7aS',
5799     duration: 6,
5800 },
5801 ],
5802 },
5803 {
5804     id: 'intermediate',
5805     name: '安全资产配置',
5806     duration: 20,
5807     nodes: [
5808         {
5809             id: 'insurance-guide',
5810             name: '保险配置避坑指南',
5811             description: '科学配置保险产品',
5812             stage: 'intermediate',
5813             platform: PLATFORM_MAP['B站'],
5814             url: 'https://www.bilibili.com/video/BV1Vt411o72w',
5815             duration: 7,
5816         },
5817         {
5818             id: 'low-risk-investment',
5819             name: '国债/货币基金/债券基金入门',
5820             description: '学习低风险投资产品',
5821             stage: 'intermediate',
5822             platform: PLATFORM_MAP['中国大学MOOC'],
5823             url: 'https://www.icourse163.org/course/JLU-1001777003',

```

```

5824     duration: 7,
5825 },
5826 {
5827     id: 'emergency-fund',
5828     name: '家庭应急金+低风险组合',
5829     description: '建立应急储备与安全资产组合',
5830     stage: 'intermediate',
5831     platform: PLATFORM_MAP['B站'],
5832     url: 'https://www.bilibili.com/video/BV1Mt411o7aW',
5833     duration: 6,
5834 },
5835 ],
5836 },
5837 {
5838     id: 'advanced',
5839     name: '基金进阶+长期财富规划',
5840     duration: 20,
5841     nodes: [
5842     {
5843         id: 'index-fund-investing',
5844         name: '指数基金定投+止盈止损',
5845         description: '掌握指数基金投资策略',
5846         stage: 'advanced',
5847         platform: PLATFORM_MAP['B站'],
5848         url: 'https://www.bilibili.com/video/BV1At411o74U',
5849         duration: 7,
5850     },
5851     {
5852         id: 'asset-allocation',
5853         name: '资产配置模型+基金组合搭建',
5854         description: '学习科学的资产配置方法',
5855         stage: 'advanced',
5856         platform: PLATFORM_MAP['中国大学MOOC'],
5857         url: 'https://www.icourse163.org/course/HUBEIU-1003206001',
5858         duration: 7,
5859     },
5860     {
5861         id: 'advanced-tools',
5862         name: '可转债/REITs高阶工具+投资心态',
5863         description: '掌握进阶投资工具与心态管理',
5864         stage: 'advanced',
5865         platform: PLATFORM_MAP['B站'],
5866         url: 'https://www.bilibili.com/video/BV1Rt411o78X',
5867         duration: 6,
5868     },
5869     ],
5870     },
5871 ],
5872 };
5873

```

```

5874 export const englishCommunicationTree: SkillTree = {
5875   id: 'english-communication',
5876   domain: 'english-communication',
5877   title: '英语学习',
5878   description: '开口→听懂→实战，三阶告别哑巴英语',
5879   totalDuration: 21,
5880   learningMethod: '1. 每天B站音标跟读15min; 2. BBC 6 Minute English精听每周3篇; 3. English with Lucy学日常表达; 4. 最终用HelloTalk找老外',
5881   learningGoal: '能开口做英文自我介绍、听懂慢速英语播客、用英语完成日常购物/点餐/电话沟通，最终用HelloTalk和老外聊上半小时',
5882   frameworkExplanation: '一阶觉醒：纠正发音+听慢速→二阶锻造：精听BBC+模仿母语者日常对话→三阶出师：职场英语+找老外实战',
5883
5884   stages: [
5885     {
5886       id: 'beginner',
5887       name: '音标+词汇+基础语法',
5888       duration: 20,
5889       nodes: [
5890         {
5891           id: 'phonetics',
5892           name: '美式音标发音纠正',
5893           description: '掌握标准美式发音',
5894           stage: 'beginner',
5895           platform: PLATFORM_MAP['B站'],
5896           url: 'https://www.bilibili.com/video/BV16s411o79B',
5897           duration: 7,
5898         },
5899         {
5900           id: 'english-grammar',
5901           name: '零基础英语语法系统入门',
5902           description: '系统学习英语基础语法',
5903           stage: 'beginner',
5904           platform: PLATFORM_MAP['中国大学MOOC'],
5905           url: 'https://www.icourse163.org/course/CPU-1002345001',
5906           duration: 7,
5907         },
5908         {
5909           id: 'core-vocabulary',
5910           name: '口语高频2000核心词汇',
5911           description: '掌握日常口语必备词汇',
5912           stage: 'beginner',
5913           platform: PLATFORM_MAP['B站'],
5914           url: 'https://www.bilibili.com/video/BV1Ds411o77k',
5915           duration: 6,
5916         },
5917       ],
5918     },
5919     {
5920       id: 'intermediate',
5921       name: '场景听力+日常口语',
5922       duration: 20,
5923       nodes: [

```

```

5924 {
5925   id: 'daily-conversation',
5926   name: '日常生活情景口语全套',
5927   description: '掌握日常场景对话能力',
5928   stage: 'intermediate',
5929   platform: PLATFORM_MAP['B站'],
5930   url: 'https://www.bilibili.com/video/BV1ms411o78X',
5931   duration: 7,
5932 },
5933 {
5934   id: 'listening-training',
5935   name: '慢速→常速听力磨耳朵训练',
5936   description: '循序渐进提升听力水平',
5937   stage: 'intermediate',
5938   platform: PLATFORM_MAP['中国大学MOOC'],
5939   url: 'https://www.icourse163.org/course/SDU-1001888002',
5940   duration: 7,
5941 },
5942 {
5943   id: 'pronunciation-improvement',
5944   name: '连读弱读摆脱中式口音',
5945   description: '改善发音, 摆脱口音问题',
5946   stage: 'intermediate',
5947   platform: PLATFORM_MAP['B站'],
5948   url: 'https://www.bilibili.com/video/BV1rs411o7aJ',
5949   duration: 6,
5950 },
5951 ],
5952 },
5953 {
5954   id: 'advanced',
5955   name: '职场口语+无障碍实战',
5956   duration: 20,
5957   nodes: [
5958     {
5959       id: 'business-english',
5960       name: '职场商务会议/面试口语',
5961       description: '掌握职场英语沟通技巧',
5962       stage: 'advanced',
5963       platform: PLATFORM_MAP['B站'],
5964       url: 'https://www.bilibili.com/video/BV1ts411o79b',
5965       duration: 7,
5966     },
5967     {
5968       id: 'cross-cultural',
5969       name: '跨文化交际+老外聊天逻辑',
5970       description: '理解西方文化与交流习惯',
5971       stage: 'advanced',
5972       platform: PLATFORM_MAP['中国大学MOOC'],
5973       url: 'https://www.icourse163.org/course/ZJUT-1002123001',

```



```

5974     duration: 7,
5975 },
5976 {
5977     id: 'fluent-speaking',
5978     name: '无字幕跟读+即兴自由表达',
5979     description: '实现流利口语与即兴表达',
5980     stage: 'advanced',
5981     platform: PLATFORM_MAP['B站'],
5982     url: 'https://www.bilibili.com/video/BV1ss411o79b',
5983     duration: 6,
5984 },
5985 ],
5986 ],
5987 ],
5988 };
5989
5990 export const programmingBasicsTree: SkillTree = {
5991     id: 'programming-basics',
5992     domain: 'programming-basics',
5993     title: '编程基础',
5994     description: 'Python入门→算法思维→项目实战，三阶通关编程',
5995     totalDuration: 21,
5996     learningMethod: '1. B站零基础Python视频跟练；2. MOOC《程序设计基础》系统学数据结构；3. LeetCode每周刷3道简单题；4. 最终独立完成一个命令行小项目',
5997     learningGoal: '能独立编写Python脚本解决实际问题、理解基础算法与数据结构、具备自学其他编程语言的能力',
5998     frameworkExplanation: '一阶：Python语法+环境配置+基础IO → 二阶：数据结构+算法入门+面向对象 → 三阶：项目实战+代码规范+Git协作',
5999     stages: [
6000     {
6001         id: 'beginner',
6002         name: 'Python语法入门',
6003         duration: 20,
6004         nodes: [
6005             { id: 'python-zero', name: '零基础Python快速入门', description: '从安装到写出第一个程序', stage: 'beginner', platform: P
6006             { id: 'cs-fundamentals', name: '计算机科学基础导论', description: '理解计算机工作原理', stage: 'beginner', platform: PLA
6007             { id: 'python-practice', name: 'Python基础编程练习30题', description: '通过习题巩固语法基础', stage: 'beginner', platfo
6008         ],
6009     },
6010     {
6011         id: 'intermediate',
6012         name: '数据结构与算法',
6013         duration: 20,
6014         nodes: [
6015             { id: 'data-structures', name: '数据结构入门：列表/栈/队列/树', description: '掌握基础数据结构', stage: 'intermediate', pla
6016             { id: 'algorithms-basic', name: '基础算法：排序/查找/递归', description: '学习经典算法思想', stage: 'intermediate', platf
6017             { id: 'oop-python', name: 'Python面向对象编程', description: '掌握类/继承/多态', stage: 'intermediate', platform: P
6018         ],
6019     },
6020     {
6021         id: 'advanced',
6022         name: '项目实战与工程化',
6023         duration: 20,

```

```

6024     nodes: [
6025         { id: 'project-crawler', name: '实战：Python爬虫+数据分析', description: '完成一个数据爬取分析项目', stage: 'advanced', pla
6026         { id: 'project-flask', name: '实战：Flask搭建简易Web应用', description: '用Python框架构建Web服务', stage: 'advanced',
6027         { id: 'git-github', name: 'Git版本控制+GitHub协作', description: '掌握团队协作开发工具', stage: 'advanced', platform:
6028     ],
6029 },
6030 ],
6031 };
6032
6033 export const financeBasicsTree: SkillTree = {
6034   id: 'finance-basics',
6035   domain: 'finance-basics',
6036   title: '理财入门',
6037   description: '记账→储蓄→基础投资，从零管好每一笔钱',
6038   totalDuration: 21,
6039   learningMethod: '1. 下载记账App坚持记一个月；2. 读《小狗钱钱》建立理财思维；3. 支付宝体验货币基金；4. 制定个人月度预算表',
6040   learningGoal: '养成记账习惯、建立3-6个月应急金、了解货币基金/定期存款等低风险产品、能制定个人预算并每月复盘',
6041   frameworkExplanation: '一阶：理财观念+记账实操 → 二阶：储蓄策略+防御性资产 → 三阶：指数基金入门+财务规划',
6042   stages: [
6043     {
6044       id: 'beginner',
6045       name: '理财启蒙+记账习惯',
6046       duration: 20,
6047       nodes: [
6048         { id: 'money-mindset', name: '《小狗钱钱》财商思维入门', description: '建立正确金钱观与理财心态', stage: 'beginner', platform:
6049         { id: 'accounting-101', name: '记账实操：收支分类+预算制定', description: '学会系统记账与预算管理', stage: 'beginner', platfor
6050         { id: 'consumer-psychology', name: '消费心理学：区分需要和想要', description: '避免冲动消费，理性购物', stage: 'beginner', pla
6051       ],
6052     },
6053     {
6054       id: 'intermediate',
6055       name: '储蓄策略+安全资产',
6056       duration: 20,
6057       nodes: [
6058         { id: 'emergency-fund-101', name: '应急金规划：3-6个月生活费', description: '建立人生第一笔安全垫', stage: 'intermediate',
6059         { id: 'saving-tools', name: '定期/大额存单/货币基金入门', description: '了解银行的存款类产品', stage: 'intermediate', platfo
6060         { id: 'credit-card', name: '信用卡正确使用+征信科普', description: '学会用信用卡而非被用', stage: 'intermediate', platform:
6061       ],
6062     },
6063     {
6064       id: 'advanced',
6065       name: '投资入门+财务规划',
6066       duration: 20,
6067       nodes: [
6068         { id: 'index-fund-101', name: '指数基金入门：定投策略实操', description: '开始你的第一笔基金定投', stage: 'advanced', platform
6069         { id: 'insurance-101', name: '保险入门：四大险种怎么买', description: '了解基础保险配置', stage: 'advanced', platform: PLA
6070         { id: 'financial-plan', name: '个人年度财务规划+复盘模板', description: '制定并执行个人理财计划', stage: 'advanced', platform
6071       ],
6072     },
6073   ],

```

```

6074  });
6075
6076  export const cetExamTree: SkillTree = {
6077    id: 'cet-exam',
6078    domain: 'cet-exam',
6079    title: '四六级备考',
6080    description: '单词→真题→技巧, 高效通关四六级',
6081    totalDuration: 21,
6082    learningMethod: '1. 每天墨墨背单词打卡50个高频词; 2. 华研外语真题每周2套精做; 3. B站听力真题精听磨耳朵; 4. 考前一周背诵作文万能模板',
6083    learningGoal: '四级425+或六级425+通过, 掌握考试核心词汇、听力常考场景、阅读快速定位技巧、作文高分模板',
6084    frameworkExplanation: '一阶: 高频词汇+语法补漏 → 二阶: 分项突破听力/阅读/翻译 → 三阶: 真题模考+作文模板背诵',
6085    stages: [
6086      {
6087        id: 'beginner',
6088        name: '词汇积累+语法补漏',
6089        duration: 20,
6090        nodes: [
6091          { id: 'cet-vocabulary', name: '四六级高频核心词汇突破', description: '掌握考试必考高频词', stage: 'beginner', platform: PL
6092          { id: 'english-grammar-cet', name: '四六级语法考点速通', description: '快速过一遍常考语法点', stage: 'beginner', platform
6093          { id: 'reading-skills', name: '长篇阅读快速定位技巧', description: '10分钟做完匹配题', stage: 'beginner', platform: PLA
6094        ],
6095      },
6096      {
6097        id: 'intermediate',
6098        name: '分项突破听力+翻译',
6099        duration: 20,
6100        nodes: [
6101          { id: 'listening-cet', name: '四六级听力真题精听训练', description: '系统提升听力分数', stage: 'intermediate', platform:
6102          { id: 'translation-cet', name: '汉译英翻译模板+高频句型', description: '掌握翻译得分套路', stage: 'intermediate', platfor
6103          { id: 'writing-template', name: '作文万能模板+高分替换词', description: '背诵即可上考场', stage: 'intermediate', platfor
6104        ],
6105      },
6106      {
6107        id: 'advanced',
6108        name: '真题模考+考前冲刺',
6109        duration: 20,
6110        nodes: [
6111          { id: 'mock-exam', name: '四六级真题全真模考', description: '限时模拟真实考试', stage: 'advanced', platform: PLATFORM_M
6112          { id: 'final-review', name: '考前7天冲刺计划', description: '临门一脚拿下425+', stage: 'advanced', platform: PLATFO
6113          { id: 'error-review', name: '错题回顾+高频易错点复盘', description: '避免考场重复踩坑', stage: 'advanced', platform: PLAT
6114        ],
6115      },
6116    ],
6117  });
6118
6119  export const skillTrees: SkillTree[] = [
6120    aiProductManagerTree,
6121    programmingBasicsTree,
6122    personalFinanceTree,
6123    financeBasicsTree,

```

```

6124   englishCommunicationTree,
6125   cetExamTree,
6126 ];
6127
6128 export const getSkillTreeByDomain = (domain: string): SkillTree | undefined => {
6129   return skillTrees.find(tree => tree.domain === domain);
6130 };
6131
6132 export const getAllSkillTreeDomains = (): { id: string; title: string; description: string }[] => {
6133   return skillTrees.map(tree => ({
6134     id: tree.domain,
6135     title: tree.title,
6136     description: tree.description,
6137   }));
6138 };
6139
6140 export const STAGE_NAME_MAP = STAGE_MAP;
6141 export const PLATFORM_NAME_MAP: Record<string, string> = {
6142   bilibili: 'B站',
6143   xiaohongshu: '小红书',
6144   mooc: '中国大学MOOC',
6145 };
6146
6147 === mobile/src/hooks/index.ts ===
6148 export { useSkillTree } from './useSkillTree';
6149 export { useSearch } from './useSearch';
6150 export { useStatistics } from './useStatistics';
6151 export { useKeyboardPositioning } from './useKeyboardPositioning';
6152
6153 === mobile/src/hooks/useKeyboardPositioning.ts ===
6154 import { useState, useEffect, useRef } from 'react';
6155 import { Keyboard, Platform, Dimensions, LayoutAnimation, Animated } from 'react-native';
6156
6157 /**
6158  * 智能键盘定位 Hook
6159  * 实现点击输入框时，输入框自动上升到屏幕中心位置
6160  */
6161 export const useKeyboardPositioning = () => {
6162   const [keyboardHeight, setKeyboardHeight] = useState(0);
6163   const [activeInputY, setActiveInputY] = useState(0);
6164   const translateY = useRef(new Animated.Value(0)).current;
6165
6166   const screenHeight = Dimensions.get('window').height;
6167
6168   // 计算输入框需要移动的偏移量
6169   const calculateInputOffset = (keyboardH: number, inputY: number): number => {
6170     const visibleScreenHeight = screenHeight - keyboardH;
6171     const inputHeight = 56; // 标准输入框高度
6172     const keyboardOffset = 40; // 键盘上方保留1cm左右的间距 (约40px)
6173     const targetY = visibleScreenHeight - inputHeight - keyboardOffset;
6174
6175     // 如果输入框已经在目标位置之上，不需要移动

```

```

6172   if (inputY <= targetY) {
6173     return 0;
6174   }
6175
6176   return targetY - inputY;
6177 };
6178
6179 // 动画移动输入框
6180 const animateInput = (offset: number) => {
6181   Animated.timing(translateY, {
6182     toValue: offset,
6183     duration: 300,
6184     useNativeDriver: true,
6185   }).start();
6186 };
6187
6188 useEffect(() => {
6189   const keyboardDidShowListener = Keyboard.addListener(
6190     Platform.OS === 'ios' ? 'keyboardWillShow' : 'keyboardDidShow',
6191     (e) => {
6192       const keyboardH = e.endCoordinates.height;
6193       setKeyboardHeight(keyboardH);
6194
6195       // 计算输入框偏移量
6196       const offset = calculateInputOffset(keyboardH, activeInputY);
6197       animateInput(offset);
6198     }
6199   );
6200
6201   const keyboardDidHideListener = Keyboard.addListener(
6202     Platform.OS === 'ios' ? 'keyboardWillHide' : 'keyboardDidHide',
6203     () => {
6204       setKeyboardHeight(0);
6205       animateInput(0);
6206     }
6207   );
6208
6209   return () => {
6210     keyboardDidShowListener.remove();
6211     keyboardDidHideListener.remove();
6212   };
6213 }, [activeInputY]);
6214
6215 // 设置活动输入框位置
6216 const setInputPosition = (yPosition: number) => {
6217   setActiveInputY(yPosition);
6218 };
6219
6220 // 获取输入框定位样式
6221 const getInputStyle = () => ({

```

```

6222   transform: [{ translateY }]
6223   });
6224
6225   return {
6226     keyboardHeight,
6227     setInputPosition,
6228     getInputStyle,
6229   };
6230 };
6231
6232 export default useKeyboardPositioning;
=== mobile/src/hooks/useSearch.ts ===
6233 import { useState } from 'react';
6234 import { Alert } from 'react-native';
6235 import { skillService } from '../services/api';
6236
6237 export const useSearch = () => {
6238   const [loading, setLoading] = useState(false);
6239
6240   const searchDomains = async (keyword: string) => {
6241     console.log('[useSearch] 搜索领域 - 关键词:', keyword);
6242     if (!keyword.trim()) {
6243       Alert.alert('提示', '请输入搜索关键词');
6244       return null;
6245     }
6246
6247     setLoading(true);
6248     try {
6249       const result = await skillService.searchPopularDomains(keyword.trim());
6250       console.log('[useSearch] 搜索完成');
6251       return result;
6252     } catch (error) {
6253       console.error('[useSearch] 搜索失败:', error);
6254       Alert.alert('错误', '搜索失败, 请检查网络连接');
6255       return null;
6256     } finally {
6257       setLoading(false);
6258     }
6259   };
6260
6261   const getSkillTreeList = async (page: number = 1, limit: number = 5) => {
6262     console.log('[useSearch] 获取技能树列表 - 页码:', page);
6263     setLoading(true);
6264     try {
6265       const result = await skillService.getSkillTreeList(page, limit);
6266       console.log('[useSearch] 技能树列表获取成功');
6267       return result;
6268     } catch (error) {
6269       console.error('[useSearch] 获取技能树列表失败:', error);
6270       Alert.alert('错误', '获取技能树列表失败');

```

```

6271     return null;
6272   } finally {
6273     setLoading(false);
6274   }
6275 };
6276
6277 return {
6278   loading,
6279   searchDomains,
6280   getSkillTreeList
6281 };
6282 };

```

=== mobile/src/hooks/useSkillTree.ts ===

```

6283 import { useState } from 'react';
6284 import { Alert } from 'react-native';
6285 import { router } from 'expo-router';
6286 import { skillService } from '../services/api';
6287
6288 export const useSkillTree = () => {
6289   const [loading, setLoading] = useState(false);
6290
6291   const generateSkillTree = async (domain: string) => {
6292     console.log('[useSkillTree] 生成技能树 - 领域:', domain);
6293     if (!domain.trim()) {
6294       Alert.alert('错误', '请输入领域名称');
6295       return false;
6296     }
6297
6298     setLoading(true);
6299     try {
6300       await skillService.generateSkillTree({ domain: domain.trim() });
6301       console.log('[useSkillTree] 技能树生成成功');
6302       router.push({
6303         pathname: '/skill-tree',
6304         params: { domain: domain.trim() }
6305       });
6306       return true;
6307     } catch (error) {
6308       console.error('[useSkillTree] 生成技能树失败:', error);
6309       Alert.alert('错误', '生成技能树失败，请检查网络连接');
6310       return false;
6311     } finally {
6312       setLoading(false);
6313     }
6314   };
6315
6316   const getRecommendedPath = async (domain: string) => {
6317     console.log('[useSkillTree] 获取推荐路径 - 领域:', domain);
6318     if (!domain.trim()) {
6319       Alert.alert('错误', '请输入领域名称');

```

```

6320     return null;
6321   }
6322
6323   setLoading(true);
6324   try {
6325     const path = await skillService.getRecommendedPath(domain.trim());
6326     console.log('[useSkillTree] 推荐路径获取成功');
6327     return path;
6328   } catch (error) {
6329     console.error('[useSkillTree] 获取推荐路径失败:', error);
6330     Alert.alert('错误', '获取推荐路径失败');
6331     return null;
6332   } finally {
6333     setLoading(false);
6334   }
6335   };
6336
6337   return {
6338     loading,
6339     generateSkillTree,
6340     getRecommendedPath
6341   };
6342 };

```

=== mobile/src/hooks/useStatistics.ts ===

```

6343 import { useState } from 'react';
6344 import { Alert } from 'react-native';
6345 import { skillService } from '../services/api';
6346
6347 export const useStatistics = () => {
6348   const [loading, setLoading] = useState(false);
6349
6350   const getStatistics = async () => {
6351     console.log('[useStatistics] 获取统计信息');
6352     setLoading(true);
6353     try {
6354       const stats = await skillService.getStatistics();
6355       console.log('[useStatistics] 统计信息获取成功');
6356       return stats;
6357     } catch (error) {
6358       console.error('[useStatistics] 获取统计信息失败:', error);
6359       Alert.alert('错误', '获取统计信息失败');
6360       return null;
6361     } finally {
6362       setLoading(false);
6363     }
6364   };
6365
6366   const getLearningReport = async (userId: string = 'user1') => {
6367     console.log('[useStatistics] 获取学习报告 - 用户:', userId);
6368     setLoading(true);

```



```

6369     try {
6370         const report = await skillService.getUserLearningReport(userId);
6371         console.log('[useStatistics] 学习报告获取成功');
6372         return report;
6373     } catch (error) {
6374         console.error('[useStatistics] 获取学习报告失败:', error);
6375         Alert.alert('错误', '获取学习报告失败');
6376         return null;
6377     } finally {
6378         setLoading(false);
6379     }
6380 };
6381
6382 return {
6383     loading,
6384     getStatistics,
6385     getLearningReport
6386 };
6387 };

```

=== mobile/src/services/api.ts ===

```

6388 import axios from 'axios';
6389 import {
6390     SkillNode,
6391     SkillTreeRequest,
6392     CreateUserRequest,
6393     LoginRequest,
6394     UserResponse,
6395     AuthResponse,
6396     MonsterSetupRequest,
6397     MonsterResponse,
6398     MonsterChatRequest,
6399     MonsterChatResponse,
6400     MonsterMessagesResponse,
6401     // 新增: 笔记相关类型
6402     CreateNoteRequest,
6403     UpdateNoteRequest,
6404     Note,
6405     // 新增: 奖励相关类型
6406     CreateRewardRequest,
6407     ClaimRewardRequest,
6408     Reward,
6409     // 新增: 领域/学习域相关类型
6410     CreateDomainRequest,
6411     UpdateNodeProgressRequest,
6412     StartLearningRequest,
6413     FinishLearningRequest,
6414     Domain,
6415 } from '../types/skill';
6416 import { API_BASE_URL } from '../utils/constants';
6417

```

```

6418 const api = axios.create({
6419   baseURL: API_BASE_URL,
6420   timeout: 10000,
6421 });
6422
6423 // 添加请求拦截器，方便调试
6424 api.interceptors.request.use(
6425   (config) => {
6426     console.log('API Request: ${config.method?.toUpperCase()} ${config.url}');
6427     return config;
6428   },
6429   (error) => {
6430     console.error('API Request Error:', error);
6431     return Promise.reject(error);
6432   }
6433 );
6434
6435 // 添加响应拦截器，方便调试
6436 api.interceptors.response.use(
6437   (response) => {
6438     console.log('API Response: ${response.status} ${response.config.url}');
6439     return response;
6440   },
6441   (error) => {
6442     // Axios 无法建连时常无 response，仅 message 为 Network Error
6443     if (!error.response && error.message === 'Network Error') {
6444       return Promise.reject(
6445         new Error('无法连接服务器，请检查手机网络、服务器是否在线，以及 API 地址是否正确')
6446       );
6447     }
6448
6449     // 网络连接错误
6450     if (error.code === 'NETWORK_ERROR' || error.code === 'ECONNREFUSED') {
6451       console.error('网络连接失败，请检查网络连接和服务器状态');
6452       console.error('当前API地址:', API_BASE_URL);
6453       return Promise.reject(new Error('网络连接失败，请检查网络连接'));
6454     }
6455
6456     // DNS解析错误
6457     if (error.code === 'ENOTFOUND') {
6458       console.error('DNS解析失败，无法连接到服务器');
6459       console.error('域名解析失败:', error.hostname);
6460       return Promise.reject(new Error('无法连接到服务器，请检查网络设置'));
6461     }
6462
6463     // 超时错误
6464     if (error.code === 'ECONNABORTED') {
6465       console.error('请求超时，服务器响应过慢');
6466       return Promise.reject(new Error('请求超时，请检查网络连接'));
6467     }

```

```

6468
6469 // 对于400错误（客户端错误），不记录错误日志，只提取错误信息
6470 if (error.response && error.response.status === 400) {
6471     // 提取后端返回的具体错误信息
6472     if (error.response.data) {
6473         const backendError = error.response.data;
6474         if (backendError.error) {
6475             // 返回后端的具体错误消息
6476             return Promise.reject(new Error(backendError.error));
6477         } else if (backendError.message) {
6478             return Promise.reject(new Error(backendError.message));
6479         }
6480     }
6481     // 如果没有具体错误信息，返回通用错误
6482     return Promise.reject(new Error('注册失败，请检查输入信息'));
6483 }
6484
6485 // 对于其他错误（如网络错误、500错误等），记录日志
6486 console.error('API Response Error:', error.message);
6487 return Promise.reject(error);
6488 }
6489 );
6490
6491 export const authService = {
6492     // 用户注册（邮箱验证模式，返回 { message, email }）
6493     register: async (request: CreateUserRequest): Promise<{ message: string; email: string }> => {
6494         const response = await api.post('/auth/register', request);
6495         return response.data.data;
6496     },
6497
6498     // 验证邮箱
6499     verifyEmail: async (email: string, token: string): Promise<AuthResponse> => {
6500         const response = await api.post('/auth/verify-email', { email, token });
6501         return response.data.data;
6502     },
6503
6504     // 重新发送验证码
6505     resendVerification: async (email: string): Promise<{ message: string }> => {
6506         const response = await api.post('/auth/resend-verification', { email });
6507         return response.data.data;
6508     },
6509
6510     // 用户登录
6511     login: async (request: LoginRequest): Promise<AuthResponse> => {
6512         const response = await api.post('/auth/login', request);
6513         return response.data.data;
6514     },
6515
6516     // 刷新令牌
6517     refreshToken: async (refreshToken: string, deviceId: string): Promise<AuthResponse> => {

```

```

6518     const response = await api.post('/auth/refresh-token', { refreshToken, deviceId });
6519     return response.data.data;
6520 },
6521
6522 // 验证令牌
6523 verifyToken: async (token: string): Promise<any> => {
6524     const response = await api.post('/auth/verify-token', { token });
6525     return response.data.data;
6526 },
6527
6528 // 用户登出
6529 logout: async (): Promise<void> => {
6530     await api.post('/auth/logout');
6531 },
6532 };
6533
6534 export const proService = {
6535     // 激活 Pro
6536     activate: async (code: string, userId: number): Promise<{ success: boolean; planId?: string; expiresAt?: str
6537         const response = await api.post('/pro/activate', { code, userId });
6538         return response.data.data;
6539     },
6540
6541     // 查询 Pro 状态
6542     getStatus: async (userId: number): Promise<{ isPro: boolean; planId?: string; expiresAt?: string }> => {
6543         const response = await api.get('/pro/status/${userId}');
6544         return response.data.data;
6545     },
6546 };
6547
6548 export const skillService = {
6549     // 生成技能树
6550     generateSkillTree: async (request: SkillTreeRequest): Promise<SkillNode> => {
6551         const response = await api.post('/skills/generate', request);
6552         return response.data.data;
6553     },
6554
6555     // 获取技能树列表
6556     getSkillTreeList: async (page: number = 1, limit: number = 10, search?: string) => {
6557         const response = await api.get('/skills/list', {
6558             params: { page, limit, search }
6559         });
6560         return response.data.data;
6561     },
6562
6563     // 根据ID获取技能树
6564     getSkillTreeById: async (id: string): Promise<SkillNode> => {
6565         const response = await api.get('/skills/${id}');
6566         return response.data.data;
6567     },

```

```

6568
6569 // 保存用户技能树
6570 saveUserSkillTree: async (userId: string, skillTree: SkillNode, title?: string, tags?: string[]) => {
6571     const response = await api.post('/skills/save', {
6572         userId,
6573         skillTree,
6574         title,
6575         tags
6576     });
6577     return response.data.data;
6578 },
6579
6580 // 获取用户进度
6581 getUserProgress: async (userId: string, skillTreeld?: string) => {
6582     const response = await api.get(`/skills/progress/${userId}`, {
6583         params: { skillTreeld }
6584     });
6585     return response.data.data;
6586 },
6587
6588 // 更新用户进度
6589 updateUserProgress: async (userId: string, skillTreeld: string, completedNodes: string[], completedLinks: st
6590     const response = await api.put(`/skills/progress/${userId}`, {
6591         skillTreeld,
6592         completedNodes,
6593         completedLinks
6594     });
6595     return response.data.data;
6596 },
6597
6598 // 搜索热门领域
6599 searchPopularDomains: async (keyword: string) => {
6600     const response = await api.get(`/skills/search/domains`, {
6601         params: { keyword }
6602     });
6603     return response.data.data;
6604 },
6605
6606 // 获取推荐学习路径
6607 getRecommendedPath: async (domain: string, currentLevel: string = 'beginner', targetLevel: string = 'advance
6608     const response = await api.get(`/skills/recommendations/path`, {
6609         params: { domain, currentLevel, targetLevel }
6610     });
6611     return response.data.data;
6612 },
6613
6614 // 获取统计信息
6615 getStatistics: async () => {
6616     const response = await api.get(`/skills/stats/overview`);
6617     return response.data.data;

```

```

6618 },
6619
6620 // 获取用户学习报告
6621 getUserLearningReport: async (userId: string, period: string = 'week') => {
6622   const response = await api.get(`/skills/report/${userId}`, {
6623     params: { period }
6624   });
6625   return response.data.data;
6626 }
6627 };
6628
6629 export const monsterService = {
6630   // 用户创建/初始化自己的小怪兽
6631   createMonster: async (request: MonsterSetupRequest): Promise<MonsterResponse> => {
6632     const response = await api.post('/monster/create', request);
6633     return response.data.data;
6634   },
6635
6636   // 扣除体力（学习时调用）
6637   consumeStamina: async (userId: number, amount: number): Promise<{ success: boolean }> => {
6638     const response = await api.post('/monster/stamina/consume', { userId, amount });
6639     return response.data;
6640   },
6641
6642   // 与小怪兽对话
6643   chat: async (request: MonsterChatRequest): Promise<MonsterChatResponse> => {
6644     const response = await api.post('/monster/chat', request);
6645     return response.data;
6646   },
6647
6648   // 获取历史对话消息
6649   getMessages: async (userId: number): Promise<MonsterMessagesResponse> => {
6650     const response = await api.get('/monster/messages/${userId}');
6651     return response.data;
6652   },
6653
6654   // 获取怪兽状态
6655   getMonsterStatus: async (userId: number): Promise<MonsterResponse> => {
6656     const response = await api.get('/monster/status/${userId}');
6657     return response.data;
6658   },
6659 };
6660
6661 // =====
6662 // 笔记服务
6663 // 对应服务端 POST/GET/PUT /api/notes/*
6664 // 用于怪兽页笔记的持久化存储，替代纯 AsyncStorage 方案
6665 // =====
6666 export const noteService = {
6667   /** 创建笔记 */

```

```

6668 createNote: async (request: CreateNoteRequest): Promise<Note> => {
6669   const response = await api.post('/notes/create', request);
6670   return response.data.data;
6671 },
6672
6673 /** 获取某用户的所有笔记列表 */
6674 getNotes: async (userId: number): Promise<Note[]> => {
6675   const response = await api.get('/notes/list/${userId}');
6676   return response.data.data;
6677 },
6678
6679 /** 按日期获取某用户的笔记 */
6680 getNoteByDate: async (userId: number, date: string): Promise<Note | null> => {
6681   const response = await api.get('/notes/${userId}/${date}');
6682   return response.data.data;
6683 },
6684
6685 /** 更新笔记内容 */
6686 updateNote: async (request: UpdateNoteRequest): Promise<void> => {
6687   await api.put('/notes/update', request);
6688 },
6689 };
6690
6691 // =====
6692 // 奖励服务
6693 // 对应服务端 POST/GET /api/rewards/*
6694 //
6695 // 奖励类型仅支持 stamina（体力）和 energy（能量），不支持经验值
6696 // 奖励来源仅两种：
6697 //   - node_complete: 完成学习节点，时间长短对应不同奖励
6698 //   - game_win: 小游戏胜利获得奖励
6699 //
6700 // 叛逆型小怪双倍：调用方在传入 amount 时自行 *= 2，
6701 // rewardService 只做如实记录，不再二次翻倍
6702 // =====
6703 export const rewardService = {
6704   /** 创建奖励记录（完成节点或小游戏胜利后发放体力/能量） */
6705   createReward: async (request: CreateRewardRequest): Promise<{ success: boolean; rewardId: number }> => {
6706     const response = await api.post('/rewards/create', request);
6707     return response.data.data;
6708   },
6709
6710   /** 获取某用户的奖励列表 */
6711   getRewards: async (userId: number): Promise<{ rewards: Reward[] }> => {
6712     const response = await api.get('/rewards/list/${userId}');
6713     return response.data.data;
6714   },
6715
6716   /** 领取/兑换奖励 */
6717   claimReward: async (request: ClaimRewardRequest): Promise<{ success: boolean }> => {

```

```

6718     const response = await api.post('/rewards/claim', request);
6719     return response.data;
6720 },
6721 };
6722
6723 // =====
6724 // 领域/学习域服务
6725 // 对应服务端 POST/GET/PUT /api/domains/*
6726 //
6727 // 领域包括预设模块（理财、AI 等）和用户自定义模块
6728 // 自定义模块同样支持添加阶段和节点，通过本服务持久化到服务端
6729 // 支持记录节点学习进度、开始/完成学习
6730 // =====
6731 export const domainService = {
6732     /** 创建学习领域（用户选择预设模块或自定义模块时调用） */
6733     createDomain: async (request: CreateDomainRequest): Promise<Domain> => {
6734         const response = await api.post('/domains/create', request);
6735         return response.data.data;
6736     },
6737
6738     /** 获取某用户的所有学习领域列表 */
6739     getDomains: async (userId: number): Promise<Domain[]> => {
6740         const response = await api.get('/domains/list/${userId}');
6741         return response.data.data;
6742     },
6743
6744     /** 根据 ID 获取单个学习领域详情 */
6745     getDomainById: async (id: number): Promise<Domain> => {
6746         const response = await api.get('/domains/${id}');
6747         return response.data.data;
6748     },
6749
6750     /** 更新节点学习进度（标记为待学习/进行中/已完成） */
6751     updateNodeProgress: async (request: UpdateNodeProgressRequest): Promise<void> => {
6752         await api.put('/domains/nodes/progress', request);
6753     },
6754
6755     /** 开始学习（创建学习记录，返回 recordId 供 finishLearning 使用） */
6756     startLearning: async (request: StartLearningRequest): Promise<{ success: boolean; recordId: number }> => {
6757         const response = await api.post('/domains/learning/start', request);
6758         return response.data.data;
6759     },
6760
6761     /** 完成学习（更新学习记录的实际时长和进度） */
6762     finishLearning: async (request: FinishLearningRequest): Promise<void> => {
6763         await api.post('/domains/learning/finish', request);
6764     },
6765 };

```

=== mobile/src/services/gameService.ts ===

```

6766 export interface GameResult {

```



```

6767 success: boolean;
6768 rewards: {
6769   stamina: number;
6770   energy: number;
6771 };
6772 message: string;
6773 level: number;
6774 totalLevels: number;
6775 }
6776
6777 export interface SokobanLevel {
6778   id: number;
6779   name: string;
6780   difficulty: 'easy' | 'medium' | 'hard' | 'expert';
6781   map: string[];
6782   answer: string[];
6783 }
6784
6785 export const sokobanLevels: SokobanLevel[] = [
6786   {
6787     id: 1,
6788     name: '第一步',
6789     difficulty: 'easy',
6790     map: ['#####', '#@$#', '#####'],
6791     answer: ['右'],
6792   },
6793   {
6794     id: 2,
6795     name: '小试牛刀',
6796     difficulty: 'easy',
6797     map: ['#####', '# #', '#@$ #', '# . #', '#####'],
6798     answer: ['右', '右', '下'],
6799   },
6800   {
6801     id: 3,
6802     name: '简单推进',
6803     difficulty: 'easy',
6804     map: ['#####', '# #', '# @$ #', '# . #', '#####'],
6805     answer: ['右', '右', '右', '下'],
6806   },
6807   {
6808     id: 4,
6809     name: '双箱入门',
6810     difficulty: 'easy',
6811     map: ['#####', '# . #', '# @$ #', '# . #', '#####'],
6812     answer: ['右', '右', '下', '左', '左', '下'],
6813   },
6814   {
6815     id: 5,
6816     name: '双箱挑战',

```

```

6817   difficulty: 'easy',
6818   map: ['#####', '#   #', '# @$ $ #', '# .. #', '#####'],
6819   answer: ['右', '右', '下', '右', '下', '左', '左'],
6820 },
6821 {
6822   id: 6,
6823   name: '转角遇箱',
6824   difficulty: 'easy',
6825   map: ['#####', '# @ #', '# $ #', '# $ #', '# .. #', '#####'],
6826   answer: ['下', '左', '下', '右', '右', '上', '右', '下', '左'],
6827 },
6828 {
6829   id: 7,
6830   name: '之字形',
6831   difficulty: 'easy',
6832   map: ['#####', '#@   #', '# $   #', '# $   #', '# ... #', '#####'],
6833   answer: ['右', '右', '下', '右', '右', '下', '右', '下', '左', '左', '左'],
6834 },
6835 {
6836   id: 8,
6837   name: '迷宫初探',
6838   difficulty: 'easy',
6839   map: ['#####', '#@   #', '# $$$$ #', '#   #', '# ..... #', '#####'],
6840   answer: ['右', '下', '右', '右', '右', '右', '上', '右', '下', '下', '左', '左', '左', '左'],
6841 },
6842 {
6843   id: 9,
6844   name: '三箱齐推',
6845   difficulty: 'easy',
6846   map: ['#####', '#   #', '# @$ $ #', '# ... #', '#####'],
6847   answer: ['右', '右', '右', '右', '下', '左', '左', '左', '左'],
6848 },
6849 {
6850   id: 10,
6851   name: '回字形',
6852   difficulty: 'easy',
6853   map: ['#####', '# . #', '# $@$ #', '# . #', '#####'],
6854   answer: ['右', '右', '下', '右', '上', '左', '左', '下', '右'],
6855 },
6856 ];
6857
6858 export const getLevelsByDifficulty = (difficulty: SokobanLevel['difficulty']): SokobanLevel[] => {
6859   return sokobanLevels.filter(level => level.difficulty === difficulty);
6860 };
6861
6862 export const getTotalLevels = (): number => {
6863   return sokobanLevels.length;
6864 };
6865
6866 export const getLevelById = (id: number): SokobanLevel | undefined => {

```

```

6867   return sokobanLevels.find(level => level.id === id);
6868 };
6869
6870 export const getLevelByIndex = (index: number): SokobanLevel => {
6871   return sokobanLevels[index];
6872 };
6873
6874 export const calculateRewards = (level: number, monsterType?: string): GameResult['rewards'] => {
6875   const baseStamina = 2 + Math.floor(level / 3);
6876   const baseEnergy = 1 + Math.floor(level / 5);
6877
6878   if (monsterType === 'rebel') {
6879     return {
6880       stamina: baseStamina * 2,
6881       energy: baseEnergy * 2,
6882     };
6883   }
6884
6885   return {
6886     stamina: baseStamina,
6887     energy: baseEnergy,
6888   };
6889 };
6890
6891 export const mockSubmitGameResult = async (
6892   level: number,
6893   completed: boolean,
6894   monsterType?: string
6895 ): Promise<GameResult> => {
6896   return new Promise((resolve) => {
6897     setTimeout(() => {
6898       if (completed) {
6899         const rewards = calculateRewards(level, monsterType);
6900         resolve({
6901           success: true,
6902           rewards,
6903           message: `恭喜完成第 ${level} 关!`,
6904           level,
6905           totalLevels: sokobanLevels.length,
6906         });
6907       } else {
6908         resolve({
6909           success: false,
6910           rewards: { stamina: 0, energy: 0 },
6911           message: '游戏未完成',
6912           level,
6913           totalLevels: sokobanLevels.length,
6914         });
6915       }
6916     }, 500);

```

```

6917 });
6918 };
6919
6920 export const mockGetDailyPlays = async (): Promise<{ dailyPlays: number; maxPlays: number }> => {
6921   return new Promise((resolve) => {
6922     setTimeout(() => {
6923       resolve({
6924         dailyPlays: 0,
6925         maxPlays: 3,
6926       });
6927     }, 300);
6928   });
6929 };
6930
6931 === mobile/src/types/skill.ts ===
6932 export type PlatformType = 'bilibili' | 'xiaohongshu' | 'mooc';
6933
6934 export type StageType = 'beginner' | 'intermediate' | 'advanced';
6935
6936 export interface SkillNode {
6937   id: string;
6938   name: string;
6939   description: string;
6940   stage: StageType;
6941   platform: PlatformType;
6942   url: string;
6943   duration: number;
6944   children?: SkillNode[];
6945 }
6946
6947 export interface SkillTree {
6948   id: string;
6949   domain: string;
6950   title: string;
6951   description: string;
6952   stages: SkillStage[];
6953   totalDuration: number;
6954   learningMethod: string;
6955   learningGoal: string;
6956   frameworkExplanation: string;
6957 }
6958
6959 export interface SkillStage {
6960   id: StageType;
6961   name: string;
6962   duration: number;
6963   nodes: SkillNode[];
6964 }
6965
6966 export interface SkillLink {
6967   title: string;

```

```

6966 url: string;
6967 type: 'video' | 'article' | 'course' | 'documentation';
6968 }
6969
6970 export interface SkillTreeRequest {
6971   domain: string;
6972   level?: 'beginner' | 'intermediate' | 'advanced';
6973 }
6974
6975 // 认证相关类型
6976 export interface CreateUserRequest {
6977   username: string;
6978   email: string;
6979   password: string;
6980 }
6981
6982 export interface LoginRequest {
6983   type: 'email';
6984   email?: string;
6985   password?: string;
6986   deviceId: string;
6987   deviceType: 'ios' | 'android' | 'web';
6988   deviceName?: string;
6989 }
6990
6991 export interface UserResponse {
6992   id: number;
6993   username?: string;
6994   email?: string;
6995   phone?: string;
6996   nickname?: string;
6997   avatarUrl?: string;
6998   loginCount: number;
6999   lastLoginAt?: Date;
7000   createdAt: Date;
7001 }
7002
7003 export interface AuthResponse {
7004   user: UserResponse;
7005   accessToken: string;
7006   refreshToken: string;
7007   expiresAt: string;
7008 }
7009
7010 export interface MonsterSetupRequest {
7011   userId: number;
7012   name: string;
7013   style?: string;
7014   personality: 'lively' | 'calm' | 'rebel';
7015 }

```

```

7016
7017 export interface MonsterResponse {
7018   id: number;
7019   name: string;
7020   style: string;
7021   level: number;
7022   exp: number;
7023   stamina: number;
7024   maxStamina: number;
7025   energy: number;
7026   maxEnergy: number;
7027   personality: 'lively' | 'calm' | 'rebel';
7028   personalityParams?: Record<string, number>;
7029   lastEnergyRecover?: Date;
7030   lastStaminaRecover?: Date;
7031 }
7032
7033 // 怪兽对话相关类型
7034 export interface MonsterChatRequest {
7035   userId: number;
7036   message: string;
7037 }
7038
7039 export interface MonsterChatResponse {
7040   success: boolean;
7041   data: {
7042     message: string;
7043     remainingEnergy?: number;
7044   };
7045 }
7046
7047 export interface MonsterMessageItem {
7048   id: number;
7049   userId: number;
7050   message: string;
7051   isUser: boolean;
7052   createdAt: string;
7053 }
7054
7055 export interface MonsterMessagesResponse {
7056   success: boolean;
7057   data: {
7058     messages: MonsterMessageItem[];
7059   };
7060 }
7061
7062 // =====
7063 // 笔记相关类型
7064 // 对应服务端 noteController / noteRoutes (POST/GET/PUT /api/notes/*)
7065 // =====

```

```

7066
7067 /** 创建笔记请求 */
7068 export interface CreateNoteRequest {
7069   userId: number;
7070   date: string;    // ISO 日期字符串, 如 '2026-05-23'
7071   content?: string; // 笔记内容 (可选)
7072 }
7073
7074 /** 更新笔记请求 */
7075 export interface UpdateNoteRequest {
7076   noteId: number;
7077   content?: string; // 新的笔记内容
7078 }
7079
7080 /** 笔记数据 (服务端返回) */
7081 export interface Note {
7082   id: number;
7083   user_id: number;
7084   date: string;
7085   content: string;
7086   created_at: string;
7087   updated_at: string;
7088 }
7089
7090 // =====
7091 // 奖励相关类型
7092 // 对应服务端 rewardController / rewardRoutes (POST/GET /api/rewards/*)
7093 //
7094 // 奖励类型仅支持 stamina (体力) 和 energy (能量), 不支持经验值
7095 // 奖励来源仅两种:
7096 //   - node_complete: 完成学习节点, 时间长短对应不同奖励
7097 //   - game_win: 小游戏胜利获得奖励
7098 //
7099 // 注意: 叛逆型小怪的游戏奖励会在调用前由移动端将 amount 乘以 2,
7100 //       rewardService 只做如实记录, 不再二次翻倍
7101 // =====
7102
7103 /** 奖励类型: 体力或能量 */
7104 export type RewardType = 'stamina' | 'energy';
7105
7106 /** 奖励来源: 完成节点 或 小游戏胜利 */
7107 export type RewardSource = 'node_complete' | 'game_win';
7108
7109 /** 创建奖励请求 */
7110 export interface CreateRewardRequest {
7111   userId: number;
7112   type: RewardType;    // 'stamina' (体力) | 'energy' (能量)
7113   amount: number;      // 奖励数量 (叛逆型双倍由调用方计算后传入)
7114   source: RewardSource; // 'node_complete' | 'game_win'
7115 }

```

```

7116
7117 /** 领取奖励请求 */
7118 export interface ClaimRewardRequest {
7119     rewardId: number;    // 要领取的奖励记录 ID
7120 }
7121
7122 /** 奖励数据（服务端返回） */
7123 export interface Reward {
7124     id: number;
7125     user_id: number;
7126     type: string;        // 'stamina' | 'energy'
7127     amount: number;      // 奖励数量
7128     source: string;       // 'node_complete' | 'game_win'
7129     claimed: boolean;     // 是否已领取
7130     created_at: string;   // 创建时间
7131 }
7132
7133 // =====
7134 // 领域/学习域相关类型
7135 // 对应服务端 domainController / domainRoutes (POST/GET/PUT /api/domains/*)
7136 //
7137 // 领域包括预设模块（理财、AI 等）和用户自定义模块
7138 // 自定义模块同样支持添加阶段和节点，数据通过 domainService 持久化到服务端
7139 // =====
7140
7141 /** 领域类型：预设 或 自定义 */
7142 export type DomainType = 'preset' | 'custom';
7143
7144 /** 创建领域请求 */
7145 export interface CreateDomainRequest {
7146     userId: number;
7147     name: string;        // 领域名称，如 'JavaScript'、'Python'、'我的自定义领域'
7148     type?: DomainType;   // 领域类型，默认 'preset'
7149 }
7150
7151 /** 更新节点进度请求 */
7152 export interface UpdateNodeProgressRequest {
7153     userId: number;
7154     domainId: number;
7155     nodeId: string;      // 节点 ID
7156     status: 'pending' | 'doing' | 'done'; // 节点状态
7157     studyTime?: number;  // 学习时长（秒），可选
7158     notes?: string;      // 学习笔记，可选
7159 }
7160
7161 /** 开始学习请求（创建学习记录，返回 recordId 供 finishLearning 使用） */
7162 export interface StartLearningRequest {
7163     userId: number;
7164     domainId: number;
7165     nodeId: string;

```



```

7166 }
7167
7168 /** 完成学习请求（更新学习记录的时长和进度） */
7169 export interface FinishLearningRequest {
7170   recordId: number;    // startLearning 返回的学习记录 ID
7171   duration: number;    // 实际学习时长（秒）
7172   progressAfter: number; // 学习后的进度百分比
7173 }
7174
7175 /** 领域数据（服务端返回） */
7176 export interface Domain {
7177   id: number;
7178   user_id: number;
7179   name: string;    // 领域名称
7180   type: string;    // 'preset' | 'custom'
7181   created_at: string; // 创建时间
7182 }
7183
7184 === mobile/src/utils/auth.ts ===
7185 import { AuthResponse } from '../types/skill';
7186
7187 // 使用内存存储作为临时解决方案，避免Expo Go环境下的权限问题
7188 let memoryStorage: { [key: string]: string } = {};
7189 const AUTH_STORAGE_KEY = 'auth_data';
7190
7191 /**
7192  * 保存用户认证信息到内存存储（临时解决方案）
7193  */
7194 export const saveAuthData = async (authData: AuthResponse): Promise<void> => {
7195   try {
7196     memoryStorage[AUTH_STORAGE_KEY] = JSON.stringify(authData);
7197     console.log('[Auth] 认证信息保存到内存成功');
7198   } catch (error) {
7199     console.error('保存认证信息失败:', error);
7200   }
7201 };
7202
7203 /**
7204  * 从内存存储获取用户认证信息
7205  */
7206 export const getAuthData = async (): Promise<AuthResponse | null> => {
7207   try {
7208     const authDataString = memoryStorage[AUTH_STORAGE_KEY];
7209     if (authDataString) {
7210       const authData = JSON.parse(authDataString);
7211       // 转换日期字符串为Date对象
7212       if (authData.user && authData.user.createdAt) {
7213         authData.user.createdAt = new Date(authData.user.createdAt);
7214       }
7215       if (authData.user && authData.user.lastLoginAt) {
7216         authData.user.lastLoginAt = new Date(authData.user.lastLoginAt);
7217       }
7218     }
7219   }
7220 }

```

```

7215     }
7216     console.log('[Auth] 从内存获取认证信息成功');
7217     return authData;
7218   }
7219   console.log('[Auth] 未找到认证信息');
7220   return null;
7221 } catch (error) {
7222   console.error('获取认证信息失败:', error);
7223   return null;
7224 }
7225 };
7226
7227 /**
7228  * 清除用户认证信息
7229  */
7230 export const clearAuthData = async (): Promise<void> => {
7231   try {
7232     delete memoryStorage[AUTH_STORAGE_KEY];
7233     // 同时清除 Pro 缓存，防止其他用户登录后复用旧的 Pro 状态
7234     try {
7235       const AsyncStorage = require('@react-native-async-storage/async-storage').default;
7236       await AsyncStorage.removeItem('user_subscription');
7237     } catch {}
7238     console.log('[Auth] 认证信息和 Pro 缓存已清除');
7239   } catch (error) {
7240     console.error('清除认证信息失败:', error);
7241   }
7242 };
7243
7244 /**
7245  * 检查token是否过期
7246  */
7247 export const isTokenExpired = (expiresAt: string): boolean => {
7248   return new Date() > new Date(expiresAt);
7249 };
7250
7251 /**
7252  * 检查用户是否已登录
7253  */
7254 export const checkAuthStatus = async (): Promise<boolean> => {
7255   try {
7256     const authData = await getAuthData();
7257
7258     if (!authData) {
7259       return false;
7260     }
7261
7262     // 检查token是否过期
7263     if (isTokenExpired(authData.expiresAt)) {
7264       // token过期，尝试刷新

```

```

7265     try {
7266         // 这里可以添加刷新token的逻辑
7267         // 暂时先清除过期数据
7268         await clearAuthData();
7269         return false;
7270     } catch (error) {
7271         console.error('刷新token失败:', error);
7272         await clearAuthData();
7273         return false;
7274     }
7275 }
7276
7277 return true;
7278 } catch (error) {
7279     console.error('检查认证状态失败:', error);
7280     return false;
7281 }
7282 };
7283
7284 /**
7285  * 获取当前用户信息
7286  */
7287 export const getCurrentUser = async () => {
7288     try {
7289         const authData = await getAuthData();
7290         console.log('[Auth] 当前用户信息:', authData?.user || '无');
7291         return authData?.user || null;
7292     } catch (error) {
7293         console.error('获取当前用户信息失败:', error);
7294         return null;
7295     }
7296 };
7297
7298 /**
7299  * 获取访问令牌
7300  */
7301 export const getAccessToken = async (): Promise<string | null> => {
7302     try {
7303         const authData = await getAuthData();
7304         console.log('[Auth] 获取访问令牌:', authData?.accessToken || '无');
7305         return authData?.accessToken || null;
7306     } catch (error) {
7307         console.error('获取访问令牌失败:', error);
7308         return null;
7309     }
7310 };
7311
7312 === mobile/src/utils/constants.ts ===
7313 import Constants from 'expo-constants';
7314
7315 // 应用常量配置

```

```

7314 // 公网 / 跨网调试: 优先读 Expo extra.apiUrl, 其次读 EXPO_PUBLIC_API_URL
7315 // 都未配置时才回退到 localhost。
7316 const extraApiBaseUrl = Constants.expoConfig?.extra?.apiBaseUrl;
7317 const rawApiBaseUrl =
7318   (typeof extraApiBaseUrl === 'string' ? extraApiBaseUrl : undefined) ??
7319   process.env.EXPO_PUBLIC_API_URL?.trim();
7320
7321 export const API_BASE_URL =
7322   rawApiBaseUrl && rawApiBaseUrl.length > 0 ? rawApiBaseUrl : 'http://localhost:3001/api';
7323 export const APP_NAME = 'LearnFlow';
7324 export const APP_VERSION = '1.0.0';
7325
7326 console.log('[Config] extra.apiUrl =', extraApiBaseUrl);
7327 console.log('[Config] EXPO_PUBLIC_API_URL =', process.env.EXPO_PUBLIC_API_URL);
7328 console.log('[Config] API_BASE_URL =', API_BASE_URL);
7329
7330 // 页面路由常量
7331 export const ROUTES = {
7332   HOME: '/',
7333   SKILL_TREE: '/skill-tree',
7334 } as const;
7335
7336 // 像素风格颜色常量 — 对齐新 ThemeContext 深色模式默认值
7337 export const PIXEL_COLORS = {
7338   PRIMARY: '#7B75D8',
7339   SUCCESS: '#4A9840',
7340   WARNING: '#D4A058',
7341   ERROR: '#D05858',
7342   BACKGROUND: '#0D0D1A',
7343   BACKGROUND_LIGHT: '#141428',
7344   BACKGROUND_DARK: '#0A0A14',
7345   WHITE: '#E8E8F8',
7346   TEXT_PRIMARY: '#E8E8F8',
7347   TEXT_SECONDARY: '#8A8AA8',
7348   TEXT_TERTIARY: '#5A5A78',
7349   BORDER: 'rgba(255, 255, 255, 0.08)',
7350   BORDER_LIGHT: 'rgba(255, 255, 255, 0.04)',
7351   BORDER_DARK: 'rgba(120, 100, 220, 0.18)',
7352
7353   PIXEL_DARK_BLUE: '#1A1A35',
7354   PIXEL_PURPLE: '#7B75D8',
7355   PIXEL_PINK: '#D07090',
7356   PIXEL_ORANGE: '#D4A058',
7357   PIXEL_YELLOW: '#E0C068',
7358   PIXEL_GREEN: '#4A9840',
7359   PIXEL_CYAN: '#50B8D8',
7360   PIXEL_GRAY: '#2A2A48',
7361   PIXEL_LIGHT_GRAY: '#4A4A68',
7362
7363   NODE_PENDING: '#2A2A48',

```

```

7364  NODE_DOING: '#D4A058',
7365  NODE_DONE: '#4A9840',
7366 } as const;
7367
7368 // 颜色常量
7369 export const COLORS = {
7370  PRIMARY: PIXEL_COLORS.PRIMARY,
7371  SUCCESS: PIXEL_COLORS.SUCCESS,
7372  WARNING: PIXEL_COLORS.WARNING,
7373  ERROR: PIXEL_COLORS.ERROR,
7374  BACKGROUND: PIXEL_COLORS.BACKGROUND,
7375  BACKGROUND_LIGHT: PIXEL_COLORS.BACKGROUND_LIGHT,
7376  BACKGROUND_DARK: PIXEL_COLORS.BACKGROUND_DARK,
7377  WHITE: PIXEL_COLORS.WHITE,
7378  TEXT_PRIMARY: PIXEL_COLORS.TEXT_PRIMARY,
7379  TEXT_SECONDARY: PIXEL_COLORS.TEXT_SECONDARY,
7380  TEXT_TERTIARY: PIXEL_COLORS.TEXT_TERTIARY,
7381  BORDER: PIXEL_COLORS.BORDER,
7382  BORDER_LIGHT: PIXEL_COLORS.BORDER_LIGHT,
7383  BORDER_DARK: PIXEL_COLORS.BORDER_DARK,
7384  ORANGE: PIXEL_COLORS.PIXEL_ORANGE,
7385  PURPLE: PIXEL_COLORS.PIXEL_PURPLE,
7386  PINK: PIXEL_COLORS.PIXEL_PINK,
7387 } as const;
7388
7389 // 风格颜色常量
7390 export const CUTE_COLORS = {
7391  WARM_WHITE: PIXEL_COLORS.BACKGROUND_LIGHT,
7392  WHITE: PIXEL_COLORS.WHITE,
7393  PINK: PIXEL_COLORS.PIXEL_PINK,
7394  MINT: PIXEL_COLORS.PIXEL_GREEN,
7395  SKY_BLUE: PIXEL_COLORS.PIXEL_CYAN,
7396  LAVENDER: PIXEL_COLORS.PIXEL_PURPLE,
7397  GOLD: PIXEL_COLORS.PIXEL_YELLOW,
7398  CORAL: PIXEL_COLORS.PIXEL_ORANGE,
7399  PURPLE: PIXEL_COLORS.PIXEL_PURPLE,
7400  TEAL: PIXEL_COLORS.PIXEL_CYAN,
7401  PEACH: PIXEL_COLORS.PIXEL_ORANGE,
7402  CREAM: PIXEL_COLORS.BACKGROUND_LIGHT,
7403  LIGHT_PINK: PIXEL_COLORS.PIXEL_PINK,
7404  DARK_GRAY: PIXEL_COLORS.TEXT_PRIMARY,
7405  GRAY: PIXEL_COLORS.TEXT_SECONDARY,
7406  LIGHT_GRAY: PIXEL_COLORS.PIXEL_GRAY,
7407  LIGHT_BLUE: PIXEL_COLORS.PIXEL_DARK_BLUE,
7408  BUTTER_YELLOW: PIXEL_COLORS.PIXEL_YELLOW,
7409 } as const;
7410
7411 // 样式常量
7412 export const SPACING = {
7413  SMALL: 8,

```

```

7414 MEDIUM: 16,
7415 LARGE: 24,
7416 XLARGE: 32,
7417 } as const;
7418
7419 // 边框圆角常量 — 更柔和的现代圆角
7420 export const BORDER_RADIUS = {
7421   SMALL: 8,
7422   MEDIUM: 12,
7423   LARGE: 16,
7424   XLARGE: 20,
7425 } as const;
7426
7427 // 阴影样式常量
7428 export const SHADOWS = {
7429   SOFT: {
7430     shadowColor: '#000',
7431     shadowOffset: { width: 0, height: 2 },
7432     shadowOpacity: 0.1,
7433     shadowRadius: 4,
7434     elevation: 3,
7435   },
7436   MEDIUM: {
7437     shadowColor: '#000',
7438     shadowOffset: { width: 0, height: 4 },
7439     shadowOpacity: 0.15,
7440     shadowRadius: 8,
7441     elevation: 5,
7442   },
7443   STRONG: {
7444     shadowColor: '#000',
7445     shadowOffset: { width: 0, height: 6 },
7446     shadowOpacity: 0.2,
7447     shadowRadius: 12,
7448     elevation: 8,
7449   },
7450 } as const;
7451
7452 // 像素边框样式常量
7453 export const PIXEL_BORDERS = {
7454   SMALL: 2,
7455   MEDIUM: 4,
7456   LARGE: 6,
7457 } as const;
7458
7459 // 默认配置
7460 export const DEFAULT_CONFIG = {
7461   SKILL_TREE_LIMIT: 5,
7462   SEARCH_LIMIT: 10,
7463   DEFAULT_USER_ID: 'user1',

```

```

7464 ENERGY_RECOVERY_HOURS: 6,
7465 MAX_ENERGY: 10,
7466 POMODORO_MINUTES: 25,
7467 } as const;
7468
7469 // 怪兽配置
7470 export const MONSTER_CONFIG = {
7471   TYPES: {
7472     LIVELY: 'lively',
7473     CALM: 'calm',
7474     REBEL: 'rebel',
7475   } as const,
7476
7477   STAMINA: {
7478     BASE_MAX: 100,
7479     CALM_BONUS: 20,
7480     COST_PER_JUMP: 10,
7481     GAME_BONUS: 20,
7482     REBEL_MULTIPLIER: 2,
7483     // Pro 版体力上限 (远超正常消耗)
7484     PRO_MAX: 1000,
7485   } as const,
7486
7487   ENERGY: {
7488     DAILY_FREE: 50,
7489     COST_PER_TOKEN: 0.05,
7490     GAME_BONUS: 5,
7491     // Pro 版每日能量增加量
7492     DAILY_PRO: 1000,
7493   } as const,
7494
7495   DAILY_RESET: {
7496     HOUR: 5,
7497     MINUTE: 0,
7498   } as const,
7499
7500   POMODORO: {
7501     TIME_OPTIONS: [25, 45, 60, 90, 120, 180] as const,
7502   } as const,
7503
7504   // 小游戏配置
7505   // DAILY_LIMIT: 免费版每日游戏次数上限
7506   // PRO_DAILY_LIMIT: PRO 版每日游戏次数上限 (预留, 后续根据用户等级读取不同值)
7507   GAME: {
7508     DAILY_LIMIT: 3,
7509     PRO_DAILY_LIMIT: 10,
7510   } as const,
7511
7512   PERSONALITIES: {
7513     lively: {

```

```

7514     name: '活力小怪',
7515     traits: ['单次学习任务时长 -5分钟', '适合快节奏碎片化学习'],
7516   },
7517   calm: {
7518     name: '沉稳小怪',
7519     traits: ['每日体力额外 +20点', '可多跳转2次', '擅长深度思考'],
7520   },
7521   rebel: {
7522     name: '叛逆小怪',
7523     traits: ['小游戏获得的体力、能量全部双倍', '勇于探索挑战'],
7524   },
7525 } as const,
7526
7527 COLORS: {
7528   lively: { primary: '#D4A058', secondary: '#B08040' },
7529   calm: { primary: '#7B75D8', secondary: '#5A54B0' },
7530   rebel: { primary: '#D07090', secondary: '#A05068' },
7531 } as const,
7532 };
7533
7534 === mobile/src/utils/helpers.ts ===
7535 import { Alert, Keyboard, Platform, Dimensions, LayoutAnimation } from 'react-native';
7536
7537 // 通用工具函数
7538
7539 /** 将 unknown 错误转为可展示、可传给原生 Alert 的纯文本 */
7540 export const toErrorMessage = (error: unknown): string => {
7541   if (error instanceof Error) return error.message || '请稍后重试';
7542   if (typeof error === 'string') return error;
7543   return '请稍后重试';
7544 };
7545
7546 /**
7547  * 显示错误提示 (参数顺序与 Alert.alert 一致: 标题、正文)
7548  */
7549 export const showErrorAlert = (title: string, message: string) => {
7550   Alert.alert(title, message);
7551 };
7552
7553 /**
7554  * 显示成功提示
7555  */
7556 export const showSuccessAlert = (title: string, message: string) => {
7557   Alert.alert(title, message);
7558 };
7559
7560 /**
7561  * 验证输入是否为空
7562  */
7563 export const validateInput = (input: string, fieldName: string): boolean => {
7564   if (!input.trim()) {

```



```

7563   showErrorAlert('错误', `请输入${fieldName}`);
7564   return false;
7565 }
7566 return true;
7567 };
7568
7569 /**
7570  * 格式化时间显示
7571  */
7572 export const formatTime = (seconds: number): string => {
7573   const hours = Math.floor(seconds / 3600);
7574   const minutes = Math.floor((seconds % 3600) / 60);
7575
7576   if (hours > 0) {
7577     return `${hours}小时${minutes}分钟`;
7578   }
7579   return `${minutes}分钟`;
7580 };
7581
7582 /**
7583  * 截断长文本
7584  */
7585 export const truncateText = (text: string, maxLength: number): string => {
7586   if (text.length <= maxLength) return text;
7587   return text.substring(0, maxLength) + '...';
7588 };
7589
7590 /**
7591  * 创建键盘定位监听器
7592  * 实现点击输入框时，输入框自动上升到屏幕中心位置
7593  */
7594 export const createKeyboardPositioningListener = (
7595   onKeyboardShow: (keyboardHeight: number, inputOffset: number) => void,
7596   onKeyboardHide: () => void,
7597   inputYPosition: number = 0
7598 ) => {
7599   const screenHeight = Dimensions.get('window').height;
7600
7601   // 计算输入框需要移动的偏移量
7602   const calculateInputOffset = (keyboardHeight: number, yPosition: number): number => {
7603     const visibleScreenHeight = screenHeight - keyboardHeight;
7604     const inputHeight = 56; // 标准输入框高度
7605     const keyboardOffset = 40; // 键盘上方保留1cm左右的间距 (约40px)
7606     const targetY = visibleScreenHeight - inputHeight - keyboardOffset;
7607
7608     // 如果输入框已经在目标位置之上，不需要移动
7609     if (yPosition <= targetY) {
7610       return 0;
7611     }
7612

```

```

7613     return targetY - yPosition;
7614 };
7615
7616 // 配置动画
7617 const configureAnimation = () => {
7618     LayoutAnimation.configureNext({
7619         duration: 300,
7620         create: {
7621             type: LayoutAnimation.Types.easeInEaseOut,
7622             property: LayoutAnimation.Properties.opacity,
7623         },
7624         update: {
7625             type: LayoutAnimation.Types.spring,
7626             springDamping: 0.7,
7627         },
7628     });
7629 };
7630
7631 const keyboardDidShowListener = Keyboard.addListener(
7632     Platform.OS === 'ios' ? 'keyboardWillShow' : 'keyboardDidShow',
7633     (e) => {
7634         const keyboardHeight = e.endCoordinates.height;
7635         const inputOffset = calculateInputOffset(keyboardHeight, inputYPosition);
7636         configureAnimation();
7637         onKeyboardShow(keyboardHeight, inputOffset);
7638     }
7639 );
7640
7641 const keyboardDidHideListener = Keyboard.addListener(
7642     Platform.OS === 'ios' ? 'keyboardWillHide' : 'keyboardDidHide',
7643     () => {
7644         configureAnimation();
7645         onKeyboardHide();
7646     }
7647 );
7648
7649 // 返回清理函数
7650 return () => {
7651     keyboardDidShowListener.remove();
7652     keyboardDidHideListener.remove();
7653 };
7654 };
7655
7656 /**
7657  * 测量输入框位置
7658  */
7659 export const measureInputPosition = (event: any, callback: (yPosition: number) => void) => {
7660     if (event.nativeEvent && event.nativeEvent.layout) {
7661         const { layout } = event.nativeEvent;
7662         callback(layout.y);

```

```

7663 }
7664 };
7665
7666 /**
7667  * 测量输入框位置 (通过ref)
7668  */
7669 export const measureInputPositionByRef = (inputRef: any, callback: (yPosition: number) => void) => {
7670   if (inputRef.current) {
7671     inputRef.current.measure((x: number, y: number, width: number, height: number, pageX: number, pageY: numbe
7672       callback(pageY);
7673   });
7674 }
7675 };
7676
7677 /**
7678  * 获取输入框定位样式
7679  */
7680 export const getInputPositioningStyle = (offset: number) => ({
7681   transform: [{ translateY: offset }]
7682 });
7683
7684 /**
7685  * 生成唯一ID
7686  */
7687 export const generateId = (): string => {
7688   return Date.now().toString(36) + Math.random().toString(36).substr(2);
7689 };
7690
7691 /**
7692  * 格式化秒数为 MM:SS 格式
7693  */
7694 export const formatTimer = (seconds: number): string => {
7695   const mins = Math.floor(seconds / 60);
7696   const secs = seconds % 60;
7697   return `${mins.toString().padStart(2, '0')}:${secs.toString().padStart(2, '0')}`;
7698 };
7699
7700 /**
7701  * 防抖函数
7702  */
7703 export const debounce = <T extends (...args: any[]) => any> (
7704   func: T,
7705   delay: number
7706 ): (...args: Parameters<T>) => void => {
7707   let timeoutId: NodeJS.Timeout;
7708   return (...args: Parameters<T>) => {
7709     clearTimeout(timeoutId);
7710     timeoutId = setTimeout(() => func.apply(null, args), delay);
7711   };
7712 };

```

```

=== mobile/src/utils/index.ts ===
7713 export * from './constants';
7714 export * from './helpers';
7715 export * from './auth';
7716 export * from './storage';
7717 export * from './useKeyboardAvoidScroll';
=== mobile/src/utils/pricing.ts ===
7718 // ===== 付费模式 — 定价与权益 =====
7719
7720 export type PlanId = 'monthly' | 'yearly' | 'lifetime';
7721
7722 export interface PricingPlan {
7723   id: PlanId;
7724   label: string;
7725   price: number;
7726   originalPrice?: number; // 划线原价
7727   renewPrice?: number; // 续费价格 (连续包月用)
7728   unit: string; // '月' | '年' | '永久'
7729   badge?: string;
7730   isPopular?: boolean;
7731 }
7732
7733 export const PRICING_PLANS: PricingPlan[] = [
7734   {
7735     id: 'monthly',
7736     label: '连续包月',
7737     price: 9.9,
7738     renewPrice: 16,
7739     unit: '月',
7740     badge: '首月特惠',
7741     isPopular: true,
7742   },
7743   {
7744     id: 'yearly',
7745     label: '年卡',
7746     price: 148,
7747     originalPrice: 192,
7748     unit: '年',
7749     badge: '省¥44',
7750   },
7751   {
7752     id: 'lifetime',
7753     label: '永久会员',
7754     price: 228,
7755     unit: '永久',
7756     badge: '超值',
7757   },
7758 ];
7759
7760 export interface ProFeature {

```

```

7761 icon: string;
7762 title: string;
7763 freeLimit: string;
7764 proBenefit: string;
7765 }
7766
7767 export const PRO_FEATURES: ProFeature[] = [
7768 {
7769   icon: 'grid-outline',
7770   title: '官方模块',
7771   freeLimit: '1',
7772   proBenefit: '无限',
7773 },
7774 {
7775   icon: 'sparkles-outline',
7776   title: 'AI生成领域',
7777   freeLimit: '1次',
7778   proBenefit: '无限',
7779 },
7780 {
7781   icon: 'flash-outline',
7782   title: '每日基础体力',
7783   freeLimit: '100 点 (沉稳+20=120) ',
7784   proBenefit: '无限',
7785 },
7786 {
7787   icon: 'arrow-redo-outline',
7788   title: '每日可跳转次数',
7789   freeLimit: '10 次 (沉稳12次) ',
7790   proBenefit: '无限',
7791 },
7792 {
7793   icon: 'diamond-outline',
7794   title: '每日基础能量 Π',
7795   freeLimit: '50 点',
7796   proBenefit: '1000 点/天',
7797 },
7798 {
7799   icon: 'shirt-outline',
7800   title: '怪兽装扮',
7801   freeLimit: '无',
7802   proBenefit: '每月更新3套',
7803 },
7804 {
7805   icon: 'lock-closed-outline',
7806   title: '学习模块权限',
7807   freeLimit: '1个官方 + 无限自定义',
7808   proBenefit: '全量解锁 + 无限自定义',
7809 },
7810 {

```

```

7811   icon: 'ban-outline',
7812   title: '广告体验',
7813   freeLimit: '可能有广告',
7814   proBenefit: '全程免广告',
7815 },
7816 ];
7817
7818 export type PaymentMethod = 'alipay' | 'wechat';
7819
7820 export const PAYMENT_METHODS: { id: PaymentMethod; label: string; icon: string }[] = [
7821   { id: 'alipay', label: '支付宝', icon: 'logo-alipay' },
7822   { id: 'wechat', label: '微信支付', icon: 'logo-wechat' },
7823 ];
7824
7825 // AsyncStorage key
7826 export const SUBSCRIPTION_STORAGE_KEY = 'user_subscription';
7827
7828 === mobile/src/utils/storage.ts ===
7829
7830 export const STORAGE_KEYS = {
7831   MONSTER: 'monster',
7832   NOTES: 'learningNotes',
7833   TASKS: 'learningTasks',
7834   LAST_RESET: 'lastDailyReset',
7835   IS_NEW_USER: 'isNewUser',
7836 } as const;
7837
7838 export const storage = {
7839   async getItem<T>(key: string): Promise<T | null> {
7840     try {
7841       const value = await AsyncStorage.getItem(key);
7842       return value ? JSON.parse(value) : null;
7843     } catch (error) {
7844       console.error(`Storage get error: ${key}`, error);
7845       return null;
7846     }
7847   },
7848
7849   async setItem(key: string, value: any): Promise<void> {
7850     try {
7851       await AsyncStorage.setItem(key, JSON.stringify(value));
7852     } catch (error) {
7853       console.error(`Storage set error: ${key}`, error);
7854     }
7855   },
7856
7857   async removeItem(key: string): Promise<void> {
7858     try {
7859       await AsyncStorage.removeItem(key);

```

```

7860   } catch (error) {
7861     console.error(`Storage remove error: ${key}`, error);
7862   }
7863 },
7864 };
7865
7866 === mobile/src/Utils/useKeyboardAvoidScroll.ts ===
7867 import { useRef, useCallback, useEffect } from 'react';
7868 import {
7869   Keyboard,
7870   LayoutAnimation,
7871   Platform,
7872   KeyboardEvent,
7873   ScrollView,
7874   Dimensions,
7875   TextInput,
7876 } from 'react-native';
7877 // =====
7878 // useKeyboardAvoidScroll
7879 // iOS 水流式丝滑键盘避让 Hook
7880 //
7881 // 核心设计:
7882 // 1. measureInWindow 绝对定位 — 聚焦时用原生 measureInWindow 获取输入框
7883 // 在屏幕上的绝对坐标, 换算为 ScrollView 内容坐标, 彻底解决 onLayout
7884 // 相对坐标导致的偏移错误
7885 // 2. keyboardWillShow 抢占 — 键盘动画开始前获取 duration, LayoutAnimation
7886 // 匹配动画曲线, scrollTo 在动画上下文中丝滑同步
7887 // 3. onScroll 追踪 — 记录当前滚动偏移, 确保切换输入框时计算准确
7888 // 4. ref 避免闭包 — 可变数据全部通过 useRef 存储, 键盘监听器只挂载一次
7889 //
7890 // 使用方式:
7891 // const { scrollViewRef, handleInputFocus, handleInputBlur, onScroll } = useKeyboardAvoidScroll();
7892 // const emailRef = useRef<TextInput>(null);
7893 //
7894 // <ScrollView ref={scrollViewRef} onScroll={onScroll}>
7895 //   <TextInput ref={emailRef} onFocus={() => handleInputFocus(emailRef)} onBlur={handleInputBlur} />
7896 // </ScrollView>
7897 // =====
7898
7899 const KEYBOARD_TOP_PADDING = 12; // 输入框底部到键盘顶部的舒适间距
7900
7901 export const useKeyboardAvoidScroll = () => {
7902   const scrollViewRef = useRef<ScrollView>(null);
7903   const activeInputRef = useRef<TextInput | null>(null);
7904   const keyboardHeightRef = useRef(0);
7905   const currentScrollYRef = useRef(0);
7906
7907   // =====
7908   // 测量 + 滚动: 获取输入框在 ScrollView 内容中的真实坐标

```

```

7909 // 并滚动到键盘上方合适位置
7910 // =====
7911 const measureAndScroll = useCallback(() => {
7912   const input = activeInputRef.current;
7913   const sv = scrollViewRef.current;
7914   if (!input || !sv) return;
7915
7916   const kbHeight = keyboardHeightRef.current;
7917   if (kbHeight === 0) return;
7918
7919   // Step 1: 获取输入框的屏幕绝对坐标
7920   input.measureInWindow((_ix: number, _iy: number, _iw: number, _ih: number) => {
7921     // Step 2: 获取 ScrollView 的屏幕绝对坐标
7922     sv.measureInWindow((_sx: number, _sy: number) => {
7923       const screenH = Dimensions.get('window').height;
7924       const visibleBottom = screenH - kbHeight;
7925       const inputWindowBottom = _iy + _ih;
7926
7927       if (inputWindowBottom > visibleBottom - KEYBOARD_TOP_PADDING) {
7928         // Step 3: 换算为 ScrollView 内容坐标
7929         // contentY = (input窗口Y - scrollView窗口Y) + 当前滚动偏移
7930         // 例如: 输入框在屏幕 y=500, ScrollView 在屏幕 y=100, 已滚动 0
7931         // contentY = 500 - 100 + 0 = 400 ← 这才是 scrollTo 需要的值
7932         const contentY = _iy - _sy + currentScrollYRef.current;
7933
7934         // Step 4: 计算目标滚动量
7935         // 让输入框底部刚好在 visibleBottom - padding 处
7936         const targetY = Math.max(
7937           0,
7938           contentY + _ih - visibleBottom + KEYBOARD_TOP_PADDING
7939         );
7940
7941         sv.scrollTo({ y: targetY, animated: false });
7942       }
7943     });
7944   });
7945 }, []);
7946
7947 // =====
7948 // 追踪 ScrollView 的当前滚动偏移
7949 // =====
7950 const onScroll = useCallback((event: any) => {
7951   currentScrollYRef.current = event.nativeEvent?.contentOffset?.y || 0;
7952 }, []);
7953
7954 // =====
7955 // 输入框 onFocus: 记录活跃输入框, 键盘已显示时直接滚动
7956 // =====
7957 const handleInputFocus = useCallback(
7958   (ref: React.RefObject<TextInput>) => {

```



```

7959     activeInputRef.current = ref.current as TextInput | null;
7960
7961     // 键盘已显示，用户正在切换输入框 → 立即测量并滚动
7962     if (keyboardHeightRef.current > 0) {
7963         // rAF 确保布局稳定后测量
7964         requestAnimationFrame(() => {
7965             measureAndScroll();
7966         });
7967     }
7968 },
7969 [measureAndScroll]
7970 );
7971
7972 // =====
7973 // 输入框 onBlur: 清除活跃引用
7974 // =====
7975 const handleInputBlur = useCallback(() => {
7976     activeInputRef.current = null;
7977 }, []);
7978
7979 // =====
7980 // 键盘事件监听（只挂载一次）
7981 // =====
7982 useEffect(() => {
7983     const showEvent =
7984         Platform.OS === 'ios' ? 'keyboardWillShow' : 'keyboardDidShow';
7985     const hideEvent =
7986         Platform.OS === 'ios' ? 'keyboardWillHide' : 'keyboardDidHide';
7987
7988     const showSub = Keyboard.addListener(showEvent, (e: KeyboardEvent) => {
7989         keyboardHeightRef.current = e.endCoordinates.height;
7990
7991         // 匹配键盘动画参数
7992         if (Platform.OS === 'ios') {
7993             LayoutAnimation.configureNext({
7994                 duration: e.duration || 280,
7995                 create: {
7996                     type: LayoutAnimation.Types.easeInEaseOut,
7997                     property: LayoutAnimation.Properties.opacity,
7998                 },
7999                 update: {
8000                     type: LayoutAnimation.Types.easeInEaseOut,
8001                 },
8002                 delete: {
8003                     type: LayoutAnimation.Types.easeInEaseOut,
8004                     property: LayoutAnimation.Properties.opacity,
8005                 },
8006             });
8007         } else {
8008             LayoutAnimation.configureNext(LayoutAnimation.Presets.easeInEaseOut);

```

```

8009     }
8010
8011     // measureInWindow 需要布局稳定，用 rAF 确保
8012     requestAnimationFrame(() => {
8013         measureAndScroll();
8014     });
8015 });
8016
8017 const hideSub = Keyboard.addListener(hideEvent, (e: KeyboardEvent) => {
8018     keyboardHeightRef.current = 0;
8019
8020     if (Platform.OS === 'ios') {
8021         LayoutAnimation.configureNext({
8022             duration: e.duration || 280,
8023             update: { type: LayoutAnimation.Types.easeInEaseOut },
8024         });
8025     } else {
8026         LayoutAnimation.configureNext(LayoutAnimation.Presets.easeInEaseOut);
8027     }
8028
8029     scrollViewRef.current?.scrollTo({ y: 0, animated: false });
8030 });
8031
8032 return () => {
8033     showSub.remove();
8034     hideSub.remove();
8035 };
8036 }, [measureAndScroll]);
8037
8038 return {
8039     scrollViewRef,
8040     handleInputFocus,
8041     handleInputBlur,
8042     onScroll,
8043 };
8044 };
8045
8046 === mobile/app/(tabs)/_layout.tsx ===
8047 import { Tabs } from 'expo-router';
8048 import { useTheme } from '../src/contexts/ThemeContext';
8049 import { Ionicons } from '@expo/vector-icons';
8050
8051 export default function TabLayout() {
8052     const { colors } = useTheme();
8053
8054     return (
8055         <Tabs
8056             screenOptions={({ route }) => ({
8057                 tabBarActiveTintColor: colors.primary,
8058                 tabBarInactiveTintColor: colors.textTertiary,

```

```

8058     headerShown: false,
8059     tabBarStyle: {
8060       backgroundColor: colors.tabBar,
8061       borderTopColor: colors.tabBarBorder,
8062       borderTopWidth: 1,
8063       paddingTop: 8,
8064       paddingBottom: 8,
8065       height: 64,
8066       elevation: 0,
8067       shadowColor: 'transparent',
8068     },
8069     tabBarLabelStyle: {
8070       fontSize: 11,
8071       fontWeight: '500',
8072     },
8073     tabBarIcon: ({ focused, color }) => {
8074       let iconName: keyof typeof Ionicons.glyphMap;
8075
8076       if (route.name === 'index') {
8077         iconName = focused ? 'map' : 'map-outline';
8078       } else if (route.name === 'monster') {
8079         iconName = focused ? 'sparkles' : 'sparkles-outline';
8080       } else if (route.name === 'profile') {
8081         iconName = focused ? 'person' : 'person-outline';
8082       } else {
8083         iconName = 'ellipse';
8084       }
8085
8086       return <Ionicons name={iconName} size={focused ? 24 : 20} color={color} />;
8087     },
8088   )))
8089 >
8090 <Tabs.Screen
8091   name="index"
8092   options={{
8093     title: '地图',
8094   }}
8095 />
8096 <Tabs.Screen
8097   name="monster"
8098   options={{
8099     title: '怪兽',
8100   }}
8101 />
8102 <Tabs.Screen
8103   name="profile"
8104   options={{
8105     title: '我的',
8106   }}
8107 />

```

```

8108   </Tabs>
8109 );
8110 }

    === mobile/app/(tabs)/index.tsx ===
8111 import React, { useState, useCallback, useMemo, useEffect } from 'react';
8112 import { View, Text, StyleSheet, ScrollView, TouchableOpacity, Alert } from 'react-native';
8113 import { SafeAreaView } from 'react-native-safe-area-context';
8114 import { router, useFocusEffect } from 'expo-router';
8115 import { Ionicons } from '@expo/vector-icons';
8116 import AsyncStorage from '@react-native-async-storage/async-storage';
8117 import { useTheme } from '../src/contexts/ThemeContext';
8118 import SubscriptionModal from '../src/components/SubscriptionModal';
8119 import { proService } from '../src/services/api';
8120 import { SUBSCRIPTION_STORAGE_KEY } from '../src/utils/pricing';
8121 import { getCurrentUser } from '../src/utils/auth';
8122
8123 interface Module {
8124   id: string;
8125   name: string;
8126   progress: number;
8127   totalNodes: number;
8128   completedNodes: number;
8129   isLocked: boolean;
8130   isCustom?: boolean;
8131 }
8132
8133 // 自定义模块调色板 — 像素莫兰迪版 (深色/浅色各一套)
8134 const CUSTOM_PALETTE_DARK = [
8135   { color: '#B56A6A', icon: 'flame' },
8136   { color: '#C4A27A', icon: 'sunny' },
8137   { color: '#6B9AA8', icon: 'water' },
8138   { color: '#6C5B7B', icon: 'flash' },
8139   { color: '#D48A6A', icon: 'bulb' },
8140   { color: '#7A9B6A', icon: 'leaf' },
8141   { color: '#9B7AA8', icon: 'diamond' },
8142   { color: '#5A8A98', icon: 'earth' },
8143   { color: '#C48AA0', icon: 'heart' },
8144   { color: '#7A8EB0', icon: 'compass' },
8145   { color: '#B8A060', icon: 'star' },
8146   { color: '#C07AC0', icon: 'sparkles' },
8147   { color: '#6A9B6A', icon: 'leaf' },
8148   { color: '#C87A5A', icon: 'bonfire' },
8149 ];
8150 const CUSTOM_PALETTE_LIGHT = [
8151   { color: '#C48A7A', icon: 'flame' },
8152   { color: '#D3B89F', icon: 'sunny' },
8153   { color: '#6A9AA8', icon: 'water' },
8154   { color: '#A3C4B5', icon: 'flash' },
8155   { color: '#D89A7A', icon: 'bulb' },
8156   { color: '#8FAA7D', icon: 'leaf' },

```

```

8157 { color: '#A58AB8', icon: 'diamond' },
8158 { color: '#6A9AA0', icon: 'earth' },
8159 { color: '#C49AAC', icon: 'heart' },
8160 { color: '#8AA8C0', icon: 'compass' },
8161 { color: '#C0A860', icon: 'star' },
8162 { color: '#C08AC0', icon: 'sparkles' },
8163 { color: '#7F9B89', icon: 'leaf' },
8164 { color: '#C87A5A', icon: 'bonfire' },
8165 ];
8166
8167 const staticStyles = StyleSheet.create({
8168   safeArea: { flex: 1 },
8169   container: { flex: 1 },
8170   scrollContent: { flexGrow: 1, paddingBottom: 20 },
8171   // 背景装饰
8172   decorCircleTL: {
8173     position: 'absolute', top: -40, left: -50, width: 160, height: 160,
8174     borderRadius: 80, opacity: 0.5,
8175   },
8176   decorCircleBR: {
8177     position: 'absolute', bottom: -60, right: -40, width: 180, height: 180,
8178     borderRadius: 90, opacity: 0.4,
8179   },
8180   decorDot1: {
8181     position: 'absolute', top: 120, right: 20, width: 4, height: 4, borderRadius: 2,
8182   },
8183   decorDot2: {
8184     position: 'absolute', top: 280, right: 45, width: 3, height: 3, borderRadius: 1.5,
8185   },
8186   header: {
8187     paddingHorizontal: 24, paddingTop: 20, paddingBottom: 24,
8188     flexDirection: 'row', justifyContent: 'space-between', alignItems: 'flex-start',
8189   },
8190   modulesGrid: { paddingHorizontal: 24, gap: 12 },
8191   moduleCard: {
8192     borderRadius: 16, padding: 16, position: 'relative', overflow: 'hidden',
8193   },
8194   decorationCircle: {
8195     position: 'absolute', top: -20, right: -10, width: 100, height: 100, borderRadius: 50,
8196   },
8197   darkGlow: {
8198     position: 'absolute', top: -30, right: -20, width: 120, height: 120, borderRadius: 60,
8199   },
8200   moduleCardContent: { position: 'relative', zIndex: 1 },
8201   moduleRow: { flexDirection: 'row', alignItems: 'center', gap: 12, marginBottom: 12 },
8202   moduleIcon: {
8203     width: 48, height: 48, borderRadius: 10,
8204     alignItems: 'center', justifyContent: 'center', flexShrink: 0,
8205   },
8206   moduleInfo: { flex: 1 },

```

```

8207 moduleProgressBar: { width: '100%', height: 6, borderRadius: 9999, overflow: 'hidden' },
8208 moduleProgressFill: { height: '100%', borderRadius: 9999 },
8209 addModuleCard: {
8210   flexDirection: 'row', alignItems: 'center', borderRadius: 16,
8211   borderWidth: 2, borderStyle: 'dashed', padding: 16, gap: 16,
8212 },
8213 addModuleIcon: {
8214   width: 52, height: 52, borderRadius: 10,
8215   alignItems: 'center', justifyContent: 'center', flexShrink: 0,
8216 },
8217 section: { paddingHorizontal: 24, marginTop: 32 },
8218 tipsContainer: { gap: 12 },
8219 tipCard: {
8220   flexDirection: 'row', alignItems: 'center', borderRadius: 14, padding: 16, gap: 12,
8221 },
8222 bottomPadding: { height: 32 },
8223 });
8224
8225 const MapScreen = () => {
8226   const { isDark, colors } = useTheme();
8227   const [modules, setModules] = useState<Module[]>([]);
8228   const [showProModal, setShowProModal] = useState(false);
8229   const [isPro, setIsPro] = useState(false);
8230
8231   useEffect(() => {
8232     (async () => {
8233       try {
8234         const user = await getCurrentUser();
8235         if (user?.id) {
8236           const status = await proService.getStatus(user.id);
8237           setIsPro(status.isPro);
8238         }
8239       } catch {}
8240     })();
8241   }, []);
8242
8243   // 动态样式
8244   const dynamicStyles = useMemo(() => ({
8245     container: { flex: 1, backgroundColor: colors.background },
8246     greeting: { fontSize: 28, fontWeight: '600' as const, color: colors.textPrimary, marginBottom: 6 },
8247     subtitle: { fontSize: 15, color: colors.textSecondary },
8248     moduleName: { fontSize: 17, fontWeight: '600' as const, color: colors.textPrimary },
8249     moduleCategory: { fontSize: 13, color: colors.textSecondary, marginTop: 2 },
8250     progressBadge: {
8251       paddingHorizontal: 10, paddingVertical: 4, borderRadius: 6,
8252       borderWidth: 1, borderColor: colors.badgeBorder, backgroundColor: colors.badgeBg,
8253     },
8254     progressBadgeText: { fontSize: 12, fontWeight: '600' as const, color: colors.badgeText },
8255     addModuleCard: {
8256       backgroundColor: colors.cardAdd, borderColor: colors.border,

```

```

8257     shadowColor: colors.shadow, shadowOffset: { width: 0, height: 1 },
8258     shadowOpacity: 0.04, shadowRadius: 3, elevation: 2,
8259   },
8260   addModuleIcon: { backgroundColor: colors.borderLight },
8261   addModuleName: { fontSize: 17, fontWeight: '600' as const, color: colors.textPrimary, marginBottom: 4 },
8262   addModuleHint: { fontSize: 13, color: colors.textSecondary },
8263   sectionTitle: { fontSize: 15, fontWeight: '600' as const, color: colors.textSecondary, marginBottom: 16 },
8264   tipCard: {
8265     backgroundColor: colors.tipBg, borderWidth: 1, borderColor: colors.hairline,
8266     shadowColor: colors.shadow, shadowOffset: { width: 0, height: 1 },
8267     shadowOpacity: 0.04, shadowRadius: 3, elevation: 2,
8268   },
8269   tipText: { fontSize: 15, color: colors.textPrimary, lineHeight: 22 },
8270 )), [colors]);
8271
8272 const customPalette = isDark ? CUSTOM_PALETTE_DARK : CUSTOM_PALETTE_LIGHT;
8273
8274 const moduleConfigs = useMemo(() => ({
8275   'ai-product-manager': {
8276     icon: 'hardware-chip' as const, color: colors.primary, cardBg: colors.cardAi,
8277     category: '专业技能', badgeBorder: colors.badgeBorderAi,
8278     progressFill: isDark ? colors.progressFillAi : colors.progressFill,
8279   },
8280   'personal-finance': {
8281     icon: 'trending-up' as const, color: colors.success, cardBg: colors.cardLife,
8282     category: '生活技能', badgeBorder: colors.badgeBorderLife,
8283     progressFill: isDark ? colors.progressFillLife : colors.progressFill,
8284   },
8285   'english-communication': {
8286     icon: 'globe' as const, color: colors.orange, cardBg: colors.cardLang,
8287     category: '语言学习', badgeBorder: colors.badgeBorderLang,
8288     progressFill: isDark ? colors.progressFillLang : colors.accentOrange,
8289   },
8290   'programming-basics': {
8291     icon: 'code-slash' as const, color: colors.primary, cardBg: colors.cardAi,
8292     category: '专业技能', badgeBorder: colors.badgeBorderAi,
8293     progressFill: isDark ? colors.progressFillAi : colors.progressFill,
8294   },
8295   'finance-basics': {
8296     icon: 'wallet' as const, color: colors.success, cardBg: colors.cardLife,
8297     category: '生活技能', badgeBorder: colors.badgeBorderLife,
8298     progressFill: isDark ? colors.progressFillLife : colors.progressFill,
8299   },
8300   'cet-exam': {
8301     icon: 'school' as const, color: colors.orange, cardBg: colors.cardLang,
8302     category: '语言学习', badgeBorder: colors.badgeBorderLang,
8303     progressFill: isDark ? colors.progressFillLang : colors.accentOrange,
8304   },
8305 )), [colors, isDark]);
8306

```

```

8307 const loadModules = useCallback(async () => {
8308   try {
8309     console.log('[Map] 开始加载地图数据, isDark=', isDark);
8310     const selectedModules = await AsyncStorage.getItem('selectedModules');
8311     const customModulesStr = await AsyncStorage.getItem('customModules');
8312     const customModules: Record<string, { name: string }> = customModulesStr ? JSON.parse(customModulesStr)
8313
8314     const presetNames: Record<string, string> = {
8315       'ai-product-manager': 'AI产品经理',
8316       'personal-finance': '个人理财',
8317       'english-communication': '英语沟通',
8318       'programming-basics': '编程基础',
8319       'finance-basics': '理财入门',
8320       'cet-exam': '四六级过关',
8321     };
8322
8323     if (selectedModules) {
8324       const selected = JSON.parse(selectedModules) as string[];
8325       const loadedModules = selected.map((id) => {
8326         if (presetNames[id]) {
8327           return { id, name: presetNames[id], progress: 0, totalNodes: 9, completedNodes: 0, isLocked: false
8328         }
8329         const custom = customModules[id];
8330         if (custom) {
8331           return { id, name: custom.name, progress: 0, totalNodes: 9, completedNodes: 0, isLocked: false, is
8332         }
8333         return null;
8334       }).filter(Boolean) as Module[];
8335       if (loadedModules.length > 0) {
8336         console.log('[Map] 加载模块:', loadedModules.map(m => m.name));
8337         setModules(loadedModules);
8338         return;
8339       }
8340     }
8341     console.log('[Map] 使用默认模块');
8342     setModules([
      { id: 'ai-product-manager', name: 'AI产品经理', progress: 0, totalNodes: 9, completedNodes: 0, i
8343   } catch (error) {
8344     console.error('[Map] 加载数据失败:', error);
8345     setModules([
      { id: 'ai-product-manager', name: 'AI产品经理', progress: 0, totalNodes: 9, completedNodes: 0, i
8346   }
8347   ], []);
8348
8349   useFocusEffect(useCallback(() => { loadModules(); }, [loadModules]));
8350
8351   const handleModulePress = (moduleId: string) => {
8352     console.log('[Map] 点击模块:', moduleId);
8353     router.push({ pathname: '/skill-tree', params: { domain: moduleId } });
8354   };
8355
8356   const handleModuleLongPress = async (moduleId: string, moduleName: string) => {

```



```

8357   Alert.alert(
8358     '删除模块',
8359     `确定要删除「${moduleName}」吗? \n学习进度将被清除。`,
8360     [
8361       { text: '取消', style: 'cancel' },
8362       {
8363         text: '删除', style: 'destructive',
8364         onPress: async () => {
8365           const stored = await AsyncStorage.getItem('selectedModules');
8366           if (stored) {
8367             const selected = JSON.parse(stored) as string[];
8368             const updated = selected.filter((id: string) => id !== moduleId);
8369             await AsyncStorage.setItem('selectedModules', JSON.stringify(updated));
8370           }
8371           const cm = await AsyncStorage.getItem('customModules');
8372           if (cm) {
8373             const parsed = JSON.parse(cm);
8374             delete parsed[moduleId];
8375             await AsyncStorage.setItem('customModules', JSON.stringify(parsed));
8376           }
8377           await AsyncStorage.multiRemove(['customStages_${moduleId}', 'customNodes_${moduleId}']);
8378           try {
8379             const { domainService } = await import('../src/services/api');
8380             await domainService.deleteDomain(moduleId);
8381           } catch (e) {
8382             console.warn('[Map] 删除服务端领域失败:', e);
8383           }
8384           setModules(prev => prev.filter(m => m.id !== moduleId));
8385         },
8386       ],
8387     ]
8388   );
8389 };
8390
8391 return (
8392   <SafeAreaView style={[staticStyles.safeArea, dynamicStyles.container]} edges={['top']}>
8393     { /* 背景装饰 */ }
8394     <View style={{ position: 'absolute', top: 0, left: 0, right: 0, bottom: 0, overflow: 'hidden', pointerEv
8395       <View style={[staticStyles.decorCircleTL, { backgroundColor: colors.decorCircle }} />
8396       <View style={[staticStyles.decorCircleBR, { backgroundColor: colors.decorCircle }} />
8397       <View style={[staticStyles.decorDot1, { backgroundColor: isDark ? '#5A6AAA' : '#8BA892', opacity: 0.6
8398       <View style={[staticStyles.decorDot2, { backgroundColor: isDark ? '#3A7060' : '#C49A6C', opacity: 0.4
8399     </View>
8400     <ScrollView contentContainerStyle={staticStyles.scrollContent} showsVerticalScrollIndicator={false}>
8401       { /* 顶部标题栏 */ }
8402       <View style={staticStyles.header}>
8403         <View>
8404           <Text style={dynamicStyles.greeting}>学习地图</Text>
8405           <Text style={dynamicStyles.subtitle}>选择一个模块开始学习</Text>
8406         </View>

```

```

8407 <TouchableOpacity
8408   onPress={() => { if (!isPro) setShowProModal(true); }}
8409   activeOpacity={isPro ? 1 : 0.7}
8410   style={{
8411     flexDirection: 'row', alignItems: 'center', gap: 4,
8412     paddingHorizontal: 10, paddingVertical: 6,
8413     backgroundColor: isPro ? colors.pro : colors.proBg,
8414     borderWidth: 1.5, borderColor: colors.proBorder,
8415     borderRadius: 2,
8416   }}
8417 >
8418   <Ionicons name="diamond" size={14} color={isPro ? '#FFFFFF' : colors.pro} />
8419   <Text style={{ fontSize: 11, fontWeight: '700' as const, color: isPro ? '#FFFFFF' : colors.pro, fo
8420 </TouchableOpacity>
8421 </View>
8422
8423 {/* 模块列表 */}
8424 <View style={staticStyles.modulesGrid}>
8425   {modules.map((module, index) => {
8426     const config = moduleConfigs[module.id];
8427     const paletteItem = customPalette[index % customPalette.length];
8428     const icon = config?.icon ?? paletteItem.icon;
8429     const color = config?.color ?? paletteItem.color;
8430     const cardBg = config?.cardBg ?? colors.card;
8431     const category = config?.category ?? '';
8432     return (
8433       <TouchableOpacity
8434         key={module.id}
8435         style={{
8436           staticStyles.moduleCard,
8437           { backgroundColor: cardBg },
8438           { borderWidth: 1, borderColor: colors.hairline },
8439           { shadowColor: colors.shadow, shadowOffset: { width: 0, height: 1 }, shadowOpacity: 0.04, sh
8440         }}
8441         onPress={() => handleModulePress(module.id)}
8442         onLongPress={() => handleModuleLongPress(module.id, module.name)}
8443         activeOpacity={0.8}
8444       >
8445         {isDark && <View style={[staticStyles.darkGlow, { backgroundColor: color + '10' }] } />}
8446         {!isDark && <View style={[staticStyles.decorationCircle, { backgroundColor: color + '14' }] } /
8447       <View style={staticStyles.moduleCardContent}>
8448         <View style={staticStyles.moduleRow}>
8449           <View style={[staticStyles.moduleIcon, { backgroundColor: color, borderWidth: 0, borderRad
8450             <Ionicons name={icon as any} size={22} color="#FFFFFF" />
8451           </View>
8452           <View style={staticStyles.moduleInfo}>
8453             <Text style={dynamicStyles.moduleName}>{module.name}</Text>
8454             {category ? <Text style={dynamicStyles.moduleCategory}>{category}</Text> : null}
8455           </View>
8456           <View style={[dynamicStyles.progressBadge, { borderColor: config?.badgeBorder ?? colors.ba

```

```

8457         <Text style={dynamicStyles.progressBarText}>{module.progress}%</Text>
8458     </View>
8459 </View>
8460 <View style={[staticStyles.moduleProgressBar, { backgroundColor: colors.progressTrack }]}>
8461     <View style={[staticStyles.moduleProgressFill, { width: `${module.progress}%`, backgroundC
8462 </View>
8463 </View>
8464 </TouchableOpacity>
8465 );
8466 ]}]
8467
8468 {/* 添加模块 */}
8469 <TouchableOpacity
8470     style={[staticStyles.addModuleCard, dynamicStyles.addModuleCard]}
8471     onPress={() => router.push('/module-selection?mode=add')}
8472     activeOpacity={0.8}
8473 >
8474     <View style={[staticStyles.addModuleIcon, dynamicStyles.addModuleIcon]}>
8475         <Ionicons name="add" size={28} color={colors.textSecondary} />
8476     </View>
8477     <View style={staticStyles.moduleInfo>
8478         <Text style={dynamicStyles.addModuleName}>添加模块</Text>
8479         <Text style={dynamicStyles.addModuleHint}>选择预设模块或创建自定义领域</Text>
8480     </View>
8481     <Ionicons name="chevron-forward" size={20} color={colors.textTertiary} />
8482 </TouchableOpacity>
8483 </View>
8484
8485 {/* 学习提示 */}
8486 <View style={staticStyles.section}>
8487     <Text style={dynamicStyles.sectionTitle}>学习提示</Text>
8488     <View style={staticStyles.tipsContainer}>
8489         {[
8490             { icon: 'checkmark-circle' as const, color: colors.accentGreen, text: '完成每个节点的学习任务' },
8491             { icon: 'flash' as const, color: colors.accentOrange, text: '消耗体力获取知识能量' },
8492             { icon: 'help-circle' as const, color: colors.textTertiary, text: '玩游戏恢复体力' },
8493         ].map((tip, i) => (
8494             <TouchableOpacity key={i} style={[staticStyles.tipCard, dynamicStyles.tipCard]} activeOpacity={0
8495                 <Ionicons name={tip.icon} size={18} color={tip.color} />
8496                 <Text style={dynamicStyles.tipText}>{tip.text}</Text>
8497             </TouchableOpacity>
8498         )))
8499     </View>
8500 </View>
8501
8502 <View style={staticStyles.bottomPadding} />
8503 </ScrollView>
8504
8505 <SubscriptionModal visible={showProModal} onClose={() => setShowProModal(false)} onProActivated={() => s
8506 </SafeAreaView>

```

```

8507   });
8508 };
8509
8510 export default MapScreen;
8511
8512 === mobile/app/(tabs)/monster.tsx ===
8512 import React, { useState, useCallback, useMemo, useRef, useEffect } from 'react';
8513 import { View, Text, StyleSheet, ScrollView, TouchableOpacity, TextInput, Alert, Modal, KeyboardAvoidingView,
8514 import { useTheme } from '../src/contexts/ThemeContext';
8515 import { SafeAreaView } from 'react-native-safe-area-context';
8516 import { Ionicons } from '@expo/vector-icons';
8517 import AsyncStorage from '@react-native-async-storage/async-storage';
8518 import MonsterIcon from '../src/components/MonsterIcon';
8519 import MiniGames from '../src/components/MiniGames';
8520 import SubscriptionModal from '../src/components/SubscriptionModal';
8521 import { storage, STORAGE_KEYS } from '../src/utls/storage';
8522 import { MONSTER_CONFIG } from '../src/utls/constants';
8523 import { formatTimer } from '../src/utls/helpers';
8524 import { useFocusEffect, router } from 'expo-router';
8525 import { noteService, rewardService, monsterService } from '../src/services/api';
8526 import { proService } from '../src/services/api';
8527 import { getCurrentUser } from '../src/utls/auth';
8528 import { SUBSCRIPTION_STORAGE_KEY } from '../src/utls/pricing';
8529
8530 type ActiveTab = 'tasks' | 'notes' | 'chat';
8531 type MonsterMessageItem = { id: number; userId: number; message: string; isUser: boolean; createdAt: string; e
8532
8533 const staticStyles = StyleSheet.create({
8534   safeArea: { flex: 1 },
8535   container: { flex: 1 },
8536   scrollContent: { flexGrow: 1 },
8537   loadingContainer: { flex: 1, alignItems: 'center', justifyContent: 'center' },
8538   loadingText: { fontSize: 14 },
8539   header: { paddingHorizontal: 24, paddingTop: 20, paddingBottom: 20 },
8540   pixelBackground: { position: 'absolute', top: 0, left: 0, right: 0, bottom: 0, opacity: 0.95 },
8541   headerContent: { position: 'relative', zIndex: 1 },
8542   title: { fontSize: 28, fontWeight: '600', marginBottom: 4 }, subtitle: { fontSize: 15, marginBottom: 20 },
8543   monsterCard: {
8544     borderRadius: 24, padding: 24, position: 'relative', overflow: 'hidden',
8545     borderWidth: 1, shadowColor: '#000', shadowOffset: { width: 0, height: 1 },
8546     shadowOpacity: 0.04, shadowRadius: 3, elevation: 2,
8547   },
8548   monsterPixelPattern: { position: 'absolute', top: 0, left: 0, right: 0, bottom: 0, opacity: 0.05 },
8549   monsterCardContent: { position: 'relative', zIndex: 1 },
8550   monsterTopRow: { flexDirection: 'row', alignItems: 'center', gap: 16, marginBottom: 16 },
8551   monsterActionRow: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', gap: 12, ma
8552   monsterIconContainer: {
8553     width: 108, height: 108, alignItems: 'center', justifyContent: 'center',
8554     borderRadius: 24, borderWidth: 2, flexShrink: 0,
8555   },

```

```

8556 monsterInfo: { flex: 1, justifyContent: 'center', gap: 8 },
8557 gameButton: {
8558   paddingVertical: 8, paddingHorizontal: 12, borderRadius: 10,
8559   borderWidth: 2, flexDirection: 'row', alignItems: 'center', gap: 4,
8560   flex: 1, shadowColor: '#000', shadowOffset: { width: 0, height: 1 },
8561   shadowOpacity: 0.04, shadowRadius: 3, elevation: 2,
8562 },
8563 gameButtonDisabled: { opacity: 0.5 },
8564 gameButtonContent: { flexDirection: 'column', gap: 2, alignItems: 'flex-start', flex: 1 },
8565 gameButtonText: { fontSize: 12, fontWeight: '600' },
8566 gameButtonSubText: { fontSize: 10 },
8567 infoButton: {
8568   width: 36, height: 36, borderRadius: 10, alignItems: 'center', justifyContent: 'center',
8569   borderWidth: 2, flexShrink: 0,
8570 },
8571 infoCard: { marginBottom: 16, padding: 16, borderRadius: 14, borderWidth: 2 },
8572 infoItem: { marginBottom: 12 },
8573 infoTitle: { fontSize: 13, fontWeight: '600', marginBottom: 6 },
8574 infoText: { fontSize: 11, lineHeight: 16 },
8575 monsterNameRow: { flexDirection: 'row', alignItems: 'center', gap: 8, marginBottom: 4, flexWrap: 'wrap' },
8576 monsterName: { fontSize: 18, fontWeight: '600', flexShrink: 1 },
8577 monsterPersonality: { fontSize: 12, lineHeight: 18 },
8578 statsRow: { gap: 12 },
8579 statContainer: { gap: 6 },
8580 statHeader: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between' },
8581 statLabelRow: { flexDirection: 'row', alignItems: 'center', gap: 6 },
8582 statLabel: { fontSize: 12 },
8583 statValue: { fontSize: 12, fontWeight: '600' },
8584 statBar: { height: 8, borderRadius: 0, overflow: 'hidden' },
8585 statBarFill: { height: '100%', borderRadius: 0 },
8586 tabsContainer: { paddingHorizontal: 20, marginBottom: 16 },
8587 tabs: { flexDirection: 'row', gap: 8, padding: 4, borderRadius: 12, borderWidth: 1 },
8588 tab: {
8589   flex: 1, flexDirection: 'row', paddingVertical: 10, borderRadius: 10,
8590   alignItems: 'center', justifyContent: 'center', gap: 6, borderWidth: 1,
8591 },
8592 tabText: { fontSize: 13 },
8593 tabContent: { paddingHorizontal: 20, gap: 16 },
8594 taskCard: {
8595   borderRadius: 14, padding: 20, borderWidth: 1.5,
8596   shadowColor: '#000', shadowOffset: { width: 0, height: 0 },
8597   shadowOpacity: 0.4, shadowRadius: 12, elevation: 4,
8598 },
8599 taskHeader: { flexDirection: 'row', alignItems: 'center', gap: 8, marginBottom: 20 },
8600 taskTitle: { fontSize: 14, fontWeight: '600', letterSpacing: 0.5 },
8601 addTaskContainer: { flexDirection: 'row', gap: 8, marginBottom: 16 },
8602 addTaskInput: {
8603   flex: 1, paddingHorizontal: 12, paddingVertical: 10, borderRadius: 10,
8604   borderWidth: 1.5, fontSize: 14,
8605 },

```

```

8606 addTaskButton: { width: 44, height: 44, borderRadius: 10, alignItems: 'center', justifyContent: 'center' },
8607 taskList: { gap: 8 },
8608 emptyTasks: { paddingVertical: 24, alignItems: 'center' },
8609 emptyTasksText: { fontSize: 13 },
8610 taskItem: { flexDirection: 'row', alignItems: 'center', gap: 10, paddingVertical: 8, paddingHorizontal: 4 },
8611 taskCheckbox: {
8612   flexShrink: 0, width: 20, height: 20, borderRadius: 6, borderWidth: 1.5,
8613   alignItems: 'center', justifyContent: 'center', backgroundColor: 'transparent',
8614 },
8615 taskCheckboxDone: { borderColor: 'transparent' },
8616 taskCheckboxInner: { width: 12, height: 12, backgroundColor: '#FFFFFF', borderRadius: 0 },
8617 taskName: { flex: 1, flexShrink: 1, fontSize: 13, lineHeight: 18, flexWrap: 'wrap' },
8618 taskNameDone: { textDecorationLine: 'line-through' },
8619 editTaskInput: {
8620   flex: 1, flexShrink: 1, fontSize: 13, lineHeight: 18, paddingVertical: 2,
8621   paddingHorizontal: 6, borderRadius: 6, borderWidth: 1,
8622 },
8623 editActions: { flexDirection: 'row', flexShrink: 0, gap: 4 },
8624 deleteTaskButton: { flexShrink: 0, padding: 4 },
8625 editTaskButton: { flexShrink: 0, padding: 4 },
8626 timeOptionsContainer: { flexDirection: 'row', flexWrap: 'wrap', gap: 8, marginBottom: 20 },
8627 timeOption: { paddingHorizontal: 12, paddingVertical: 6, borderRadius: 6 },
8628 timeOptionText: { fontSize: 12, fontWeight: '600' },
8629 pomodoroContainer: { alignItems: 'center', gap: 16 },
8630 pomodoroTimer: {
8631   width: 140, height: 140, borderRadius: 70, borderWidth: 1,
8632   alignItems: 'center', justifyContent: 'center',
8633   shadowColor: '#000', shadowOffset: { width: 0, height: 2 },
8634   shadowOpacity: 0.08, shadowRadius: 8, elevation: 3,
8635 },
8636 pomodoroTime: { fontSize: 36, fontWeight: '700' },
8637 pomodoroButtons: { width: '100%' },
8638 pomodoroButton: {
8639   flexDirection: 'row', alignItems: 'center', justifyContent: 'center', gap: 8,
8640   paddingVertical: 12, paddingHorizontal: 24, borderRadius: 10, borderWidth: 1,
8641   shadowColor: '#000', shadowOffset: { width: 0, height: 2 },
8642   shadowOpacity: 0.08, shadowRadius: 8, elevation: 3,
8643 },
8644 pomodoroButtonText: { fontSize: 14, fontWeight: '600' },
8645 noteCard: {
8646   borderRadius: 14, padding: 16, borderWidth: 1,
8647   shadowColor: '#000', shadowOffset: { width: 0, height: 1 },
8648   shadowOpacity: 0.04, shadowRadius: 3, elevation: 2,
8649 },
8650 noteHeader: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', marginBottom: 12 },
8651 noteTitle: { fontSize: 14, fontWeight: '600' },
8652 noteInput: {
8653   width: '100%', paddingHorizontal: 16, paddingVertical: 12, borderRadius: 10,
8654   borderWidth: 2, fontSize: 14, minHeight: 100,
8655 },

```

```

8656 saveNoteButton: { width: '100%', marginTop: 12, paddingVertical: 10, borderRadius: 10, borderWidth: 1 },
8657 saveNoteText: { color: '#FFFFFF', fontSize: 14, fontWeight: '600', textAlign: 'center' },
8658 notesHistory: { marginTop: 16 },
8659 historyTitle: { fontSize: 12, marginBottom: 12 },
8660 emptyNotes: {
8661   borderRadius: 14, padding: 24, alignItems: 'center',
8662   borderWidth: 1, borderStyle: 'dashed',
8663 },
8664 emptyNotesText: { fontSize: 13 },
8665 notesList: { gap: 12 },
8666 savedNote: { borderRadius: 14, padding: 16, borderWidth: 1 },
8667 savedNoteContent: { fontSize: 13, lineHeight: 20, marginBottom: 8 },
8668 savedNoteDate: { fontSize: 11 },
8669 chatCard: { borderRadius: 14, padding: 24, alignItems: 'center', borderWidth: 1 },
8670 chatMonsterIcon: { marginBottom: 16 },
8671 chatTitle: { fontSize: 16, fontWeight: '600', marginBottom: 8 },
8672 chatDescription: { fontSize: 13, lineHeight: 20, textAlign: 'center' },
8673 // 聊天列表
8674 chatList: { flex: 1 },
8675 chatListContent: { padding: 16, flexGrow: 1 },
8676 chatEmpty: { flex: 1, alignItems: 'center', justifyContent: 'center', paddingVertical: 60 },
8677 chatEmptyText: { fontSize: 16, fontWeight: '600', marginTop: 16 },
8678 chatEmptyHint: { fontSize: 13, marginTop: 8 },
8679 // 消息气泡
8680 messageBubble: { flexDirection: 'row', marginBottom: 12, maxWidth: '80%' },
8681 messageUser: { alignSelf: 'flex-end', justifyContent: 'flex-end' },
8682 messageMonster: { alignSelf: 'flex-start' },
8683 messageAvatar: { marginRight: 8, alignSelf: 'flex-end' },
8684 messageContent: { borderRadius: 18, paddingHorizontal: 14, paddingVertical: 10 },
8685 messageContentUser: { borderBottomRightRadius: 4 },
8686 messageContentMonster: { borderBottomLeftRadius: 4, borderWidth: 1 },
8687 messageText: { fontSize: 14, lineHeight: 20 },
8688 energyCostText: { fontSize: 11, textAlign: 'right', marginTop: 2, paddingRight: 4 },
8689 // 输入栏
8690 chatInputBar: {
8691   flexDirection: 'row', alignItems: 'flex-end', padding: 12,
8692   borderTopWidth: 1,
8693 },
8694 chatTextInput: {
8695   flex: 1, borderWidth: 1, borderRadius: 10,
8696   paddingHorizontal: 16, paddingTop: 10, paddingBottom: 10,
8697   maxHeight: 100, fontSize: 14,
8698 },
8699 sendButton: { width: 40, height: 40, borderRadius: 20, alignItems: 'center', justifyContent: 'center', margin: 0 10 },
8700 sendButtonDisabled: { opacity: 0.5 },
8701 proBadge: { marginTop: 16, paddingHorizontal: 12, paddingVertical: 6, borderRadius: 6 },
8702 proText: { fontSize: 11, fontWeight: '600' },
8703 bottomPadding: { height: 100 },
8704 modalFullScreen: { flex: 1 },
8705 statBarContainer: { marginBottom: 4 },

```

```

8706   monsterNameContainer: { flex: 1 },
8707   monsterStats: { marginTop: 8 },
8708   statItem: { gap: 4 },
8709 });
8710
8711 const MonsterManageScreen = () => {
8712   const { colors, isDark } = useTheme();
8713
8714   const dynamicStyles = useMemo(() => ({
8715     container: { flex: 1, backgroundColor: colors.background },
8716     loadingText: { color: colors.textSecondary },
8717     title: { color: colors.textPrimary, fontSize: 28, fontWeight: '600' as const, marginBottom: 6 },
8718     subtitle: { color: colors.textSecondary },
8719     monsterCard: { backgroundColor: colors.surface, borderColor: colors.hairline },
8720     monsterIconContainer: { backgroundColor: colors.borderLight, borderColor: colors.borderDark },
8721     gameButton: { backgroundColor: colors.borderLight, borderColor: colors.borderDark },
8722     infoButton: { backgroundColor: colors.borderLight, borderColor: colors.border },
8723     infoCard: { backgroundColor: colors.borderLight, borderColor: colors.borderDark },
8724     infoTitle: { color: colors.textPrimary },
8725     infoText: { color: colors.textSecondary },
8726     monsterName: { color: colors.textPrimary },
8727     monsterPersonality: { color: colors.textSecondary },
8728     statLabel: { color: colors.textSecondary },
8729     statBar: { backgroundColor: colors.border },
8730     tabs: { backgroundColor: colors.surface, borderColor: colors.borderLight },
8731     tab: { borderColor: colors.borderLight },
8732     taskCard: { backgroundColor: colors.surface, borderColor: colors.borderDark },
8733     taskTitle: { color: colors.textPrimary },
8734     addTaskInput: { backgroundColor: colors.inputBg, borderColor: colors.borderDark, color: colors.textPrimary },
8735     addTaskButton: { backgroundColor: colors.primary },
8736     taskCheckbox: { borderColor: colors.primary },
8737     taskCheckboxDone: { backgroundColor: colors.primary, borderColor: colors.primary },
8738     taskName: { color: colors.textPrimary },
8739     taskNameDone: { color: colors.primary },
8740     timeOptionText: { color: colors.textSecondary },
8741     pomodoroTimer: { borderColor: colors.primary, backgroundColor: colors.primary },
8742     pomodoroTime: { color: colors.onPrimary },
8743     pomodoroButton: { backgroundColor: colors.primary, borderColor: colors.primary },
8744     pomodoroButtonText: { color: colors.onPrimary },
8745     noteCard: { backgroundColor: colors.surface, borderColor: colors.hairline },
8746     noteTitle: { color: colors.textPrimary },
8747     noteInput: { backgroundColor: colors.inputBg, borderColor: colors.borderDark, color: colors.textPrimary },
8748     saveNoteButton: { backgroundColor: colors.primary, borderColor: colors.primary },
8749     historyTitle: { color: colors.textSecondary },
8750     emptyNotes: { backgroundColor: colors.borderLight, borderColor: colors.border },
8751     emptyNotesText: { color: colors.textTertiary },
8752     savedNote: { backgroundColor: colors.surface, borderColor: colors.borderLight },
8753     savedNoteContent: { color: colors.textPrimary },
8754     savedNoteDate: { color: colors.textSecondary },
8755     chatCard: { backgroundColor: colors.surface, borderColor: colors.borderDark },

```



```

8756   chatTitle: { color: colors.textPrimary },
8757   chatDescription: { color: colors.textSecondary },
8758   chatEmptyText: { color: colors.textPrimary },
8759   chatEmptyHint: { color: colors.textSecondary },
8760   messageContentMonster: { backgroundColor: colors.surfaceLight, borderColor: colors.borderLight },
8761   messageText: { color: colors.textPrimary },
8762   energyCostText: { color: colors.textTertiary },
8763   chatInputBar: { backgroundColor: colors.surface, borderTopColor: colors.borderLight },
8764   chatTextInput: { backgroundColor: colors.inputBg, borderColor: colors.inputBorder, color: colors.textPrima
8765   sendButton: { backgroundColor: colors.primary },
8766   proBadge: { backgroundColor: colors.borderLight },
8767   proText: { color: colors.warning },
8768   pixelBackground: { backgroundColor: colors.backgroundDark },
8769   statBarContainer: { backgroundColor: 'rgba(255,255,255,0.06)' },
8770   )), [colors]);
8771
8772   const [monsterData, setMonsterData] = useState<any>(null);
8773   const [activeTab, setActiveTab] = useState<ActiveTab>('tasks');
8774   const [notes, setNotes] = useState('');
8775   const [savedNotes, setSavedNotes] = useState<any[]>([]);
8776   const [showInfo, setShowInfo] = useState(false);
8777   const [showProModal, setShowProModal] = useState(false);
8778   const [tasks, setTasks] = useState<any[]>([]);
8779   const [newTaskText, setNewTaskText] = useState('');
8780   const [editingTaskId, setEditingTaskId] = useState<number | null>(null);
8781   const [editTaskText, setEditTaskText] = useState('');
8782   const [selectedTime, setSelectedTime] = useState(MONSTER_CONFIG.POMODORO.TIME_OPTIONS[0]);
8783   const [showGameModal, setShowGameModal] = useState(false);
8784   const [showChatFullscreen, setShowChatFullscreen] = useState(false);
8785   const [dailyPlays, setDailyPlays] = useState(0);
8786
8787   // 聊天相关
8788   const [chatMessages, setChatMessages] = useState<MonsterMessageItem[]>([]);
8789   const [chatInput, setChatInput] = useState('');
8790   const [isSending, setIsSending] = useState(false);
8791   const [showChatInput, setShowChatInput] = useState(false);
8792   const [isPro, setIsPro] = useState(false);
8793   const flatListRef = useRef<FlatList>(null);
8794
8795   useEffect(
8796     useCallback(() => {
8797       loadData();
8798       loadChatMessages();
8799       checkProStatus();
8800     }, [])
8801   );
8802
8803   useEffect(() => {
8804     loadData();
8805     loadChatMessages();

```

```

8806   checkProStatus();
8807 }, []);
8808
8809 // 检查 Pro 状态 (以数据库 is_pro 字段为准)
8810 const checkProStatus = async () => {
8811   try {
8812     const user = await getCurrentUser();
8813     if (user?.id) {
8814       const status = await proService.getStatus(user.id);
8815       setIsPro(status.isPro);
8816     }
8817   } catch {}
8818 };
8819
8820 const pomodoroTimeLeft = selectedTime * 60;
8821
8822 // Pro 激活时更新怪兽上限
8823 useEffect(() => {
8824   if (!isPro || !monsterData) return;
8825   const proStaminaMax = MONSTER_CONFIG.STAMINA.PRO_MAX;
8826   const proEnergyDaily = MONSTER_CONFIG.ENERGY.DAILY_PRO;
8827   // 如果已经是 Pro 上限, 则跳过
8828   if (monsterData.maxStamina === proStaminaMax && monsterData.maxPaiEnergy === proEnergyDaily) return;
8829
8830   // 体力: 提升到 Pro 上限, 并补满差值
8831   const stamina = Math.min(monsterData.stamina + (proStaminaMax - monsterData.maxStamina), proStaminaMax);
8832   // 能量: 没有上限, 但给 Pro 用户一次性奖励 (补足到至少 DAILY_PRO), 并移除上限
8833   const energyBonus = proEnergyDaily - (monsterData.maxPaiEnergy || MONSTER_CONFIG.ENERGY.DAILY_FREE);
8834   const paiEnergy = Math.max(monsterData.paiEnergy, proEnergyDaily) + Math.max(energyBonus, 0);
8835   const updated = {
8836     ...monsterData,
8837     stamina,
8838     maxStamina: proStaminaMax,
8839     paiEnergy,
8840     // 移除能量上限, 设为极大值表示无限
8841     maxPaiEnergy: paiEnergy,
8842   };
8843   setMonsterData(updated);
8844   storage.setItem(STORAGE_KEYS.MONSTER, updated);
8845 }, [isPro]);
8846
8847 const loadData = async () => {
8848   try {
8849     console.log('[Monster] 开始加载数据');
8850     const monster = await storage.getItem(STORAGE_KEYS.MONSTER);
8851     if (monster) {
8852       const resetData = await checkAndResetDaily(monster, isPro);
8853       setMonsterData(resetData);
8854     } else {
8855       console.log('[Monster] 无怪物数据, 创建默认怪物');

```

```

8856 const proChecked = isPro; // 读取当前已加载的 pro 状态
8857 const staminaMax = proChecked ? MONSTER_CONFIG.STAMINA.PRO_MAX : MONSTER_CONFIG.STAMINA.BASE_MAX + MON
8858 const energyMax = proChecked ? MONSTER_CONFIG.ENERGY.DAILY_PRO : MONSTER_CONFIG.ENERGY.DAILY_FREE;
8859 const newMonster = {
8860   name: '小怪兽',
8861   type: MONSTER_CONFIG.TYPES.CALM,
8862   level: 1,
8863   exp: 0,
8864   stamina: staminaMax,
8865   maxStamina: staminaMax,
8866   paiEnergy: energyMax,
8867   maxPaiEnergy: energyMax,
8868   knowledgePoints: 0,
8869   createdAt: new Date().toISOString(),
8870 };
8871 setMonsterData(newMonster);
8872 await storage.setItem(STORAGE_KEYS.MONSTER, newMonster);
8873 }
8874
8875 // 从服务端加载笔记（按用户ID）
8876 try {
8877   const user = await getCurrentUser();
8878   if (user?.id) {
8879     const serverNotes = await noteService.getNotes(user.id);
8880     if (serverNotes && serverNotes.length > 0) {
8881       setSavedNotes(serverNotes.map((n: any) => ({
8882         id: n.id,
8883         content: n.content,
8884         date: n.createdAt || n.date,
8885       })));
8886     }
8887   }
8888 } catch {
8889   console.log('[Monster] 笔记从服务端加载失败');
8890 }
8891
8892 const savedTasks = await storage.getItem<any[]>(STORAGE_KEYS.TASKS);
8893 if (savedTasks) setTasks(savedTasks);
8894
8895 const plays = await storage.getItem<number>('dailyGamePlays');
8896 const lastDate = await storage.getItem<string>('lastPlayDate');
8897 const today = new Date().toDateString();
8898
8899 if (lastDate === today && plays !== null) {
8900   setDailyPlays(plays);
8901 } else {
8902   setDailyPlays(0);
8903   await storage.setItem('dailyGamePlays', 0);
8904   await storage.setItem('lastPlayDate', today);
8905 }

```

```

8906   } catch (error) {
8907     console.error('加载数据失败:', error);
8908   }
8909 };
8910
8911 const checkAndResetDaily = async (data: any, isPro: boolean) => {
8912   try {
8913     const lastResetStr = await storage.getItem<string>(STORAGE_KEYS.LAST_RESET);
8914     const lastReset = lastResetStr ? new Date(lastResetStr) : null;
8915     const now = new Date();
8916     const today5AM = new Date(now);
8917     today5AM.setHours(MONSTER_CONFIG.DAILY_RESET.HOUR, MONSTER_CONFIG.DAILY_RESET.MINUTE, 0, 0);
8918
8919     const maxStamina = data.type === MONSTER_CONFIG.TYPES.CALM
8920       ? MONSTER_CONFIG.STAMINA.BASE_MAX + MONSTER_CONFIG.STAMINA.CALM_BONUS
8921       : MONSTER_CONFIG.STAMINA.BASE_MAX;
8922
8923     // 每日能量增加量: Pro 用户 1000, 免费用户 50
8924     const dailyEnergyGrant = isPro ? MONSTER_CONFIG.ENERGY.DAILY_PRO : MONSTER_CONFIG.ENERGY.DAILY_FREE;
8925
8926     if (!lastReset || lastReset < today5AM) {
8927       const resetData = {
8928         ...data,
8929         stamina: maxStamina,
8930         maxStamina,
8931         paiEnergy: (data.paiEnergy || 0) + dailyEnergyGrant,
8932         // 不再有 maxPaiEnergy 上限, 但保留字段以兼容现有代码, 设为当前能量值 (表示无限)
8933         maxPaiEnergy: (data.paiEnergy || 0) + dailyEnergyGrant,
8934       };
8935       await storage.setItem(STORAGE_KEYS.MONSTER, resetData);
8936       await storage.setItem(STORAGE_KEYS.LAST_RESET, now.toISOString());
8937       return resetData;
8938     }
8939
8940     return { ...data, maxStamina };
8941   } catch (error) {
8942     console.error('重置检查失败:', error);
8943     return data;
8944   }
8945 };
8946
8947 const handleSaveNote = async () => {
8948   if (!notes.trim()) return;
8949
8950   const newNote = { id: Date.now(), content: notes, date: new Date().toISOString() };
8951   setSavedNotes(prev => [newNote, ...prev]);
8952   setNotes("");
8953
8954   try {
8955     const user = await getCurrentUser();

```

```

8956     if (user?.id) {
8957         await noteService.createNote({ userId: user.id, date: newNote.date, content: newNote.content });
8958         console.log('[Monster] 笔记已保存至服务端');
8959     }
8960 } catch (error) {
8961     console.warn('[Monster] 笔记保存至服务端失败:', error);
8962 }
8963 };
8964
8965 const handlePlayGame = () => {
8966     if (!monsterData) return;
8967     const dailyGameLimit = isPro ? MONSTER_CONFIG.GAME.PRO_DAILY_LIMIT : MONSTER_CONFIG.GAME.DAILY_LIMIT;
8968     if (dailyPlays >= dailyGameLimit) {
8969         Alert.alert('提示', '今日体力补充已达上限, 明天再来吧');
8970         return;
8971     }
8972     setShowGameModal(true);
8973 };
8974
8975 const handleGameComplete = async (rewards: { stamina: number; energy: number }) => {
8976     const storedJson = await AsyncStorage.getItem(STORAGE_KEYS.MONSTER);
8977     if (!storedJson) return;
8978     const latestData = JSON.parse(storedJson);
8979
8980     let staminaBonus = rewards.stamina;
8981     let energyBonus = rewards.energy;
8982     const user = await getCurrentUser();
8983
8984     if (latestData.type === MONSTER_CONFIG.TYPES.REBEL) {
8985         staminaBonus *= 2;
8986         energyBonus *= 2;
8987     }
8988
8989     try {
8990         if (user?.id) {
8991             await rewardService.createReward({ userId: user.id, type: 'stamina', source: 'game_win', amount: stami
8992             await rewardService.createReward({ userId: user.id, type: 'energy', source: 'game_win', amount: energy
8993             console.log('[Monster] 奖励已同步至服务端');
8994         }
8995     } catch (error) {
8996         console.warn('[Monster] 奖励同步服务端失败, 仅本地记录:', error);
8997     }
8998
8999     const newStamina = Math.min(latestData.stamina + staminaBonus, latestData.maxStamina);
9000     const newPai = latestData.paiEnergy + energyBonus; // 能量无上限, 直接累加
9001     const updated = { ...latestData, stamina: newStamina, paiEnergy: newPai };
9002
9003     setMonsterData(updated);
9004     await storage.setItem(STORAGE_KEYS.MONSTER, updated);
9005

```

```

9006   const newPlays = dailyPlays + 1;
9007   setDailyPlays(newPlays);
9008   await storage.setItem('dailyGamePlays', newPlays);
9009
9010   setShowGameModal(false);
9011   Alert.alert('游戏完成! ', '获得 ${staminaBonus} 体力值和 ${energyBonus} 能量!');
9012 };
9013
9014 const addTask = async () => {
9015   if (!newTaskText.trim()) return;
9016   const newTask = { id: Date.now(), text: newTaskText.trim(), completed: false, createdAt: new Date().toISOString() };
9017   const updatedTasks = [...tasks, newTask];
9018   setTasks(updatedTasks);
9019   await storage.setItem(STORAGE_KEYS.TASKS, updatedTasks);
9020   setNewTaskText("");
9021 };
9022
9023 const toggleTask = async (taskId: number) => {
9024   const updatedTasks = tasks.map(task => task.id === taskId ? { ...task, completed: !task.completed } : task);
9025   setTasks(updatedTasks);
9026   await storage.setItem(STORAGE_KEYS.TASKS, updatedTasks);
9027 };
9028
9029 const deleteTask = async (taskId: number) => {
9030   Alert.alert('确认删除', '确定要删除这个任务吗? ', [
9031     { text: '取消', style: 'cancel' },
9032     {
9033       text: '删除',
9034       onPress: async () => {
9035         const updatedTasks = tasks.filter(task => task.id !== taskId);
9036         setTasks(updatedTasks);
9037         await storage.setItem(STORAGE_KEYS.TASKS, updatedTasks);
9038       },
9039     },
9040   ]);
9041 };
9042
9043 const startEditTask = (taskId: number, text: string) => {
9044   setEditingTaskId(taskId);
9045   setEditTaskText(text);
9046 };
9047
9048 const saveEditTask = async (taskId: number) => {
9049   if (!editTaskText.trim()) return;
9050   const updatedTasks = tasks.map(task =>
9051     task.id === taskId ? { ...task, text: editTaskText.trim() } : task
9052   );
9053   setTasks(updatedTasks);
9054   await storage.setItem(STORAGE_KEYS.TASKS, updatedTasks);
9055   setEditingTaskId(null);

```

```

9056   setEditTaskText("");
9057   };
9058
9059   const cancelEdit = () => {
9060     setEditingTaskId(null);
9061     setEditTaskText("");
9062   };
9063
9064   // 聊天功能
9065   const loadChatMessages = async () => {
9066     try {
9067       const user = await getCurrentUser();
9068       if (!user?.id) return;
9069       const res = await monsterService.getMessages(user.id);
9070       if (res.success) {
9071         const serverMessages: MonsterMessageItem[] = (res.data.messages || []).map((msg: any) => {
9072           const isUser = typeof msg.isUser === 'boolean' ? msg.isUser : msg.is_user === true || msg.is_user ==
9073           return { ...msg, isUser, energyCost: msg.energyCost ?? undefined };
9074         });
9075         setChatMessages(prev => {
9076           const serverIds = new Set(serverMessages.map(m => m.id));
9077           const localOnly = prev.filter(m => !serverIds.has(m.id) && m.id > 0);
9078           return [...serverMessages, ...localOnly];
9079         });
9080       }
9081     } catch (error) {
9082       console.log("[Monster] 加载聊天记录失败:", error);
9083     }
9084   };
9085
9086   const sendMessage = async () => {
9087     const text = chatInput.trim();
9088     if (!text || isSending) return;
9089
9090     const user = await getCurrentUser();
9091     if (!user?.id) {
9092       Alert.alert('提示', '请先登录后再与小怪兽聊天');
9093       return;
9094     }
9095
9096     setChatInput("");
9097     setIsSending(true);
9098
9099     const tempUserMsg: MonsterMessageItem = {
9100       id: Date.now(), userId: user.id, message: text, isUser: true, createdAt: new Date().toISOString(),
9101     };
9102     setChatMessages(prev => [...prev, tempUserMsg]);
9103
9104     try {
9105       const res = await monsterService.chat({ userId: user.id, message: text });

```

```

9106   if (res.success && res.data) {
9107     const cost = Math.max(1, Math.round(res.data.message.length * 0.05 * 10) / 10);
9108     const monsterMsg: MonsterMessageItem = {
9109       id: Date.now() + 1, userId: user.id, message: res.data.message, isUser: false,
9110       createdAt: new Date().toISOString(), energyCost: cost,
9111     };
9112     setChatMessages(prev => [...prev, monsterMsg]);
9113     if (typeof res.data.remainingEnergy === 'number') {
9114       setMonsterData((prev: any) => {
9115         const updated = { ...prev, paiEnergy: res.data.remainingEnergy };
9116         storage.setItem(STORAGE_KEYS.MONSTER, updated).catch(() => {});
9117         return updated;
9118       });
9119     }
9120   }
9121 } catch (error) {
9122   console.error('[Monster] 发送消息失败:', error);
9123   Alert.alert('发送失败', '网络似乎不太好, 请稍后重试 ~ ');
9124 } finally {
9125   setIsSending(false);
9126 }
9127 };
9128
9129 const handleInputBarPress = () => {
9130   setShowChatInput(true);
9131 };
9132
9133 if (!monsterData) {
9134   return (
9135     <SafeAreaView style={{staticStyles.safeArea, dynamicStyles.container}} edges={['top']}>
9136       <View style={staticStyles.loadingContainer}>
9137         <Text style={dynamicStyles.loadingText}>加载中...</Text>
9138       </View>
9139     </SafeAreaView>
9140   );
9141 }
9142
9143 const staminaPercent = (monsterData.stamina / monsterData.maxStamina) * 100;
9144 const paiPercent = 100; // 能量无上限, 进度条始终满格 (已隐藏)
9145
9146 const renderTasksTab = () => (
9147   <View style={staticStyles.tabContent}>
9148     <View style={[staticStyles.taskCard, dynamicStyles.taskCard]}>
9149       <View style={staticStyles.taskHeader}>
9150         <Ionicons name="checkmark-circle" size={20} color={colors.primary} />
9151         <Text style={dynamicStyles.taskTitle}>Keys to Success</Text>
9152       </View>
9153       <View style={staticStyles.addTaskContainer}>
9154         <TextInput
9155           style={[staticStyles.addTaskInput, dynamicStyles.addTaskInput]}

```



```

9156     placeholder="添加新任务..."
9157     placeholderTextColor={colors.textTertiary}
9158     value={newTaskText}
9159     onChangeText={setNewTaskText}
9160     onSubmitEditing={addTask}
9161   />
9162   <TouchableOpacity
9163     style={[staticStyles.addTaskButton, dynamicStyles.addTaskButton, { opacity: newTaskText.trim() ? 1
9164     onPress={addTask}
9165     disabled={!newTaskText.trim()}
9166   >
9167     <Ionicons name="add" size={20} color="#FFFFFF" />
9168   </TouchableOpacity>
9169 </View>
9170 <View style={staticStyles.taskList}>
9171   {tasks.length === 0 ? (
9172     <View style={staticStyles.emptyTasks}>
9173       <Text style={dynamicStyles.emptyTasksText}>还没有任务，快来添加一个吧! </Text>
9174     </View>
9175   ) : (
9176     tasks.map(task => (
9177       <View key={task.id} style={staticStyles.taskItem}>
9178         <TouchableOpacity
9179           style={[staticStyles.taskCheckbox, task.completed && staticStyles.taskCheckboxDone, dynamicS
9180           onPress={() => toggleTask(task.id)}
9181         >
9182           {task.completed ? <View style={staticStyles.taskCheckboxInner} /> : null}
9183         </TouchableOpacity>
9184         {editingTaskId === task.id ? (
9185           <>
9186             <TextInput
9187               style={[staticStyles.editTaskInput, { backgroundColor: colors.inputBg, borderColor: colo
9188               value={editTaskText}
9189               onChangeText={setEditTaskText}
9190               onSubmitEditing={() => saveEditTask(task.id)}
9191               autoFocus
9192             />
9193             <View style={staticStyles.editActions}>
9194               <TouchableOpacity style={staticStyles.editTaskButton} onPress={() => saveEditTask(task.i
9195                 <Ionicons name="checkmark" size={18} color={colors.success} />
9196             </TouchableOpacity>
9197               <TouchableOpacity style={staticStyles.editTaskButton} onPress={cancelEdit}>
9198                 <Ionicons name="close" size={18} color={colors.textTertiary} />
9199             </TouchableOpacity>
9200           </View>
9201         </>
9202       ) : (
9203         <>
9204           <Text
9205             style={[dynamicStyles.taskName, task.completed && dynamicStyles.taskNameDone]}

```

```

9206         numberOfLines={3}
9207       >{task.text}</Text>
9208     <TouchableOpacity style={staticStyles.editTaskButton} onPress={() => startEditTask(task.id
9209       <Ionicons name="pencil" size={16} color={colors.textTertiary} />
9210     </TouchableOpacity>
9211     <TouchableOpacity style={staticStyles.deleteTaskButton} onPress={() => deleteTask(task.id)
9212       <Ionicons name="trash-outline" size={18} color="#FF6B6B" />
9213     </TouchableOpacity>
9214   </>
9215   })
9216 </View>
9217 })
9218 })
9219 </View>
9220 </View>
9221
9222 <View style={[staticStyles.taskCard, dynamicStyles.taskCard]}>
9223   <View style={staticStyles.taskHeader}>
9224     <Ionicons name="timer" size={20} color={colors.error} />
9225     <Text style={dynamicStyles.taskTitle}>番茄钟</Text>
9226   </View>
9227   <View style={staticStyles.timeOptionsContainer}>
9228     {MONSTER_CONFIG.POMODORO.TIME_OPTIONS.map(time => (
9229       <TouchableOpacity
9230         key={time}
9231         style={[staticStyles.timeOption, { backgroundColor: selectedTime === time ? colors.primary : col
9232         onPress={() => setSelectedTime(time)}
9233       >
9234         <Text style={[dynamicStyles.timeOptionText, { color: selectedTime === time ? 'FFFFFF' : colors.
9235       </TouchableOpacity>
9236     )}}
9237   </View>
9238   <View style={staticStyles.pomodoroContainer}>
9239     <View style={[staticStyles.pomodoroTimer, dynamicStyles.pomodoroTimer]}>
9240       <Text style={dynamicStyles.pomodoroTime}>{formatTimer(pomodoroTimeLeft)}</Text>
9241     </View>
9242     <View style={staticStyles.pomodoroButtons}>
9243       <TouchableOpacity
9244         style={[staticStyles.pomodoroButton, dynamicStyles.pomodoroButton]}
9245         onPress={() => {
9246           router.push({ pathname: '/pomodoro', params: { duration: String(selectedTime) } });
9247         }}
9248       >
9249         <Ionicons name="timer" size={20} color={colors.onPrimary} />
9250         <Text style={dynamicStyles.pomodoroButtonText}>开始专注</Text>
9251       </TouchableOpacity>
9252     </View>
9253   </View>
9254 </View>
9255 </View>

```

```

9256 );
9257
9258 const renderNotesTab = () => (
9259   <View style={staticStyles.tabContent}>
9260     <View style={[staticStyles.noteCard, dynamicStyles.noteCard]}>
9261       <View style={staticStyles.noteHeader}>
9262         <Text style={dynamicStyles.noteTitle}> 学习笔记</Text>
9263         <Ionicons name="create-outline" size={16} color={colors.primary} />
9264       </View>
9265       <TextInput
9266         style={[staticStyles.noteInput, dynamicStyles.noteInput]}
9267         value={notes}
9268         onChangeText={setNotes}
9269         placeholder="记录今天的学习心得..."
9270         placeholderTextColor={colors.textTertiary}
9271         multiline
9272         numberOfLines={4}
9273         textAlignVertical="top"
9274       />
9275       <TouchableOpacity
9276         style={[staticStyles.saveNoteButton, dynamicStyles.saveNoteButton, { opacity: notes.trim() ? 1 : 0.5}
9277         onPress={handleSaveNote}
9278         disabled={!notes.trim()}
9279         activeOpacity={0.7}
9280       >
9281         <Text style={staticStyles.saveNoteText}>保存笔记</Text>
9282       </TouchableOpacity>
9283     </View>
9284
9285     <View style={staticStyles.notesHistory}>
9286       <Text style={dynamicStyles.historyTitle}>历史记录 ({savedNotes.length})</Text>
9287       {savedNotes.length === 0 ? (
9288         <View style={[staticStyles.emptyNotes, dynamicStyles.emptyNotes]}>
9289           <Text style={dynamicStyles.emptyNotesText}>还没有笔记记录</Text>
9290         </View>
9291       ) : (
9292         <View style={staticStyles.notesList}>
9293           {savedNotes.map((note) => (
9294             <View key={note.id} style={[staticStyles.savedNote, dynamicStyles.savedNote]}>
9295               <View style={{ flex: 1 }}>
9296                 <Text style={dynamicStyles.savedNoteContent}>{note.content}</Text>
9297                 <Text style={dynamicStyles.savedNoteDate}>{new Date(note.date).toLocaleString('zh-CN')}</Text>
9298               </View>
9299               <TouchableOpacity
9300                 onPress={() => setSavedNotes(prev => prev.filter(n => n.id !== note.id))}
9301                 style={{ padding: 8 }}
9302                 hitSlop={{ top: 8, bottom: 8, left: 8, right: 8 }}
9303               >
9304                 <Ionicons name="close-circle" size={18} color={colors.textTertiary} />
9305               </TouchableOpacity>

```

```

9306     </View>
9307   )))
9308   </View>
9309   })
9310 </View>
9311 </View>
9312 );
9313
9314 const renderChatTab = () => {
9315
9316   // Pro 用户：完整聊天界面
9317   return (
9318     <KeyboardAvoidingView
9319       style={{ flex: 1 }}
9320       behavior={Platform.OS === 'ios' ? 'padding' : undefined}
9321       keyboardVerticalOffset={Platform.OS === 'ios' ? 90 : 0}
9322     >
9323       {/* 聊天头部 */}
9324       <View style={{ flexDirection: 'row', alignItems: 'center', paddingHorizontal: 12, paddingVertical: 10, b
9325         <TouchableOpacity
9326           onPress={() => setActiveTab('tasks')}
9327           style={{ width: 40, height: 40, borderRadius: 10, alignItems: 'center', justifyContent: 'center', bo
9328         >
9329           <Ionicons name="arrow-back" size={20} color={colors.textPrimary} />
9330         </TouchableOpacity>
9331         <View style={{ flexDirection: 'row', alignItems: 'center', gap: 8, marginLeft: 12 }}>
9332           <MonsterIcon type={monsterData.type} size={28} />
9333           <Text style={{ color: colors.textPrimary, fontSize: 16, fontWeight: '600' }}>{monsterData.name}</Tex
9334         </View>
9335       </View>
9336
9337       {/* 非 Pro 用户提示条 */}
9338       {!isPro && (
9339         <View style={{ paddingHorizontal: 16, paddingVertical: 12, backgroundColor: colors.surface, borderBott
9340           <Text style={{ color: colors.textSecondary, fontSize: 14, textAlign: 'center' }}>
9341             每日能量 +50，升级 Pro 可领取 <Text style={{ color: colors.primary, fontWeight: '600' }}>1000 点/天</Text>
9342           </Text>
9343           <TouchableOpacity
9344             style={{ marginTop: 8, paddingHorizontal: 16, paddingVertical: 8, backgroundColor: colors.primary,
9345             onPress={() => setShowProModal(true)}
9346           >
9347             <Text style={{ color: 'FFFFFF', fontSize: 14, fontWeight: '600' }}>升级 Pro</Text>
9348           </TouchableOpacity>
9349         </View>
9350       )}
9351
9352     <FlatList
9353       ref={flatListRef}
9354       data={chatMessages}
9355       keyExtractor={({item}) => item.id.toString()}

```

```

9356 onContentSizeChange={() => flatListRef.current?.scrollToEnd({ animated: true })}
9357 keyboardShouldPersistTaps="handled"
9358 keyboardDismissMode="on-drag"
9359 style={{ flex: 1 }}
9360 containerViewStyle={[staticStyles.chatListContent, { flexGrow: 1 }}]
9361 ListEmptyComponent={
9362   <View style={staticStyles.chatEmpty}>
9363     <MonsterIcon type={monsterData.type} size={60} />
9364     <Text style={[staticStyles.chatEmptyText, dynamicStyles.chatEmptyText]}>
9365       和 {monsterData.name} 打个招呼吧~
9366     </Text>
9367     <Text style={[staticStyles.chatEmptyHint, dynamicStyles.chatEmptyHint]}>
9368       聊聊学习、吐槽烦恼、求安慰都行
9369     </Text>
9370   </View>
9371 }
9372 renderItem={({ item }) => (
9373   <View style={[
9374     staticStyles.messageBubble,
9375     item.isUser ? staticStyles.messageUser : staticStyles.messageMonster,
9376   ]}>
9377     {!item.isUser && (
9378       <View style={staticStyles.messageAvatar}>
9379         <MonsterIcon type={monsterData.type} size={28} />
9380       </View>
9381     )}
9382     <View style={[
9383       staticStyles.messageContent,
9384       item.isUser ? staticStyles.messageContentUser : staticStyles.messageContentMonster,
9385       item.isUser ? { backgroundColor: colors.primary } : dynamicStyles.messageContentMonster,
9386     ]}>
9387       <Text style={[
9388         staticStyles.messageText,
9389         { color: item.isUser ? 'FFFFFF' : colors.textPrimary },
9390       ]}>
9391         {item.message}
9392       </Text>
9393     </View>
9394   </View>
9395 )}
9396 />
9397 /* 临时输入栏： 点击后弹出浮动输入框 */
9398 <TouchableOpacity
9399   style={[staticStyles.chatInputBar, dynamicStyles.chatInputBar]}
9400   onPress={() => handleInputBarPress()}
9401   activeOpacity={0.8}
9402 >
9403   <Text style={{ flex: 1, color: colors.textTertiary, fontSize: 14 }}>
9404     {chatInput || `和 ${monsterData.name} 说点什么...`}
9405   </Text>

```

```

9406     <View style={{staticStyles.sendButton, dynamicStyles.sendButton, !chatInput.trim() && staticStyles.sendButton}}>
9407         <Ionicons name="send" size={18} color="#FFFFFF" />
9408     </View>
9409 </TouchableOpacity>
9410
9411 /* 浮动输入模式 */
9412 <Modal visible={showChatInput} transparent animationType="fade" onRequestClose={() => { setShowChatInput(false); }}>
9413     <TouchableOpacity style={{ flex: 1, backgroundColor: 'rgba(0,0,0,0.3)' }} activeOpacity={1} onPress={() => { setShowChatInput(false); }}>
9414         <View style={{ flex: 1, justifyContent: 'flex-end' }} pointerEvents="box-none">
9415             <TouchableOpacity activeOpacity={1} onPress={() => {}}>
9416                 <KeyboardAvoidingView behavior={Platform.OS === 'ios' ? 'padding' : undefined} keyboardVerticalOffset={20}>
9417                     <View style={{ backgroundColor: colors.surface, borderTopLeftRadius: 16, borderTopRightRadius: 16, padding: 10 }}>
9418                         <View style={{ flexDirection: 'row', alignItems: 'flex-end', gap: 10 }}>
9419                             <TextInput
9420                                 style={{staticStyles.chatTextInput, dynamicStyles.chatTextInput, { flex: 1, maxHeight: 100, padding: 5, margin: 5 0 0 0 }}
9421                                 value={chatInput}
9422                                 onChangeText={setChatInput}
9423                                 placeholder={`和 ${monsterData.name} 说点什么...`}
9424                                 placeholderTextColor={colors.textTertiary}
9425                                 multiline
9426                                 maxLength={500}
9427                                 autoFocus
9428                             />
9429                             <TouchableOpacity
9430                                 style={{staticStyles.sendButton, dynamicStyles.sendButton, (!chatInput.trim() || isSending) ? { opacity: 0.5 } : {}}
9431                                 onPress={() => { sendMessage(); setShowChatInput(false); Keyboard.dismiss(); }}
9432                                 disabled={!chatInput.trim() || isSending}
9433                             >
9434                                 {isSending ? (
9435                                     <ActivityIndicator size="small" color="#FFFFFF" />
9436                                 ) : (
9437                                     <Ionicons name="send" size={18} color="#FFFFFF" />
9438                                 )}
9439                             </TouchableOpacity>
9440                         </View>
9441                     </View>
9442                 </KeyboardAvoidingView>
9443             </TouchableOpacity>
9444         </View>
9445     </TouchableOpacity>
9446 </Modal>
9447 </KeyboardAvoidingView>
9448 );
9449 };
9450
9451 return (
9452     <SafeAreaView style={{staticStyles.safeArea, dynamicStyles.container}} edges={['top']}>
9453         {activeTab === 'chat' ? (
9454             renderChatTab()
9455         ) : (

```

```

9456 <ScrollView containerViewStyle={staticStyles.scrollContent} showsVerticalScrollIndicator={false}>
9457   <View style={staticStyles.header}>
9458     <View style={staticStyles.pixelBackground} />
9459     <View style={staticStyles.headerContent}>
9460       <Text style={dynamicStyles.title}>我的怪兽</Text>
9461       <Text style={[staticStyles.subtitle, dynamicStyles.subtitle]}>查看你的学习伙伴</Text>
9462       <View style={[staticStyles.monsterCard, dynamicStyles.monsterCard]}>
9463         <View style={staticStyles.monsterPixelPattern} />
9464         <View style={staticStyles.monsterCardContent}>
9465           <View style={staticStyles.monsterTopRow}>
9466             <View style={[staticStyles.monsterIconContainer, dynamicStyles.monsterIconContainer]}>
9467               <MonsterIcon type={monsterData.type} size={80} />
9468             </View>
9469             <View style={staticStyles.monsterInfo}>
9470               <Text style={[staticStyles.monsterName, dynamicStyles.monsterName]} numberOfLines={1}>{mon
9471               <Text style={[staticStyles.monsterPersonality, dynamicStyles.monsterPersonality]}>
9472                 {monsterData.type === MONSTER_CONFIG.TYPES.LIVELY ? '活力型怪兽'
9473                 : monsterData.type === MONSTER_CONFIG.TYPES.CALM ? '沉稳型怪兽'
9474                 : '叛逆型怪兽'}
9475               </Text>
9476             </View>
9477           </View>
9478
9479           <View style={staticStyles.monsterActionRow}>
9480             <TouchableOpacity
9481               style={[staticStyles.gameButton, dynamicStyles.gameButton, dailyPlays >= (isPro ? MONSTER_
9482               onPress={handlePlayGame}
9483               activeOpacity={0.7}
9484             >
9485               <Ionicons name="game-controller-outline" size={16} color={dailyPlays >= MONSTER_CONFIG.GAM
9486               <View style={staticStyles.gameButtonContent}>
9487                 <Text style={[staticStyles.gameButtonText, { color: dailyPlays >= (isPro ? MONSTER_CONFI
9488                 <Text style={[staticStyles.gameButtonSubText, dynamicStyles.infoText]}>
9489                 剩余: {(isPro ? MONSTER_CONFIG.GAME.PRO_DAILY_LIMIT : MONSTER_CONFIG.GAME.DAILY_LIMIT) -
9490               </Text>
9491             </View>
9492           </TouchableOpacity>
9493           <TouchableOpacity style={[staticStyles.infoButton, dynamicStyles.infoButton]} onPress={() =>
9494             <Ionicons name="information" size={18} color={colors.textSecondary} />
9495           </TouchableOpacity>
9496         </View>
9497
9498       {showInfo && (
9499         <View style={[staticStyles.infoCard, dynamicStyles.infoCard]}>
9500         <View style={staticStyles.infoItem}>
9501           <Text style={[staticStyles.infoTitle, dynamicStyles.infoTitle]}> 体力值</Text>
9502           <Text style={[staticStyles.infoText, dynamicStyles.infoText]}>
9503             • 单次知识节点跳转消耗 10 体力{'\n'}
9504             • 每日凌晨 5:00 自动恢复至上限{'\n'}
9505             • 小游戏可额外补充体力

```

```

9506         </Text>
9507     </View>
9508     <View style={staticStyles.infoItem}>
9509         <Text style={{staticStyles.infoTitle, dynamicStyles.infoTitle}}>□ 能量</Text>
9510         <Text style={{staticStyles.infoText, dynamicStyles.infoText}}>
9511             • AI对话消耗 = 对话Token数 × 0.05{'\n'}
9512             • 每日凌晨 5:00 自动恢复至上限{'\n'}
9513             • 小游戏可额外补充能量
9514         </Text>
9515     </View>
9516 </View>
9517 }}
9518
9519 <View style={staticStyles.statsRow}>
9520     <View style={staticStyles.statContainer}>
9521         <View style={staticStyles.statHeader}>
9522             <View style={staticStyles.statLabelRow}>
9523                 <Icons name="flash" size={14} color={colors.orange} />
9524                 <Text style={dynamicStyles.statLabel}>体力值</Text>
9525             </View>
9526             <Text style={{dynamicStyles.statValue, { color: colors.orange }}}>
9527                 {monsterData.stamina}/{monsterData.maxStamina}
9528             </Text>
9529         </View>
9530         <View style={staticStyles.statBar}>
9531             <View style={{staticStyles.statBarFill, { width: `${staminaPercent}%`, backgroundColor:
9532             </View>
9533         </View>
9534     </View>
9535     <View style={staticStyles.statContainer}>
9536         <View style={staticStyles.statHeader}>
9537             <View style={staticStyles.statLabelRow}>
9538                 <Text style={{ color: colors.purple }}>□</Text>
9539                 <Text style={dynamicStyles.statLabel}>能量</Text>
9540             </View>
9541             <Text style={{dynamicStyles.statValue, { color: colors.purple }}}>
9542                 {Number(monsterData.paiEnergy).toFixed(1)}
9543                 <Text style={{ fontSize: 12, color: colors.textTertiary }}>
9544                     {isPro ? ' (每日 +1000)' : ' (每日 +50)'}
9545                 </Text>
9546             </Text>
9547         </View>
9548         { /* 能量无上限，移除进度条 */ }
9549         <View style={staticStyles.statBar}>
9550             <View style={{staticStyles.statBarFill, { width: '100%', backgroundColor: colors.purple,
9551             </View>
9552         </View>
9553     </View>
9554 </View>
9555 </View>

```



```

9556     </View>
9557 </View>
9558
9559 <View style={staticStyles.tabsContainer}>
9560   <View style={[staticStyles.tabs, dynamicStyles.tabs]}>
9561     {[
9562       { id: 'tasks' as ActiveTab, label: '任务', icon: 'layers' },
9563       { id: 'chat' as ActiveTab, label: '对话', icon: 'chatbubbles' },
9564       { id: 'notes' as ActiveTab, label: '笔记', icon: 'document-text' },
9565     ].map((tab) => (
9566       <TouchableOpacity
9567         key={tab.id}
9568         onPress={() => setActiveTab(tab.id)}
9569         activeOpacity={0.7}
9570         style={[
9571           staticStyles.tab,
9572           {
9573             backgroundColor: activeTab === tab.id ? colors.borderDark : 'transparent',
9574             borderColor: activeTab === tab.id ? colors.borderDark : 'transparent',
9575           },
9576         ])}
9577       >
9578         <Ionicons name={tab.icon as any} size={16} color={activeTab === tab.id ? colors.primary : colors.borderDark} />
9579         <Text style={[staticStyles.tabText, dynamicStyles.infoText, { color: activeTab === tab.id ? colors.primary : colors.borderDark }]}>
9580           {tab.label}
9581         </Text>
9582       </TouchableOpacity>
9583     )]}
9584   </View>
9585 </View>
9586
9587 {activeTab === 'tasks' && renderTasksTab()}
9588 {activeTab === 'notes' && renderNotesTab()}
9589 <View style={staticStyles.bottomPadding} />
9590 </ScrollView>
9591 ))
9592
9593 <Modal visible={showGameModal} animationType="slide" statusBarTranslucent onRequestClose={() => setShowGameModal(false)} />
9594 <View style={staticStyles.modalFullScreen}>
9595   <MiniGames onGameComplete={handleGameComplete} onClose={() => setShowGameModal(false)} />
9596 </View>
9597 </Modal>
9598
9599 <Modal visible={showChatFullscreen} animationType="slide" statusBarTranslucent onRequestClose={() => setShowChatFullscreen(false)} />
9600 <SafeAreaView style={{ flex: 1, backgroundColor: colors.background }} edges={['top']}>
9601   <View style={{ flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', padding: 10 }}>
9602     <View style={{ flexDirection: 'row', alignItems: 'center', gap: 8 }}>
9603       <MonsterIcon type={monsterData.type} size={28} />
9604       <Text style={{ color: colors.textPrimary, fontSize: 16, fontWeight: '600' }}>{monsterData.name}</Text>
9605     </View>

```

```

9606     <TouchableOpacity
9607         style={{ width: 36, height: 36, borderRadius: 10, alignItems: 'center', justifyContent: 'center'
9608         onPress={() => setShowChatFullscreen(false)}
9609     >
9610         <Ionicons name="close" size={20} color={colors.textPrimary} />
9611     </TouchableOpacity>
9612 </View>
9613 <KeyboardAvoidingView style={{ flex: 1 }} behavior={Platform.OS === 'ios' ? 'padding' : undefined}>
9614     <FlatList
9615         ref={flatListRef}
9616         data={chatMessages}
9617         keyExtractor={(item) => item.id.toString()}
9618         onContentSizeChange={() => flatListRef.current?.scrollToEnd({ animated: true })}
9619         keyboardShouldPersistTaps="handled"
9620         keyboardDismissMode="on-drag"
9621         style={{ flex: 1 }}
9622         contentContainerStyle={{ padding: 16, paddingBottom: 8 }}
9623         ListEmptyComponent={
9624             <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center', paddingTop: 80 }}>
9625                 <MonsterIcon type={monsterData.type} size={80} />
9626                 <Text style={{ color: colors.textSecondary, fontSize: 14, marginTop: 16 }}>和 {monsterData.name}
9627                 <Text style={{ color: colors.textTertiary, fontSize: 12, marginTop: 8 }}>聊聊学习、吐槽烦恼、求安慰都行</Text>
9628             </View>
9629         }
9630         renderItem={({ item }) => (
9631             <View style={{ marginBottom: 12 }}>
9632                 <View style={{staticStyles.messageBubble, item.isUser ? staticStyles.messageUser : staticStyles.messageBot}}>
9633                     {!item.isUser && (
9634                         <View style={staticStyles.messageAvatar}>
9635                             <MonsterIcon type={monsterData.type} size={28} />
9636                         </View>
9637                     )}
9638                     <View style={{staticStyles.messageContent, item.isUser ? staticStyles.messageContentUser : staticStyles.messageContentBot}}>
9639                         <Text style={{staticStyles.messageText, { color: item.isUser ? colors.onPrimary : colors.onSecondary }}}>
9640                             {item.message}
9641                         </Text>
9642                         {!item.isUser && typeof item.energyCost === 'number' && (
9643                             <Text style={staticStyles.energyCostText}>- {item.energyCost}</Text>
9644                         )}
9645                     </View>
9646                 </View>
9647             </View>
9648             <View style={{staticStyles.chatInputBar, dynamicStyles.chatInputBar}}>
9649                 <TextInput
9650                     style={{staticStyles.chatTextInput, dynamicStyles.chatTextInput}}
9651                     value={chatInput}
9652                     onChangeText={setChatInput}
9653                     placeholder={`和 ${monsterData.name} 说点什么...`}
9654                     placeholderTextColor={colors.textTertiary}
9655                     multiline

```

```

9656         maxLength={500}
9657         returnKeyType="send"
9658         onSubmitEditing={sendMessage}
9659         editable={!isSending}
9660     />
9661     <TouchableOpacity
9662         style={[staticStyles.sendButton, dynamicStyles.sendButton, (!chatInput.trim() || isSending) &&
9663         onPress={sendMessage}
9664         disabled={!chatInput.trim() || isSending}
9665     >
9666         {isSending ? <ActivityIndicator size="small" color="#FFFFFF" /> : <Ionicons name="send" size={
9667     </TouchableOpacity>
9668 </View>
9669 </KeyboardAvoidingView>
9670 </SafeAreaView>
9671 </Modal>
9672
9673 <SubscriptionModal visible={showProModal} onClose={() => setShowProModal(false)} onProActivated={() => {
9674 </SafeAreaView>
9675 });
9676 };
9677
9678 export default MonsterManageScreen;
9679
=== mobile/app/(tabs)/profile.tsx ===
9680 import React, { useState, useEffect, useMemo } from 'react';
9681 import { View, Text, StyleSheet, ScrollView, Alert, TouchableOpacity, Switch } from 'react-native';
9682 import { SafeAreaView } from 'react-native-safe-area-context';
9683 import { router } from 'expo-router';
9684 import { Ionicons } from '@expo/vector-icons';
9685 import { useTheme } from '../src/contexts/ThemeContext';
9686 import { getCurrentUser, clearAuthData } from '../src/utls/auth';
9687 import AsyncStorage from '@react-native-async-storage/async-storage';
9688 import HelpModal from '../src/components/HelpModal';
9689 import SubscriptionModal from '../src/components/SubscriptionModal';
9690 import { proService } from '../src/services/api';
9691 import { SUBSCRIPTION_STORAGE_KEY } from '../src/utls/pricing';
9692
9693 const ProfileScreen = () => {
9694     const { isDark, colors, toggleTheme } = useTheme();
9695     const [user, setUser] = useState<any>(null);
9696     const [isLoggedIn, setIsLoggedIn] = useState(false);
9697     const [notifications, setNotifications] = useState(true);
9698     const [showHelpModal, setShowHelpModal] = useState(false);
9699     const [showGuideModal, setShowGuideModal] = useState(false);
9700     const [showAvatarModal, setShowAvatarModal] = useState(false);
9701     const [avatarType, setAvatarType] = useState<'male' | 'female' | 'monster'>('male');
9702     const [pendingAvatarType, setPendingAvatarType] = useState<'male' | 'female' | 'monster'>('male');
9703     const [daysSinceJoin, setDaysSinceJoin] = useState(0);
9704     const [domainCount, setDomainCount] = useState(0);

```

```

9705 const [activeDomainCount, setActiveDomainCount] = useState(0);
9706 const [showProModal, setShowProModal] = useState(false);
9707 const [isPro, setIsPro] = useState(false);
9708
9709 useEffect(() => {
9710   const loadUserData = async () => {
9711     try {
9712       console.log('[Profile] 开始加载用户数据');
9713       const currentUser = await getCurrentUser();
9714       setUser(currentUser);
9715       setIsLoggedIn(!currentUser);
9716       console.log('[Profile] 用户登录状态:', !!currentUser);
9717     } catch (error) {
9718       console.error('[Profile] 加载用户数据失败:', error);
9719       setIsLoggedIn(false);
9720     }
9721   };
9722
9723   loadUserData();
9724   checkPro();
9725 }, []);
9726
9727 const checkPro = async () => {
9728   try {
9729     const user = await getCurrentUser();
9730     if (user?.id) {
9731       const status = await proService.getStatus(user.id);
9732       setIsPro(status.isPro);
9733     }
9734   } catch {}
9735 };
9736
9737 const userData = {
9738   name: user?.username || 'LearnFlow用户',
9739   title: '学习探索者',
9740   level: 5,
9741   xp: 2450,
9742   xpToNextLevel: 5000,
9743   streak: 7,
9744   completedSkills: 12,
9745   studyHours: 48,
9746 };
9747
9748 const xpPercent = Math.round((userData.xp / userData.xpToNextLevel) * 100);
9749
9750 const settings = [
9751   { icon: 'notifications' as const, label: '学习提醒', value: notifications, toggle: () => setNotifications(!not
9752   { icon: 'moon' as const, label: isDark ? '深色模式' : '浅色模式', value: isDark, toggle: toggleTheme },
9753   { icon: 'globe' as const, label: '语言', text: '简体中文' },
9754   { icon: 'shield' as const, label: '隐私设置' },

```

```

9755   ];
9756
9757   const handleLogout = () => {
9758     console.log('[Profile] 用户点击退出登录');
9759     Alert.alert('确认退出', '您确定要退出登录吗? ', [
9760       { text: '取消', style: 'cancel' },
9761       {
9762         text: '确定',
9763         onPress: async () => {
9764           // 清除内存中的认证信息 (token + 用户数据)
9765           // 业务数据 (怪兽、笔记、进度等) 保留不清除, 重新登录后仍可访问
9766           await clearAuthData();
9767           // 重置组件状态, 防止登录态残留
9768           setUser(null);
9769           setIsLoggedIn(false);
9770           console.log('[Profile] 退出登录完成, 已清除认证数据');
9771           router.replace('/login');
9772         }
9773       }
9774     ]);
9775   };
9776
9777   const handleLogin = () => {
9778     router.replace('/login');
9779   };
9780
9781   const renderSettings = () => (
9782     <View style={styles.tabContent}>
9783       /* 设置卡片组: 学习提醒、深浅模式 */
9784       <View style={styles.settingsGroup}>
9785         {settings.slice(0, 2).map((s, i) => (
9786           <View key={s.label} style={styles.settingItem, i < 1 && styles.settingItemBorder}>
9787             <View style={styles.settingIconContainer}>
9788               <Ionicons name={s.icon} size={16} color={colors.primary} />
9789             </View>
9790             <Text style={styles.settingLabel}>{s.label}</Text>
9791             {s.value !== undefined && s.toggle ? (
9792               <Switch
9793                 value={s.value}
9794                 onChange={s.toggle}
9795                 trackColor={{ false: 'rgba(255,255,255,0.1)', true: colors.primary }}
9796                 thumbColor="#fff"
9797               />
9798             ) : (
9799               <View style={styles.settingRight}>
9800                 {s.text && <Text style={styles.settingText}>{s.text}</Text>}
9801                 <Ionicons name="chevron-forward" size={16} color={colors.textTertiary} />
9802               </View>
9803             )}
9804             </View>

```

```

9805     )))
9806     { /* 了解充值 — 在深浅模式下方、隐私设置上方 */ }
9807     <View style={ [styles.settingItem, styles.settingItemBorder] } >
9808         <TouchableOpacity
9809             onPress={() => setShowProModal(true)}
9810             activeOpacity={0.7}
9811             style={{ flexDirection: 'row', alignItems: 'center', flex: 1, gap: 12 }}
9812         >
9813             <View style={ [styles.settingIconContainer, { backgroundColor: colors.proBg, borderWidth: 1.5, bord
9814                 <Ionicons name="diamond" size={16} color={colors.pro} />
9815             </View>
9816             <Text style={ [styles.settingLabel, { color: colors.pro, fontWeight: '700' as const } ] } >了解充值</Text>
9817             <View style={styles.settingRight}>
9818                 <Ionicons name="chevron-forward" size={16} color={colors.pro} />
9819             </View>
9820         </TouchableOpacity>
9821     </View>
9822     { /* 隐私设置 */ }
9823     <View style={styles.settingItem}>
9824         <View style={styles.settingIconContainer}>
9825             <Ionicons name="shield" size={16} color={colors.textSecondary} />
9826         </View>
9827         <Text style={styles.settingLabel}>隐私设置</Text>
9828         <View style={styles.settingRight}>
9829             <Ionicons name="chevron-forward" size={16} color={colors.textTertiary} />
9830         </View>
9831     </View>
9832 </View>
9833
9834 <View style={styles.settingsGroup}>
9835     [
9836         { label: 'Learnflow指南', icon: 'star' as const, onPress: () => setShowGuideModal(true) },
9837         { label: '使用帮助', icon: 'help-circle' as const, onPress: () => setShowHelpModal(true) },
9838     ].map((item, i) => (
9839         <TouchableOpacity
9840             key={item.label}
9841             style={ [styles.settingItem, i === 0 && styles.settingItemBorder] }
9842             onPress={item.onPress}
9843             disabled={!item.onPress}
9844             activeOpacity={item.onPress ? 0.7 : 1}
9845         >
9846             <View style={ [styles.settingIconContainer, { backgroundColor: 'rgba(255,255,255,0.06)' } ] } >
9847                 <Ionicons name={item.icon} size={16} color={colors.textSecondary} />
9848             </View>
9849             <Text style={styles.settingLabel}>{item.label}</Text>
9850             <Ionicons name="chevron-forward" size={16} color={colors.textTertiary} />
9851         </TouchableOpacity>
9852     )))
9853 </View>
9854

```

```

9855     {isLoggedIn ? (
9856       <TouchableOpacity style={styles.logoutButton} onPress={handleLogout}>
9857         <Ionicons name="log-out" size={18} color={colors.error} />
9858         <Text style={styles.logoutText}>退出登录</Text>
9859       </TouchableOpacity>
9860     ) : (
9861       <TouchableOpacity style={styles.loginButton} onPress={handleLogin}>
9862         <Ionicons name="log-in" size={18} color={colors.primary} />
9863         <Text style={styles.loginText}>登录账号</Text>
9864       </TouchableOpacity>
9865     )}
9866   </View>
9867 );
9868
9869 const styles = useMemo(() => StyleSheet.create({
9870   container: {
9871     flex: 1,
9872     backgroundColor: colors.background,
9873   },
9874   scrollContent: {
9875     flexGrow: 1,
9876   },
9877   header: {
9878     position: 'relative',
9879     paddingTop: 20,
9880     paddingBottom: 24,
9881   },
9882   headerGradient: {
9883     position: 'absolute',
9884     top: 0,
9885     left: 0,
9886     right: 0,
9887     bottom: 0,
9888     backgroundColor: colors.backgroundDark,
9889   },
9890   headerContent: {
9891     paddingHorizontal: 20,
9892   },
9893   headerTop: {
9894     flexDirection: 'row',
9895     alignItems: 'center',
9896     justifyContent: 'space-between',
9897     marginBottom: 20,
9898   },
9899   headerTitle: {
9900     color: colors.textPrimary,
9901     fontWeight: '600',
9902     fontSize: 28,
9903     marginBottom: 4,
9904   },

```

```

9905 settingsButton: {
9906   width: 40,
9907   height: 40,
9908   borderRadius: 10,
9909   backgroundColor: colors.surface,
9910   borderWidth: 1,
9911   borderColor: colors.borderLight,
9912   alignItems: 'center',
9913   justifyContent: 'center',
9914 },
9915 profileCard: {
9916   borderRadius: 16,
9917   backgroundColor: colors.surface,
9918   borderWidth: 1,
9919   borderColor: colors.hairline,
9920   shadowColor: colors.shadow,
9921   shadowOffset: { width: 0, height: 1 },
9922   shadowOpacity: 0.04,
9923   shadowRadius: 3,
9924   elevation: 2,
9925 },
9926 profileCardDark: {
9927   backgroundColor: colors.surface,
9928   borderWidth: 2,
9929   borderColor: 'rgba(108,91,123,0.5)',
9930   shadowOpacity: 1,
9931   elevation: 4,
9932 },
9933 profileCardContent: {
9934   padding: 20,
9935 },
9936 profileInfo: {
9937   flexDirection: 'row',
9938   alignItems: 'flex-start',
9939   gap: 16,
9940   marginBottom: 20,
9941 },
9942 avatarContainer: { position: 'relative' },
9943 avatar: {
9944   width: 64, height: 64, borderRadius: 16,
9945   backgroundColor: colors.borderLight, alignItems: 'center', justifyContent: 'center',
9946   borderWidth: 1, borderColor: colors.hairline,
9947 },
9948 avatarDark: {
9949   backgroundColor: colors.borderDark,
9950   shadowColor: colors.shadow, shadowOffset: { width: 0, height: 1 },
9951   shadowOpacity: 0.04, shadowRadius: 3, elevation: 2,
9952 },
9953 avatarEmoji: { fontSize: 32 },
9954 avatarEditBadge: {

```



```

9955   position: 'absolute', bottom: -2, right: -2,
9956   width: 22, height: 22, borderRadius: 6,
9957   backgroundColor: colors.primary, alignItems: 'center', justifyContent: 'center',
9958   borderWidth: 1.5, borderColor: colors.background,
9959 },
9960 levelBadge: {
9961   position: 'absolute', bottom: -4, right: -4,
9962   width: 24, height: 24, borderRadius: 6,
9963   backgroundColor: colors.primary, alignItems: 'center', justifyContent: 'center',
9964   borderWidth: 1.5, borderColor: colors.background,
9965 },
9966 levelBadgeDark: {
9967   borderColor: '#0D0D1A',
9968   shadowColor: colors.primary, shadowOffset: { width: 0, height: 2 },
9969   shadowOpacity: 0.4, shadowRadius: 8, elevation: 4,
9970 },
9971 levelBadgeText: { color: '#FFFFFF', fontSize: 10, fontWeight: '600', },
9972 userInfo: { flex: 1, paddingTop: 4 },
9973 nameRow: { flexDirection: 'row', alignItems: 'center', gap: 8, marginBottom: 6 },
9974 userName: { color: colors.textPrimary, fontSize: 22, fontWeight: '600', },
9975 titleBadge: { backgroundColor: colors.borderLight, paddingHorizontal: 10, paddingVertical: 3, borderRadius:
9976 titleBadgeDark: {
9977   backgroundColor: 'rgba(120,100,220,0.2)', borderWidth: 0.5, borderColor: 'rgba(160,140,240,0.15)',
9978 },
9979 titleBadgeText: { color: colors.primary, fontSize: 11, fontWeight: '500', },
9980 userMeta: { flexDirection: 'row', alignItems: 'center', gap: 4 },
9981 streakText: { color: colors.textSecondary, fontSize: 12, },
9982 metaSeparator: { color: colors.textTertiary, fontSize: 12, marginHorizontal: 4 },
9983 levelText: { color: colors.textSecondary, fontSize: 12, },
9984 xpSection: {},
9985 xpHeader: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', marginBottom: 8 },
9986 xpText: { color: colors.textSecondary, fontSize: 12, },
9987 xpPercent: { color: colors.primary, fontSize: 12, fontWeight: '600', },
9988 xpBar: { height: 8, backgroundColor: colors.border, borderRadius: 0, overflow: 'hidden' },
9989 xpBarDark: { backgroundColor: 'rgba(255,255,255,0.06)' },
9990 xpProgress: { height: '100%', backgroundColor: colors.primary, borderRadius: 0 },
9991 statsRow: {
9992   paddingHorizontal: 20,
9993   marginBottom: 20,
9994   flexDirection: 'row',
9995   gap: 8,
9996 },
9997 statCard: {
9998   flex: 1,
9999   borderRadius: 14,
10000   backgroundColor: colors.surface,
10001   borderWidth: 1,
10002   borderColor: colors.hairline,
10003   padding: 12,
10004   alignItems: 'center',

```

```

10005  shadowColor: colors.shadow,
10006  shadowOffset: { width: 0, height: 1 },
10007  shadowOpacity: 0.04,
10008  shadowRadius: 3,
10009  elevation: 2,
10010 },
10011 statIcon: {
10012   fontSize: 18,
10013   marginBottom: 2,
10014 },
10015 statValue: {
10016   fontWeight: '600',
10017   fontSize: 16,
10018   marginTop: 2,
10019 },
10020 statLabel: {
10021   color: colors.textSecondary,
10022   fontSize: 10,
10023 },
10024 tabsContainer: {
10025   paddingHorizontal: 20,
10026   marginBottom: 16,
10027 },
10028 tabs: {
10029   flexDirection: 'row',
10030   gap: 4,
10031   padding: 4,
10032   borderRadius: 12,
10033   backgroundColor: colors.surface,
10034 },
10035 tabButton: {
10036   flex: 1,
10037   paddingVertical: 8,
10038   borderRadius: 10,
10039   alignItems: 'center',
10040 },
10041 tabButtonActive: {
10042   backgroundColor: colors.primary + '15',
10043   borderWidth: 1,
10044   borderColor: colors.primary + '30',
10045   flex: 1,
10046   paddingVertical: 8,
10047   borderRadius: 10,
10048   alignItems: 'center',
10049 },
10050 tabButtonText: {
10051   color: colors.textSecondary,
10052   fontSize: 12,
10053   fontWeight: '400',
10054 },

```

```

10055 tabButtonTextActive: {
10056   color: colors.primary,
10057   fontWeight: '600',
10058 },
10059 tabContent: {
10060   paddingHorizontal: 20,
10061 },
10062 settingsGroup: {
10063   borderRadius: 14,
10064   backgroundColor: colors.surface,
10065   borderWidth: 1,
10066   borderColor: colors.hairline,
10067   overflow: 'hidden',
10068   marginBottom: 12,
10069   shadowColor: colors.shadow,
10070   shadowOffset: { width: 0, height: 1 },
10071   shadowOpacity: 0.04,
10072   shadowRadius: 3,
10073   elevation: 2,
10074 },
10075 settingItem: {
10076   flexDirection: 'row',
10077   alignItems: 'center',
10078   gap: 12,
10079   paddingHorizontal: 16,
10080   paddingVertical: 14,
10081 },
10082 settingItemBorder: {
10083   borderBottomWidth: 1,
10084   borderBottomColor: 'rgba(255,255,255,0.04)',
10085 },
10086 settingIconContainer: {
10087   width: 32,
10088   height: 32,
10089   borderRadius: 8,
10090   backgroundColor: colors.borderLight,
10091   alignItems: 'center',
10092   justifyContent: 'center',
10093 },
10094 settingLabel: {
10095   flex: 1,
10096   color: colors.textPrimary,
10097   fontSize: 14,
10098 },
10099 settingRight: {
10100   flexDirection: 'row',
10101   alignItems: 'center',
10102   gap: 8,
10103 },
10104 settingText: {

```

```

10105   color: colors.textSecondary,
10106   fontSize: 13,
10107 },
10108 loginButton: {
10109   width: '100%',
10110   paddingVertical: 16,
10111   borderRadius: 14,
10112   flexDirection: 'row',
10113   alignItems: 'center',
10114   justifyContent: 'center',
10115   gap: 8,
10116   backgroundColor: colors.primary + '12',
10117   borderWidth: 1,
10118   borderColor: colors.primary + '25',
10119   marginTop: 12,
10120 },
10121 loginText: {
10122   color: colors.primary,
10123   fontWeight: '600',
10124   fontSize: 14,
10125 },
10126 loginPrompt: {
10127   alignItems: 'center',
10128   padding: 20,
10129 },
10130 loginPromptTitle: {
10131   color: colors.textPrimary,
10132   fontSize: 18,
10133   fontWeight: '600',
10134   marginBottom: 8,
10135 },
10136 loginPromptDesc: {
10137   color: colors.textSecondary,
10138   fontSize: 14,
10139   marginBottom: 16,
10140 },
10141 loginPromptButton: {
10142   flexDirection: 'row',
10143   alignItems: 'center',
10144   gap: 8,
10145   paddingHorizontal: 24,
10146   paddingVertical: 12,
10147   borderRadius: 14,
10148   backgroundColor: colors.primary,
10149   shadowColor: colors.shadow,
10150   shadowOffset: { width: 0, height: 2 },
10151   shadowOpacity: 0.08,
10152   shadowRadius: 8,
10153   elevation: 2,
10154 },

```

```

10155 loginPromptButtonText: {
10156   color: colors.onPrimary,
10157   fontWeight: '600',
10158   fontSize: 14,
10159 },
10160 logoutButton: {
10161   width: '100%',
10162   paddingVertical: 16,
10163   borderRadius: 14,
10164   flexDirection: 'row',
10165   alignItems: 'center',
10166   justifyContent: 'center',
10167   gap: 8,
10168   backgroundColor: colors.error + '12',
10169   borderWidth: 1,
10170   borderColor: colors.error + '25',
10171   marginTop: 12,
10172 },
10173 logoutText: {
10174   color: colors.error,
10175   fontWeight: '600',
10176   fontSize: 15,
10177 },
10178 // ---- Pro 升级板块 ----
10179 proSection: {
10180   marginHorizontal: 24,
10181   marginTop: 4,
10182   marginBottom: 20,
10183 },
10184 proCard: {
10185   borderRadius: 16,
10186   padding: 20,
10187   borderWidth: 1.5,
10188   borderColor: colors.proBorder,
10189   backgroundColor: colors.proBg,
10190   overflow: 'hidden',
10191   position: 'relative',
10192 },
10193 proCardGlow: {
10194   position: 'absolute',
10195   top: -30,
10196   right: -30,
10197   width: 120,
10198   height: 120,
10199   borderRadius: 60,
10200   opacity: 0.12,
10201 },
10202 proCardHeader: {
10203   flexDirection: 'row',
10204   alignItems: 'center',

```

```

10205   justifyContent: 'space-between',
10206   marginBottom: 14,
10207 },
10208 proCardTitleRow: {
10209   flexDirection: 'row',
10210   alignItems: 'center',
10211   gap: 10,
10212 },
10213 proCardIcon: {
10214   width: 40,
10215   height: 40,
10216   borderRadius: 12,
10217   backgroundColor: colors.pro,
10218   alignItems: 'center',
10219   justifyContent: 'center',
10220 },
10221 proCardTitle: {
10222   fontSize: 18,
10223   fontWeight: '700' as const,
10224   color: colors.textPrimary,
10225 },
10226 proCardBadge: {
10227   backgroundColor: colors.pro,
10228   paddingHorizontal: 10,
10229   paddingVertical: 3,
10230   borderRadius: 12,
10231 },
10232 proCardBadgeText: {
10233   fontSize: 11,
10234   fontWeight: '700' as const,
10235   color: '#FFFFFF',
10236 },
10237 proFeatureList: {
10238   gap: 10,
10239   marginBottom: 16,
10240 },
10241 proFeatureItem: {
10242   flexDirection: 'row',
10243   alignItems: 'center',
10244   gap: 10,
10245 },
10246 proFeatureCheck: {
10247   width: 20,
10248   height: 20,
10249   borderRadius: 10,
10250   backgroundColor: colors.pro + '20',
10251   alignItems: 'center',
10252   justifyContent: 'center',
10253 },
10254 proFeatureText: {

```

```

10255     fontSize: 14,
10256     color: colors.textPrimary,
10257   },
10258   proUpgradeBtn: {
10259     height: 46,
10260     borderRadius: 23,
10261     alignItems: 'center',
10262     justifyContent: 'center',
10263     flexDirection: 'row',
10264     gap: 8,
10265   },
10266   proUpgradeBtnText: {
10267     fontSize: 15,
10268     fontWeight: '700' as const,
10269     color: '#FFFFFF',
10270   },
10271   // 头像选择弹窗
10272   modalOverlay: {
10273     position: 'absolute', top: 0, left: 0, right: 0, bottom: 0,
10274     backgroundColor: 'rgba(0,0,0,0.5)', zIndex: 100,
10275     alignItems: 'center', justifyContent: 'center',
10276   },
10277   modalCard: {
10278     width: '80%', borderRadius: 16, padding: 24, alignItems: 'center',
10279     backgroundColor: colors.surface,
10280     borderWidth: 1,
10281     borderColor: colors.hairline,
10282   },
10283   modalTitle: {
10284     fontSize: 18, fontWeight: '600', marginBottom: 20,
10285   },
10286   avatarOptions: {
10287     flexDirection: 'row', gap: 12, marginBottom: 20,
10288   },
10289   avatarOption: {
10290     width: 80, height: 90, borderRadius: 14, borderWidth: 1,
10291     alignItems: 'center', justifyContent: 'center', gap: 6, borderColor: colors.hairline,
10292   },
10293   avatarOptionEmoji: { fontSize: 32 },
10294   avatarOptionLabel: { fontSize: 12, fontWeight: '600', },
10295   modalCloseBtn: {
10296     paddingHorizontal: 32, paddingVertical: 10, borderRadius: 12,
10297   },
10298   modalCancelBtn: {
10299     flex: 1, paddingVertical: 12, borderRadius: 14, alignItems: 'center',
10300   },
10301   modalConfirmBtn: {
10302     flex: 1, paddingVertical: 12, borderRadius: 14, alignItems: 'center',
10303   },
10304   bottomPadding: {

```

```

10305     height: 100,
10306   },
10307   )), [colors]);
10308
10309   return (
10310     <SafeAreaView style={styles.container} edges={['top']}>
10311       <ScrollView contentContainerStyle={styles.scrollContent} showsVerticalScrollIndicator={false}>
10312         <View style={styles.header}>
10313           <View style={styles.headerGradient} />
10314           <View style={styles.headerContent}>
10315             <View style={styles.headerTop}>
10316               <Text style={styles.headerTitle}>个人中心</Text>
10317             </View>
10318
10319             <View style={[styles.profileCard, isDark && styles.profileCardDark]}>
10320               <View style={styles.profileCardContent}>
10321                 {isLoggedIn ? (
10322                   <>
10323                     <View style={styles.profileInfo}>
10324                       <View style={styles.avatarContainer}>
10325                         <View style={[styles.avatar, isDark && styles.avatarDark]}>
10326                           <Text style={styles.avatarEmoji}></Text>
10327                         </View>
10328                         <View style={[styles.levelBadge, isDark && styles.levelBadgeDark]}>
10329                           <Text style={styles.levelBadgeText}>{userData.level}</Text>
10330                         </View>
10331                       </View>
10332                       <View style={styles.userInfo}>
10333                         <View style={styles.nameRow}>
10334                           <Text style={styles.userName}>{userData.name}</Text>
10335                         {isPro && (
10336                           <View style={{ backgroundColor: colors.proBg, borderWidth: 1, borderColor: colors.
10337                             <Text style={{ color: colors.pro, fontSize: 10, fontWeight: '700' }}>PRO</Text>
10338                           </View>
10339                         )}
10340                       </View>
10341                       <View style={{ flexDirection: 'row', gap: 16, marginTop: 6 }}>
10342                         <View style={{ flexDirection: 'row', alignItems: 'center', gap: 4 }}>
10343                           <Ionicons name="calendar" size={13} color={colors.textSecondary} />
10344                           <Text style={{ color: colors.textSecondary, fontSize: 12, }}>加入 {daysSinceJoin} 天
10345                         </View>
10346                         <View style={{ flexDirection: 'row', alignItems: 'center', gap: 4 }}>
10347                           <Ionicons name="grid" size={13} color={colors.textSecondary} />
10348                           <Text style={{ color: colors.textSecondary, fontSize: 12, }}>{domainCount} 个领域</T
10349                         </View>
10350                       </View>
10351                       <View style={styles.userMeta}>
10352                         <Ionicons name="flame" size={14} color={colors.warning} />
10353                         <Text style={styles.streakText}> {userData.streak}天连续</Text>
10354                         <Text style={styles.metaSeparator}>·</Text>

```



```

10355         <Text style={styles.levelText}>Lv.{userData.level}</Text>
10356     </View>
10357 </View>
10358 </View>
10359
10360 <View style={styles.xpSection}>
10361     <View style={styles.xpHeader}>
10362         <Text style={styles.xpText}>{userData.xp.toLocaleString()} / {userData.xpToNextLevel.t
10363         <Text style={styles.xpPercent}>{xpPercent}%</Text>
10364     </View>
10365     <View style={{styles.xpBar, isDark && styles.xpBarDark}}>
10366         <View style={{styles.xpProgress, { width: `${xpPercent}%` }}} />
10367     </View>
10368 </View>
10369 </>
10370 ) : (
10371     <View style={styles.loginPrompt}>
10372         <Text style={styles.loginPromptTitle}>欢迎使用 LearnFlow</Text>
10373         <Text style={styles.loginPromptDesc}>登录账号以同步您的学习数据</Text>
10374         <TouchableOpacity style={styles.loginPromptButton} onPress={handleLogin}>
10375             <Ionicons name="log-in" size={16} color="#fff" />
10376             <Text style={styles.loginPromptButtonText}>立即登录</Text>
10377         </TouchableOpacity>
10378     </View>
10379 })
10380 </View>
10381 </View>
10382 </View>
10383 </View>
10384
10385 {renderSettings()}
10386
10387 <View style={styles.bottomPadding} />
10388 </ScrollView>
10389
10390 <HelpModal
10391     visible={showHelpModal}
10392     onClose={() => setShowHelpModal(false)}
10393 />
10394
10395 {/* LearnFlow 指南弹窗 —— 样式与 HelpModal 保持一致 */}
10396 {showGuideModal && (
10397     <View style={{ position: 'absolute', top: 0, left: 0, right: 0, bottom: 0, backgroundColor: 'rgba(0,0,
10398     <View style={{ width: '90%', maxHeight: '80%', backgroundColor: colors.background, borderRadius: 4,
10399     <View style={{ flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', paddin
10400     <Text style={{ color: colors.textPrimary, fontSize: 18, fontWeight: '600', }}> Learnflow指南</Te
10401     <TouchableOpacity onPress={() => setShowGuideModal(false)} style={{ padding: 8 }}>
10402         <Ionicons name="close" size={28} color={colors.textSecondary} />
10403     </TouchableOpacity>
10404 </View>

```

```

10405 <ScrollView style={{ paddingHorizontal: 20, paddingVertical: 16, maxHeight: 400 }} contentContaine
10406 {{
10407   '体力系统：学习消耗体力，完成游戏可恢复。',
10408   '□ 能量：通过专注学习收集，无上限。对话消耗能量 = 怪兽回复字数 × 0.05。',
10409   '小游戏：每日可玩3次，恢复体力。叛逆型怪兽双倍奖励。',
10410   '学习地图：选择领域→技能树→点击节点→跳转学习。',
10411   '番茄钟：专注计时页面可点怪兽图标对话答疑。',
10412   '小怪兽：活力型学习减时，沉稳型多体力，叛逆型双倍。',
10413   '自定义模块：创建专属领域，向左滑动可删除。',
10414   '首页模块：长按模块卡片可删除，轻点进入学习。',
10415   '每日凌晨5点重置游戏次数和每日数据。',
10416   ].map((tip, i) => (
10417     <Text key={i} style={{ color: colors.textSecondary, fontSize: 13, lineHeight: 26, marginBotto
10418   )))
10419 </ScrollView>
10420 </View>
10421 </View>
10422 }}
10423
10424 /* 头像选择弹窗 */
10425 {showAvatarModal && (
10426   <View style={styles.modalOverlay}>
10427     <View style={[styles.modalCard, { backgroundColor: colors.card }]}>
10428       <Text style={styles.modalTitle, { color: colors.textPrimary }}>选择头像</Text>
10429       <View style={styles.avatarOptions}>
10430         {{
10431           { type: 'male' as const, emoji: '', label: '男生' },
10432           { type: 'female' as const, emoji: '', label: '女生' },
10433           { type: 'monster' as const, emoji: '', label: '小怪兽' },
10434         ].map(opt => (
10435           <TouchableOpacity
10436             key={opt.type}
10437             style={{
10438               styles.avatarOption,
10439               { backgroundColor: pendingAvatarType === opt.type ? (colors.primary + '20') : colors.borde
10440             }}
10441             onPress={() => handleAvatarSelect(opt.type)}
10442             activeOpacity={0.7}
10443           >
10444             <Text style={styles.avatarOptionEmoji}>{opt.emoji}</Text>
10445             <Text style={styles.avatarOptionLabel, { color: pendingAvatarType === opt.type ? colors.pri
10446           </TouchableOpacity>
10447         )))
10448 </View>
10449 <View style={{ flexDirection: 'row', gap: 12, width: '100%' }}>
10450   <TouchableOpacity style={styles.modalCancelBtn, { backgroundColor: colors.borderLight }} onPress=
10451     <Text style={{ color: colors.textSecondary, fontWeight: '600' }}>取消</Text>
10452   </TouchableOpacity>
10453   <TouchableOpacity style={styles.modalConfirmBtn, { backgroundColor: colors.primary }} onPress=
10454     <Text style={{ color: '#FFFFFF', fontWeight: '600' }}>确认修改</Text>

```

```

10455         </TouchableOpacity>
10456     </View>
10457 </View>
10458 </View>
10459 }}
10460
10461     {/* Pro 付费弹窗 */}
10462     <SubscriptionModal visible={showProModal} onClose={() => setShowProModal(false)} />
10463 </SafeAreaView>
10464 );
10465
10466
10467 };
10468
10469 export default ProfileScreen;
10470
10471 === mobile/app/_layout.tsx ===
10472 import { Stack } from 'expo-router';
10473 import { StatusBar } from 'expo-status-bar';
10474 import { ThemeProvider, useTheme } from '../src/contexts/ThemeContext';
10475
10476 function AppContent() {
10477     const { isDark } = useTheme();
10478     return (
10479         <>
10480             <Stack screenOptions={{ headerShown: false }}>
10481                 <Stack.Screen name="story" />
10482                 <Stack.Screen name="login" />
10483                 <Stack.Screen name="register" />
10484                 <Stack.Screen name="onboarding" />
10485                 <Stack.Screen name="monster-selection" />
10486                 <Stack.Screen name="module-selection" />
10487                 <Stack.Screen name="(tabs)" />
10488                 <Stack.Screen name="skill-tree" />
10489                 <Stack.Screen name="pomodoro" options={{ gestureEnabled: false }} />
10490             </Stack>
10491             <StatusBar style={isDark ? 'light' : 'dark'} />
10492         </>
10493     );
10494 }
10495
10496 export default function RootLayout() {
10497     return (
10498         <ThemeProvider>
10499             <AppContent />
10500         </ThemeProvider>
10501     );
10502 }
10503
10504 === mobile/app/identity-selection.tsx ===

```

```

10503 import React, { useState, useMemo, useCallback } from 'react';
10504 import { View, Text, StyleSheet, TouchableOpacity, Animated, ScrollView } from 'react-native';
10505 import { SafeAreaView } from 'react-native-safe-area-context';
10506 import { router } from 'expo-router';
10507 import { Ionicons } from '@expo/vector-icons';
10508 import AsyncStorage from '@react-native-async-storage/async-storage';
10509 import { useTheme } from '../src/contexts/ThemeContext';
10510 import { getCurrentUser } from '../src/utils/auth';
10511 import { userService } from '../src/services/api';
10512
10513 type IdentityType = 'student' | 'worker';
10514
10515 interface DomainOption {
10516   id: string;
10517   name: string;
10518   description: string;
10519   icon: string;
10520 }
10521
10522 const STUDENT_DOMAINS: DomainOption[] = [
10523   {
10524     id: 'programming-basics',
10525     name: '编程基础',
10526     description: '零基础学Python，从Hello World到小项目实战',
10527     icon: 'code-slash',
10528   },
10529   {
10530     id: 'finance-basics',
10531     name: '理财入门',
10532     description: '学会记账、攒钱，让零花钱开始生钱',
10533     icon: 'wallet',
10534   },
10535   {
10536     id: 'cet-exam',
10537     name: '四六级过关',
10538     description: '词汇→听力→阅读→写作→翻译，全攻略备考',
10539     icon: 'school',
10540   },
10541 ];
10542
10543 const WORKER_DOMAINS: DomainOption[] = [
10544   {
10545     id: 'ai-product-manager',
10546     name: 'AI产品经理',
10547     description: '掌握Prompt/RAG/Agent三大AI产品兵器',
10548     icon: 'hardware-chip',
10549   },
10550   {
10551     id: 'personal-finance',
10552     name: '理财进阶',

```

```

10553   description: '构建完整理财体系，从资产配置到指数定投',
10554   icon: 'trending-up',
10555 },
10556 {
10557   id: 'english-communication',
10558   name: '英语沟通',
10559   description: '开口→听懂→实战，三阶告别哑巴英语',
10560   icon: 'globe',
10561 },
10562 ];
10563
10564 const IdentitySelectionScreen = () => {
10565   const { colors, isDark } = useTheme();
10566   const [step, setStep] = useState<'identity' | 'domain'>('identity');
10567   const [selectedIdentity, setSelectedIdentity] = useState<IdentityType | null>(null);
10568   const [selectedDomain, setSelectedDomain] = useState<string | null>(null);
10569   const [fadeAnim] = useState(new Animated.Value(1));
10570
10571   const domains = selectedIdentity === 'student' ? STUDENT_DOMAINS : WORKER_DOMAINS;
10572
10573   /** 根据 domain ID 返回模块主题色和 cardBg */
10574   const getDomainStyle = useCallback((domainId: string) => {
10575     const CATEGORY_MAP: Record<string, { color: string; cardBg: string }> = {
10576       'ai-product-manager': {
10577         color: colors.primary,
10578         cardBg: isDark ? 'rgba(123,117,216,0.10)' : 'rgba(90,84,160,0.12)',
10579       },
10580       'programming-basics': {
10581         color: colors.primary,
10582         cardBg: isDark ? 'rgba(123,117,216,0.10)' : 'rgba(90,84,160,0.12)',
10583       },
10584       'personal-finance': {
10585         color: colors.success,
10586         cardBg: isDark ? 'rgba(74,152,64,0.08)' : 'rgba(90,128,64,0.12)',
10587       },
10588       'finance-basics': {
10589         color: colors.success,
10590         cardBg: isDark ? 'rgba(74,152,64,0.08)' : 'rgba(90,128,64,0.12)',
10591       },
10592       'english-communication': {
10593         color: colors.orange,
10594         cardBg: isDark ? 'rgba(212,160,88,0.08)' : 'rgba(196,154,96,0.12)',
10595       },
10596       'cet-exam': {
10597         color: colors.orange,
10598         cardBg: isDark ? 'rgba(212,160,88,0.08)' : 'rgba(196,154,96,0.12)',
10599       },
10600     };
10601     return CATEGORY_MAP[domainId] || { color: colors.primary, cardBg: colors.card };
10602   }, [colors, isDark]);

```

```

10603
10604 const handleSelectIdentity = (identity: IdentityType) => {
10605   console.log('[Identity] 选择身份:', identity);
10606   Animated.sequence([
10607     Animated.timing(fadeAnim, { toValue: 0, duration: 200, useNativeDriver: true }),
10608   ]).start(() => {
10609     setSelectedIdentity(identity);
10610     setStep('domain');
10611     Animated.timing(fadeAnim, { toValue: 1, duration: 250, useNativeDriver: true }).start();
10612   });
10613 };
10614
10615 const saveIdentityToBackend = async () => {
10616   try {
10617     const user = await getCurrentUser();
10618     if (user?.id && selectedIdentity) {
10619       await userService.updateIdentity(selectedIdentity);
10620       console.log('[Identity] 身份已保存到服务端:', selectedIdentity);
10621     }
10622   } catch (error) {
10623     console.warn('[Identity] 保存身份到服务端失败:', error);
10624   }
10625 };
10626
10627 const handleStart = useCallback(async () => {
10628   // 保存身份到后端
10629   await saveIdentityToBackend();
10630
10631   // 如果有选领域, 直接覆写 selectedModules (新手引导阶段不应保留旧数据)
10632   if (selectedDomain) {
10633     await AsyncStorage.setItem('selectedModules', JSON.stringify([selectedDomain]));
10634   }
10635
10636   console.log('[Identity] 完成新手引导, 进入主应用');
10637   router.replace('/(tabs)');
10638 }, [selectedDomain, selectedIdentity]);
10639
10640 const handleSkip = useCallback(async () => {
10641   await saveIdentityToBackend();
10642   console.log('[Identity] 跳过领域选择, 进入主应用');
10643   router.replace('/(tabs)');
10644 }, []);
10645
10646 const styles = useMemo(() => StyleSheet.create({
10647   container: {
10648     flex: 1,
10649     backgroundColor: colors.background,
10650   },
10651   content: {
10652     flex: 1,

```

```

10653     paddingHorizontal: 24,
10654     paddingTop: 48,
10655     paddingBottom: 32,
10656   },
10657   stepIndicator: {
10658     flexDirection: 'row',
10659     justifyContent: 'center',
10660     gap: 8,
10661     marginBottom: 32,
10662   },
10663   stepDot: {
10664     width: 8,
10665     height: 8,
10666     borderRadius: 4,
10667   },
10668   header: {
10669     alignItems: 'center',
10670     marginBottom: 32,
10671   },
10672   title: {
10673     color: colors.textPrimary,
10674     fontSize: 26,
10675     fontWeight: '600',
10676
10677     marginBottom: 8,
10678   },
10679   subtitle: {
10680     color: colors.textSecondary,
10681     fontSize: 14,
10682
10683     textAlign: 'center',
10684     lineHeight: 22,
10685   },
10686   identityCards: {
10687     gap: 16,
10688   },
10689   identityCard: {
10690     borderRadius: 20,
10691     borderWidth: 2,
10692     padding: 24,
10693     flexDirection: 'row',
10694     alignItems: 'center',
10695     gap: 16,
10696   },
10697   identityIconContainer: {
10698     width: 64,
10699     height: 64,
10700     borderRadius: 20,
10701     alignItems: 'center',
10702     justifyContent: 'center',

```

```

10703     flexShrink: 0,
10704 },
10705 identityInfo: {
10706     flex: 1,
10707 },
10708 identityName: {
10709     fontSize: 20,
10710     fontWeight: '600',
10711
10712     marginBottom: 4,
10713 },
10714 identityDesc: {
10715     fontSize: 13,
10716
10717 },
10718 domainCards: {
10719     gap: 16,
10720 },
10721 domainCard: {
10722     borderRadius: 20,
10723     borderWidth: 2,
10724     padding: 20,
10725     flexDirection: 'row',
10726     alignItems: 'center',
10727     gap: 16,
10728     overflow: 'hidden',
10729     position: 'relative',
10730 },
10731 domainCardSelected: {
10732     borderWidth: 2.5,
10733 },
10734 domainIconContainer: {
10735     width: 52,
10736     height: 52,
10737     borderRadius: 16,
10738     alignItems: 'center',
10739     justifyContent: 'center',
10740     flexShrink: 0,
10741 },
10742 domainInfo: {
10743     flex: 1,
10744 },
10745 domainName: {
10746     fontSize: 17,
10747     fontWeight: '600',
10748
10749     marginBottom: 4,
10750 },
10751 domainDescription: {
10752     fontSize: 12,

```



```

10753
10754     lineHeight: 18,
10755   },
10756   checkCircle: {
10757     width: 28,
10758     height: 28,
10759     borderRadius: 14,
10760     alignItems: 'center',
10761     justifyContent: 'center',
10762   },
10763   skipSection: {
10764     marginTop: 24,
10765     alignItems: 'center',
10766   },
10767   skipButton: {
10768     flexDirection: 'row',
10769     alignItems: 'center',
10770     gap: 6,
10771     paddingVertical: 12,
10772     paddingHorizontal: 20,
10773   },
10774   skipText: {
10775     fontSize: 14,
10776     color: colors.textSecondary,
10777
10778   },
10779   spacer: {
10780     flex: 1,
10781   },
10782   startButton: {
10783     paddingVertical: 16,
10784     borderRadius: 16,
10785     alignItems: 'center',
10786     justifyContent: 'center',
10787     marginTop: 24,
10788   },
10789   startButtonText: {
10790     color: 'FFFFFF',
10791     fontSize: 16,
10792     fontWeight: '600',
10793
10794   },
10795   ), [colors]);
10796
10797   return (
10798     <SafeAreaView style={styles.container} edges={['top', 'bottom']}>
10799       <ScrollView contentContainerStyle={styles.content} showsVerticalScrollIndicator={false}>
10800         {/* 步骤指示器 */}
10801         <View style={styles.stepIndicator}>
10802           <View style={['stepDot', { backgroundColor: step === 'identity' ? colors.primary : colors.prima

```

```

10803     <View style={{styles.stepDot, { backgroundColor: step === 'domain' ? colors.primary : colors.border
10804 </View>
10805
10806 <Animated.View style={{ opacity: fadeAnim }}>
10807 {step === 'identity' ? (
10808     <>
10809         {/* 第一步：选择身份 */}
10810         <View style={styles.header}>
10811             <Text style={styles.title}>选择你的身份</Text>
10812             <Text style={styles.subtitle}>告诉我们你的状态，我们为你推荐最适合的学习内容</Text>
10813         </View>
10814
10815         <View style={styles.identityCards}>
10816             <TouchableOpacity
10817                 activeOpacity={0.7}
10818                 onPress={() => handleSelectIdentity('student')}
10819             >
10820                 <View style={{styles.identityCard, {
10821                     backgroundColor: colors.card,
10822                     borderColor: colors.border,
10823                 }}>
10824                     <View style={{styles.identityIconContainer, { backgroundColor: '#4A90D9' + '20' }}}>
10825                         <Text style={{ fontSize: 32 }}></Text>
10826                     </View>
10827                     <View style={styles.identityInfo}>
10828                         <Text style={{styles.identityName, { color: colors.textPrimary }}}>我在上学</Text>
10829                         <Text style={{styles.identityDesc, { color: colors.textSecondary }}}>大学生/研究生，利用课余时间充电</T
10830                     </View>
10831                     <Ionicons name="chevron-forward" size={20} color={colors.textSecondary} />
10832                 </View>
10833             </TouchableOpacity>
10834
10835             <TouchableOpacity
10836                 activeOpacity={0.7}
10837                 onPress={() => handleSelectIdentity('worker')}
10838             >
10839                 <View style={{styles.identityCard, {
10840                     backgroundColor: colors.card,
10841                     borderColor: colors.border,
10842                 }}>
10843                     <View style={{styles.identityIconContainer, { backgroundColor: '#7B75D8' + '20' }}}>
10844                         <Text style={{ fontSize: 32 }}></Text>
10845                     </View>
10846                     <View style={styles.identityInfo}>
10847                         <Text style={{styles.identityName, { color: colors.textPrimary }}}>我已工作</Text>
10848                         <Text style={{styles.identityDesc, { color: colors.textSecondary }}}>职场人士，利用碎片时间提升竞争力</T
10849                     </View>
10850                     <Ionicons name="chevron-forward" size={20} color={colors.textSecondary} />
10851                 </View>
10852             </TouchableOpacity>

```

```

10853     </View>
10854   </>
10855   ): (
10856     <>
10857     /* 第二步：选择学习领域 */
10858     <View style={styles.header}>
10859       <Text style={styles.title}>推荐学习领域</Text>
10860       <Text style={styles.subtitle}>
10861         根据你的身份，为你精选了以下学习方向
10862       </Text>
10863     </View>
10864
10865     <View style={styles.domainCards}>
10866       {domains.map((domain) => {
10867         const isSelected = selectedDomain === domain.id;
10868         const ds = getDomainStyle(domain.id);
10869         return (
10870           <TouchableOpacity
10871             key={domain.id}
10872             activeOpacity={0.7}
10873             onPress={() => setSelectedDomain(domain.id)}
10874           >
10875             <View style={[
10876               styles.domainCard,
10877               {
10878                 backgroundColor: isSelected ? ds.color + '15' : colors.card,
10879                 borderColor: isSelected ? ds.color : colors.border,
10880               },
10881               isDark && isSelected && { borderColor: ds.color },
10882             ]}>
10883               <View style={[styles.domainIconContainer, {
10884                 backgroundColor: ds.color,
10885                 shadowColor: isSelected ? ds.color : 'transparent',
10886                 shadowOpacity: 0.3,
10887                 shadowRadius: 8,
10888                 elevation: isSelected ? 4 : 0,
10889               }]}>
10890                 <Ionicons name={domain.icon as any} size={26} color="#FFFFFF" />
10891               </View>
10892               <View style={styles.domainInfo}>
10893                 <Text style={[styles.domainName, { color: isSelected ? ds.color : colors.textPrimary
10894                   {domain.name}
10895                 </Text>
10896                 <Text style={[styles.domainDescription, { color: colors.textSecondary }]}>
10897                   {domain.description}
10898                 </Text>
10899               </View>
10900               {isSelected && (
10901                 <View style={[styles.checkCircle, { backgroundColor: ds.color }]}>
10902                   <Ionicons name="checkmark" size={18} color="#FFFFFF" />

```

```

10903         </View>
10904     })
10905     </View>
10906     </TouchableOpacity>
10907 );
10908 }}}
10909 </View>
10910
10911 {/* 跳过 */}
10912 <View style={styles.skipSection}>
10913     <TouchableOpacity style={styles.skipButton} onPress={handleSkip}>
10914         <Icons name="arrow-forward" size={14} color={colors.textSecondary} />
10915         <Text style={styles.skipText}>以上没有感兴趣的，可在app内自主添加</Text>
10916     </TouchableOpacity>
10917 </View>
10918 </>
10919 })
10920 </Animated.View>
10921
10922 <View style={styles.spacer} />
10923
10924 {/* 底部按钮 */}
10925 {step === 'domain' && (
10926     <TouchableOpacity
10927         style={[
10928             styles.startButton,
10929             {
10930                 backgroundColor: selectedDomain ? colors.primary : colors.surface,
10931                 opacity: selectedDomain ? 1 : 0.5,
10932                 shadowColor: selectedDomain ? colors.primary : 'transparent',
10933                 shadowOffset: { width: 0, height: 8 },
10934                 shadowOpacity: 0.35,
10935                 shadowRadius: 24,
10936                 elevation: selectedDomain ? 5 : 0,
10937             },
10938         ]}
10939         onPress={handleStart}
10940         disabled={!selectedDomain}
10941         activeOpacity={0.7}
10942     >
10943         <Text style={styles.startButtonText}>开始学习</Text>
10944     </TouchableOpacity>
10945 )}
10946 </ScrollView>
10947 </SafeAreaView>
10948 );
10949 };
10950
10951 export default IdentitySelectionScreen;
10952

```

```

=== mobile/app/index.tsx ===
10953 import React, { useEffect } from 'react';
10954 import { View, Text, StyleSheet } from 'react-native';
10955 import { SafeAreaView } from 'react-native-safe-area-context';
10956 import { router, useRootNavigationState } from 'expo-router';
10957 import { Loading } from '../src/components/ui';
10958 import { SPACING } from '../src/utils/constants';
10959 import { useTheme } from '../src/contexts/ThemeContext';
10960
10961 const IndexScreen = () => {
10962   const { colors } = useTheme();
10963   const rootNavState = useRootNavigationState();
10964
10965   useEffect(() => {
10966     if (rootNavState?.key) {
10967       // 延迟一帧确保导航栈就绪
10968       const timer = setTimeout(() => {
10969         router.replace('/login');
10970       }, 0);
10971       return () => clearTimeout(timer);
10972     }
10973     }, [rootNavState?.key]);
10974
10975   const styles = StyleSheet.create({
10976     container: {
10977       flex: 1,
10978       backgroundColor: colors.background,
10979     },
10980     content: {
10981       flex: 1,
10982       justifyContent: 'center',
10983       alignItems: 'center',
10984       padding: SPACING.LARGE,
10985     },
10986     title: {
10987       fontSize: 32,
10988       fontWeight: 'bold' as const,
10989       color: colors.primary,
10990       marginBottom: SPACING.SMALL,
10991     },
10992     subtitle: {
10993       fontSize: 16,
10994       color: colors.textSecondary,
10995       marginBottom: SPACING.XLARGE,
10996     },
10997   });
10998
10999   return (
11000     <SafeAreaView style={styles.container}>
11001     <View style={styles.content}>

```

```

11002     <Text style={styles.title}>LearnFlow</Text>
11003     <Text style={styles.subtitle}>技能学习路径管理</Text>
11004     <Loading visible={true} message="加载中..." />
11005   </View>
11006 </SafeAreaView>
11007 );
11008 };
11009
11010 export default IndexScreen;
11011
11012 === mobile/app/login.tsx ===
11011 import React, { useState, useMemo, useCallback } from 'react';
11012 import { View, Text, StyleSheet, Alert, ScrollView, TouchableOpacity, Dimensions } from 'react-native';
11013 import { SafeAreaView } from 'react-native-safe-area-context';
11014 import { router } from 'expo-router';
11015 import { Icons } from '@expo/vector-icons';
11016 import { useTheme } from '../src/contexts/ThemeContext';
11017 import { authService } from '../src/services/api';
11018 import { saveAuthData } from '../src/utils/auth';
11019 import AsyncStorage from '@react-native-async-storage/async-storage';
11020 import { STORAGE_KEYS } from '../src/utils/storage';
11021 import { showErrorAlert, toErrorMessage } from '../src/utils';
11022 import { FloatingInputBar, InputFieldConfig } from '../src/components/FloatingInputBar';
11023
11024 const LoginScreen = () => {
11025   const { colors } = useTheme();
11026   const [loginType, setLoginType] = useState<'login' | 'register'>('login');
11027   const [username, setUsername] = useState('');
11028   const [email, setEmail] = useState('');
11029   const [password, setPassword] = useState('');
11030   const [loading, setLoading] = useState(false);
11031   const [showPassword, setShowPassword] = useState(false);
11032   const [activeFieldId, setActiveFieldId] = useState<string | null>(null);
11033
11034   const handleSubmit = async () => {
11035     console.log('[Login] handleSubmit - 类型: ${loginType}, 邮箱: ${email}');
11036     if (!email || !password) {
11037       Alert.alert('错误', '请输入邮箱和密码');
11038       return;
11039     }
11040
11041     // 邮箱格式校验
11042     const emailRegex = /^[^s@]+@[^s@]+\.[^s@]+$/;
11043     if (!emailRegex.test(email)) {
11044       Alert.alert('错误', '请输入有效的邮箱地址');
11045       return;
11046     }
11047
11048     if (loginType === 'register') {
11049       if (!username.trim()) {
11050         Alert.alert('错误', '请输入用户名');

```

```

11051     return;
11052 }
11053 if (password.length < 6) {
11054     Alert.alert('错误', '密码长度至少6位');
11055     return;
11056 }
11057 }
11058
11059 setLoading(true);
11060 try {
11061
11062     if (loginType === 'register') {
11063         console.log('[Login] 开始注册 - 用户名:', username, '邮箱:', email);
11064         await authService.register({
11065             username: username.trim(),
11066             email,
11067             password
11068         });
11069         // 注册成功, 跳转验证码页面
11070         setLoading(false);
11071         router.push({ pathname: '/verify-email', params: { email, username: username.trim() } });
11072         return;
11073     }
11074
11075     // 登录
11076     console.log('[Login] 开始登录 - 邮箱:', email);
11077     const authResponse = await authService.login({
11078         email,
11079         password,
11080         deviceId: 'mobile-device',
11081         type: 'email',
11082         deviceType: 'android',
11083         deviceName: '移动设备'
11084     });
11085
11086     // 登录成功
11087     console.log('[Login] 登录成功');
11088     await saveAuthData(authResponse);
11089
11090     setLoading(false);
11091     Alert.alert('登录成功', '欢迎回来! ');
11092     router.replace('/(tabs)');
11093 } catch (error) {
11094     console.error('[Login] ${loginType === 'register' ? '注册' : '登录'}失败:', error);
11095     setLoading(false);
11096     showErrorAlert(
11097         loginType === 'register' ? '注册失败' : '登录失败',
11098         toErrorMessage(error)
11099     );
11100 }

```

```

11101  });
11102
11103  // ---- 浮动输入栏字段配置 ----
11104  const inputFields: InputFieldConfig[] = useMemo(() => {
11105    const fields: InputFieldConfig[] = [];
11106    if (loginType === 'register') {
11107      fields.push({
11108        id: 'username',
11109        label: '用户名',
11110        icon: 'person',
11111        value: username,
11112        onChangeText: setUsername,
11113        placeholder: '请输入用户名',
11114      });
11115    }
11116    fields.push(
11117      {
11118        id: 'email',
11119        label: '邮箱',
11120        icon: 'mail',
11121        value: email,
11122        onChangeText: setEmail,
11123        keyboardType: 'email-address',
11124        placeholder: '请输入邮箱',
11125      },
11126      {
11127        id: 'password',
11128        label: '密码',
11129        icon: 'lock-closed',
11130        value: password,
11131        onChangeText: setPassword,
11132        secureTextEntry: !showPassword,
11133        placeholder: loginType === 'register' ? '至少6位密码' : '请输入密码',
11134      },
11135    );
11136    return fields;
11137  }, [loginType, username, email, password, showPassword]);
11138
11139  const styles = useMemo(() => StyleSheet.create({
11140    container: {
11141      flex: 1,
11142      backgroundColor: colors.background,
11143    },
11144    scrollContent: {
11145      flexGrow: 1,
11146      paddingHorizontal: 20,
11147      paddingBottom: 0,
11148      minHeight: Dimensions.get('window').height,
11149    },
11150    backgroundDecorations: {

```



```

11151   position: 'absolute',
11152   top: 0,
11153   left: 0,
11154   right: 0,
11155   bottom: 0,
11156   overflow: 'hidden',
11157 },
11158 gradientCircle1: {
11159   position: 'absolute',
11160   top: -80,
11161   right: -80,
11162   width: 256,
11163   height: 256,
11164   borderRadius: 128,
11165   backgroundColor: '#D4A574',
11166   opacity: 0.08,
11167 },
11168 gradientCircle2: {
11169   position: 'absolute',
11170   top: '33%',
11171   left: -80,
11172   width: 192,
11173   height: 192,
11174   borderRadius: 96,
11175   backgroundColor: '#C89070',
11176   opacity: 0.06,
11177 },
11178 gradientCircle3: {
11179   position: 'absolute',
11180   bottom: 80,
11181   right: 40,
11182   width: 128,
11183   height: 128,
11184   borderRadius: 64,
11185   backgroundColor: '#D4A574',
11186   opacity: 0.04,
11187 },
11188 logoSection: {
11189   alignItems: 'center',
11190   paddingTop: 48,
11191   paddingBottom: 32,
11192 },
11193 logoContainer: {
11194   width: 80,
11195   height: 80,
11196   borderRadius: 2,
11197   backgroundColor: '#D4A574',
11198   justifyContent: 'center',
11199   alignItems: 'center',
11200   marginBottom: 16,

```

```

11201   shadowColor: '#D4A574',
11202   shadowOffset: { width: 0, height: 8 },
11203   shadowOpacity: 0.4,
11204   shadowRadius: 32,
11205   elevation: 5,
11206 },
11207 appTitle: {
11208   fontSize: 28,
11209   fontWeight: '600',
11210   color: '#fff',
11211   marginBottom: 4,
11212   letterSpacing: -0.5,
11213 },
11214 appSubtitle: {
11215   fontSize: 14,
11216   color: colors.textSecondary,
11217 },
11218 tabContainer: {
11219   paddingHorizontal: 20,
11220   marginBottom: 24,
11221 },
11222 tabSwitcher: {
11223   flexDirection: 'row',
11224   backgroundColor: colors.surface,
11225   borderRadius: 2,
11226   padding: 4,
11227 },
11228 tabButton: {
11229   flex: 1,
11230   paddingVertical: 12,
11231   borderRadius: 2,
11232   alignItems: 'center',
11233 },
11234 activeTabButton: {
11235   backgroundColor: '#D4A574',
11236   shadowColor: '#D4A574',
11237   shadowOffset: { width: 0, height: 2 },
11238   shadowOpacity: 0.15,
11239   shadowRadius: 8,
11240   elevation: 2,
11241 },
11242 tabText: {
11243   fontSize: 15,
11244   color: colors.textSecondary,
11245   fontWeight: '400',
11246 },
11247 activeTabText: {
11248   color: '#fff',
11249   fontWeight: '600',
11250 },

```

```

11251 formContainer: {
11252   gap: 16,
11253 },
11254 inputContainer: {
11255   height: 56,
11256   flexDirection: 'row',
11257   alignItems: 'center',
11258   backgroundColor: colors.inputBg,
11259   borderWidth: 1,
11260   borderColor: colors.inputBorder,
11261   borderRadius: 10,
11262   paddingHorizontal: 16,
11263 },
11264 inputIcon: {
11265   marginRight: 12,
11266 },
11267 textInput: {
11268   flex: 1,
11269   fontSize: 15,
11270   color: colors.textPrimary,
11271   paddingVertical: 0,
11272 },
11273 displayText: {
11274   flex: 1,
11275   fontSize: 15,
11276   paddingVertical: 0,
11277 },
11278 passwordToggle: {
11279   padding: 4,
11280 },
11281 forgotPassword: {
11282   alignSelf: 'flex-end',
11283 },
11284 forgotPasswordText: {
11285   fontSize: 13,
11286   color: '#D4A574',
11287 },
11288 loginButton: {
11289   height: 56,
11290   borderRadius: 2,
11291   backgroundColor: '#D4A574',
11292   justifyContent: 'center',
11293   alignItems: 'center',
11294   shadowColor: '#D4A574',
11295   shadowOffset: { width: 0, height: 4 },
11296   shadowOpacity: 0.3,
11297   shadowRadius: 12,
11298   elevation: 5,
11299 },
11300 loginButtonDisabled: {

```

```

11301     backgroundColor: colors.border,
11302     shadowOpacity: 0,
11303   },
11304   loginButtonText: {
11305     color: '#fff',
11306     fontSize: 16,
11307     fontWeight: '600',
11308   },
11309   loadingSpinner: {
11310     width: 20,
11311     height: 20,
11312     borderRadius: 10,
11313     borderWidth: 2,
11314     borderColor: '#fff',
11315     borderTopColor: 'transparent',
11316   },
11317   termsText: {
11318     textAlign: 'center',
11319     color: colors.textSecondary,
11320     fontSize: 12,
11321     marginBottom: 20,
11322   },
11323   termsLink: {
11324     color: colors.primary,
11325   },
11326   )), [colors]);
11327
11328   return (
11329     <SafeAreaView style={styles.container} edges={['top', 'bottom']}>
11330       <ScrollView
11331         style={styles.container}
11332         contentContainerStyle={styles.scrollContent}
11333         showsVerticalScrollIndicator={false}
11334         keyboardShouldPersistTaps="handled"
11335       >
11336         {/* 背景装饰 */}
11337         <View style={styles.backgroundDecorations}>
11338           <View style={styles.gradientCircle1} />
11339           <View style={styles.gradientCircle2} />
11340           <View style={styles.gradientCircle3} />
11341         </View>
11342
11343         {/* Logo 区域 */}
11344         <View style={styles.logoSection}>
11345           <Text style={styles.appTitle}>LearnFlow</Text>
11346           <Text style={styles.appSubtitle}>开启你的技能冒险之旅</Text>
11347         </View>
11348
11349         {/* Tab 切换器 */}
11350         <View style={styles.tabContainer}>

```

```

11351 <View style={styles.tabSwitcher}>
11352   <TouchableOpacity
11353     style={[styles.tabButton, loginType === 'login' && styles.activeTabButton]}
11354     onPress={() => setLoginType('login')}
11355   >
11356     <Text style={[styles.tabText, loginType === 'login' && styles.activeTabText]}>
11357       登录
11358     </Text>
11359   </TouchableOpacity>
11360   <TouchableOpacity
11361     style={[styles.tabButton, loginType === 'register' && styles.activeTabButton]}
11362     onPress={() => setLoginType('register')}
11363   >
11364     <Text style={[styles.tabText, loginType === 'register' && styles.activeTabText]}>
11365       注册
11366     </Text>
11367   </TouchableOpacity>
11368 </View>
11369 </View>
11370
11371 /* 表单 */
11372 <View style={styles.formContainer}>
11373   {loginType === 'register' && (
11374     <TouchableOpacity
11375       style={styles.inputContainer}
11376       activeOpacity={0.7}
11377       onPress={() => setActiveFieldId('username')}
11378     >
11379       <Ionicons name="person" size={20} color={colors.primary} style={styles.inputIcon} />
11380       <Text
11381         style={[styles.displayText, username ? { color: colors.textPrimary } : { color: colors.textS
11382           numberOfLines={1}
11383       >
11384         {username || '用户名'}
11385       </Text>
11386     </TouchableOpacity>
11387   )}
11388
11389   <TouchableOpacity
11390     style={styles.inputContainer}
11391     activeOpacity={0.7}
11392     onPress={() => setActiveFieldId('email')}
11393   >
11394     <Ionicons name="mail" size={20} color={colors.primary} style={styles.inputIcon} />
11395     <Text
11396       style={[styles.displayText, email ? { color: colors.textPrimary } : { color: colors.textSecond
11397       numberOfLines={1}
11398     >
11399       {email || '邮箱'}
11400     </Text>

```

```

11401     </TouchableOpacity>
11402
11403     <TouchableOpacity
11404         style={styles.inputContainer}
11405         activeOpacity={0.7}
11406         onPress={() => setActiveFieldId('password')}
11407     >
11408         <Ionicons name="lock-closed" size={20} color={colors.primary} style={styles.inputIcon} />
11409         <Text
11410             style={{styles.displayText, password ? { color: colors.textPrimary } : { color: colors.textSec
11411                 numberOfLines={1}
11412         >
11413             {password ? '●'.repeat(Math.min(password.length, 12)) : '密码'}
11414         </Text>
11415         <TouchableOpacity
11416             style={styles.passwordToggle}
11417             onPress={() => setShowPassword(!showPassword)}
11418             hitSlop={{ top: 8, bottom: 8, left: 8, right: 8 }}
11419         >
11420             <Ionicons
11421                 name={showPassword ? "eye-off" : "eye"}
11422                 size={20}
11423                 color={colors.textSecondary}
11424             />
11425         </TouchableOpacity>
11426     </TouchableOpacity>
11427
11428     {loginType === 'login' && (
11429         <TouchableOpacity style={styles.forgotPassword}>
11430             <Text style={styles.forgotPasswordText}>忘记密码? </Text>
11431         </TouchableOpacity>
11432     )}
11433
11434     <TouchableOpacity
11435         style={{styles.loginButton, loading && styles.loginButtonDisabled}}
11436         onPress={handleSubmit}
11437         disabled={loading}
11438         activeOpacity={0.7}
11439     >
11440         {loading ? (
11441             <View style={styles.loadingSpinner} />
11442         ) : (
11443             <Text style={styles.loginButtonText}>{loginType === 'login' ? '登录' : '创建账号'}</Text>
11444         )}
11445     </TouchableOpacity>
11446 </View>
11447
11448 {/* 底部条款 */}
11449 <Text style={styles.termsText}>
11450     继续即表示你同意我们的{' '}

```

```

11451     <Text style={styles.termsLink}>服务条款</Text> 和{' '}
11452     <Text style={styles.termsLink}>隐私政策</Text>
11453   </Text>
11454 </ScrollView>
11455
11456   {/* 键盘上方浮动输入栏 */}
11457   <FloatingInputBar
11458     fields={inputFields}
11459     activeFieldId={activeFieldId}
11460     onDismiss={() => setActiveFieldId(null)}
11461     colors={colors}
11462   />
11463 </SafeAreaView>
11464   );
11465
11466
11467 };
11468
11469 export default LoginScreen;
11470
11471 === mobile/app/module-selection.tsx ===
11472 import React, { useState, useEffect, useMemo } from 'react';
11473 import {
11474   View, Text, StyleSheet, TouchableOpacity, Animated, ScrollView,
11475   TextInput, Alert, KeyboardAvoidingView, Platform, Modal, ActivityIndicator,
11476 } from 'react-native';
11477 import { SafeAreaView } from 'react-native-safe-area-context';
11478 import { router, useLocalSearchParams } from 'expo-router';
11479 import { Ionicons } from '@expo/vector-icons';
11480 import { useTheme } from '../src/contexts/ThemeContext';
11481 import AsyncStorage from '@react-native-async-storage/async-storage';
11482 import MonsterIcon from '../src/components/MonsterIcon';
11483 import { PlatformType } from '../src/types/skill';
11484 import { API_BASE_URL, MONSTER_CONFIG } from '../src/utils/constants';
11485 import { storage, STORAGE_KEYS } from '../src/utils/storage';
11486 import SubscriptionModal from '../src/components/SubscriptionModal';
11487 import { getCurrentUser } from '../src/utils/auth';
11488 import { proService } from '../src/services/api';
11489 import { SUBSCRIPTION_STORAGE_KEY } from '../src/utils/pricing';
11490
11491 // ---- 平台选项 (与 skill-tree 保持一致) ----
11492 const PLATFORM_OPTIONS: { key: PlatformType; label: string; icon: string; color: string }[] = [
11493   { key: 'bilibili', label: 'B站', icon: 'play-circle', color: '#FB7299' },
11494   { key: 'xiaohongshu', label: '小红书', icon: 'book', color: '#FF2442' },
11495   { key: 'mooc', label: '中国大学MOOC', icon: 'school', color: '#5D9BFA' },
11496 ];
11497
11498 // ---- 类型定义 ----
11499 type ModuleType = 'ai-product-manager' | 'personal-finance' | 'english-communication' | string;

```

```

11500
11501 type ModuleCategory = 'professional' | 'life' | 'language';
11502
11503 interface Module {
11504   id: ModuleType;
11505   name: string;
11506   icon: string;
11507   category: ModuleCategory;
11508   description: string;
11509   difficulty: string;
11510 }
11511
11512 /** 自定义模块中一个大标题（阶段） */
11513 interface CustomStageDraft {
11514   localId: string; // 临时 ID，提交时替换为正式 ID
11515   name: string;
11516 }
11517
11518 /** 自定义模块中一个小结点 */
11519 interface CustomNodeDraft {
11520   localId: string;
11521   stageLocalId: string; // 归属哪个大标题
11522   name: string;
11523   url: string;
11524   platform: PlatformType;
11525 }
11526
11527 // ---- 预设模块列表 ----
11528 const predefinedModules: Module[] = [
11529   // 专业技能类（primary）
11530   { id: 'ai-product-manager', name: 'AI产品经理', icon: 'hardware-chip',
11531     category: 'professional', description: '掌握AI产品设计与落地', difficulty: '中级' },
11532   { id: 'programming-basics', name: '编程基础', icon: 'code-slash',
11533     category: 'professional', description: '从零开始学编程', difficulty: '初级' },
11534   // 生活技能类（success）
11535   { id: 'personal-finance', name: '理财进阶', icon: 'trending-up',
11536     category: 'life', description: '深入投资理财实战', difficulty: '中级' },
11537   { id: 'finance-basics', name: '理财入门', icon: 'wallet',
11538     category: 'life', description: '从0到1管好钱', difficulty: '初级' },
11539   // 语言学习类（orange）
11540   { id: 'english-communication', name: '英语沟通', icon: 'language',
11541     category: 'language', description: '提升英语听说能力', difficulty: '初级' },
11542   { id: 'cet-exam', name: '四六级过关', icon: 'school',
11543     category: 'language', description: '高效备战四六级', difficulty: '初级' },
11544 ];
11545
11546 // ---- 工具函数 ----
11547 let draftIdCounter = 0;
11548 const generateDraftId = () => `draft_${Date.now()}_${draftIdCounter++}`;
11549

```



```

11550 const ModuleSelectionScreen = () => {
11551   const { colors } = useTheme();
11552
11553   // 根据分类获取颜色
11554   const getCategoryColor = (category: ModuleCategory): string => {
11555     switch (category) {
11556       case 'professional': return colors.primary;
11557       case 'life': return colors.success;
11558       case 'language': return colors.orange;
11559     }
11560   };
11561   const { mode } = useLocalSearchParams<{ mode?: string }>();
11562   const isAddMode = mode === 'add';
11563
11564   // ---- 预设模块选择相关状态 ----
11565   const [selectedModules, setSelectedModules] = useState<ModuleType[]>([]);
11566   const [existingModules, setExistingModules] = useState<ModuleType[]>([]);
11567   const [fadeAnim] = useState(new Animated.Value(1));
11568
11569   // ---- 自定义模块创建相关状态 ----
11570   const [customName, setCustomName] = useState("");
11571   const [customDescription, setCustomDescription] = useState("");
11572   // 大标题草稿列表，确保至少有一个
11573   const [stageDrafts, setStageDrafts] = useState<CustomStageDraft[]>([]);
11574   // 小结点草稿列表
11575   const [nodeDrafts, setNodeDrafts] = useState<CustomNodeDraft[]>([]);
11576   // toast 提示
11577   const [toastMessage, setToastMessage] = useState("");
11578   // AI 加载状态
11579   const [aiLoading, setAiLoading] = useState(false);
11580   // 添加模式下的 tab 切换
11581   const [activeAddTab, setActiveAddTab] = useState<'official' | 'aicustom'>('official');
11582   // Pro 弹窗
11583   const [showProModal, setShowProModal] = useState(false);
11584
11585   // ---- 初始化 ----
11586   useEffect(() => {
11587     if (isAddMode) {
11588       const loadExisting = async () => {
11589         const stored = await AsyncStorage.getItem('selectedModules');
11590         if (stored) {
11591           const parsed = JSON.parse(stored) as ModuleType[];
11592           setExistingModules(parsed);
11593           // 进入添加模式时，初始选中已有模块，防止覆盖
11594           setSelectedModules(parsed);
11595         }
11596       };
11597       loadExisting();
11598       // 初始化时预创建 1 个大标题和 1 个小结点
11599       const stageId = generateDraftId();

```

```

11600     const nodeId = generateDraftId();
11601     setStageDrafts([{ localId: stageId, name: " " }]);
11602     setNodeDrafts([{ localId: nodeId, stageLocalId: stageId, name: "", url: "", platform: 'bilibili' }]);
11603   } else {
11604     setSelectedModules(['ai-product-manager']);
11605   }
11606 }, [isAddMode]);
11607
11608 // ---- 预设模块：列表（全部展示，不消失） ----
11609 const availableModules = predefinedModules;
11610
11611 // ---- 预设模块：切换选中 ----
11612 const toggleModule = (id: ModuleType) => {
11613   if (isAddMode) {
11614     // 添加模式：官方领域 tab 下只能单选，AI自定义 tab 下可多选
11615     if (activeAddTab === 'official') {
11616       // 单选：点击已选中的取消，点击新的替换
11617       if (selectedModules.includes(id)) {
11618         setSelectedModules(selectedModules.filter(m => m !== id));
11619       } else {
11620         setSelectedModules([id]);
11621       }
11622     } else {
11623       // AI自定义 tab 下多选（追加已有模块）
11624       if (selectedModules.includes(id)) {
11625         setSelectedModules(selectedModules.filter(m => m !== id));
11626       } else {
11627         setSelectedModules([...selectedModules, id]);
11628       }
11629     }
11630   } else {
11631     // 初始选择模式下多选（最多3个）
11632     if (selectedModules.includes(id)) {
11633       setSelectedModules(selectedModules.filter(m => m !== id));
11634     } else {
11635       if (selectedModules.length >= 3) return;
11636       setSelectedModules([...selectedModules, id]);
11637     }
11638   }
11639 };
11640
11641 // ---- Pro 检测（以数据库 is_pro 字段为准） ----
11642 const [isPro, setIsPro] = useState(false);
11643
11644 useEffect(() => {
11645   const checkPro = async () => {
11646     try {
11647       const user = await getCurrentUser();
11648       if (user?.id) {
11649         const proStatus = await proService.getStatus(user.id);

```

```

11650     setIsPro(proStatus.isPro);
11651   }
11652 } catch {}
11653 };
11654 checkPro();
11655 }, []);
11656
11657 // ===== 自定义模块：大标题操作 =====
11658
11659 /** 添加一个大标题草稿 */
11660 const handleAddStage = () => {
11661   const newId = generateDraftId();
11662   setStageDrafts(prev => [...prev, { localId: newId, name: '' }]);
11663 };
11664
11665 /** 更新大标题名称 */
11666 const handleStageNameChange = (stageId: string, name: string) => {
11667   setStageDrafts(prev => prev.map(s => s.localId === stageId ? { ...s, name } : s));
11668 };
11669
11670 /** 删除一个大标题，同时连带删除其下的小结点草稿 */
11671 const handleRemoveStage = (stageId: string) => {
11672   setStageDrafts(prev => prev.filter(s => s.localId !== stageId));
11673   setNodeDrafts(prev => prev.filter(n => n.stageLocalId !== stageId));
11674 };
11675
11676 // ===== 自定义模块：小结点操作 =====
11677
11678 /** 为指定大标题添加一个小结点草稿 */
11679 const handleAddNode = (stageLocalId: string) => {
11680   setNodeDrafts(prev => [...prev, {
11681     localId: generateDraftId(),
11682     stageLocalId,
11683     name: '',
11684     url: '',
11685     platform: 'bilibili',
11686   }]);
11687 };
11688
11689 /** 更新小结点字段 */
11690 const handleNodeChange = (nodeId: string, field: keyof CustomNodeDraft, value: string) => {
11691   setNodeDrafts(prev => prev.map(n => n.localId === nodeId ? { ...n, [field]: value } : n));
11692 };
11693
11694 /** 删除小结点草稿 */
11695 const handleRemoveNode = (nodeId: string) => {
11696   setNodeDrafts(prev => prev.filter(n => n.localId !== nodeId));
11697 };
11698
11699 // ===== 验证 & 提交 =====

```

```

11700
11701 /** 验证自定义模块创建表单 */
11702 const validateCustomForm = (): string | null => {
11703   if (!customName.trim()) return '请输入模块名称';
11704   if (!customDescription.trim()) return '请输入模块介绍';
11705   // 验证每个大标题都有名称
11706   for (const stage of stageDrafts) {
11707     if (!stage.name.trim()) return '每个大标题都需要填写名称';
11708   }
11709   // 验证每个大标题下至少有一个有效的小结点
11710   for (const stage of stageDrafts) {
11711     const stageNodes = nodeDrafts.filter(n => n.stageLocalId === stage.localId);
11712     if (stageNodes.length === 0) return `大标题「${stage.name} || '未命名'」下至少需要一个结点`;
11713     for (const node of stageNodes) {
11714       if (!node.name.trim()) return '每个结点都需要填写名称';
11715       if (!node.url.trim()) return '每个结点都需要填写链接';
11716       const isValidUrl = node.url.trim().startsWith('http://') || node.url.trim().startsWith('https://');
11717       if (!isValidUrl) return '链接格式不正确，需要以 http:// 或 https:// 开头';
11718     }
11719   }
11720   return null; // 验证通过
11721 };
11722
11723 // ---- AI 一键填充接口响应类型 ----
11724 interface AIFillNode {
11725   nodeName: string;
11726   subNodes: { subName: string; link: string }[];
11727 }
11728 interface AIFillResponse {
11729   data: {
11730     moduleDescription: string;
11731     nodes: AIFillNode[];
11732   };
11733 }
11734
11735 /** AI 一键填充自定义模块内容 */
11736 const handleAIFill = async () => {
11737   if (!customName.trim()) {
11738     Alert.alert('提示', '请先填写模块名称');
11739     return;
11740   }
11741   if (!customDescription.trim()) {
11742     Alert.alert('提示', '请先填写模块介绍');
11743     return;
11744   }
11745
11746   // 免费用户：检查是否已有自定义模块
11747   if (!isPro) {
11748     const hasCustom = existingModules.some(id => id.startsWith('custom-'));
11749     if (hasCustom) {

```

```

11750     Alert.alert('提示', '免费额度已用完, 请升级 Pro 无限畅用', [
11751         { text: '取消', style: 'cancel' },
11752         { text: '了解 Pro', onPress: () => setShowProModal(true) },
11753     ]);
11754     return;
11755 }
11756 }
11757
11758 setAiLoading(true);
11759 try {
11760     const controller = new AbortController();
11761     const timeoutId = setTimeout(() => controller.abort(), 10000);
11762
11763     const response = await fetch(`${API_BASE_URL}/ai/fill-module`, {
11764         method: 'POST',
11765         headers: { 'Content-Type': 'application/json' },
11766         body: JSON.stringify({
11767             moduleName: customName.trim(),
11768             moduleDescription: customDescription.trim(),
11769         }),
11770         signal: controller.signal,
11771     });
11772
11773     clearTimeout(timeoutId);
11774
11775     if (!response.ok) {
11776         throw new Error('AI生成失败, 请重试');
11777     }
11778
11779     const result: AIFillResponse = await response.json();
11780
11781     if (!result.data || !result.data.moduleDescription || !Array.isArray(result.data.nodes)) {
11782         throw new Error('数据解析异常, 请重试');
11783     }
11784
11785     // 保留用户填写的模块介绍, 不覆盖
11786     // 只填充 AI 生成的大标题和小结点
11787
11788     // 清空现有草稿并填充 AI 生成的大标题和小结点
11789     const newStages: CustomStageDraft[] = [];
11790     const newNodes: CustomNodeDraft[] = [];
11791
11792     result.data.nodes.forEach((node, si) => {
11793         const stageId = `ai_stage_${Date.now()}_${si}`;
11794         newStages.push({ localId: stageId, name: node.nodeName });
11795
11796         (node.subNodes || []).forEach((sub) => {
11797             newNodes.push({
11798                 localId: `ai_node_${Date.now()}_${si}_${newNodes.length}`,
11799                 stageLocalId: stageId,

```

```

11800         name: sub.subName || '',
11801         url: sub.link || '',
11802         platform: 'bilibili',
11803     });
11804 });
11805 });
11806
11807 setStageDrafts(newStages);
11808 setNodeDrafts(newNodes);
11809
11810 Alert.alert('生成成功', 'AI 已为你自动填充模块内容，请检查并修改后提交');
11811 } catch (error: any) {
11812     if (error.name === 'AbortError') {
11813         Alert.alert('生成失败', '网络异常，请稍后重试');
11814     } else {
11815         Alert.alert('生成失败', error.message || 'AI生成失败，请重试');
11816     }
11817 } finally {
11818     setAiLoading(false);
11819 }
11820 };
11821
11822 /** 提交自定义模块 */
11823 const handleCreateCustomModule = async () => {
11824     const error = validateCustomForm();
11825     if (error) {
11826         Alert.alert('提示', error);
11827         return;
11828     }
11829
11830     // 免费用户：检查是否已有自定义模块
11831     if (!isPro) {
11832         const hasCustom = existingModules.some(id => id.startsWith('custom-'));
11833         if (hasCustom) {
11834             Alert.alert('提示', '免费额度已用完，请升级 Pro 无限畅用', [
11835                 { text: '取消', style: 'cancel' },
11836                 { text: '了解 Pro', onPress: () => setShowProModal(true) },
11837             ]);
11838             return;
11839         }
11840     }
11841
11842     const moduleId = `custom-${Date.now()}`;
11843
11844     // 1. 保存自定义模块元信息（名称 + 介绍）
11845     const customModulesStr = await AsyncStorage.getItem('customModules');
11846     const customModules: Record<string, { name: string; description: string }> = customModulesStr
11847       ? JSON.parse(customModulesStr) : {};
11848     customModules[moduleId] = {
11849         name: customName.trim(),

```

```

11850     description: customDescription.trim(),
11851   };
11852   await AsyncStorage.setItem('customModules', JSON.stringify(customModules));
11853
11854   // 2. 将草稿转换为正式的自定义 stages/nodes 并保存到 AsyncStorage
11855   const formalStages: { id: string; name: string }[] = stageDrafts.map(s => ({
11856     id: `custom-stage_${s.localId}`,
11857     name: s.name.trim(),
11858   }));
11859
11860   const formalNodes: {
11861     id: string; stageId: string; name: string;
11862     url: string; platform: PlatformType; duration: number;
11863   }[] = nodeDrafts.map(n => {
11864     return {
11865       id: `custom-node_${n.localId}`,
11866       stageId: `custom-stage_${n.stageLocalId}`,
11867       name: n.name.trim(),
11868       url: n.url.trim(),
11869       platform: n.platform,
11870       duration: 1, // 默认1小时, 用户后续可在结点中调整
11871     };
11872   });
11873
11874   await AsyncStorage.setItem(`customStages_${moduleId}`, JSON.stringify(formalStages));
11875   await AsyncStorage.setItem(`customNodes_${moduleId}`, JSON.stringify(formalNodes));
11876
11877   // 3. 更新 selectedModules
11878   const newSelected = [...existingModules, moduleId];
11879   await AsyncStorage.setItem('selectedModules', JSON.stringify(newSelected));
11880
11881   setToastMessage('模块创建成功');
11882   setTimeout(() => {
11883     setToastMessage('');
11884     router.back();
11885   }, 1500);
11886 };
11887
11888 // ---- 预设模块: 确认选择 ----
11889 const handleConfirmPreset = async () => {
11890   const newIds = selectedModules.filter(id => !existingModules.includes(id));
11891   if (newIds.length === 0) return;
11892   // 免费用户检测: 已有官方模块则禁止添加
11893   if (!isPro && existingModules.length > 0) {
11894     Alert.alert('提示', '免费用户仅可选择/拥有一个官方模块, 请先删除原有模块');
11895     return;
11896   }
11897   await AsyncStorage.setItem('selectedModules', JSON.stringify(selectedModules));
11898   if (isAddMode) {
11899     router.back();

```

```

11900   } else {
11901     await AsyncStorage.setItem('onboardingCompleted', 'true');
11902     router.replace('/(tabs)');
11903   }
11904 };
11905
11906 // ---- styles ----
11907 const styles = useMemo(() => StyleSheet.create({
11908   container: { flex: 1, backgroundColor: colors.background },
11909   scrollContent: { flexGrow: 1 },
11910   content: { flex: 1, paddingHorizontal: 24, paddingTop: 20, paddingBottom: 32 },
11911   // 标题区
11912   title: { color: colors.textPrimary, fontSize: 24, fontWeight: '600', marginBottom: 4 },
11913   subtitle: { color: colors.textSecondary, fontSize: 13, marginBottom: 24 },
11914   // 输入框通用
11915   input: {
11916     backgroundColor: colors.inputBg, borderRadius: 10, borderWidth: 1,
11917     borderColor: colors.inputBorder, paddingHorizontal: 16, paddingVertical: 14,
11918     fontSize: 16, color: colors.textPrimary, marginBottom: 16,
11919   },
11920   inputMultiline: {
11921     minHeight: 80, textAlignVertical: 'top',
11922   },
11923   // 分区标题
11924   sectionTitle: {
11925     color: colors.textPrimary, fontSize: 16, fontWeight: '600',
11926     marginBottom: 12, marginTop: 8,
11927   },
11928   sectionHint: {
11929     color: colors.textTertiary, fontSize: 12,
11930     marginBottom: 16, marginTop: -8,
11931   },
11932   // 大标题卡片
11933   stageCard: {
11934     backgroundColor: colors.card, borderRadius: 16, borderWidth: 1,
11935     borderColor: colors.border, padding: 16, marginBottom: 12,
11936   },
11937   stageHeader: {
11938     flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', marginBottom: 12,
11939   },
11940   stageLabel: { color: colors.primary, fontSize: 14, fontWeight: '600' },
11941   stageRemoveBtn: {
11942     width: 28, height: 28, borderRadius: 14, backgroundColor: colors.border,
11943     alignItems: 'center', justifyContent: 'center',
11944   },
11945   stageInput: {
11946     backgroundColor: colors.inputBg, borderRadius: 10, borderWidth: 1,
11947     borderColor: colors.inputBorder, paddingHorizontal: 14, paddingVertical: 12,
11948     fontSize: 15, color: colors.textPrimary, marginBottom: 14,
11949   },

```



```

11950 // 小结点区域
11951 nodeSection: { marginTop: 4 },
11952 nodeLabel: { color: colors.textSecondary, fontSize: 13, fontWeight: '600', marginBottom: 8 },
11953 nodeCard: {
11954   backgroundColor: colors.surface, borderRadius: 14, padding: 12,
11955   marginBottom: 8, position: 'relative',
11956 },
11957 nodeHeader: {
11958   flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', marginBottom: 8,
11959 },
11960 nodeIndex: { color: colors.textTertiary, fontSize: 12 },
11961 nodeNameInput: {
11962   backgroundColor: colors.inputBg, borderRadius: 10, borderWidth: 1,
11963   borderColor: colors.inputBorder, paddingHorizontal: 12, paddingVertical: 10,
11964   fontSize: 14, color: colors.textPrimary, marginBottom: 6,
11965 },
11966 nodeUrlInput: {
11967   backgroundColor: colors.inputBg, borderRadius: 10, borderWidth: 1,
11968   borderColor: colors.inputBorder, paddingHorizontal: 12, paddingVertical: 10,
11969   fontSize: 13, color: colors.textPrimary, marginBottom: 6,
11970 },
11971 // 平台选择器
11972 platformRow: { flexDirection: 'row', gap: 6, marginTop: 4 },
11973 platformChip: {
11974   flexDirection: 'row', alignItems: 'center', gap: 3, paddingHorizontal: 10, paddingVertical: 6,
11975   borderRadius: 8, borderWidth: 1, borderColor: colors.border,
11976 },
11977 platformChipActive: { borderColor: 'transparent' },
11978 platformChipText: { fontSize: 11, fontWeight: '600' },
11979 // 添加按钮
11980 addNodeBtn: {
11981   borderWidth: 1, borderStyle: 'dashed', borderColor: colors.borderDark,
11982   borderRadius: 10, paddingVertical: 10, alignItems: 'center', marginTop: 4,
11983 },
11984 addNodeBtnText: { color: colors.primary, fontSize: 13, fontWeight: '600' },
11985 addStageBtn: {
11986   borderWidth: 1.5, borderStyle: 'dashed', borderColor: colors.borderDark,
11987   borderRadius: 14, paddingVertical: 14, alignItems: 'center', marginTop: 8, marginBottom: 24,
11988 },
11989 addStageBtnText: { color: colors.primary, fontSize: 14, fontWeight: '600' },
11990 // 确认按钮
11991 confirmBtn: {
11992   backgroundColor: colors.primary, borderRadius: 16, paddingVertical: 16,
11993   alignItems: 'center', marginTop: 4, marginBottom: 32,
11994   shadowColor: colors.shadow, shadowOffset: { width: 0, height: 2 },
11995   shadowOpacity: 0.08, shadowRadius: 12, elevation: 3,
11996 },
11997 confirmBtnDisabled: { backgroundColor: colors.border, opacity: 0.5, shadowOpacity: 0 },
11998 confirmBtnText: { color: '#FFFFFF', fontSize: 16, fontWeight: '600' },
11999 // Toast

```

```

12000 toast: {
12001   position: 'absolute', bottom: 40, left: 20, right: 20,
12002   flexDirection: 'row', alignItems: 'center', justifyContent: 'center', gap: 8,
12003   backgroundColor: 'rgba(0,0,0,0.85)', paddingVertical: 12, paddingHorizontal: 20, borderRadius: 12,
12004 },
12005 toastText: { color: '#FFFFFF', fontSize: 14, fontWeight: '600' },
12006 // 添加模式 tab 切换
12007 addTabs: {
12008   flexDirection: 'row',
12009   backgroundColor: colors.surface,
12010   borderRadius: 2,
12011   padding: 3,
12012   marginBottom: 20,
12013 },
12014 addTab: {
12015   flex: 1,
12016   paddingVertical: 10,
12017   borderRadius: 2,
12018   alignItems: 'center',
12019 },
12020 addTabActive: {
12021   backgroundColor: colors.primary,
12022 },
12023 addTabText: {
12024   fontSize: 13,
12025   fontWeight: '600',
12026   fontFamily: 'Courier',
12027 },
12028 // 预设模块卡片 — 主页风格（水平布局）
12029 modulesContainer: {
12030   gap: 10,
12031 },
12032 moduleCard: {
12033   borderRadius: 2,
12034   borderWidth: 2,
12035   overflow: 'hidden',
12036   padding: 14,
12037   flexDirection: 'row',
12038   alignItems: 'center',
12039   gap: 12,
12040 },
12041 moduleCardIcon: {
12042   width: 44,
12043   height: 44,
12044   borderRadius: 2,
12045   alignItems: 'center',
12046   justifyContent: 'center',
12047   flexShrink: 0,
12048 },
12049 moduleCardInfo: {

```

```

12050     flex: 1,
12051 },
12052 moduleCardName: {
12053     fontSize: 14,
12054     fontWeight: '700',
12055     fontFamily: 'Courier',
12056     marginBottom: 2,
12057 },
12058 moduleCardCategory: {
12059     fontSize: 11,
12060     fontWeight: '500',
12061     fontFamily: 'Courier',
12062 },
12063 moduleCardCheck: {
12064     width: 24,
12065     height: 24,
12066     borderRadius: 2,
12067     alignItems: 'center',
12068     justifyContent: 'center',
12069     flexShrink: 0,
12070 },
12071 selectedCount: { textAlign: 'center', color: colors.textSecondary, fontSize: 13, marginTop: 24 },
12072 spacer: { flex: 1 },
12073 startButton: {
12074     paddingVertical: 16, borderRadius: 16, alignItems: 'center',
12075     justifyContent: 'center', marginTop: 24,
12076 },
12077 startButtonText: { color: colors.textPrimary, fontSize: 16, fontWeight: '600' },
12078 // no more modules hint
12079 emptyHint: {
12080     color: colors.textSecondary, fontSize: 14,
12081     textAlign: 'center', marginTop: 40,
12082 },
12083 // 标头小怪兽
12084 header: { alignItems: 'center', marginBottom: 24 },
12085 monsterAvatarContainer: { marginBottom: 16 },
12086 monsterAvatar: { width: 80, height: 80, position: 'relative' },
12087 monsterHeadAvatar: { width: 40, height: 24, backgroundColor: colors.primary, position: 'absolute', top: 16
12088 monsterEyesAvatar: { flexDirection: 'row', gap: 8, position: 'absolute', top: 8, left: 8 },
12089 eyeAvatar: { width: 10, height: 10, backgroundColor: '#FFFFFF', position: 'relative' },
12090 pupilAvatar: { width: 4, height: 5, backgroundColor: colors.background, position: 'absolute', top: 2, left
12091 mouthAvatar: { width: 16, height: 4, backgroundColor: colors.background, position: 'absolute', top: 18, le
12092 bodyAvatar: { width: 32, height: 8, backgroundColor: colors.primary, position: 'absolute', top: 40, left:
12093 bubble: {
12094     backgroundColor: colors.card, borderRadius: 16, borderWidth: 2, borderColor: colors.primary,
12095     paddingHorizontal: 24, paddingVertical: 12, marginBottom: 20,
12096 },
12097 bubbleTail: {
12098     width: 0, height: 0, borderLeftWidth: 8, borderRightWidth: 8, borderBottomWidth: 8,
12099     borderLeftColor: 'transparent', borderRightColor: 'transparent', borderBottomColor: colors.primary,

```

```

12100     position: 'absolute', top: -8, left: '50%', marginLeft: -8,
12101   },
12102   bubbleText: { color: colors.textPrimary, fontSize: 14 },
12103
12104   // ---- AI 填充 ----
12105   nameRow: { flexDirection: 'row', alignItems: 'center', gap: 10 },
12106   nameInput: { flex: 1 },
12107   aiFillBtn: {
12108     backgroundColor: colors.primary, borderRadius: 14, paddingHorizontal: 14,
12109     paddingVertical: 14, justifyContent: 'center', alignItems: 'center',
12110     shadowColor: colors.shadow, shadowOffset: { width: 0, height: 2 },
12111     shadowOpacity: 0.08, shadowRadius: 8, elevation: 3, minWidth: 110,
12112   },
12113   aiFillBtnDisabled: { backgroundColor: colors.border, shadowOpacity: 0 },
12114   aiFillBtnText: { color: '#fff', fontSize: 13, fontWeight: '700' },
12115
12116   // ---- AI 加载 Modal ----
12117   aiModalOverlay: {
12118     flex: 1, backgroundColor: 'rgba(0,0,0,0.6)',
12119     justifyContent: 'center', alignItems: 'center',
12120   },
12121   aiModalCard: {
12122     backgroundColor: colors.surface, borderRadius: 20, padding: 40,
12123     alignItems: 'center', borderWidth: 1, borderColor: colors.border,
12124     marginHorizontal: 40,
12125   },
12126   aiModalTitle: {
12127     color: colors.textPrimary, fontSize: 18, fontWeight: '700', marginTop: 20,
12128   },
12129   aiModalSubtitle: {
12130     color: colors.textSecondary, fontSize: 14, marginTop: 8, textAlign: 'center',
12131   },
12132   }], [colors]);
12133
12134   // ---- 渲染：预设模块选择界面（主页风格，水平卡片） ----
12135   const renderPresetSelection = () => (
12136     <View style={styles.modulesContainer}>
12137       {availableModules.length === 0 && isAddMode ? (
12138         <Text style={styles.emptyHint}>所有预设模块已添加</Text>
12139       ) : (
12140         availableModules.map((module) => {
12141           const isSelected = selectedModules.includes(module.id);
12142           const catColor = getCategoryColor(module.category);
12143           return (
12144             <TouchableOpacity
12145               key={module.id}
12146               onPress={() => toggleModule(module.id)}
12147               activeOpacity={0.7}
12148             >
12149             <View

```

```

12150     style=[{
12151         styles.moduleCard,
12152     {
12153         backgroundColor: isSelected ? catColor : colors.surface,
12154         borderColor: catColor,
12155         shadowColor: isSelected ? catColor : colors.shadow,
12156         shadowOffset: { width: isSelected ? 3 : 0, height: isSelected ? 3 : 0 },
12157         shadowOpacity: isSelected ? 0.5 : 0,
12158         shadowRadius: 0,
12159         elevation: isSelected ? 3 : 0,
12160     },
12161     ]}
12162 >
12163 <View style={[styles.moduleCardIcon, {
12164     backgroundColor: isSelected ? 'rgba(255,255,255,0.2)' : catColor,
12165 }]}>
12166     <Ionicons name={module.icon as any} size={22} color={isSelected ? '#FFFFFF' : colors.backgro
12167 </View>
12168 <View style={styles.moduleCardInfo}>
12169     <Text
12170         style={[styles.moduleCardName, { color: isSelected ? '#FFFFFF' : colors.textPrimary }]}
12171     >
12172         {module.name}
12173     </Text>
12174     <Text
12175         style={[styles.moduleCardCategory, { color: isSelected ? 'rgba(255,255,255,0.8)' : colors.
12176     >
12177         {module.category === 'professional' ? '专业技能' : module.category === 'life' ? '生活技能' : '语言学习
12178     </Text>
12179 </View>
12180 {isSelected && (
12181     <View style={[styles.moduleCardCheck, { backgroundColor: 'rgba(255,255,255,0.3)' }]}>
12182         <Ionicons name="checkmark" size={16} color="#FFFFFF" />
12183     </View>
12184     )}
12185 </View>
12186 </TouchableOpacity>
12187 );
12188 })
12189 })
12190 </View>
12191 );
12192
12193 // ---- 渲染: 自定义模块创建表单 ----
12194 const renderCustomForm = () => (
12195     <>
12196         {/* ===== 第一步: 输入模块名称 ===== */}
12197         <Text style={styles.sectionTitle}>模块名称</Text>
12198         <TextInput
12199             style={[styles.input, styles.nameInput]}

```

```

12200     placeholder="输入自定义模块名称，如「Swift 开发」"
12201     placeholderTextColor={colors.textTertiary}
12202     value={customName}
12203     onChangeText={setCustomName}
12204     maxLength={20}
12205     returnKeyType="done"
12206 />
12207
12208 /* ===== 第二步：输入模块介绍 ===== */
12209 <Text style={styles.sectionTitle}>模块介绍</Text>
12210 <Text style={styles.sectionHint}>描述该模块的学习内容和目标</Text>
12211 <TextInput
12212     style={[styles.input, styles.inputMultiline]}
12213     placeholder="输入一段模块介绍，如「从零开始学习 Swift 语言与 iOS 开发」"
12214     placeholderTextColor={colors.textTertiary}
12215     value={customDescription}
12216     onChangeText={setCustomDescription}
12217     maxLength={200}
12218     multiline
12219     numberOfLines={3}
12220 />
12221
12222 /* ===== 第三步：AI 一键生成 ===== */
12223 <TouchableOpacity
12224     style={[styles.aiFillBtn, { width: '100%' }, aiLoading && styles.aiFillBtnDisabled]}
12225     onPress={handleAIFill}
12226     disabled={aiLoading}
12227     activeOpacity={0.7}
12228 >
12229     {aiLoading ? (
12230         <ActivityIndicator size="small" color="#fff" />
12231     ) : (
12232         <Text style={styles.aiFillBtnText}> AI一键生成</Text>
12233     )}
12234 </TouchableOpacity>
12235
12236 /* ===== 第四步：创建模块结点 ===== */
12237 <Text style={styles.sectionTitle}>模块结点</Text>
12238 <Text style={styles.sectionHint}>至少需要一个大标题和一个学习结点</Text>
12239
12240 /* 遍历所有大标题草稿 */
12241 {stageDrafts.map((stage, stageIndex) => {
12242     const stageNodes = nodeDrafts.filter(n => n.stageLocalId === stage.localId);
12243     return (
12244         <View key={stage.localId} style={styles.stageCard}>
12245             /* 大标题头部：标签 + 删除按钮 */
12246             <View style={styles.stageHeader}>
12247                 <Text style={styles.stageLabel}>大标题 {stageIndex + 1}</Text>
12248                 {stageDrafts.length > 1 && (
12249                     <TouchableOpacity

```

```

12250         style={styles.stageRemoveBtn}
12251         onPress={() => handleRemoveStage(stage.localId)}
12252         hitSlop={{ top: 8, bottom: 8, left: 8, right: 8 }}
12253     >
12254         <Ionicons name="close" size={14} color={colors.textSecondary} />
12255     </TouchableOpacity>
12256 }}
12257 </View>
12258
12259 {/* 大标题名称输入 */}
12260 <TextInput
12261     style={styles.stageInput}
12262     placeholder="输入大标题，如「初级阶段」「基础入门」"
12263     placeholderTextColor={colors.textTertiary}
12264     value={stage.name}
12265     onChangeText={(text) => handleStageNameChange(stage.localId, text)}
12266     maxLength={20}
12267 />
12268
12269 {/* 该大标题下的小结点列表 */}
12270 <View style={styles.nodeSection}>
12271     <Text style={styles.nodeLabel}>学习结点</Text>
12272     {stageNodes.map((node, nodeIndex) => (
12273         <View key={node.localId} style={styles.nodeCard}>
12274             {/* 结点头部：序号 + 删除 */}
12275             <View style={styles.nodeHeader}>
12276                 <Text style={styles.nodeIndex}>结点 {nodeIndex + 1}</Text>
12277                 {stageNodes.length > 1 && (
12278                     <TouchableOpacity
12279                         onPress={() => handleRemoveNode(node.localId)}
12280                         hitSlop={{ top: 6, bottom: 6, left: 6, right: 6 }}
12281                     >
12282                         <Ionicons name="close-circle" size={16} color={colors.textTertiary} />
12283                     </TouchableOpacity>
12284                 )}
12285             </View>
12286
12287             {/* 结点名称 */}
12288             <TextInput
12289                 style={styles.nodeNameInput}
12290                 placeholder="输入结点名称，如「Swift 基础语法」"
12291                 placeholderTextColor={colors.textTertiary}
12292                 value={node.name}
12293                 onChangeText={(text) => handleNodeChange(node.localId, 'name', text)}
12294                 maxLength={30}
12295             />
12296
12297             {/* 结点链接 */}
12298             <TextInput
12299                 style={styles.nodeUrlInput}

```

```

12300     placeholder="输入学习链接 (https://...)"
12301     placeholderTextColor={colors.textTertiary}
12302     value={node.url}
12303     onChangeText={({text}) => handleNodeChange(node.localId, 'url', text)}
12304     autoCapitalize="none"
12305     keyboardType="url"
12306   />
12307
12308   {/* 平台选择器 */}
12309   <View style={styles.platformRow}>
12310     {PLATFORM_OPTIONS.map(opt => {
12311       const isActive = node.platform === opt.key;
12312       return (
12313         <TouchableOpacity
12314           key={opt.key}
12315           style={[
12316             styles.platformChip,
12317             isActive && { backgroundColor: opt.color + '20', borderColor: opt.color },
12318           ]}
12319           onPress={() => handleNodeChange(node.localId, 'platform', opt.key)}
12320         >
12321           <Ionicons name={opt.icon as any} size={14} color={opt.color} />
12322           <Text style={[
12323             styles.platformChipText,
12324             { color: isActive ? opt.color : colors.textSecondary },
12325           ]}>
12326             {opt.label}
12327           </Text>
12328         </TouchableOpacity>
12329       );
12330     })}
12331   </View>
12332 </View>
12333 )))
12334
12335   {/* 为当前大标题添加更多结点 */}
12336   <TouchableOpacity
12337     style={styles.addNodeBtn}
12338     onPress={() => handleAddNode(stage.localId)}
12339     activeOpacity={0.7}
12340   >
12341     <Text style={styles.addNodeBtnText}>+ 添加结点</Text>
12342   </TouchableOpacity>
12343 </View>
12344 </View>
12345 );
12346 )))
12347
12348   {/* ===== 添加更多大标题 ===== */}
12349   <TouchableOpacity

```



```

12350     style={styles.addStageBtn}
12351     onPress={handleAddStage}
12352     activeOpacity={0.7}
12353   >
12354     <Text style={styles.addStageBtnText}>+ 添加大标题</Text>
12355   </TouchableOpacity>
12356
12357   /* ===== 第四步：确认添加 ===== */
12358   <TouchableOpacity
12359     style={styles.confirmBtn}
12360     onPress={handleCreateCustomModule}
12361     activeOpacity={0.8}
12362   >
12363     <Text style={styles.confirmBtnText}>确认添加</Text>
12364   </TouchableOpacity>
12365 </>
12366 );
12367
12368 // ---- 小怪兽头像（复用） ----
12369 const MonsterAvatar = () => (
12370   <MonsterIcon type="calm" size={80} />
12371 );
12372
12373 return (
12374   <SafeAreaView style={styles.container} edges={['top', 'bottom']}>
12375     <KeyboardAvoidingView
12376       style={{ flex: 1 }}
12377       behavior={Platform.OS === 'ios' ? 'padding' : undefined}
12378       keyboardVerticalOffset={Platform.OS === 'ios' ? 10 : 0}
12379     >
12380       <ScrollView
12381         contentContainerStyle={styles.scrollContent}
12382         showsVerticalScrollIndicator={false}
12383         keyboardShouldPersistTaps="handled"
12384       >
12385         <View style={styles.content}>
12386           /* ---- 头部小怪兽 ---- */
12387           {!isAddMode && (
12388             <Animated.View style={[styles.header, { opacity: fadeAnim }]}>
12389               <View style={styles.monsterAvatarContainer}>
12390                 <MonsterAvatar />
12391               </View>
12392               <View style={styles.bubble}>
12393                 <View style={styles.bubbleTail} />
12394                 <Text style={styles.bubbleText}>小怪兽，我们要探索哪些领域呢？</Text>
12395               </View>
12396             </Animated.View>
12397           )}
12398
12399           /* ---- 标题 ---- */

```

```

12400 <Text style={styles.title}>{isAddMode ? '创建自定义模块' : '选择学习模块'}</Text>
12401 <Text style={styles.subtitle}>
12402   {isAddMode
12403     ? '填写模块信息并创建学习结点，构建属于你的学习地图'
12404     : '最多选择 3 个模块开始学习'}
12405 </Text>
12406
12407 {/* ---- 根据模式渲染不同内容 ---- */}
12408 {isAddMode ? (
12409   <>
12410     {/* Tab 切换器 */}
12411     <View style={styles.addTabs}>
12412       <TouchableOpacity
12413         style={[styles.addTab, activeAddTab === 'official' && styles.addTabActive]}
12414         onPress={() => setActiveAddTab('official')}
12415         activeOpacity={0.7}
12416       >
12417         <Text style={[styles.addTabText, {
12418           color: activeAddTab === 'official' ? '#FFFFFF' : colors.textSecondary,
12419         }]}>官方模块</Text>
12420       </TouchableOpacity>
12421       <TouchableOpacity
12422         style={[styles.addTab, activeAddTab === 'aicustom' && styles.addTabActive]}
12423         onPress={() => setActiveAddTab('aicustom')}
12424         activeOpacity={0.7}
12425       >
12426         <Text style={[styles.addTabText, {
12427           color: activeAddTab === 'aicustom' ? '#FFFFFF' : colors.textSecondary,
12428         }]}>AI 自定义</Text>
12429       </TouchableOpacity>
12430     </View>
12431
12432     {/* 官方模块 tab */}
12433     {activeAddTab === 'official' && (
12434       <>
12435         {availableModules.length === 0 ? (
12436           <Text style={styles.emptyHint}>所有预设模块已添加</Text>
12437         ) : (
12438           <>
12439             {renderPresetSelection()}
12440             {selectedModules.length > 0 && (
12441               <>
12442                 <Text style={styles.selectedCount}>已选中 1 个预设模块</Text>
12443                 <TouchableOpacity
12444                   style={[styles.startButton, {
12445                     backgroundColor: colors.primary,
12446                     shadowColor: colors.shadow,
12447                     shadowOffset: { width: 0, height: 2 },
12448                     shadowOpacity: 0.08, shadowRadius: 12, elevation: 3,
12449                     marginBottom: 16,

```

```

12450         }}}
12451         onPress={handleConfirmPreset}
12452         activeOpacity={0.7}
12453     >
12454         <Text style={styles.startButtonText}>确认添加预设模块 </Text>
12455     </TouchableOpacity>
12456 </>
12457     })
12458 </>
12459 })
12460 </>
12461 })
12462
12463 { /* AI 自定义 tab */
12464 {activeAddTab === 'aicustom' && renderCustomForm()}
12465 </>
12466 ) : (
12467 <>{ /* 初始选择模式：只显示预设模块 */
12468     <Animated.View style={{ opacity: fadeAnim }}>
12469         {renderPresetSelection()}
12470     </Animated.View>
12471
12472     <Text style={styles.selectedCount}>
12473         已选择 {selectedModules.length}/3 个模块
12474     </Text>
12475
12476     <View style={styles.spacer} />
12477
12478     <TouchableOpacity
12479         style={[
12480             styles.startButton,
12481             {
12482                 backgroundColor: selectedModules.length === 0 ? colors.border : colors.primary,
12483                 opacity: selectedModules.length === 0 ? 0.5 : 1,
12484                 shadowColor: selectedModules.length === 0 ? 'transparent' : colors.shadow,
12485                 shadowOffset: { width: 0, height: 2 },
12486                 shadowOpacity: selectedModules.length === 0 ? 0 : 0.08,
12487                 shadowRadius: 12,
12488                 elevation: selectedModules.length === 0 ? 0 : 3,
12489             },
12490         ]}
12491         onPress={handleConfirmPreset}
12492         disabled={selectedModules.length === 0}
12493         activeOpacity={0.7}
12494     >
12495         <Text style={styles.startButtonText}>开始学习之旅 </Text>
12496     </TouchableOpacity>
12497 </>
12498 })
12499 </View>

```

```

12500     </ScrollView>
12501   </KeyboardAvoidingView>
12502
12503   {/* ---- Toast 提示 ---- */}
12504   {toastMessage !== '' && (
12505     <View style={styles.toast}>
12506       <Icons name="checkmark-circle" size={18} color="#3AE374" />
12507       <Text style={styles.toastText}>{toastMessage}</Text>
12508     </View>
12509   )}
12510
12511   {/* ---- AI 生成进度弹窗（不可关闭） ---- */}
12512   <Modal visible={aiLoading} transparent animationType="fade">
12513     <View style={styles.aiModalOverlay}>
12514       <View style={styles.aiModalCard}>
12515         <ActivityIndicator size="large" color="#5D9BFA" />
12516         <Text style={styles.aiModalTitle}>AI 正在生成</Text>
12517         <Text style={styles.aiModalSubtitle}>
12518           AI正在生成学习内容，请稍候...
12519         </Text>
12520       </View>
12521     </View>
12522   </Modal>
12523   <SubscriptionModal visible={showProModal} onClose={() => setShowProModal(false)} />
12524 </SafeAreaView>
12525 );
12526 };
12527
12528 export default ModuleSelectionScreen;
12529
12530 === mobile/app/monster-selection.tsx ===
12529 import React, { useState, useMemo } from 'react';
12530 import { View, Text, StyleSheet, TouchableOpacity, TextInput, Animated, ScrollView } from 'react-native';
12531 import { SafeAreaView } from 'react-native-safe-area-context';
12532 import { router } from 'expo-router';
12533 import { Icons } from '@expo/vector-icons';
12534 import { useTheme } from '../src/contexts/ThemeContext';
12535 import { storage, STORAGE_KEYS } from '../src/utils/storage';
12536 import { MONSTER_CONFIG } from '../src/utils/constants';
12537 import { monsterService } from '../src/services/api';
12538 import { getCurrentUser } from '../src/utils/auth';
12539 import MonsterIcon from '../src/components/MonsterIcon';
12540
12541 type MonsterType = 'lively' | 'calm' | 'rebel';
12542
12543 const MonsterSelectionScreen = () => {
12544   const { colors } = useTheme();
12545   const [selectedType, setSelectedType] = useState<MonsterType>('calm');
12546   const [monsterName, setMonsterName] = useState('');
12547   const [step, setStep] = useState<'select' | 'name'>('select');
12548   const [fadeAnim] = useState(new Animated.Value(1));

```

```

12549
12550 const monsters = useMemo() => [
12551   {
12552     id: 'lively' as MonsterType,
12553     name: '活力小怪',
12554     personality: '元气满满',
12555     color: colors.orange,
12556     // 选中背景用实色暖橘，与怪兽本身的亮橙拉开差异
12557     selectedBg: '#f1ddc5fe',
12558     description: '适合快节奏碎片化学习',
12559     trait: '专属特权：单次学习任务时长直接减少 5 分钟',
12560   },
12561   {
12562     id: 'calm' as MonsterType,
12563     name: '沉稳小怪',
12564     personality: '冷静沉着',
12565     color: MONSTER_CONFIG.COLORS.calm.primary,
12566     // 选中背景用实色中蓝，与怪兽的深天蓝拉开差异
12567     selectedBg: '#81c3e1ff',
12568     description: '擅长深度思考',
12569     trait: '专属特权：每日额外赠送 20 点体力，可多 2 次跳转',
12570   },
12571   {
12572     id: 'rebel' as MonsterType,
12573     name: '叛逆小怪',
12574     personality: '个性独立',
12575     color: MONSTER_CONFIG.COLORS.rebel.primary,
12576     // 选中背景用实色中粉，与怪兽的玫红拉开差异
12577     selectedBg: '#f8b2dbff',
12578     description: '有主见爱探索、敢于挑战',
12579     trait: '专属特权：小游戏获得的体力、能量双倍',
12580   },
12581 ], [colors]);
12582
12583 const handleSelectMonster = (type: MonsterType) => {
12584   console.log('[MonsterSelection] 选择怪物类型:', type);
12585   setSelectedType(type);
12586 };
12587
12588 const handleContinue = async () => {
12589   if (step === 'select') {
12590     console.log('[MonsterSelection] 进入命名步骤');
12591     Animated.sequence([
12592       Animated.timing(fadeAnim, {
12593         toValue: 0,
12594         duration: 200,
12595         useNativeDriver: true,
12596       })),
12597     ]).start(() => {
12598       setStep('name');

```

```

12599     Animated.timing(fadeAnim, {
12600       toValue: 1,
12601       duration: 200,
12602       useNativeDriver: true,
12603     }).start();
12604   });
12605 } else {
12606   const maxStamina = selectedType === 'calm' ? 120 : 100;
12607   const monsterData = {
12608     type: selectedType,
12609     name: monsterName || monsters.find(m => m.id === selectedType)?.name || '小怪兽',
12610     level: 1,
12611     exp: 0,
12612     stamina: maxStamina,
12613     maxStamina,
12614     paiEnergy: 50,
12615     maxPaiEnergy: 50,
12616     knowledgePoints: 0,
12617     createdAt: new Date().toISOString(),
12618   };
12619   console.log('[MonsterSelection] 创建怪物数据:', monsterData.name, '类型:', selectedType);
12620   await storage.setItem(STORAGE_KEYS.MONSTER, monsterData);
12621   console.log('[MonsterSelection] 怪物数据已保存到本地');
12622
12623   try {
12624     const user = await getCurrentUser();
12625     if (user?.id) {
12626       console.log('[MonsterSelection] 同步怪物到服务端 - 用户ID:', user.id);
12627       await monsterService.createMonster({
12628         userId: user.id,
12629         name: monsterData.name,
12630         personality: selectedType,
12631       });
12632       console.log('[MonsterSelection] 怪物已同步到服务端');
12633     }
12634   } catch (error) {
12635     console.error('[MonsterSelection] 同步怪物到服务端失败:', error);
12636   }
12637
12638   router.replace('/identity-selection');
12639 }
12640 };
12641
12642 const selectedMonster = monsters.find(m => m.id === selectedType);
12643
12644 const styles = useMemo(() => StyleSheet.create({
12645   container: {
12646     flex: 1,
12647     backgroundColor: colors.background,
12648   },

```

```

12649   scrollContent: {
12650     flexGrow: 1,
12651   },
12652   content: {
12653     flex: 1,
12654     paddingHorizontal: 24,
12655     paddingTop: 32,
12656     paddingBottom: 32,
12657     position: 'relative',
12658   },
12659   pixelBackground: {
12660     position: 'absolute',
12661     top: 0,
12662     left: 0,
12663     right: 0,
12664     bottom: 0,
12665     opacity: 0.1,
12666   },
12667   header: {
12668     alignItems: 'center',
12669     marginBottom: 32,
12670   },
12671   title: {
12672     color: colors.textPrimary,
12673     fontSize: 26,
12674     fontWeight: '600',
12675
12676     marginBottom: 8,
12677   },
12678   subtitle: {
12679     color: colors.textSecondary,
12680     fontSize: 14,
12681
12682   },
12683   monstersContainer: {
12684     gap: 16,
12685   },
12686   monsterCard: {
12687     borderRadius: 16,
12688     borderWidth: 2,
12689     overflow: 'hidden',
12690   },
12691   monsterCardContent: {
12692     padding: 20,
12693     flexDirection: 'row',
12694     alignItems: 'center',
12695     gap: 16,
12696   },
12697   monsterIconContainer: {
12698     flexShrink: 0,

```

```

12699 },
12700 monsterIcon: {
12701     position: 'relative',
12702 },
12703 monsterHeadIcon: {
12704     position: 'absolute',
12705 },
12706 earIcon: {
12707     position: 'absolute',
12708 },
12709 eyeIcon: {
12710     position: 'absolute',
12711 },
12712 pupilIcon: {
12713     position: 'absolute',
12714 },
12715 mouthIcon: {
12716     position: 'absolute',
12717 },
12718 bodyIcon: {
12719     position: 'absolute',
12720 },
12721 sparkleIcon: {
12722     position: 'absolute',
12723 },
12724 lightningIcon: {
12725     position: 'absolute',
12726 },
12727 monsterInfo: {
12728     flex: 1,
12729 },
12730 monsterName: {
12731     fontSize: 18,
12732     fontWeight: '600',
12733
12734     marginBottom: 4,
12735 },
12736 monsterDescription: {
12737     fontSize: 12,
12738
12739     marginBottom: 6,
12740 },
12741 traitBadge: {
12742     alignSelf: 'flex-start',
12743     paddingHorizontal: 8,
12744     paddingVertical: 4,
12745     borderRadius: 8,
12746 },
12747 traitText: {
12748     fontSize: 11,

```



```

12749     fontWeight: '600',
12750
12751   },
12752   tipContainer: {
12753     flexDirection: 'row',
12754     alignItems: 'center',
12755     justifyContent: 'center',
12756     backgroundColor: 'rgba(255,214,10,0.1)',
12757     borderRadius: 12,
12758     paddingVertical: 12,
12759     paddingHorizontal: 16,
12760     marginTop: 16,
12761     borderWidth: 1,
12762     borderColor: 'rgba(255,214,10,0.3)',
12763   },
12764   tiplcon: {
12765     fontSize: 16,
12766     marginRight: 8,
12767   },
12768   tipText: {
12769     color: colors.warning,
12770     fontSize: 12,
12771
12772     fontWeight: '600',
12773   },
12774   checkMark: {
12775     width: 32,
12776     height: 32,
12777     borderRadius: 16,
12778     backgroundColor: 'rgba(255,255,255,0.3)',
12779     alignItems: 'center',
12780     justifyContent: 'center',
12781   },
12782   nameInputContainer: {
12783     alignItems: 'center',
12784   },
12785   selectedMonsterIcon: {
12786     marginBottom: 24,
12787   },
12788   nameInput: {
12789     width: '100%',
12790     paddingHorizontal: 24,
12791     paddingVertical: 16,
12792     backgroundColor: colors.inputBg,
12793     borderRadius: 10,
12794     borderWidth: 1,
12795     borderColor: colors.inputBorder,
12796     color: colors.textPrimary,
12797     fontSize: 20,
12798

```

```

12799     fontWeight: '600',
12800     textAlign: 'center',
12801 },
12802 charCount: {
12803     marginTop: 12,
12804     color: colors.textSecondary,
12805     fontSize: 12,
12806
12807 },
12808 spacer: {
12809     flex: 1,
12810 },
12811 continueButton: {
12812     backgroundColor: colors.primary,
12813     paddingVertical: 16,
12814     borderRadius: 16,
12815     alignItems: 'center',
12816     justifyContent: 'center',
12817     marginTop: 24,
12818     shadowColor: colors.shadow,
12819     shadowOffset: { width: 0, height: 2 },
12820     shadowOpacity: 0.08,
12821     shadowRadius: 12,
12822     elevation: 3,
12823 },
12824 continueButtonText: {
12825     color: colors.onPrimary,
12826     fontSize: 16,
12827     fontWeight: '600',
12828
12829 },
12830 )), [colors]);
12831
12832 return (
12833   <SafeAreaView style={styles.container} edges={['top', 'bottom']}>
12834     <ScrollView contentContainerStyle={styles.scrollContent} showsVerticalScrollIndicator={false}>
12835       <View style={styles.content}>
12836         <View style={styles.pixelBackground} />
12837
12838         <Animated.View style={[styles.header, { opacity: fadeAnim }]}>
12839           <Text style={styles.title}>
12840             {step === 'select' ? '选择你的怪兽伙伴' : '给它起个名字吧'}
12841           </Text>
12842           <Text style={styles.subtitle}>
12843             {step === 'select' ? '每种怪兽都有独特的性格' : '这将是你最好的学习伙伴'}
12844           </Text>
12845         </Animated.View>
12846
12847         {step === 'select' ? (
12848           <Animated.View style={[styles.monstersContainer, { opacity: fadeAnim }]}>

```

```

12849 {monsters.map((monster) => {
12850   const isSelected = selectedType === monster.id;
12851   return (
12852     <TouchableOpacity
12853       key={monster.id}
12854       onPress={() => handleSelectMonster(monster.id)}
12855       activeOpacity={0.7}
12856     >
12857       <View
12858         style={[
12859           styles.monsterCard,
12860           {
12861             backgroundColor: isSelected ? monster.selectedBg : colors.card,
12862             borderColor: monster.color,
12863           },
12864         ]}
12865       >
12866         <View style={styles.monsterCardContent}>
12867           <View style={styles.monsterIconContainer}>
12868             <MonsterIcon type={monster.id} size={80} />
12869           </View>
12870
12871           <View style={styles.monsterInfo}>
12872             <Text style={[styles.monsterName, { color: isSelected ? monster.color : colors.textP
12873               {monster.name}
12874             </Text>
12875             <Text style={styles.monsterDescription, { color: isSelected ? (monster.color + 'CC'
12876               {monster.description}
12877             </Text>
12878             <View
12879               style={[
12880                 styles.traitBadge,
12881                 {
12882                   backgroundColor: isSelected ? 'rgba(255,255,255,0.2)' : colors.borderLight,
12883                 },
12884               ]}
12885             >
12886               <Text style={[styles.traitText, { color: monster.color }]}>
12887                 {monster.trait}
12888               </Text>
12889             </View>
12890           </View>
12891
12892           {isSelected && (
12893             <View style={styles.checkMark}>
12894               <Ionicons name="checkmark" size={18} color="#FFFFFF" />
12895             </View>
12896           )}
12897         </View>
12898       </View>

```

```

12899         </TouchableOpacity>
12900     );
12901 }}}
12902 <View style={styles.tipContainer}>
12903     <Text style={styles.tipIcon}> </Text>
12904     <Text style={styles.tipText}>温馨提示：选定小怪兽后，暂不支持更换修改</Text>
12905 </View>
12906 </Animated.View>
12907 ) : (
12908 <Animated.View style={[styles.nameInputContainer, { opacity: fadeAnim }]}>
12909     <View style={styles.selectedMonsterIcon}>
12910         <MonsterIcon type={selectedType} size={120} />
12911     </View>
12912
12913     <TextInput
12914         style={[
12915             styles.nameInput,
12916             { borderColor: selectedMonster?.color },
12917         ]}
12918         value={monsterName}
12919         onChangeText={setMonsterName}
12920         placeholder={selectedMonster?.name || '小怪兽'}
12921         placeholderTextColor={colors.textTertiary}
12922         maxLength={12}
12923         autoFocus
12924     />
12925
12926     <Text style={styles.charCount}>
12927         {monsterName.length}/12 字符
12928     </Text>
12929 </Animated.View>
12930 )}
12931
12932 <View style={styles.spacer} />
12933
12934 <TouchableOpacity
12935     style={[
12936         styles.continueButton,
12937         {
12938             backgroundColor: step === 'name' && !monsterName.trim()
12939               ? colors.surface
12940             : colors.primary,
12941             opacity: step === 'name' && !monsterName.trim() ? 0.5 : 1,
12942             shadowColor: step === 'name' && !monsterName.trim() ? 'transparent' : colors.primary,
12943             shadowOffset: { width: 0, height: 8 },
12944             shadowOpacity: step === 'name' && !monsterName.trim() ? 0 : 0.35,
12945             shadowRadius: 24,
12946             elevation: step === 'name' && !monsterName.trim() ? 0 : 5,
12947         },
12948     ]}

```

```

12949     onPress={handleContinue}
12950     disabled={step === 'name' && !monsterName.trim()}
12951     activeOpacity={0.7}
12952   >
12953     <Text style={styles.continueButtonText}>
12954       {step === 'select' ? '确认选择' : '开始冒险'}
12955     </Text>
12956   </TouchableOpacity>
12957 </View>
12958 </ScrollView>
12959 </SafeAreaView>
12960 );
12961 };
12962
12963 export default MonsterSelectionScreen;
12964
12965 === mobile/app/onboarding.tsx ===
12966 import React, { useMemo } from 'react';
12967 import { View, Text, StyleSheet, TouchableOpacity } from 'react-native';
12968 import { SafeAreaView } from 'react-native-safe-area-context';
12969 import { router } from 'expo-router';
12970 import { Ionicons } from '@expo/vector-icons';
12971 import { SPACING } from '../src/utils/constants';
12972 import { useTheme } from '../src/contexts/ThemeContext';
12973
12974 const OnboardingScreen = () => {
12975   const { colors } = useTheme();
12976   const handleStart = () => {
12977     console.log('[Onboarding] 用户点击开始探索, 跳转至故事页');
12978     router.replace('/story');
12979   };
12980
12981   const styles = useMemo(() => StyleSheet.create({
12982     container: {
12983       flex: 1,
12984       backgroundColor: colors.background,
12985     },
12986     header: {
12987       flexDirection: 'row',
12988       alignItems: 'center',
12989       paddingHorizontal: SPACING.MEDIUM,
12990       paddingVertical: SPACING.SMALL,
12991     },
12992     backBtn: {
12993       width: 40,
12994       height: 40,
12995       borderRadius: 20,
12996       alignItems: 'center',
12997       justifyContent: 'center',
12998       backgroundColor: 'rgba(255,255,255,0.1)',

```

```

12998 },
12999 content: {
13000   flex: 1,
13001   alignItems: 'center',
13002   justifyContent: 'center',
13003   padding: SPACING.XLARGE,
13004 },
13005 illustrationContainer: {
13006   width: 160,
13007   height: 160,
13008   marginBottom: 40,
13009   position: 'relative',
13010   alignItems: 'center',
13011   justifyContent: 'center',
13012 },
13013 planet: {
13014   width: 120,
13015   height: 120,
13016   position: 'relative',
13017 },
13018 planetSurface: {
13019   width: 120,
13020   height: 120,
13021   borderRadius: 60,
13022   backgroundColor: colors.primary,
13023 },
13024 planetRing: {
13025   position: 'absolute',
13026   top: 45,
13027   left: 0,
13028   width: 120,
13029   height: 30,
13030   borderWidth: 4,
13031   borderColor: 'rgba(255,255,255,0.3)',
13032   borderRadius: 60,
13033 },
13034 piSymbol: {
13035   position: 'absolute',
13036   top: 10,
13037   right: 10,
13038 },
13039 piText: {
13040   fontSize: 32,
13041   fontWeight: 'bold',
13042   color: '#FFD60A',
13043 },
13044 title: {
13045   fontSize: 28,
13046   fontWeight: 'bold',
13047   color: colors.textPrimary,

```

```

13048     textAlign: 'center',
13049     marginBottom: SPACING.SMALL,
13050   },
13051   subtitle: {
13052     fontSize: 16,
13053     color: colors.textSecondary,
13054     textAlign: 'center',
13055     marginBottom: 40,
13056   },
13057   featureList: {
13058     width: '100%',
13059     marginBottom: 40,
13060   },
13061   featureItem: {
13062     flexDirection: 'row',
13063     alignItems: 'center',
13064     backgroundColor: 'rgba(255,255,255,0.05)',
13065     borderRadius: 12,
13066     padding: SPACING.MEDIUM,
13067     marginBottom: SPACING.SMALL,
13068   },
13069   featureIcon: {
13070     fontSize: 24,
13071     marginRight: SPACING.MEDIUM,
13072   },
13073   featureText: {
13074     fontSize: 16,
13075     color: colors.textPrimary,
13076     fontWeight: '500',
13077   },
13078   startButton: {
13079     backgroundColor: colors.primary,
13080     paddingVertical: 16,
13081     paddingHorizontal: 48,
13082     borderRadius: 30,
13083     shadowColor: colors.primary,
13084     shadowOffset: { width: 0, height: 8 },
13085     shadowOpacity: 0.35,
13086     shadowRadius: 24,
13087     elevation: 5,
13088   },
13089   startButtonText: {
13090     color: 'FFFFFF',
13091     fontSize: 18,
13092     fontWeight: '700',
13093   },
13094   }), [colors]);
13095
13096   return (
13097     <SafeAreaView style={styles.container}>

```

```

13098 <View style={styles.header}>
13099   <TouchableOpacity style={styles.backBtn} onPress={() => router.back()} activeOpacity={0.7}>
13100     <Ionicons name="arrow-back" size={22} color={colors.textSecondary} />
13101   </TouchableOpacity>
13102 </View>
13103 <View style={styles.content}>
13104   <View style={styles.illustrationContainer}>
13105     <View style={styles.planet}>
13106       <View style={styles.planetSurface} />
13107       <View style={styles.planetRing} />
13108     </View>
13109     <View style={styles.piSymbol}>
13110       <Text style={styles.piText}>Π</Text>
13111     </View>
13112   </View>
13113
13114   <Text style={styles.title}>欢迎来到 LearnFlow</Text>
13115   <Text style={styles.subtitle}>你的个性化学习伙伴</Text>
13116
13117   <View style={styles.featureList}>
13118     <View style={styles.featureItem}>
13119       <Text style={styles.featureIcon}></Text>
13120       <Text style={styles.featureText}>智能学习路径规划</Text>
13121     </View>
13122     <View style={styles.featureItem}>
13123       <Text style={styles.featureIcon}></Text>
13124       <Text style={styles.featureText}>趣味游戏恢复体力</Text>
13125     </View>
13126     <View style={styles.featureItem}>
13127       <Text style={styles.featureIcon}></Text>
13128       <Text style={styles.featureText}>学习数据可视化</Text>
13129     </View>
13130   </View>
13131
13132   <TouchableOpacity style={styles.startButton} onPress={handleStart}>
13133     <Text style={styles.startButtonText}>开始探索</Text>
13134   </TouchableOpacity>
13135 </View>
13136 </SafeAreaView>
13137 );
13138 };
13139
13140 export default OnboardingScreen;
13141
13142 === mobile/app/pomodoro.tsx ===
13141 import React, { useState, useEffect, useRef, useMemo, useCallback } from 'react';
13142 import {
13143   View, Text, StyleSheet, TouchableOpacity, AppState, Alert, TextInput,
13144   AppStateStatus, Modal, Animated, Easing, Dimensions, Linking, BackHandler,
13145   ScrollView, KeyboardAvoidingView, Platform,
13146 } from 'react-native';

```



```

13147 import { SafeAreaView } from 'react-native-safe-area-context';
13148 import { router, useLocalSearchParams, useNavigation } from 'expo-router';
13149 import { Ionicons } from '@expo/vector-icons';
13150 import AsyncStorage from '@react-native-async-storage/async-storage';
13151 import * as Brightness from 'expo-brightness';
13152 import { activateKeepAwakeAsync, deactivateKeepAwake } from 'expo-keep-awake';
13153 import { useTheme } from '../src/contexts/ThemeContext';
13154 import MonsterIcon from '../src/components/MonsterIcon';
13155 import { getCurrentUser } from '../src/utils/auth';
13156 import { monsterService } from '../src/services/api';
13157
13158 const { width: SCREEN_WIDTH } = Dimensions.get('window');
13159 const PROGRESS_SIZE = SCREEN_WIDTH * 0.55;
13160 const STROKE_WIDTH = 6;
13161 const TICK_MS = 100;
13162
13163 const POMODORO_STATE_KEY = 'pomodoro_runtime';
13164
13165 const formatDisplay = (seconds: number): string => {
13166   const m = Math.floor(seconds / 60);
13167   const s = seconds % 60;
13168   return `${m.toString().padStart(2, '0')}:${s.toString().padStart(2, '0')}`;
13169 };
13170
13171 const PomodoroScreen = () => {
13172   const { nodeName, url, duration } = useLocalSearchParams<{
13173     nodeName?: string; url?: string; duration: string;
13174   }>();
13175   const { colors } = useTheme();
13176
13177   const totalSeconds = parseInt(duration || '25', 10) * 60;
13178
13179   console.log('[Pomodoro] 启动番茄钟', { nodeName, url, duration, totalSeconds });
13180
13181   const [timerSeconds, setTimerSeconds] = useState(totalSeconds);
13182   const [isPaused, setIsPaused] = useState(false);
13183   const [isCompleted, setIsCompleted] = useState(false);
13184   const [showExitConfirm, setShowExitConfirm] = useState(false);
13185   const [actionConfirm, setActionConfirm] = useState<{
13186     type: 'pause' | 'resume' | 'reset' | 'stop';
13187   } | null>(null);
13188   const [showBrightness, setShowBrightness] = useState(false);
13189   const [brightness, setBrightness] = useState(0.5);
13190   const [rewards, setRewards] = useState({ stamina: 0, energy: 0 });
13191
13192   // 怪兽对话状态
13193   const [showMonsterChat, setShowMonsterChat] = useState(false);
13194   const [chatMessages, setChatMessages] = useState<{ text: string; isUser: boolean }[]>([]);
13195   const [chatInput, setChatInput] = useState('');
13196   const [isSending, setIsSending] = useState(false);

```

```

13197 const [energyInfo, setEnergyInfo] = useState<{ cost: number; remaining: number } | null>(null);
13198 const [monsterType, setMonsterType] = useState<'lively' | 'calm' | 'rebel'>('lively');
13199
13200 // 加载怪兽类型
13201 useEffect(() => {
13202   (async () => {
13203     try {
13204       const user = await getCurrentUser();
13205       if (!user?.id) return;
13206       const res = await monsterService.getMonsterStatus(user.id);
13207       if (res?.success && res.data?.personality) {
13208         setMonsterType(res.data.personality);
13209       }
13210     } catch {}
13211   })();
13212 }, []);
13213
13214 const completedRef = useRef(false);
13215 const exitedRef = useRef(false);
13216 const startRef = useRef(Date.now());
13217 const pausedAtRef = useRef<number | null>(null);
13218 const totalPausedRef = useRef(0);
13219 const isPausedRef = useRef(false);
13220 const timerRef = useRef(totalSeconds);
13221 const appStateRef = useRef<AppStateStatus>('active');
13222 const initialBrightnessRef = useRef(0.5);
13223
13224 const navigation = useNavigation();
13225
13226 // 拦截硬件返回 & 手势返回
13227 useEffect(() => {
13228   const backAction = () => {
13229     if (completedRef.current) return false;
13230     setShowExitConfirm(true);
13231     return true;
13232   };
13233   const backHandler = BackHandler.addEventListener('hardwareBackPress', backAction);
13234
13235   const unsubscribe = navigation.addListener('beforeRemove', (e) => {
13236     if (completedRef.current) return;
13237     e.preventDefault();
13238     setShowExitConfirm(true);
13239   });
13240
13241   return () => {
13242     backHandler.remove();
13243     unsubscribe();
13244   };
13245 }, [navigation]);
13246

```

```

13247 // 环形进度动画
13248 const progressAnim = useRef(new Animated.Value(1)).current;
13249 const pulseAnim = useRef(new Animated.Value(1)).current;
13250
13251 useEffect(() => {
13252   Animated.loop(
13253     Animated.sequence([
13254       Animated.timing(pulseAnim, { toValue: 1.03, duration: 1800, easing: Easing.inOut(Easing.ease), useNativeDriver: true,
13255       Animated.timing(pulseAnim, { toValue: 1, duration: 1800, easing: Easing.inOut(Easing.ease), useNativeDriver: true,
13256     ]),
13257   ).start();
13258 }, []);
13259
13260 // KeepAwake + Brightness
13261 useEffect(() => {
13262   activateKeepAwakeAsync('pomodoro');
13263   Brightness.getBrightnessAsync().then(b => {
13264     initialBrightnessRef.current = b;
13265     setBrightness(b);
13266   });
13267   return () => {
13268     deactivateKeepAwake('pomodoro');
13269     Brightness.setBrightnessAsync(initialBrightnessRef.current);
13270   };
13271 }, []);
13272
13273 const handleBrightnessChange = useCallback(async (val: number) => {
13274   setBrightness(val);
13275   await Brightness.setBrightnessAsync(val);
13276 }, []);
13277
13278 // 持久化运行时状态（用于崩溃/杀进程恢复判断）
13279 const persistRuntime = useCallback(async () => {
13280   if (completedRef.current || exitedRef.current) return;
13281   await AsyncStorage.setItem(POMODORO_STATE_KEY, JSON.stringify({
13282     totalSeconds,
13283     startTime: startRef.current,
13284     pausedAt: pausedAtRef.current,
13285     totalPaused: totalPausedRef.current,
13286     nodeName,
13287     url,
13288     duration,
13289   }));
13290 }, [totalSeconds, nodeName, url, duration]);
13291
13292 const clearRuntime = useCallback(async () => {
13293   await AsyncStorage.removeItem(POMODORO_STATE_KEY);
13294 }, []);
13295
13296 // 计算已过秒数

```

```

13297 const getElapsed = useCallback(() => {
13298   const now = Date.now();
13299   let pausedMs = totalPausedRef.current;
13300   if (pausedAtRef.current) {
13301     pausedMs += now - pausedAtRef.current;
13302   }
13303   return Math.floor((now - startRef.current - pausedMs) / 1000);
13304 }, []);
13305
13306 // 主计时循环
13307 useEffect(() => {
13308   const tick = () => {
13309     if (completedRef.current || exitedRef.current) return;
13310     if (isPausedRef.current) return;
13311
13312     const elapsed = getElapsed();
13313     const remaining = Math.max(0, totalSeconds - elapsed);
13314     timerRef.current = remaining;
13315     setTimerSeconds(remaining);
13316
13317     if (remaining <= 0) {
13318       completedRef.current = true;
13319       setIsCompleted(true);
13320       setIsPaused(false);
13321       clearRuntime();
13322       // 计算奖励
13323       const staminaReward = Math.ceil(parseInt(duration || '25', 10) / 5);
13324       const energyReward = Math.floor(parseInt(duration || '25', 10) / 10);
13325       console.log('[Pomodoro] 计时完成, 发放奖励', { staminaReward, energyReward });
13326       setRewards({ stamina: staminaReward, energy: energyReward });
13327       // 发放奖励到怪兽
13328       AsyncStorage.getItem('monster').then(monster => {
13329         if (monster) {
13330           const m = JSON.parse(monster);
13331           m.stamina = Math.min(m.stamina + staminaReward, m.maxStamina);
13332           m.paiEnergy = Math.min(m.paiEnergy + energyReward, m.maxPaiEnergy);
13333           AsyncStorage.setItem('monster', JSON.stringify(m));
13334         }
13335       });
13336     }
13337   };
13338
13339   const interval = setInterval(tick, TICK_MS);
13340   tick(); // 立即执行一次
13341
13342   return () => clearInterval(interval);
13343 }, [totalSeconds, getElapsed, clearRuntime, duration]);
13344
13345 // 进度动画同步
13346 useEffect(() => {

```

```

13347     if (completedRef.current) return;
13348     const progress = timerSeconds / totalSeconds;
13349     Animated.timing(progressAnim, {
13350       toValue: progress,
13351       duration: TICK_MS * 3,
13352       useNativeDriver: false,
13353     }).start();
13354   }, [timerSeconds, totalSeconds]);
13355
13356   // AppState 监听
13357   useEffect(() => {
13358     const subscription = AppState.addListener('change', (nextState) => {
13359       const prev = appStateRef.current;
13360       appStateRef.current = nextState;
13361
13362       if (prev === 'active' && nextState !== 'active') {
13363         console.log('[Pomodoro] 切到后台, 保存运行状态');
13364         persistRuntime();
13365       } else if (prev !== 'active' && nextState === 'active') {
13366         console.log('[Pomodoro] 回到前台, 恢复计时');
13367         if (exitedRef.current || completedRef.current) return;
13368
13369         if (isPausedRef.current) return;
13370
13371         const elapsed = getElapsed();
13372         const remaining = Math.max(0, totalSeconds - elapsed);
13373         if (remaining <= 0) {
13374           completedRef.current = true;
13375           setIsCompleted(true);
13376           clearRuntime();
13377           const staminaReward = Math.ceil(parseInt(duration || '25', 10) / 5);
13378           const energyReward = Math.floor(parseInt(duration || '25', 10) / 10);
13379           setRewards({ stamina: staminaReward, energy: energyReward });
13380           AsyncStorage.getItem('monster').then(monster => {
13381             if (monster) {
13382               const m = JSON.parse(monster);
13383               m.stamina = Math.min(m.stamina + staminaReward, m.maxStamina);
13384               m.paiEnergy = Math.min(m.paiEnergy + energyReward, m.maxPaiEnergy);
13385               AsyncStorage.setItem('monster', JSON.stringify(m));
13386             }
13387           });
13388         } else {
13389           timerRef.current = remaining;
13390           setTimerSeconds(remaining);
13391         }
13392       }
13393     });
13394
13395     return () => subscription.remove();
13396   }, [totalSeconds, getElapsed, persistRuntime, clearRuntime, duration]);

```

```

13397
13398 // 页面挂载时校验：如果是杀进程重启，进度作废
13399 useEffect(() => {
13400   (async () => {
13401     const stored = await AsyncStorage.getItem(POMODORO_STATE_KEY);
13402     if (stored) {
13403       console.log('[Pomodoro] 检测到上次异常退出，进度作废');
13404       // 存在运行时状态说明上次异常退出（杀进程/崩溃），本次进度作废
13405       await clearRuntime();
13406     }
13407     startRef.current = Date.now();
13408     totalPausedRef.current = 0;
13409     pausedAtRef.current = null;
13410   })();
13411 }, []);
13412
13413 // ---- 操作处理 ----
13414
13415 const handlePause = useCallback(() => {
13416   if (completedRef.current) return;
13417   if (!isPausedRef.current) {
13418     setActionConfirm({ type: 'pause' });
13419   } else {
13420     setActionConfirm({ type: 'resume' });
13421   }
13422 }, []);
13423
13424 const confirmAction = useCallback(() => {
13425   if (!actionConfirm) return;
13426   const type = actionConfirm.type;
13427   console.log('[Pomodoro] 确认操作:', type);
13428   setActionConfirm(null);
13429
13430   switch (type) {
13431     case 'pause':
13432       isPausedRef.current = true;
13433       pausedAtRef.current = Date.now();
13434       setIsPaused(true);
13435       break;
13436     case 'resume':
13437       if (pausedAtRef.current) {
13438         totalPausedRef.current += Date.now() - pausedAtRef.current;
13439         pausedAtRef.current = null;
13440       }
13441       isPausedRef.current = false;
13442       setIsPaused(false);
13443       break;
13444     case 'reset':
13445       totalPausedRef.current = 0;
13446       pausedAtRef.current = null;

```

```

13447     isPausedRef.current = false;
13448     startRef.current = Date.now();
13449     timerRef.current = totalSeconds;
13450     setTimerSeconds(totalSeconds);
13451     setIsPaused(false);
13452     setIsCompleted(false);
13453     completedRef.current = false;
13454     break;
13455     case 'stop':
13456         exitedRef.current = true;
13457         completedRef.current = true;
13458         clearRuntime();
13459         router.back();
13460         break;
13461     }
13462 }, [actionConfirm, totalSeconds, clearRuntime]);
13463
13464 const handleReset = useCallback(() => {
13465     if (completedRef.current) return;
13466     setActionConfirm({ type: 'reset' });
13467 }, []);
13468
13469 const handleStop = useCallback(() => {
13470     if (completedRef.current) return;
13471     setActionConfirm({ type: 'stop' });
13472 }, []);
13473
13474 const confirmExit = useCallback(() => {
13475     console.log('[Pomodoro] 确认退出, 计时终止, 无奖励');
13476     setShowExitConfirm(false);
13477     exitedRef.current = true;
13478     completedRef.current = true;
13479     clearRuntime();
13480     router.back();
13481 }, [clearRuntime]);
13482
13483 const handleCompletionBack = useCallback(() => {
13484     router.back();
13485 }, []);
13486
13487 // ---- 怪兽对话发送 ----
13488 const handleSendChat = async () => {
13489     const text = chatInput.trim();
13490     if (!text || isSending) return;
13491     const user = await getCurrentUser();
13492     if (!user?.id) { Alert.alert('提示', '请先登录'); return; }
13493
13494     setChatInput("");
13495     setIsSending(true);
13496     setChatMessages(prev => [...prev, { text, isUser: true }]);

```

```

13497
13498   try {
13499     const res = await monsterService.chat({ userId: user.id, message: text });
13500     if (res.success && res.data) {
13501       setChatMessages(prev => [...prev, { text: res.data.message, isUser: false }]);
13502       if (res.data.energyCost !== undefined) {
13503         setEnergyInfo({ cost: res.data.energyCost, remaining: res.data.remainingEnergy ?? 0 });
13504       }
13505     }
13506   } catch {
13507     Alert.alert('发送失败', '网络似乎不太好~');
13508   } finally {
13509     setIsSending(false);
13510   }
13511 };
13512
13513 // ---- styles ----
13514
13515 const styles = useMemo(() => StyleSheet.create({
13516   container: { flex: 1, backgroundColor: colors.background },
13517   header: {
13518     flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between',
13519     paddingHorizontal: 20, paddingVertical: 12,
13520   },
13521   headerLeft: {
13522     flexDirection: 'row', alignItems: 'center', gap: 8,
13523   },
13524   headerTitle: { fontSize: 15, fontWeight: '600', color: colors.textSecondary, },
13525   headerNode: { fontSize: 12, color: colors.textTertiary, marginTop: 2 },
13526   brightnessBtn: {
13527     width: 36, height: 36, borderRadius: 18, alignItems: 'center', justifyContent: 'center',
13528     backgroundColor: colors.borderDark,
13529   },
13530   center: { flex: 1, alignItems: 'center', justifyContent: 'center' },
13531   progressContainer: {
13532     width: PROGRESS_SIZE, height: PROGRESS_SIZE, alignItems: 'center', justifyContent: 'center',
13533   },
13534   progressBg: {
13535     position: 'absolute', width: PROGRESS_SIZE, height: PROGRESS_SIZE,
13536     borderRadius: PROGRESS_SIZE / 2, borderWidth: STROKE_WIDTH,
13537     borderColor: colors.borderDark,
13538   },
13539   timerText: {
13540     fontSize: 40, fontWeight: '900',
13541     color: colors.textPrimary,
13542
13543     letterSpacing: 2,
13544   },
13545   timerSub: {
13546     fontSize: 14, color: colors.textSecondary, marginTop: 8,

```



```

13547   },
13548   pausedHint: {
13549     fontSize: 13, color: colors.warning, marginTop: 4, opacity: 0.8,
13550   },
13551   controls: {
13552     flexDirection: 'row', alignItems: 'center', justifyContent: 'center',
13553     gap: 20, paddingVertical: 32, paddingHorizontal: 20,
13554   },
13555   ctrlBtn: {
13556     width: 64, height: 64, borderRadius: 32, alignItems: 'center', justifyContent: 'center',
13557     backgroundColor: colors.card, borderWidth: 1, borderColor: colors.borderDark,
13558   },
13559   ctrlBtnCenter: {
13560     width: 76, height: 76, borderRadius: 38,
13561   },
13562   ctrlLabel: {
13563     fontSize: 11, color: colors.textTertiary, marginTop: 6, textAlign: 'center',
13564   },
13565   ctrlWrapper: { alignItems: 'center' },
13566   // 亮度滑块
13567   brightnessContainer: {
13568     position: 'absolute', bottom: 100, left: 40, right: 40,
13569     backgroundColor: colors.card, borderRadius: 16, padding: 16,
13570     borderWidth: 1, borderColor: colors.borderDark,
13571   },
13572   brightnessRow: {
13573     flexDirection: 'row', alignItems: 'center', gap: 12,
13574   },
13575   brightnessTrack: {
13576     flex: 1, height: 6, backgroundColor: colors.borderDark, borderRadius: 3,
13577   },
13578   brightnessFill: {
13579     height: '100%', borderRadius: 3, backgroundColor: colors.primary,
13580   },
13581   brightnessHandle: {
13582     position: 'absolute', top: -8, width: 22, height: 22, borderRadius: 11,
13583     backgroundColor: colors.primary, borderWidth: 2, borderColor: colors.textPrimary,
13584     left: '50%', marginLeft: -11, shadowColor: '#000', shadowOffset: { width: 0, height: 2 }, shadowOpacity:
13585   },
13586   // 完成覆盖层
13587   completionOverlay: {
13588     ...StyleSheet.absoluteFillObject,
13589     backgroundColor: colors.background,
13590     alignItems: 'center', justifyContent: 'center',
13591     padding: 40,
13592     zIndex: 10,
13593   },
13594   completeIcon: { fontSize: 64, marginBottom: 20 },
13595   completeTitle: {
13596     fontSize: 32, fontWeight: '600', color: colors.warning,

```

```

13597     marginBottom: 8,
13598   },
13599   completeSub: {
13600     fontSize: 16, color: colors.textSecondary,
13601     marginBottom: 32, textAlign: 'center',
13602   },
13603   rewardsContainer: {
13604     flexDirection: 'row', gap: 32, marginBottom: 40,
13605   },
13606   rewardItem: { alignItems: 'center' },
13607   rewardValue: {
13608     fontSize: 28, fontWeight: '600', color: colors.warning,
13609   },
13610   rewardLabel: {
13611     fontSize: 14, color: colors.textSecondary, marginTop: 4,
13612   },
13613   backBtn: {
13614     paddingHorizontal: 32, paddingVertical: 14, backgroundColor: colors.primary,
13615     borderRadius: 16,
13616   },
13617   backBtnText: {
13618     color: colors.textInverse, fontSize: 16, fontWeight: '600',
13619   },
13620   gotoBtn: {
13621     flexDirection: 'row', alignItems: 'center', justifyContent: 'center', gap: 8,
13622     paddingHorizontal: 32, paddingVertical: 12, backgroundColor: '#6C63FF',
13623     borderRadius: 16, marginBottom: 12,
13624   },
13625   gotoBtnText: {
13626     color: colors.textInverse, fontSize: 15, fontWeight: '600',
13627   },
13628   navLinkBtn: {
13629     flexDirection: 'row', alignItems: 'center', justifyContent: 'center', gap: 6,
13630     paddingHorizontal: 20, paddingVertical: 10, borderRadius: 12,
13631     backgroundColor: 'rgba(108,99,255,0.1)', borderWidth: 1, borderColor: 'rgba(108,99,255,0.25)',
13632     marginTop: 12,
13633   },
13634   navLinkText: {
13635     fontSize: 14, color: colors.primary, fontWeight: '600',
13636   },
13637   // 确认弹窗
13638   confirmOverlay: {
13639     flex: 1, backgroundColor: 'rgba(0,0,0,0.6)', alignItems: 'center', justifyContent: 'center', padding: 40
13640   },
13641   confirmCard: {
13642     width: '100%', maxWidth: 320, backgroundColor: colors.card, borderRadius: 20,
13643     padding: 24, borderWidth: 1, borderColor: colors.borderDark,
13644     alignItems: 'center',
13645   },
13646   confirmTitle: {

```

```

13647     fontSize: 18, fontWeight: '600', color: colors.textPrimary,
13648     marginBottom: 8,
13649 },
13650 confirmMsg: {
13651     fontSize: 14, color: colors.textSecondary,
13652     textAlign: 'center', marginBottom: 24, lineHeight: 20,
13653 },
13654 confirmMsgNote: {
13655     fontSize: 13, color: colors.textTertiary,
13656     textAlign: 'center', marginBottom: 24, lineHeight: 18,
13657 },
13658 confirmRow: {
13659     flexDirection: 'row', gap: 12, width: '100%',
13660 },
13661 confirmCancel: {
13662     flex: 1, paddingVertical: 12, borderRadius: 12, alignItems: 'center',
13663     backgroundColor: colors.borderDark,
13664 },
13665 confirmCancelText: {
13666     fontSize: 15, color: colors.textSecondary, fontWeight: '600',
13667 },
13668 confirmDanger: {
13669     flex: 1, paddingVertical: 12, borderRadius: 12, alignItems: 'center',
13670     backgroundColor: 'rgba(233,69,96,0.2)',
13671 },
13672 confirmDangerText: {
13673     fontSize: 15, color: colors.textInverse, fontWeight: '600',
13674 },
13675 confirmPrimary: {
13676     flex: 1, paddingVertical: 12, borderRadius: 12, alignItems: 'center',
13677     backgroundColor: 'rgba(123,117,216,0.2)',
13678 },
13679 confirmPrimaryText: {
13680     fontSize: 15, color: colors.textInverse, fontWeight: '600',
13681 },
13682 // 怪兽对话弹窗样式
13683 chatOverlay: { flex: 1, backgroundColor: colors.background },
13684 chatCard: {
13685     flex: 1, borderRadius: 0,
13686     paddingBottom: 20, overflow: 'hidden',
13687 },
13688 chatHeader: {
13689     flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center',
13690     padding: 16, borderBottomWidth: 1,
13691 },
13692 chatTitle: { fontSize: 16, fontWeight: '600', },
13693 chatBubble: {
13694     maxWidth: '80%', paddingHorizontal: 14, paddingVertical: 10, borderRadius: 14, marginBottom: 8,
13695 },
13696 energyBar: {

```

```

13697     marginHorizontal: 16, padding: 10, borderRadius: 10, alignItems: 'center', marginBottom: 8,
13698   },
13699   chatInputRow: {
13700     flexDirection: 'row', alignItems: 'flex-end', gap: 8,
13701     paddingHorizontal: 16, paddingTop: 8, borderTopWidth: 1,
13702   },
13703   chatTextInput: {
13704     flex: 1, borderRadius: 12, paddingHorizontal: 14, paddingVertical: 10,
13705     fontSize: 13, maxHeight: 80,
13706   },
13707   chatSendBtn: {
13708     width: 40, height: 40, borderRadius: 12, alignItems: 'center', justifyContent: 'center',
13709   },
13710 ]), [colors]);
13711
13712 // ---- 完成覆盖层 ----
13713 const renderCompletion = () => (
13714   <View style={styles.completionOverlay}>
13715     <Animated.Text style={{styles.completelcon, { transform: [{ scale: pulseAnim }] }}>
13716
13717   </Animated.Text>
13718   <Text style={styles.completeTitle}>专注完成! </Text>
13719   <Text style={styles.completeSub}>
13720     小怪兽为你感到骄傲, 你完整完成了 {duration} 分钟的专注学习
13721   </Text>
13722   <View style={styles.rewardsContainer}>
13723     <View style={styles.rewardItem}>
13724       <Text style={styles.rewardValue}>+{rewards.stamina}</Text>
13725       <Text style={styles.rewardLabel}> 体力</Text>
13726     </View>
13727     <View style={styles.rewardItem}>
13728       <Text style={styles.rewardValue}>+{rewards.energy}</Text>
13729       <Text style={styles.rewardLabel}> 能量</Text>
13730     </View>
13731   </View>
13732   {url ? (
13733     <TouchableOpacity
13734       style={styles.gotoBtn}
13735       onPress={async () => {
13736         console.log('[Pomodoro] 跳转学习网页', url);
13737         const supported = await Linking.canOpenURL(url);
13738         if (supported) {
13739           Linking.openURL(url).catch(err => {
13740             console.error('[Pomodoro] Linking error:', err);
13741             Alert.alert('错误', '无法打开链接, 请检查网络连接');
13742           });
13743         } else {
13744           Alert.alert('错误', '设备不支持打开此链接');
13745         }
13746       }}

```

```

13747     activeOpacity={0.7}
13748   >
13749     <Ionicons name="open-outline" size={18} color="#FFFFFF" />
13750     <Text style={styles.gotoBtnText}>去学习网页</Text>
13751   </TouchableOpacity>
13752 ) : null}
13753 <TouchableOpacity style={styles.backBtn} onPress={handleCompletionBack} activeOpacity={0.7}>
13754   <Text style={styles.backBtnText}>返回节点列表 </Text>
13755 </TouchableOpacity>
13756 </View>
13757 );
13758
13759 // ---- 操作确认弹窗 ----
13760 const renderActionModal = () => {
13761   if (!actionConfirm) return null;
13762   const { type } = actionConfirm;
13763
13764   let title = '';
13765   let msg = '';
13766   let confirmStyle = styles.confirmPrimary;
13767   let confirmTextStyle = styles.confirmPrimaryText;
13768
13769   switch (type) {
13770     case 'pause':
13771       title = '暂停计时';
13772       msg = '暂停后计时将停止，你可以随时继续';
13773       break;
13774     case 'resume':
13775       title = '继续计时';
13776       msg = '专注计时将继续进行';
13777       break;
13778     case 'reset':
13779       title = '重置计时';
13780       msg = '当前进度将全部清零，无法恢复';
13781       confirmStyle = styles.confirmDanger;
13782       confirmTextStyle = styles.confirmDangerText;
13783       break;
13784     case 'stop':
13785       title = '结束计时';
13786       msg = '此次专注将终止，本次无法获得任何奖励';
13787       confirmStyle = styles.confirmDanger;
13788       confirmTextStyle = styles.confirmDangerText;
13789       break;
13790   }
13791
13792   return (
13793     <Modal visible transparent onRequestClose={() => setActionConfirm(null)} animationType="fade">
13794       <View style={styles.confirmOverlay}>
13795         <View style={styles.confirmCard}>
13796           <Text style={styles.confirmTitle}>{title}</Text>

```

```

13797     <Text style={styles.confirmMsg}>{msg}</Text>
13798     <View style={styles.confirmRow}>
13799         <TouchableOpacity style={styles.confirmCancel} onPress={() => setActionConfirm(null)} activeOpac
13800             <Text style={styles.confirmCancelText}>取消</Text>
13801         </TouchableOpacity>
13802         <TouchableOpacity style={confirmStyle} onPress={confirmAction} activeOpacity={0.7}>
13803             <Text style={confirmTextStyle}>确认</Text>
13804         </TouchableOpacity>
13805     </View>
13806 </View>
13807 </View>
13808 </Modal>
13809 );
13810 };
13811
13812 // ---- 退出提醒弹窗 ----
13813 const renderExitModal = () => (
13814     <Modal visible={showExitConfirm} transparent onRequestClose={() => setShowExitConfirm(false)} animationTyp
13815     <View style={styles.confirmOverlay}>
13816         <View style={styles.confirmCard}>
13817             <Text style={styles.confirmTitle}>是否退出</Text>
13818             <Text style={styles.confirmMsgNote}>注：退出体力不再返还</Text>
13819             <View style={styles.confirmRow}>
13820                 <TouchableOpacity style={styles.confirmCancel} onPress={() => setShowExitConfirm(false)} activeOpa
13821                     <Text style={styles.confirmCancelText}>取消</Text>
13822                 </TouchableOpacity>
13823                 <TouchableOpacity style={styles.confirmDanger} onPress={confirmExit} activeOpacity={0.7}>
13824                     <Text style={styles.confirmDangerText}>确认退出</Text>
13825                 </TouchableOpacity>
13826             </View>
13827         </View>
13828     </View>
13829 </Modal>
13830 );
13831
13832 return (
13833     <SafeAreaView style={styles.container} edges={['top']}>
13834         { /* Header */ }
13835         <View style={styles.header}>
13836             <View style={styles.headerLeft}>
13837                 <View>
13838                     <Text style={styles.headerTitle}>{isCompleted ? '专注完成' : '正在专注'}</Text>
13839                     {nodeName ? <Text style={styles.headerNode}>{nodeName}</Text> : null}
13840                 </View>
13841             </View>
13842             <View style={{ flexDirection: 'row', alignItems: 'center', gap: 8 }}>
13843                 <TouchableOpacity
13844                     style={{ flexDirection: 'row', alignItems: 'center', gap: 6, backgroundColor: colors.borderLight,
13845                     onPress={async () => {
13846                         setShowMonsterChat(true);

```

```

13847     setChatInput('');
13848     setEnergyInfo(null);
13849     // 加载历史消息
13850     try {
13851         const user = await getCurrentUser();
13852         if (user?.id) {
13853             const res = await monsterService.getMessages(user.id);
13854             if (res.success && res.data?.messages) {
13855                 setChatMessages(res.data.messages.map((m: any) => ({ text: m.message, isUser: m.isUser })))
13856             }
13857         }
13858     } catch {}
13859 }
13860 activeOpacity={0.7}
13861 >
13862 <MonsterIcon type={monsterType} size={32} />
13863 <Text style={{ color: colors.textPrimary, fontSize: 12, fontWeight: '600', }}>怪兽答疑</Text>
13864 </TouchableOpacity>
13865 <TouchableOpacity
13866 style={styles.brightnessBtn}
13867 onPress={() => setShowBrightness(!showBrightness)}
13868 activeOpacity={0.7}
13869 >
13870 <Icons name="sunny" size={18} color={showBrightness ? colors.warning : colors.textSecondary} />
13871 </TouchableOpacity>
13872 </View>
13873 </View>
13874
13875 /* 计时区域 */
13876 <View style={styles.center}>
13877 <View style={styles.progressContainer}>
13878 <View style={styles.progressBg} />
13879 <Text style={styles.timerText}>{formatDisplay(timerSeconds)}</Text>
13880 <Text style={styles.timerSub}>[{nodeName}]</Text>
13881 {isPaused && <Text style={styles.pausedHint}>已暂停</Text>}
13882 </View>
13883 {url ? (
13884 <TouchableOpacity
13885 style={styles.navLinkBtn}
13886 onPress={async () => {
13887     console.log('[Pomodoro] 在计时中跳转学习网页', url);
13888     const supported = await Linking.canOpenURL(url);
13889     if (supported) {
13890         Linking.openURL(url).catch(err => {
13891             console.error('[Pomodoro] Linking error:', err);
13892             Alert.alert('错误', '无法打开链接，请检查网络连接');
13893         });
13894     } else {
13895         Alert.alert('错误', '设备不支持打开此链接');
13896     }

```

```

13897     }}
13898     activeOpacity={0.7}
13899   >
13900     <Ionicons name="open-outline" size={16} color={colors.primary} />
13901     <Text style={styles.navLinkText}>去学习网页</Text>
13902   </TouchableOpacity>
13903   ) : null}
13904 </View>
13905
13906 {/* 操作按钮 */}
13907 <View style={styles.controls}>
13908   <View style={styles.ctrlWrapper}>
13909     <TouchableOpacity style={styles.ctrlBtn} onPress={handleReset} activeOpacity={0.7}>
13910       <Ionicons name="refresh" size={24} color={colors.warning} />
13911     </TouchableOpacity>
13912     <Text style={styles.ctrlLabel}>重置</Text>
13913   </View>
13914
13915   <View style={styles.ctrlWrapper}>
13916     <TouchableOpacity
13917       style={[styles.ctrlBtn, styles.ctrlBtnCenter, {
13918         backgroundColor: isPaused ? 'rgba(58,227,116,0.15)' : 'rgba(255,215,0,0.15)',
13919         borderColor: isPaused ? colors.success : colors.warning,
13920       }]}
13921       onPress={handlePause}
13922       activeOpacity={0.7}
13923     >
13924       <Ionicons
13925         name={isPaused ? 'play' : 'pause'}
13926         size={32}
13927         color={isPaused ? colors.success : colors.warning}
13928       />
13929     </TouchableOpacity>
13930     <Text style={styles.ctrlLabel}>{isPaused ? '继续' : '暂停'}</Text>
13931   </View>
13932
13933   <View style={styles.ctrlWrapper}>
13934     <TouchableOpacity style={styles.ctrlBtn} onPress={handleStop} activeOpacity={0.7}>
13935       <Ionicons name="stop" size={24} color={colors.error} />
13936     </TouchableOpacity>
13937     <Text style={styles.ctrlLabel}>结束</Text>
13938   </View>
13939 </View>
13940
13941 {/* 亮度调节 */}
13942 {showBrightness && (
13943   <View style={styles.brightnessContainer}>
13944     <View style={styles.brightnessRow}>
13945       <Ionicons name="sunny" size={16} color={colors.textTertiary} />
13946       <TouchableOpacity

```



```

13947     style={styles.brightnessTrack}
13948     activeOpacity={1}
13949     onPress={(e) => {
13950         const { locationX } = e.nativeEvent;
13951         handleBrightnessChange(Math.max(0.05, Math.min(1, locationX / 200)));
13952     }}
13953 >
13954     <View style={[styles.brightnessFill, { width: `${brightness * 100}%` }]}>
13955         <View style={[styles.brightnessHandle, { left: '100%' }]} />
13956     </View>
13957 </TouchableOpacity>
13958 <Ionicons name="sunny" size={18} color={colors.warning} />
13959 </View>
13960 </View>
13961 )}
13962
13963 /* 完成覆盖层 */
13964 {isCompleted && renderCompletion()}
13965
13966 /* 操作确认弹窗 */
13967 {renderActionModal()}
13968
13969 /* 退出提醒弹窗 */
13970 {renderExitModal()}
13971
13972 /* 怪兽对话弹窗 */
13973 <Modal visible={showMonsterChat} animationType="slide" onRequestClose={() => setShowMonsterChat(false)}>
13974     <KeyboardAvoidingView behavior={Platform.OS === 'ios' ? 'padding' : 'height'} style={{ flex: 1 }}>
13975         <View style={styles.chatOverlay}>
13976             <View style={[styles.chatCard, { backgroundColor: colors.card }]}>
13977                 <View style={[styles.chatHeader, { borderColor: colors.borderDark }]}>
13978                     <Text style={[styles.chatTitle, { color: colors.textPrimary }]}>和小怪兽聊聊</Text>
13979                     <TouchableOpacity onPress={() => setShowMonsterChat(false)}>
13980                         <Ionicons name="close" size={24} color={colors.textSecondary} />
13981                     </TouchableOpacity>
13982                 </View>
13983                 <ScrollView style={{ flex: 1, paddingHorizontal: 16, paddingTop: 12 }} contentContainerStyle={{
13984                     {chatMessages.length === 0 && (
13985                         <Text style={{ color: colors.textTertiary, textAlign: 'center', marginTop: 20, }}>
13986                             聊聊学习、吐槽烦恼、求安慰都行~
13987                         </Text>
13988                     )}}
13989                     {chatMessages.map((msg, i) => (
13990                         <View key={i} style={[styles.chatBubble, {
13991                             alignSelf: msg.isUser ? 'flex-end' : 'flex-start',
13992                             backgroundColor: msg.isUser ? (colors.primary + '20') : colors.borderLight,
13993                         }]}>
13994                             <Text style={{ color: msg.isUser ? colors.primary : colors.textPrimary, fontSize: 13, line
13995                         </View>
13996                     )}}

```

```

13997     </ScrollView>
13998     {energyInfo && (
13999         <View style={[styles.energyBar, { backgroundColor: colors.borderLight }]}>
14000             <Text style={{ color: colors.textSecondary, fontSize: 11, }}>
14001                 消耗 Π {energyInfo.cost} · 剩余 Π {energyInfo.remaining}
14002             </Text>
14003         </View>
14004     )}
14005     <View style={[styles.chatInputRow, { borderColor: colors.borderDark }]}>
14006         <TextInput
14007             style={[styles.chatTextInput, { color: colors.textPrimary, backgroundColor: colors.borderLig
14008                 placeholder="说点什么..."
14009                 placeholderTextColor={colors.textTertiary}
14010                 value={chatInput}
14011                 onChangeText={setChatInput}
14012                 multiline
14013             />
14014         <TouchableOpacity onPress={handleSendChat} disabled={isSending} style={[styles.chatSendBtn, {
14015             <Ionicons name="send" size={16} color="#fff" />
14016         </TouchableOpacity>
14017     </View>
14018 </View>
14019 </View>
14020 </KeyboardAvoidingView>
14021 </Modal>
14022 </SafeAreaView>
14023 );
14024 };
14025
14026 export default PomodoroScreen;
14027 === mobile/app/register.tsx ===
14027 import React, { useState, useMemo } from 'react';
14028 import { View, Text, StyleSheet, Alert, ScrollView, TouchableOpacity, Dimensions } from 'react-native';
14029 import { SafeAreaView } from 'react-native-safe-area-context';
14030 import { router } from 'expo-router';
14031 import { Ionicons } from '@expo/vector-icons';
14032 import { useTheme } from '../src/contexts/ThemeContext';
14033 import { authService } from '../src/services/api';
14034 import { saveAuthData } from '../src/utls/auth';
14035 import { showErrorAlert, toErrorMessage } from '../src/utls';
14036 import { FloatingInputBar, InputFieldConfig } from '../src/components/FloatingInputBar';
14037 import AsyncStorage from '@react-native-async-storage/async-storage';
14038 import { STORAGE_KEYS } from '../src/utls/storage';
14039
14040 const RegisterScreen = () => {
14041     const { colors } = useTheme();
14042     const [username, setUsername] = useState("");
14043     const [email, setEmail] = useState("");
14044     const [password, setPassword] = useState("");
14045     const [confirmPassword, setConfirmPassword] = useState("");

```

```

14046 const [loading, setLoading] = useState(false);
14047 const [showPassword, setShowPassword] = useState(false);
14048 const [showConfirmPassword, setShowConfirmPassword] = useState(false);
14049 const [activeFieldId, setActiveFieldId] = useState<string | null>(null);
14050
14051 const inputFields: InputFieldConfig[] = useMemo(() => [
14052   {
14053     id: 'username',
14054     label: '用户名',
14055     icon: 'person',
14056     value: username,
14057     onChangeText: setUsername,
14058     placeholder: '请输入用户名',
14059   },
14060   {
14061     id: 'email',
14062     label: '邮箱',
14063     icon: 'mail',
14064     value: email,
14065     onChangeText: setEmail,
14066     keyboardType: 'email-address',
14067     placeholder: '请输入邮箱',
14068   },
14069   {
14070     id: 'password',
14071     label: '密码',
14072     icon: 'lock-closed',
14073     value: password,
14074     onChangeText: setPassword,
14075     secureTextEntry: !showPassword,
14076     placeholder: '至少6位密码',
14077   },
14078   {
14079     id: 'confirmPassword',
14080     label: '确认密码',
14081     icon: 'lock-closed',
14082     value: confirmPassword,
14083     onChangeText: setConfirmPassword,
14084     secureTextEntry: !showConfirmPassword,
14085     placeholder: '请再次输入密码',
14086   },
14087 ], [username, email, password, confirmPassword, showPassword, showConfirmPassword]);
14088
14089 const handleRegister = async () => {
14090   console.log('[Register] 开始注册 - 用户名:', username, '邮箱:', email);
14091   if (!username || !email || !password || !confirmPassword) {
14092     console.log('[Register] 验证失败 - 缺少必填字段');
14093     Alert.alert('错误', '请填写所有必填字段');
14094     return;
14095   }

```

```

14096
14097   if (username.trim().length < 2) {
14098       Alert.alert('错误', '用户名至少2个字符');
14099       return;
14100   }
14101
14102   const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
14103   if (!emailRegex.test(email)) {
14104       Alert.alert('错误', '请输入有效的邮箱地址');
14105       return;
14106   }
14107
14108   if (password !== confirmPassword) {
14109       console.log('[Register] 验证失败 - 密码不一致');
14110       Alert.alert('错误', '两次输入的密码不一致');
14111       return;
14112   }
14113
14114   if (password.length < 6) {
14115       console.log('[Register] 验证失败 - 密码长度不足');
14116       Alert.alert('错误', '密码长度至少6位');
14117       return;
14118   }
14119
14120   setLoading(true);
14121   try {
14122       await authService.register({ username: username.trim(), email, password });
14123
14124       console.log('[Register] 注册成功, 跳转验证码页面 - 邮箱:', email);
14125       setLoading(false);
14126
14127       router.push({ pathname: '/verify-email', params: { email, username: username.trim() } });
14128   } catch (error) {
14129       console.error('[Register] 注册失败:', error);
14130       setLoading(false);
14131
14132       if (error && typeof error === 'object' && 'message' in error) {
14133           const errorMessage = (error as any).message;
14134           if (errorMessage.includes('邮箱')) {
14135               Alert.alert('注册失败', '该邮箱已被注册');
14136           } else if (errorMessage.includes('用户名')) {
14137               Alert.alert('注册失败', '该用户名已被使用');
14138           } else {
14139               showErrorAlert('注册失败', errorMessage);
14140           }
14141       } else {
14142           showErrorAlert('注册失败', toErrorMessage(error));
14143       }
14144   }
14145   };

```

```

14146
14147 const handleLogin = () => {
14148   router.push('/login');
14149 };
14150
14151 const styles = useMemo(() => StyleSheet.create({
14152   container: {
14153     flex: 1,
14154     backgroundColor: colors.background,
14155   },
14156   scrollContent: {
14157     flexGrow: 1,
14158     paddingHorizontal: 20,
14159     paddingBottom: 0,
14160     minHeight: Dimensions.get('window').height,
14161   },
14162   backgroundDecorations: {
14163     position: 'absolute',
14164     top: 0,
14165     left: 0,
14166     right: 0,
14167     bottom: 0,
14168     overflow: 'hidden',
14169   },
14170   gradientCircle1: {
14171     position: 'absolute',
14172     top: -80,
14173     right: -80,
14174     width: 256,
14175     height: 256,
14176     borderRadius: 128,
14177     backgroundColor: colors.primary,
14178     opacity: 0.1,
14179   },
14180   gradientCircle2: {
14181     position: 'absolute',
14182     top: '33%',
14183     left: -80,
14184     width: 192,
14185     height: 192,
14186     borderRadius: 96,
14187     backgroundColor: colors.orange,
14188     opacity: 0.08,
14189   },
14190   gradientCircle3: {
14191     position: 'absolute',
14192     bottom: 80,
14193     right: 40,
14194     width: 128,
14195     height: 128,

```

```

14196   borderRadius: 64,
14197   backgroundColor: colors.success,
14198   opacity: 0.06,
14199 },
14200 logoSection: {
14201   alignItems: 'center',
14202   paddingTop: 60,
14203   paddingBottom: 40,
14204 },
14205 logoContainer: {
14206   width: 80,
14207   height: 80,
14208   borderRadius: 24,
14209   backgroundColor: 'linear-gradient(135deg, #5D9BFA, #7B5EA7)',
14210   justifyContent: 'center',
14211   alignItems: 'center',
14212   marginBottom: 16,
14213   shadowColor: colors.primary,
14214   shadowOffset: { width: 0, height: 8 },
14215   shadowOpacity: 0.4,
14216   shadowRadius: 32,
14217   elevation: 5,
14218 },
14219 appTitle: {
14220   fontSize: 28,
14221   fontWeight: '800',
14222   color: '#fff',
14223   marginBottom: 4,
14224   letterSpacing: -0.5,
14225 },
14226 appSubtitle: {
14227   fontSize: 14,
14228   color: colors.textSecondary,
14229 },
14230 formContainer: {
14231   gap: 16,
14232 },
14233 inputContainer: {
14234   height: 56,
14235   flexDirection: 'row',
14236   alignItems: 'center',
14237   backgroundColor: colors.inputBg,
14238   borderWidth: 1,
14239   borderColor: colors.inputBorder,
14240   borderRadius: 10,
14241   paddingHorizontal: 16,
14242 },
14243 inputIcon: {
14244   marginRight: 12,
14245 },

```

```

14246   textInput: {
14247     flex: 1,
14248     fontSize: 15,
14249     color: colors.textPrimary,
14250     paddingVertical: 0,
14251   },
14252   displayText: {
14253     flex: 1,
14254     fontSize: 15,
14255     paddingVertical: 0,
14256   },
14257   passwordToggle: {
14258     padding: 4,
14259   },
14260   registerButton: {
14261     height: 56,
14262     borderRadius: 16,
14263     backgroundColor: colors.primary,
14264     justifyContent: 'center',
14265     alignItems: 'center',
14266     shadowColor: colors.shadow,
14267     shadowOffset: { width: 0, height: 2 },
14268     shadowOpacity: 0.08,
14269     shadowRadius: 12,
14270     elevation: 3,
14271   },
14272   registerButtonDisabled: {
14273     backgroundColor: colors.border,
14274     shadowOpacity: 0,
14275   },
14276   registerButtonText: {
14277     color: '#fff',
14278     fontSize: 16,
14279     fontWeight: '700',
14280   },
14281   loadingSpinner: {
14282     width: 20,
14283     height: 20,
14284     borderRadius: 10,
14285     borderWidth: 2,
14286     borderColor: '#fff',
14287     borderTopColor: 'transparent',
14288   },
14289   divider: {
14290     flexDirection: 'row',
14291     alignItems: 'center',
14292     marginVertical: 24,
14293     gap: 12,
14294   },
14295   dividerLine: {

```

```

14296     flex: 1,
14297     height: 1,
14298     backgroundColor: colors.border,
14299 },
14300 dividerText: {
14301     color: colors.textSecondary,
14302     fontSize: 13,
14303 },
14304 socialContainer: {
14305     flexDirection: 'row',
14306     gap: 12,
14307     marginBottom: 32,
14308 },
14309 socialButton: {
14310     flex: 1,
14311     height: 48,
14312     flexDirection: 'row',
14313     alignItems: 'center',
14314     justifyContent: 'center',
14315     gap: 8,
14316     backgroundColor: colors.surface,
14317     borderWidth: 1,
14318     borderColor: colors.border,
14319     borderRadius: 12,
14320 },
14321 socialButtonText: {
14322     fontSize: 13,
14323     color: colors.textPrimary,
14324 },
14325 loginContainer: {
14326     alignItems: 'center',
14327     marginBottom: 16,
14328 },
14329 loginText: {
14330     color: colors.textSecondary,
14331     fontSize: 14,
14332 },
14333 loginLink: {
14334     color: colors.primary,
14335     fontWeight: '700',
14336     marginLeft: 4,
14337 },
14338 termsText: {
14339     textAlign: 'center',
14340     color: colors.textSecondary,
14341     fontSize: 12,
14342     marginBottom: 20,
14343 },
14344 termsLink: {
14345     color: colors.primary,

```



```

14346     },
14347   )), [colors]);
14348
14349   return (
14350     <SafeAreaView style={styles.container} edges={['top', 'bottom']}>
14351       <ScrollView
14352         style={styles.container}
14353         contentContainerStyle={styles.scrollContent}
14354         showsVerticalScrollIndicator={false}
14355         keyboardShouldPersistTaps="handled"
14356       >
14357         {/* 背景装饰 */}
14358         <View style={styles.backgroundDecorations}>
14359           <View style={styles.gradientCircle1} />
14360           <View style={styles.gradientCircle2} />
14361           <View style={styles.gradientCircle3} />
14362         </View>
14363
14364         {/* Logo 区域 */}
14365         <View style={styles.logoSection}>
14366           <View style={styles.logoContainer}>
14367             <Ionicons name="flash" size={36} color="#fff" />
14368           </View>
14369           <Text style={styles.appTitle}>创建账号</Text>
14370           <Text style={styles.appSubtitle}>开启你的技能冒险之旅</Text>
14371         </View>
14372
14373         {/* 表单 */}
14374         <View style={styles.formContainer}>
14375           <TouchableOpacity
14376             style={styles.inputContainer}
14377             activeOpacity={0.7}
14378             onPress={() => setActiveFieldId('username')}
14379           >
14380             <Ionicons name="person" size={20} color={colors.primary} style={styles.inputIcon} />
14381             <Text
14382               style={{styles.displayText, username ? { color: colors.textPrimary } : { color: colors.textSec
14383                 numberOfLines={1}
14384             >
14385               {username || '用户名'}
14386             </Text>
14387           </TouchableOpacity>
14388
14389           <TouchableOpacity
14390             style={styles.inputContainer}
14391             activeOpacity={0.7}
14392             onPress={() => setActiveFieldId('email')}
14393           >
14394             <Ionicons name="mail" size={20} color={colors.primary} style={styles.inputIcon} />
14395             <Text

```

```

14396         style={{styles.displayText, email ? { color: colors.textPrimary } : { color: colors.textSecond
14397         numberOfLines={1}
14398     >
14399         {email || '邮箱'}
14400     </Text>
14401 </TouchableOpacity>
14402
14403 <TouchableOpacity
14404     style={styles.inputContainer}
14405     activeOpacity={0.7}
14406     onPress={() => setActiveFieldId('password')}
14407 >
14408     <Ionicons name="lock-closed" size={20} color={colors.primary} style={styles.inputIcon} />
14409     <Text
14410         style={{styles.displayText, password ? { color: colors.textPrimary } : { color: colors.textSec
14411         numberOfLines={1}
14412     >
14413         {password ? '●'.repeat(Math.min(password.length, 12)) : '密码'}
14414     </Text>
14415     <TouchableOpacity
14416         style={styles.passwordToggle}
14417         onPress={() => setShowPassword(!showPassword)}
14418         hitSlop={{ top: 8, bottom: 8, left: 8, right: 8 }}
14419     >
14420         <Ionicons name={showPassword ? "eye-off" : "eye"} size={20} color={colors.textSecondary} />
14421     </TouchableOpacity>
14422 </TouchableOpacity>
14423
14424 <TouchableOpacity
14425     style={styles.inputContainer}
14426     activeOpacity={0.7}
14427     onPress={() => setActiveFieldId('confirmPassword')}
14428 >
14429     <Ionicons name="lock-closed" size={20} color={colors.primary} style={styles.inputIcon} />
14430     <Text
14431         style={{styles.displayText, confirmPassword ? { color: colors.textPrimary } : { color: colors.
14432         numberOfLines={1}
14433     >
14434         {confirmPassword ? '●'.repeat(Math.min(confirmPassword.length, 12)) : '确认密码'}
14435     </Text>
14436     <TouchableOpacity
14437         style={styles.passwordToggle}
14438         onPress={() => setShowConfirmPassword(!showConfirmPassword)}
14439         hitSlop={{ top: 8, bottom: 8, left: 8, right: 8 }}
14440     >
14441         <Ionicons name={showConfirmPassword ? "eye-off" : "eye"} size={20} color={colors.textSecondary}
14442     </TouchableOpacity>
14443 </TouchableOpacity>
14444
14445 <TouchableOpacity

```

```

14446         style={[styles.registerButton, loading && styles.registerButtonDisabled]}
14447         onPress={handleRegister}
14448         disabled={loading}
14449         activeOpacity={0.7}
14450     >
14451         {loading ? (
14452             <View style={styles.loadingSpinner} />
14453         ) : (
14454             <Text style={styles.registerButtonText}>注册</Text>
14455         )}
14456     </TouchableOpacity>
14457 </View>
14458
14459 {/* 分割线 */}
14460 <View style={styles.divider}>
14461     <View style={styles.dividerLine} />
14462     <Text style={styles.dividerText}>或者</Text>
14463     <View style={styles.dividerLine} />
14464 </View>
14465
14466 {/* 社交注册 */}
14467 <View style={styles.socialContainer}>
14468     {[
14469         { name: 'Google', icon: 'logo-google', color: '#EA4335' },
14470         { name: 'Apple', icon: 'logo-apple', color: '#ffffff' },
14471     ].map((provider) => (
14472         <TouchableOpacity
14473             key={provider.name}
14474             style={styles.socialButton}
14475             onPress={() => {}}
14476             activeOpacity={0.7}
14477         >
14478             <Ionicons name={provider.icon as any} size={20} color={provider.color} />
14479             <Text style={styles.socialButtonText}>{provider.name}</Text>
14480         </TouchableOpacity>
14481     ))}
14482 </View>
14483
14484 {/* 登录链接 */}
14485 <View style={styles.loginContainer}>
14486     <Text style={styles.loginText}>
14487         已有账号?
14488         <Text style={styles.loginLink} onPress={handleLogin}>
14489             立即登录
14490         </Text>
14491     </Text>
14492 </View>
14493
14494 {/* 底部条款 */}
14495 <Text style={styles.termsText}>

```

```

14496         继续即表示你同意我们的{' '}
14497         <Text style={styles.termsLink}>服务条款</Text> 和{' '}
14498         <Text style={styles.termsLink}>隐私政策</Text>
14499     </Text>
14500 </ScrollView>
14501
14502     /* 键盘上方浮动输入栏 */
14503     <FloatingInputBar
14504         fields={inputFields}
14505         activeFieldId={activeFieldId}
14506         onDismiss={() => setActiveFieldId(null)}
14507         colors={colors}
14508     />
14509 </SafeAreaView>
14510 );
14511
14512
14513 };
14514
14515 export default RegisterScreen;
14516
14517 === mobile/app/skill-tree.tsx ===
14518 /**
14519  * SkillTreeScreen — 技能树 / 课程详情页
14520  *
14521  * 功能概览:
14522  * 1. 展示模块的「学习方法 / 学习目标 / 框架说明」描述区域
14523  * 2. 以「大标题（阶段）→ 小结点」的树形结构展示所有学习节点
14524  * 3. 点击结点 → 弹出 DurationModal → 选择时长 → 扣除体力 → 进入番茄钟页面
14525  * 4. 支持添加 / 删除自定义大标题和自定义学习结点
14526  * 5. 支持编辑自定义结点和大纲题的文本内容
14527  * 6. 纯自定义模块（无默认 SkillTree）也可正常展示与编辑
14528  *
14529  * 数据存储（AsyncStorage）:
14530  * - customModules: { [moduleId]: { name, description } } 自定义模块元信息
14531  * - customStages_{domain}: [{ id, name }] 自定义大标题
14532  * - customNodes_{domain}: [{ id, stageId, name, url, platform, duration }] 自定义结点
14533  */
14534 import React, { useState, useEffect, useMemo } from 'react';
14535 import {
14536     View, Text, StyleSheet, ScrollView, Alert, TouchableOpacity,
14537     Modal, ActivityIndicator, TextInput,
14538 } from 'react-native';
14539 import { SafeAreaView } from 'react-native-safe-area-context';
14540 import { router, useLocalSearchParams } from 'expo-router';
14541 import { Ionicons } from '@expo/vector-icons';
14542 import { getSkillTreeByDomain, PLATFORM_NAME_MAP } from '../src/data/skillTrees';
14543 import { SkillTree, PlatformType, StageType, SkillNode, SkillStage } from '../src/types/skill';
14544 import { useTheme } from '../src/contexts/ThemeContext';
14545 import AsyncStorage from '@react-native-async-storage/async-storage';
14546 import { monsterService } from '../src/services/api';

```

```

14545 import { proService } from '../src/services/api';
14546 import { getCurrentUser } from '../src/utlis/auth';
14547 import { SUBSCRIPTION_STORAGE_KEY } from '../src/utlis/pricing';
14548 import { MONSTER_CONFIG } from '../src/utlis/constants';
14549
14550 // ---- 本地类型定义 ----
14551
14552 /** 自定义大标题（阶段）的结构 */
14553 interface CustomStage {
14554   id: string;
14555   name: string;
14556 }
14557
14558 /** 自定义学习结点的结构 */
14559 interface CustomNode {
14560   id: string;
14561   stageId: string; // 归属哪个大标题
14562   name: string;
14563   url: string;
14564   platform: PlatformType;
14565   duration: number; // 预估学习时长（小时）
14566 }
14567
14568 // ===== 主组件 =====
14569
14570 const SkillTreeScreen = () => {
14571   const { domain } = useLocalSearchParams();
14572   const { colors } = useTheme();
14573   const [isPro, setIsPro] = useState(false);
14574
14575   // 加载 Pro 状态（以数据库 is_pro 字段为准）
14576   useEffect(() => {
14577     (async () => {
14578       try {
14579         const user = await getCurrentUser();
14580         if (user?.id) {
14581           const status = await proService.getStatus(user.id);
14582           setIsPro(status.isPro);
14583         }
14584       } catch {}
14585     })();
14586   }, []);
14587
14588   // ---- 样式表（依赖 colors） ----
14589   const s = useMemo(() => StyleSheet.create({
14590     safeArea: { flex: 1 },
14591     container: { flex: 1, alignItems: 'center', justifyContent: 'center' },
14592     loadingText: { fontSize: 14, marginTop: 12 },
14593     emptyContainer: { flex: 1, alignItems: 'center', justifyContent: 'center', paddingHorizontal: 40 },
14594     emptyBtn: {

```

```

14595     paddingVertical: 10, paddingHorizontal: 20, borderRadius: 12,
14596     borderWidth: 1, borderStyle: 'dashed',
14597   },
14598   emptyBtnText: { fontSize: 13, fontWeight: '600' },
14599   scrollContent: { flexGrow: 1, paddingBottom: 20 },
14600 // 顶部栏
14601   header: {
14602     flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between',
14603     paddingHorizontal: 20, paddingVertical: 16,
14604   },
14605   backBtnRect: {
14606     width: 40, height: 40, borderRadius: 10,
14607     alignItems: 'center', justifyContent: 'center',
14608   },
14609   backButton: { fontSize: 24, color: '#FFFFFF', fontWeight: 'bold' },
14610   headerTitle: { fontSize: 18, fontWeight: '600' },
14611   headerPlaceholder: { width: 40 },
14612 // 描述区
14613   descriptionSection: { paddingHorizontal: 20, marginBottom: 20 },
14614   descriptionTitle: { fontSize: 14, fontWeight: '600', marginTop: 16, marginBottom: 8 },
14615   descriptionText: { fontSize: 13, lineHeight: 20 },
14616 // 时长徽章
14617   totalDurationBadge: {
14618     flexDirection: 'row', alignItems: 'center', gap: 6, paddingHorizontal: 16, paddingVertical: 8,
14619     borderRadius: 10, marginHorizontal: 20, marginBottom: 20, alignSelf: 'center',
14620   },
14621   totalDurationText: { fontSize: 12, fontWeight: '600' },
14622 // 技能树容器
14623   treeContainer: { paddingHorizontal: 16 },
14624   stageColumn: { marginBottom: 20 },
14625   middleStage: { marginTop: 20 },
14626 // 大标题
14627   stageNode: {
14628     paddingHorizontal: 12, paddingVertical: 8, borderRadius: 10,
14629     borderWidth: 1, alignItems: 'center', marginBottom: 12,
14630   },
14631   stageNodeRow: { flexDirection: 'row', alignItems: 'center' },
14632   stageName: { fontSize: 12, fontWeight: '600', marginBottom: 2 },
14633   stageDuration: { fontSize: 10, color: '#888' },
14634 // 结点列表
14635   skillNodesList: { gap: 8 },
14636   skillNode: {
14637     backgroundColor: colors.surface, borderWidth: 1,
14638     borderRadius: 14, padding: 12, position: 'relative',
14639     borderColor: colors.hairline,
14640     shadowColor: colors.shadow,
14641     shadowOffset: { width: 0, height: 1 },
14642     shadowOpacity: 0.04,
14643     shadowRadius: 3,
14644     elevation: 2,

```

```

14645   },
14646   skillNodeHeader: { flexDirection: 'row', alignItems: 'center', gap: 8, marginBottom: 6 },
14647   nodeBadge: {
14648     width: 20, height: 20, borderRadius: 6,
14649     alignItems: 'center', justifyContent: 'center',
14650   },
14651   nodeBadgeText: { fontSize: 10, fontWeight: '600' },
14652   skillNodeName: { fontSize: 13, fontWeight: '600', marginBottom: 8, lineHeight: 16 },
14653   skillNodeFooter: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between' },
14654   platformText: { fontSize: 10, fontWeight: '500' },
14655   durationText: { fontSize: 10, color: '#888', fontWeight: '500' },
14656   arrowIcon: { position: 'absolute', right: 12, bottom: 12 },
14657   // 添加结点按钮
14658   addNodeBtn: {
14659     alignItems: 'center', justifyContent: 'center', paddingVertical: 10,
14660     borderRadius: 10, borderWidth: 1, borderStyle: 'dashed', marginTop: 4,
14661   },
14662   addNodeText: { fontSize: 12, fontWeight: '600' },
14663   // 添加大标题按钮
14664   addStageBtn: {
14665     alignItems: 'center', justifyContent: 'center', marginHorizontal: 16,
14666     paddingVertical: 14, borderRadius: 14, borderWidth: 1, borderStyle: 'dashed', marginTop: 8,
14667   },
14668   addStageText: { fontSize: 14, fontWeight: '600' },
14669   // 返回按钮 (空状态用)
14670   backBtn: {
14671     marginTop: 24, paddingVertical: 10, paddingHorizontal: 24,
14672     borderRadius: 14,
14673   },
14674   // 输入框通用
14675   addInput: {
14676     borderWidth: 1, borderRadius: 10, paddingHorizontal: 14,
14677     paddingVertical: 12, fontSize: 14, marginBottom: 12,
14678   },
14679   addLabel: { fontSize: 12, fontWeight: '600', marginBottom: 6 },
14680   // 平台选择
14681   platformSelector: { flexDirection: 'row', flexWrap: 'wrap', gap: 8, marginBottom: 16 },
14682   platformOption: {
14683     width: '30%', flexDirection: 'row', alignItems: 'center', justifyContent: 'center',
14684     gap: 4, paddingVertical: 10, borderRadius: 10, borderWidth: 1,
14685   },
14686   platformOptionText: { fontSize: 11, fontWeight: '600' },
14687   bottomPadding: { height: 40 },
14688   // 模态框通用
14689   modalOverlay: { flex: 1, backgroundColor: 'rgba(0,0,0,0.65)', alignItems: 'center', justifyContent: 'center'
14690   modalContent: { width: '100%', borderRadius: 16, padding: 24, backgroundColor: colors.surface },
14691   // 卡片式弹出 (学习时长选择用)
14692   overlayLight: { flex: 1, backgroundColor: 'rgba(0,0,0,0.3)', alignItems: 'center', justifyContent: 'center',
14693   modalCard: {
14694     width: '100%', maxWidth: 380, borderRadius: 16, padding: 24,

```

```

14695   borderWidth: 1, borderColor: colors.hairline,
14696   backgroundColor: colors.surface,
14697   shadowColor: colors.shadow,
14698   shadowOffset: { width: 0, height: 2 },
14699   shadowOpacity: 0.08,
14700   shadowRadius: 12,
14701   elevation: 6,
14702 },
14703 modalHeader: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', marginBottom: 16
14704 modalTitle: { fontSize: 20, fontWeight: '600' },
14705 modalClose: {
14706   width: 32, height: 32, borderRadius: 8,
14707   alignItems: 'center', justifyContent: 'center',
14708 },
14709 modalNodeName: { fontSize: 16, fontWeight: '600', color: '#E2E0DF', marginBottom: 16 },
14710 // 体力信息
14711 staminaInfo: { marginBottom: 16 },
14712 staminaBar: { height: 6, backgroundColor: colors.progressTrack, borderRadius: 9999, overflow: 'hidden', marg
14713 staminaFill: { height: '100%', borderRadius: 9999 },
14714 staminaText: { fontSize: 12, color: '#B6B2B8' },
14715 modalSubtitle: { fontSize: 14, fontWeight: '600', color: '#E2E0DF', marginBottom: 12 },
14716 // 时长选项
14717 durationOptions: { flexDirection: 'row', flexWrap: 'wrap', gap: 10, marginBottom: 16 },
14718 durationButton: {
14719   paddingHorizontal: 16, paddingVertical: 10, borderRadius: 10,
14720   borderWidth: 1, borderColor: colors.hairline,
14721 },
14722 durationButtonActive: {},
14723 durationTextSmall: { fontSize: 12, color: '#888' },
14724 durationTextActive: { color: 'FFFFFF' },
14725 costInfo: { marginBottom: 20 },
14726 costText: { fontSize: 12, color: '#C4A27A', marginBottom: 4 },
14727 suggestedText: { fontSize: 12, color: '#8A86A0' },
14728 modalConfirmBtn: { paddingVertical: 14, borderRadius: 14, alignItems: 'center' },
14729 modalConfirmText: { color: colors.onPrimary, fontSize: 16, fontWeight: '600' },
14730 // Toast
14731 toast: {
14732   position: 'absolute', bottom: 40, left: 20, right: 20,
14733   flexDirection: 'row', alignItems: 'center', justifyContent: 'center', gap: 8,
14734   backgroundColor: colors.surface, paddingVertical: 12, paddingHorizontal: 20, borderRadius: 14,
14735   borderWidth: 1, borderColor: colors.hairline,
14736 },
14737 toastText: { color: 'FFFFFF', fontSize: 14, fontWeight: '600' },
14738 }}, [colors]);
14739
14740 // ===== 子组件: 大标题结点 =====
14741
14742 const StageNode = ({ name, duration, stageColor, onLongPress, onEdit, isCustom }: {
14743   name: string; duration: number;
14744   stageColor: { bg: string; border: string; text: string };

```



```

14745     onLongPress?: () => void; onEdit?: () => void;
14746     isCustom?: boolean;
14747 }) => (
14748     <TouchableOpacity
14749         onLongPress={onLongPress}
14750         onPress={isCustom ? onEdit : undefined}
14751         disabled={!isCustom}
14752         activeOpacity={isCustom ? 0.6 : 1}
14753     >
14754         <View style={[s.stageNode, { backgroundColor: stageColor.bg, borderColor: stageColor.border }]}>
14755             <View style={s.stageNodeRow}>
14756                 <Text style={[s.stageName, { color: stageColor.text }]}>{name}</Text>
14757                 {isCustom && <Icons name="create-outline" size={10} color={stageColor.text} style={{ marginLeft:
14758             </View>
14759             <Text style={s.stageDuration}>{duration}小时</Text>
14760         </View>
14761     </TouchableOpacity>
14762 );
14763
14764 // ===== 子组件: 学习结点卡片 =====
14765
14766 const SkillNodeItem = ({ name, platform, duration, url, index, stageColor, platformIcon, platformColor, onSe
14767     name: string; platform: PlatformType; duration: number; url: string;
14768     index: number; stageColor: { bg: string; border: string; text: string };
14769     platformIcon: string; platformColor: string;
14770     onSelect: (name: string, url: string, suggestedDuration: number) => void;
14771     onLongPress?: () => void; onEdit?: () => void;
14772     isCustom?: boolean;
14773 }) => (
14774     <TouchableOpacity
14775         style={[s.skillNode, { borderColor: stageColor.border }]}
14776         onPress={() => onSelect(name, url, duration)}
14777         onLongPress={onLongPress}
14778         activeOpacity={0.7}
14779     >
14780         <View style={s.skillNodeHeader}>
14781             <View style={[s.nodeBadge, { backgroundColor: stageColor.border }]}>
14782                 <Text style={[s.nodeBadgeText, { color: stageColor.text }]}>{index + 1}</Text>
14783             </View>
14784             <Icons name={platformIcon as any} size={14} color={platformColor} />
14785             {isCustom && (
14786                 <TouchableOpacity onPress={onEdit} hitSlop={{ top: 8, bottom: 8, left: 8, right: 8 }}>
14787                     <Icons name="create-outline" size={12} color="#888" />
14788                 </TouchableOpacity>
14789             )}
14790         </View>
14791         <Text style={[s.skillNodeName, { color: stageColor.text }]}>{name}</Text>
14792         <View style={s.skillNodeFooter}>
14793             <Text style={[s.platformText, { color: platformColor }]}>
14794                 {PLATFORM_NAME_MAP[platform] ?? platform}

```

```

14795     </Text>
14796     <Text style={s.durationText}>{duration}h</Text>
14797 </View>
14798 <View style={s.arrowIcon}>
14799     <Icons name="chevron-forward" size={14} color={stageColor.text} />
14800 </View>
14801 </TouchableOpacity>
14802 );
14803
14804 // ===== 模态框：选择学习时长 =====
14805
14806 const DurationModal = ({ visible, onClose, nodeName, url, suggestedDuration }: {
14807     visible: boolean; onClose: () => void;
14808     nodeName: string; url: string; suggestedDuration: number;
14809 }) => {
14810     const [selectedDuration, setSelectedDuration] = useState(25);
14811     const [monsterStamina, setMonsterStamina] = useState(100);
14812     const [maxStamina, setMaxStamina] = useState(100);
14813
14814     useEffect(() => {
14815         if (!visible) return;
14816         setSelectedDuration(25);
14817         const loadStamina = async () => {
14818             const monster = await AsyncStorage.getItem('monster');
14819             if (monster) {
14820                 const data = JSON.parse(monster);
14821                 const proChecked = isPro; // 使用当前已加载的状态
14822                 const staminaMax = proChecked ? MONSTER_CONFIG.STAMINA.PRO_MAX : (data.type === 'calm' ? 120 : 100);
14823                 setMonsterStamina(typeof data.stamina === 'number' ? Math.min(data.stamina, staminaMax) : staminaMax);
14824                 setMaxStamina(staminaMax);
14825             }
14826         };
14827         loadStamina();
14828     }, [visible]);
14829
14830     const durations = [15, 25, 30, 45, 60];
14831     const staminaCost = isPro ? 0 : Math.ceil(selectedDuration / 5);
14832
14833     const handleConfirm = async () => {
14834         if (monsterStamina < staminaCost) {
14835             Alert.alert('体力不足', `需要 ${staminaCost} 点体力, 当前体力 ${monsterStamina} 点`);
14836             return;
14837         }
14838         console.log('[DurationModal] 确认开始学习', { nodeName, url, selectedDuration, staminaCost });
14839         try {
14840             const monster = await AsyncStorage.getItem('monster');
14841             if (monster) {
14842                 const m = JSON.parse(monster);
14843                 m.stamina = Math.max(0, m.stamina - staminaCost);
14844                 await AsyncStorage.setItem('monster', JSON.stringify(m));

```

```

14845     }
14846     const user = await getCurrentUser();
14847     if (user?.id) await monsterService.consumeStamina(user.id, staminaCost).catch(() => {});
14848
14849     onClose();
14850     router.push({
14851       pathname: '/pomodoro',
14852       params: { nodeName, url, duration: String(selectedDuration) },
14853     });
14854   } catch {
14855     Alert.alert('错误', '操作失败');
14856   }
14857 };
14858
14859 if (!visible) return null;
14860 return (
14861   <Modal visible transparent onRequestClose={onClose} animationType="fade">
14862     <TouchableOpacity style={s.overlayLight} onPress={onClose} activeOpacity={1}>
14863       <View style={[s.modalCard, { backgroundColor: colors.backgroundDark || colors.background, borderColo
14864         <View style={s.modalHeader}>
14865           <Text style={s.modalTitle}>开始学习</Text>
14866           <TouchableOpacity style={[s.modalClose, { backgroundColor: colors.border }] } onPress={onClose}>
14867             <Ionicons name="close" size={20} color="#888" />
14868           </TouchableOpacity>
14869         </View>
14870         <Text style={s.modalNodeName}>{nodeName}</Text>
14871         <View style={s.staminaInfo}>
14872           <View style={s.staminaBar}>
14873             <View style={[s.staminaFill, { width: `${Math.min(100, (monsterStamina / maxStamina) * 100)}%`
14874           </View>
14875           <Text style={[s.staminaText, { color: colors.textSecondary }] }>体力: {monsterStamina}/{maxStamina}
14876         </View>
14877         <Text style={s.modalSubtitle}>选择学习时长</Text>
14878         <View style={s.durationOptions}>
14879           {durations.map(d => (
14880             <TouchableOpacity
14881               key={d}
14882               onPress={() => setSelectedDuration(d)}
14883               style={[s.durationButton, selectedDuration === d && { borderColor: colors.primary },
14884                 { backgroundColor: selectedDuration === d ? colors.primary : colors.backgroundDark || colo
14885             ]}>
14886             <Text style={[s.durationTextSmall, selectedDuration === d && s.durationTextActive]}>{d}分钟</T
14887           </TouchableOpacity>
14888         ]))
14889       </View>
14890       <View style={s.costInfo}>
14891         <Text style={s.costText}>{isPro ? 'Pro 会员 · 无限次跳转' : '预计消耗体力: ${staminaCost} 点'}</Text>
14892         <Text style={[s.suggestedText, { color: colors.textSecondary }] }>建议学习时长: {suggestedDuration}小时</
14893       </View>
14894       <TouchableOpacity style={[s.modalConfirmBtn, { backgroundColor: colors.primary }] } onPress={handle

```

```

14895         <Text style={s.modalConfirmText}>开始学习</Text>
14896     </TouchableOpacity>
14897 </View>
14898 </TouchableOpacity>
14899 </Modal>
14900 );
14901 };
14902
14903 const PLATFORM_OPTIONS: { key: PlatformType; label: string; icon: string; color: string }[] = useMemo(() =>
14904     [
14905         { key: 'bilibili' as PlatformType, label: 'B站', icon: 'play-circle', color: '#FB7299' },
14906         { key: 'xiaohongshu' as PlatformType, label: '小红书', icon: 'book', color: '#FF2442' },
14907         { key: 'mooc' as PlatformType, label: '中国大学MOOC', icon: 'school', color: colors.primary },
14908     ], [colors.primary]);
14909 /** 获取平台对应的图标和颜色 */
14910 const getPlatformIcon = (platform: PlatformType) => {
14911     switch (platform) {
14912         case 'bilibili': return { icon: 'play-circle', color: '#FB7299' };
14913         case 'xiaohongshu': return { icon: 'book', color: '#FF2442' };
14914         case 'mooc': return { icon: 'school', color: colors.primary };
14915         default: return { icon: 'link', color: '#888' };
14916     }
14917 };
14918
14919 /** 获取阶段对应的颜色配置 */
14920 const getStageColor = (stage: StageType | string) => {
14921     switch (stage) {
14922         case 'beginner': return { bg: colors.borderLight, border: colors.borderDark, text: colors.primary };
14923         case 'intermediate': return { bg: 'rgba(72,209,176,0.15)', border: 'rgba(72,209,176,0.3)', text: '#48D1B' };
14924         case 'advanced': return { bg: 'rgba(255,152,0,0.15)', border: 'rgba(255,152,0,0.3)', text: '#FF9800' };
14925         default: return { bg: colors.borderLight, border: colors.borderDark, text: colors.primary };
14926     }
14927 };
14928
14929 // ---- 技能树数据 ----
14930 const [skillTree, setSkillTree] = useState<SkillTree | null>(null);
14931 const [loading, setLoading] = useState(true);
14932
14933 // ---- DurationModal 相关 ----
14934 const [modalVisible, setModalVisible] = useState(false);
14935 const [selectedNode, setSelectedNode] = useState({ name: '', url: '', suggestedDuration: 0 });
14936
14937 // ---- 自定义数据 ----
14938 const [customStages, setCustomStages] = useState<CustomStage[]>([]);
14939 const [customNodes, setCustomNodes] = useState<CustomNode[]>([]);
14940 const [customDescription, setCustomDescription] = useState('');
14941
14942 // ---- 添加大标题 / 结点的弹窗 ----
14943 const [addStageVisible, setAddStageVisible] = useState(false);
14944 const [addNodeVisible, setAddNodeVisible] = useState(false);

```

```

14945 const [newStageName, setNewStageName] = useState("");
14946 const [newNodeName, setNewNodeName] = useState("");
14947 const [newNodeUrl, setNewNodeUrl] = useState("");
14948 const [newNodePlatform, setNewNodePlatform] = useState<PlatformType>('bilibili');
14949 const [targetStageId, setTargetStageId] = useState(""); // 当前要添加结点的目标大标题 ID
14950
14951 // ---- 编辑自定义结点 / 大标题的弹窗 ----
14952 const [editNodeVisible, setEditNodeVisible] = useState(false);
14953 const [editStageVisible, setEditStageVisible] = useState(false);
14954 // 编辑中的临时状态
14955 const [editingNode, setEditingNode] = useState<CustomNode | null>(null);
14956 const [editingStage, setEditingStage] = useState<CustomStage | null>(null);
14957 const [editNodeName, setEditNodeName] = useState("");
14958 const [editNodeUrl, setEditNodeUrl] = useState("");
14959 const [editNodePlatform, setEditNodePlatform] = useState<PlatformType>('bilibili');
14960 const [editStageName, setEditStageName] = useState("");
14961
14962 // ---- Toast ----
14963 const [toastMessage, setToastMessage] = useState("");
14964
14965 // ===== 数据加载 =====
14966
14967 useEffect(() => {
14968   loadAllData();
14969 }, [domain]);
14970
14971 /** 加载默认技能树 + 自定义数据 */
14972 const loadAllData = async () => {
14973   if (!domain) return;
14974   setLoading(true);
14975
14976   // 1. 尝试加载默认技能树
14977   const tree = getSkillTreeByDomain(domain as string);
14978   setSkillTree(tree || null);
14979
14980   // 2. 加载自定义模块元信息 (名称 + 介绍)
14981   const customModulesStr = await AsyncStorage.getItem('customModules');
14982   if (customModulesStr) {
14983     const customModules: Record<string, { name: string; description: string }> = JSON.parse(customModulesStr);
14984     if (customModules[domain as string]) {
14985       setCustomDescription(customModules[domain as string].description || "");
14986     }
14987   }
14988
14989   // 3. 加载自定义大标题和结点
14990   await loadCustomData();
14991   setLoading(false);
14992 };
14993
14994 /** 从 AsyncStorage 加载自定义的大标题和结点 */

```

```

14995 const loadCustomData = async () => {
14996   if (!domain) return;
14997   try {
14998     const stagesJson = await AsyncStorage.getItem(`customStages_${domain}`);
14999     const nodesJson = await AsyncStorage.getItem(`customNodes_${domain}`);
15000     if (stagesJson) setCustomStages(JSON.parse(stagesJson));
15001     if (nodesJson) setCustomNodes(JSON.parse(nodesJson));
15002   } catch (e) {
15003     console.error('[SkillTree] 加载自定义数据失败:', e);
15004   }
15005 };
15006
15007 /** 保存自定义大标题到 AsyncStorage */
15008 const saveCustomStages = async (stages: CustomStage[]) => {
15009   if (!domain) return;
15010   await AsyncStorage.setItem(`customStages_${domain}`, JSON.stringify(stages));
15011   setCustomStages(stages);
15012 };
15013
15014 /** 保存自定义结点到 AsyncStorage */
15015 const saveCustomNodes = async (nodes: CustomNode[]) => {
15016   if (!domain) return;
15017   await AsyncStorage.setItem(`customNodes_${domain}`, JSON.stringify(nodes));
15018   setCustomNodes(nodes);
15019 };
15020
15021 // ===== 添加大标题 =====
15022
15023 const handleAddStage = () => {
15024   if (!newStageName.trim()) return;
15025   const id = `custom_${Date.now()}`;
15026   saveCustomStages([...customStages, { id, name: newStageName.trim() }]);
15027   setNewStageName('');
15028   setAddStageVisible(false);
15029   showToast('大标题添加成功');
15030 };
15031
15032 // ===== 添加结点 =====
15033
15034 const handleAddNode = () => {
15035   if (!newNodeName.trim() || !newNodeUrl.trim()) return;
15036   const node: CustomNode = {
15037     id: `custom-node_${Date.now()}`,
15038     stagelId: targetStagelId,
15039     name: newNodeName.trim(),
15040     url: newNodeUrl.trim(),
15041     platform: newNodePlatform,
15042     duration: 1,
15043   };
15044   saveCustomNodes([...customNodes, node]);

```

```

15045     setNewNodeName(''); setNewNodeUrl(''); setNewNodePlatform('bilibili');
15046     setAddNodeVisible(false);
15047     showToast('结点添加成功');
15048 };
15049
15050 const openAddNodeForStage = (stageId: string) => {
15051     setTargetStageId(stageId);
15052     setAddNodeVisible(true);
15053 };
15054
15055 // ===== 删除大标题 / 结点 =====
15056
15057 const handleDeleteStage = (stageId: string, stageName: string) => {
15058     Alert.alert('删除大标题', `确定要删除「${stageName}」及其下所有结点吗?`, [
15059         { text: '取消', style: 'cancel' },
15060         { text: '删除', style: 'destructive',
15061             onPress: async () => {
15062                 await saveCustomStages(customStages.filter(s => s.id !== stageId));
15063                 await saveCustomNodes(customNodes.filter(n => n.stageId !== stageId));
15064             }},
15065     ]);
15066 };
15067
15068 const handleDeleteNode = (nodeId: string, nodeName: string) => {
15069     Alert.alert('删除结点', `确定要删除「${nodeName}」吗?`, [
15070         { text: '取消', style: 'cancel' },
15071         { text: '删除', style: 'destructive',
15072             onPress: async () => saveCustomNodes(customNodes.filter(n => n.id !== nodeId)),
15073         },
15074     ]);
15075 };
15076
15077 // ===== 编辑自定义大标题 =====
15078
15079 const openEditStage = (stage: CustomStage) => {
15080     setEditingStage(stage);
15081     setEditStageName(stage.name);
15082     setEditStageVisible(true);
15083 };
15084
15085 const handleSaveEditStage = () => {
15086     if (!editingStage || !editStageName.trim()) return;
15087     saveCustomStages(customStages.map(s =>
15088         s.id === editingStage.id ? { ...s, name: editStageName.trim() } : s,
15089     ));
15090     setEditStageVisible(false);
15091     showToast('大标题已更新');
15092 };
15093
15094 // ===== 编辑自定义结点 =====

```

```

15095
15096 const openEditNode = (node: CustomNode) => {
15097   setEditingNode(node);
15098   setEditNodeName(node.name);
15099   setEditNodeUrl(node.url);
15100   setEditNodePlatform(node.platform);
15101   setEditNodeVisible(true);
15102 };
15103
15104 const openEditDefaultNode = (node: SkillNode, stageld: string) => {
15105   const pseudo: CustomNode = {
15106     id: node.id,
15107     stageld,
15108     name: node.name,
15109     url: node.url,
15110     platform: node.platform,
15111     duration: node.duration,
15112   };
15113   setEditingNode(pseudo);
15114   setEditNodeName(node.name);
15115   setEditNodeUrl(node.url);
15116   setEditNodePlatform(node.platform);
15117   setEditNodeVisible(true);
15118 };
15119
15120 const handleSaveEditNode = () => {
15121   if (!editingNode || !editNodeName.trim() || !editNodeUrl.trim()) return;
15122   const exists = customNodes.some(n => n.id === editingNode.id);
15123   if (exists) {
15124     saveCustomNodes(customNodes.map(n =>
15125       n.id === editingNode.id
15126         ? { ...n, name: editNodeName.trim(), url: editNodeUrl.trim(), platform: editNodePlatform }
15127         : n,
15128     ));
15129   } else {
15130     const newNode: CustomNode = {
15131       id: editingNode.id,
15132       stageld: editingNode.stageld,
15133       name: editNodeName.trim(),
15134       url: editNodeUrl.trim(),
15135       platform: editNodePlatform,
15136       duration: 1,
15137     };
15138     saveCustomNodes([...customNodes, newNode]);
15139   }
15140   setEditNodeVisible(false);
15141   showToast('结点已更新');
15142 };
15143
15144 // ===== 结点跳转 =====

```



```

15145
15146 const handleNodeSelect = (name: string, url: string, suggestedDuration: number) => {
15147   console.log('[SkillTree] 选择结点', { name, url, suggestedDuration });
15148   setSelectedNode({ name, url, suggestedDuration });
15149   setModalVisible(true);
15150 };
15151
15152 // ---- 返回上一页 ----
15153 const handleGoBack = () => router.back();
15154
15155 // ---- Toast 辅助函数 ----
15156 const showToast = (msg: string) => {
15157   setToastMessage(msg);
15158   setTimeout(() => setToastMessage(""), 2000);
15159 };
15160
15161 // ===== 数据合并 =====
15162
15163 /**
15164  * 合并默认 SkillTree 的 stages 与自定义 stages
15165  * - 如果是纯自定义模块（无默认 SkillTree），只展示自定义数据
15166  * - 如果同时存在，默认数据在前，自定义数据追加在后
15167  */
15168 const mergedStages = useMemo(() => {
15169   // 默认阶段数据（可能为空）
15170   const defaultStages: (SkillStage & { isCustom?: boolean })[] =
15171     (skillTree?.stages ?? []).map(s => ({ ...s, isCustom: false }));
15172
15173   // 将自定义结点按 stageId 分组，构造 SkillStage 数组
15174   const customStageList: (SkillStage & { isCustom?: boolean })[] = customStages.map(cs => {
15175     const stageNodes = customNodes
15176       .filter(n => n.stageId === cs.id)
15177       .map((n): SkillNode => ({
15178         id: n.id,
15179         name: n.name,
15180         description: "",
15181         stage: 'beginner' as StageType,
15182         platform: n.platform,
15183         url: n.url,
15184         duration: n.duration,
15185       }));
15186     return {
15187       id: cs.id as StageType,
15188       name: cs.name,
15189       duration: stageNodes.reduce((sum, n) => sum + n.duration, 0),
15190       nodes: stageNodes,
15191       isCustom: true,
15192     };
15193   });
15194

```

```

15195     return [...defaultStages, ...customStageList];
15196   }, [skillTree, customStages, customNodes]);
15197
15198   /** 总学习时长: 默认基础 + 自定义结点累加 */
15199   const totalDuration = useMemo(() => {
15200     const base = skillTree?.totalDuration ?? 0;
15201     return base + customNodes.reduce((sum, n) => sum + n.duration, 0);
15202   }, [skillTree, customNodes]);
15203
15204   // ===== 加载中状态 =====
15205
15206   if (loading) {
15207     return (
15208       <SafeAreaView style={{s.safeArea, { backgroundColor: colors.background }}}>
15209         <View style={s.container}>
15210           <ActivityIndicator size="large" color={colors.primary} />
15211           <Text style={{s.loadingText, { color: colors.textSecondary }}}>加载中...</Text>
15212         </View>
15213       </SafeAreaView>
15214     );
15215   }
15216
15217   // ===== 空状态: 完全无数据 =====
15218
15219   // 如果既没有默认 SkillTree, 也没有任何自定义数据 → 显示空状态
15220   const isEmpty = !skillTree && customStages.length === 0;
15221
15222   if (isEmpty) {
15223     return (
15224       <SafeAreaView style={{s.safeArea, { backgroundColor: colors.background }}}>
15225         <View style={s.emptyContainer}>
15226           <Icons name="book-outline" size={48} color={colors.textTertiary} />
15227           <Text style={{ fontSize: 14, color: colors.textSecondary, marginTop: 12, textAlign: 'center' }}>
15228             暂无学习内容, 请添加大标题和结点
15229           </Text>
15230           <View style={{ flexDirection: 'row', gap: 12, marginTop: 20 }}>
15231             <TouchableOpacity style={{s.emptyBtn, { borderColor: colors.borderDark }}} onPress={() => setAddSt
15232               <Text style={{s.emptyBtnText, { color: colors.primary }}}>添加大标题</Text>
15233             </TouchableOpacity>
15234           </View>
15235           <TouchableOpacity style={{s.backBtn, { backgroundColor: colors.borderDark }}} onPress={handleGoBack}
15236             <Text style={s.backArrow}>← 返回</Text>
15237           </TouchableOpacity>
15238         </View>
15239       </SafeAreaView>
15240     );
15241   }
15242
15243   // ===== 正常渲染 =====
15244

```

```

15245 return (
15246   <SafeAreaView style={{[s.safeArea, { backgroundColor: colors.background }]}>
15247     <ScrollView contentContainerStyle={s.scrollContent} showsVerticalScrollIndicator={false}>
15248       {/* ---- 顶部导航栏 ---- */}
15249       <View style={{[s.header, { backgroundColor: colors.background }]}>
15250         <TouchableOpacity style={{[s.backBtnRect, { backgroundColor: colors.borderDark }]} onPress={handleGoB
15251           <Text style={s.backArrow}>←</Text>
15252         </TouchableOpacity>
15253         <Text style={{[s.headerTitle, { color: colors.textPrimary }]}>
15254           {skillTree?.title ?? (customDescription ? '自定义模块' : '学习课程')}
15255         </Text>
15256         <View style={s.headerPlaceholder} />
15257       </View>
15258
15259       {/* ---- 描述区域 ---- */}
15260       <View style={s.descriptionSection}>
15261         {/* 学习方法: 仅默认模块展示 */}
15262         {skillTree?.learningMethod ? (
15263           <>
15264             <Text style={{[s.descriptionTitle, { color: colors.primary }]}> 学习方法</Text>
15265             <Text style={{[s.descriptionText, { color: colors.textPrimary }]}>{skillTree?.learningMethod}</Tex
15266           </>
15267         ) : null}
15268
15269         {/* 学习目标: 默认模块展示, 自定义模块展示其介绍 */}
15270         <Text style={{[s.descriptionTitle, { color: colors.primary }]}> 学习目标</Text>
15271         <Text style={{[s.descriptionText, { color: colors.textPrimary }]}>
15272           {skillTree?.learningGoal ?? customDescription ?? '自定义学习模块'}
15273         </Text>
15274
15275         {/* 框架说明: 仅默认模块展示 */}
15276         {skillTree?.frameworkExplanation ? (
15277           <>
15278             <Text style={{[s.descriptionTitle, { color: colors.primary }]}> 框架说明</Text>
15279             <Text style={{[s.descriptionText, { color: colors.textPrimary }]}>
15280               {skillTree?.frameworkExplanation}
15281             </Text>
15282           </>
15283         ) : null}
15284       </View>
15285
15286       {/* ---- 总时长徽章 ---- */}
15287       <View style={{[s.totalDurationBadge, { backgroundColor: colors.borderLight }]}>
15288         <Icons name="time" size={14} color={colors.primary} />
15289         <Text style={{[s.totalDurationText, { color: colors.primary }]}>总时长 {totalDuration}小时</Text>
15290       </View>
15291
15292       {/* ---- 技能树主体 ---- */}
15293       <View style={s.treeContainer}>
15294         {mergedStages.map((stage, stageIndex) => {

```

```

15295     const stageColor = getStageColor(stage.id);
15296     // 找到对应的自定义大标题对象（用于编辑回调）
15297     const customStageObj = customStages.find(cs => cs.id === stage.id);
15298     return (
15299         <View key={stage.id} style={[s.stageColumn, stageIndex === 1 && s.middleStage]}>
15300             /* 大标题 */
15301             <StageNode
15302                 name={stage.name}
15303                 duration={stage.duration}
15304                 stageColor={stageColor}
15305                 isCustom={stage.isCustom}
15306                 onLongPress={stage.isCustom ? () => handleDeleteStage(stage.id, stage.name) : undefined}
15307                 onEdit={stage.isCustom ? () => openEditStage(customStageObj!) : undefined}
15308             />
15309
15310             /* 该大标题下的结点列表 */
15311             <View style={s.skillNodesList}>
15312                 {stage.nodes.map((node, nodeIndex) => {
15313                     const customNodeObj = customNodes.find(cn => cn.id === node.id);
15314                     const isCustomNode = !!customNodeObj;
15315                     const displayName = customNodeObj?.name ?? node.name;
15316                     const displayPlatform = customNodeObj?.platform ?? node.platform;
15317                     const displayUrl = customNodeObj?.url ?? node.url;
15318                     const platformInfo = getPlatformIcon(displayPlatform);
15319                     return (
15320                         <SkillNodeItem
15321                             key={node.id}
15322                             name={displayName}
15323                             platform={displayPlatform}
15324                             duration={node.duration}
15325                             url={displayUrl}
15326                             index={nodeIndex}
15327                             stageColor={stageColor}
15328                             platformIcon={platformInfo.icon}
15329                             platformColor={platformInfo.color}
15330                             onSelect={handleNodeSelect}
15331                             isCustom={isCustomNode}
15332                             onLongPress={isCustomNode
15333                                 ? () => handleDeleteNode(node.id, node.name)
15334                                 : () => openEditDefaultNode(node, stage.id)}
15335                             )
15336                             onEdit={isCustomNode ? () => openEditNode(customNodeObj!) : undefined}
15337                         />
15338                     );
15339                 })}
15340
15341             /* 为当前大标题添加结点 */
15342             <TouchableOpacity
15343                 style={[s.addNodeBtn, { borderColor: colors.borderDark }]}
15344                 onPress={() => openAddNodeForStage(stage.id)}

```

```

15345         activeOpacity={0.7}
15346     >
15347         <Text style={([s.addNodeText, { color: colors.primary }])}>添加结点</Text>
15348     </TouchableOpacity>
15349 </View>
15350 </View>
15351 );
15352 }}}
15353 </View>
15354
15355 {/ * ---- 添加大标题按钮 ---- */}
15356 <TouchableOpacity
15357     style={([s.addStageBtn, { borderColor: colors.primary }])}
15358     onPress={() => setAddStageVisible(true)}
15359     activeOpacity={0.7}
15360 >
15361     <Text style={([s.addStageText, { color: colors.primary }])}>添加大标题</Text>
15362 </TouchableOpacity>
15363
15364 <View style={s.bottomPadding} />
15365 </ScrollView>
15366
15367 {/ * ===== 各种模态框 ===== */}
15368
15369 {/ * 学习时长选择 */}
15370 <DurationModal
15371     visible={modalVisible}
15372     onClose={() => setModalVisible(false)}
15373     nodeName={selectedNode.name}
15374     url={selectedNode.url}
15375     suggestedDuration={selectedNode.suggestedDuration}
15376 />
15377
15378 {/ * 添加大标题 */}
15379 <Modal visible={addStageVisible} transparent onRequestClose={() => setAddStageVisible(false)} animationT
15380 <TouchableOpacity style={s.modalOverlay} onPress={() => setAddStageVisible(false)} activeOpacity={1}>
15381 <View style={([s.modalContent, { backgroundColor: colors.surface }])} onStartShouldSetResponder={() =>
15382 <View style={s.modalHeader}>
15383     <Text style={([s.modalTitle, { color: colors.textPrimary }])}>添加大标题</Text>
15384 <TouchableOpacity style={([s.modalClose, { backgroundColor: colors.border }])} onPress={() => setA
15385     <Ionicons name="close" size={20} color={colors.textSecondary} />
15386 </TouchableOpacity>
15387 </View>
15388 <TextInput
15389     style={([s.addInput, { backgroundColor: colors.inputBg, color: colors.textPrimary, borderColor: c
15390     placeholder="输入大标题名称"
15391     placeholderTextColor={colors.textTertiary}
15392     value={newStageName} onChangeText={setNewStageName}
15393     maxLength={20} autoFocus
15394 />

```

```

15395     <TouchableOpacity
15396         style={{[s.modalConfirmBtn, !newStageName.trim() ]&& { opacity: 0.5 }}}
15397         onPress={handleAddStage} disabled={!newStageName.trim()}
15398     >
15399         <Text style={s.modalConfirmText}>确认添加</Text>
15400     </TouchableOpacity>
15401 </View>
15402 </TouchableOpacity>
15403 </Modal>
15404
15405 /* 添加结点 */
15406 <Modal visible={addNodeVisible} transparent onRequestClose={() => setAddNodeVisible(false)} animationTyp
15407     <TouchableOpacity style={s.modalOverlay} onPress={() => setAddNodeVisible(false)} activeOpacity={1}>
15408     <View style={[s.modalContent, { backgroundColor: colors.surface }] } onStartShouldSetResponder={() =>
15409     <View style={s.modalHeader}>
15410         <Text style={[s.modalTitle, { color: colors.textPrimary }]}>添加结点</Text>
15411         <TouchableOpacity style={[s.modalClose, { backgroundColor: colors.border }] } onPress={() => setA
15412             <Ionicons name="close" size={20} color={colors.textSecondary} />
15413         </TouchableOpacity>
15414     </View>
15415     <Text style={[s.addLabel, { color: colors.textSecondary }]}>结点名称</Text>
15416     <TextInput
15417         style={[s.addInput, { backgroundColor: colors.inputBg, color: colors.textPrimary, borderColor: c
15418         placeholder="输入结点名称" placeholderTextColor={colors.textTertiary}
15419         value={newNodeName} onChangeText={setNewNodeName} maxLength={30}
15420     />
15421     <Text style={[s.addLabel, { color: colors.textSecondary }]}>学习链接</Text>
15422     <TextInput
15423         style={[s.addInput, { backgroundColor: colors.inputBg, color: colors.textPrimary, borderColor: c
15424         placeholder="https://..." placeholderTextColor={colors.textTertiary}
15425         value={newNodeUrl} onChangeText={setNewNodeUrl}
15426         autoCapitalize="none" keyboardType="url"
15427     />
15428     <Text style={[s.addLabel, { color: colors.textSecondary }]}>平台</Text>
15429     <View style={s.platformSelector}>
15430         {PLATFORM_OPTIONS.map(opt => (
15431             <TouchableOpacity key={opt.key}
15432                 style={[s.platformOption, { borderColor: colors.border }, newNodePlatform === opt.key && { b
15433                 onPress={() => setNewNodePlatform(opt.key)}
15434             >
15435                 <Ionicons name={opt.icon as any} size={16} color={opt.color} />
15436                 <Text style={[s.platformOptionText, { color: newNodePlatform === opt.key ? opt.color : color
15437                     {opt.label}
15438                 </Text>
15439             </TouchableOpacity>
15440         )]}
15441     </View>
15442     <TouchableOpacity
15443         style={[s.modalConfirmBtn, (!newNodeName.trim() || !newNodeUrl.trim()) && { opacity: 0.5 }]}
15444         onPress={handleAddNode} disabled={!newNodeName.trim() || !newNodeUrl.trim()}

```

```

15445     >
15446     <Text style={s.modalConfirmText}>确认添加</Text>
15447   </TouchableOpacity>
15448 </View>
15449 </TouchableOpacity>
15450 </Modal>
15451
15452 {/* 编辑大标题 */}
15453 <Modal visible={editStageVisible} transparent onRequestClose={() => setEditStageVisible(false)} animatio
15454 <TouchableOpacity style={s.modalOverlay} onPress={() => setEditStageVisible(false)} activeOpacity={1}>
15455 <View style={[s.modalContent, { backgroundColor: colors.surface }]} onStartShouldSetResponder={() =>
15456 <View style={s.modalHeader}>
15457 <Text style={[s.modalTitle, { color: colors.textPrimary }]}>编辑大标题</Text>
15458 <TouchableOpacity style={[s.modalClose, { backgroundColor: colors.border }]} onPress={() => setE
15459 <Icons name="close" size={20} color={colors.textSecondary} />
15460 </TouchableOpacity>
15461 </View>
15462 <TextInput
15463 style={[s.addInput, { backgroundColor: colors.inputBg, color: colors.textPrimary, borderColor: c
15464 placeholder="输入大标题名称" placeholderTextColor={colors.textTertiary}
15465 value={editStageName} onChangeText={setEditStageName}
15466 maxLength={20} autoFocus
15467 />
15468 <TouchableOpacity
15469 style={[s.modalConfirmBtn, !editStageName.trim() && { opacity: 0.5 }]}
15470 onPress={handleSaveEditStage} disabled={!editStageName.trim()}
15471 >
15472 <Text style={s.modalConfirmText}>保存更改</Text>
15473 </TouchableOpacity>
15474 </View>
15475 </TouchableOpacity>
15476 </Modal>
15477
15478 {/* 编辑结点 */}
15479 <Modal visible={editNodeVisible} transparent onRequestClose={() => setEditNodeVisible(false)} animatio
15480 <TouchableOpacity style={s.modalOverlay} onPress={() => setEditNodeVisible(false)} activeOpacity={1}>
15481 <View style={[s.modalContent, { backgroundColor: colors.surface }]} onStartShouldSetResponder={() =>
15482 <View style={s.modalHeader}>
15483 <Text style={[s.modalTitle, { color: colors.textPrimary }]}>编辑结点</Text>
15484 <TouchableOpacity style={[s.modalClose, { backgroundColor: colors.border }]} onPress={() => setE
15485 <Icons name="close" size={20} color={colors.textSecondary} />
15486 </TouchableOpacity>
15487 </View>
15488 <Text style={[s.addLabel, { color: colors.textSecondary }]}>结点名称</Text>
15489 <TextInput
15490 style={[s.addInput, { backgroundColor: colors.inputBg, color: colors.textPrimary, borderColor: c
15491 placeholder="结点名称" placeholderTextColor={colors.textTertiary}
15492 value={editNodeName} onChangeText={setEditNodeName} maxLength={30}
15493 />
15494 <Text style={[s.addLabel, { color: colors.textSecondary }]}>学习链接</Text>

```

```

15495     <TextInput
15496       style={([s.addInput, { backgroundColor: colors.inputBg, color: colors.textPrimary, borderColor: c
15497         placeholder="https://..." placeholderTextColor={colors.textTertiary}
15498         value={editNodeUrl} onChangeText={setEditNodeUrl}
15499         autoCapitalize="none" keyboardType="url"
15500     />
15501     <Text style={([s.addLabel, { color: colors.textSecondary }])}>平台</Text>
15502     <View style={s.platformSelector}>
15503       {PLATFORM_OPTIONS.map(opt => (
15504         <TouchableOpacity key={opt.key}
15505           style={([s.platformOption, { borderColor: colors.border }, editNodePlatform === opt.key && {
15506             onPress={() => setEditNodePlatform(opt.key)}
15507         >
15508           <Ionicons name={opt.icon as any} size={16} color={opt.color} />
15509           <Text style={([s.platformOptionText, { color: editNodePlatform === opt.key ? opt.color : colo
15510             {opt.label}
15511           </Text>
15512         </TouchableOpacity>
15513       )]}
15514     </View>
15515     <TouchableOpacity
15516       style={([s.modalConfirmBtn, (!editNodeName.trim() || !editNodeUrl.trim()) && { opacity: 0.5 }])}
15517       onPress={handleSaveEditNode} disabled={!editNodeName.trim() || !editNodeUrl.trim()}
15518     >
15519       <Text style={s.modalConfirmText}>保存更改</Text>
15520     </TouchableOpacity>
15521   </View>
15522 </TouchableOpacity>
15523 </Modal>
15524
15525   {/ * Toast */}
15526   {toastMessage !== '' && (
15527     <View style={s.toast}>
15528       <Ionicons name="checkmark-circle" size={18} color={colors.success} />
15529       <Text style={s.toastText}>{toastMessage}</Text>
15530     </View>
15531   )}
15532 </SafeAreaView>
15533 );
15534 };
15535
15536
15537
15538 export default SkillTreeScreen;
15539 === mobile/app/splash.tsx ===
15539 import React, { useEffect } from 'react';
15540 import { View, Text, StyleSheet, Animated } from 'react-native';
15541 import { SafeAreaView } from 'react-native-safe-area-context';
15542 import { router } from 'expo-router';
15543 import { useTheme } from '../src/contexts/ThemeContext';

```



```

15544
15545 const SplashScreen = () => {
15546   const { colors } = useTheme();
15547   const fadeAnim = React.useRef(new Animated.Value(0)).current;
15548   const translateYAnim = React.useRef(new Animated.Value(20)).current;
15549   const scaleAnim = React.useRef(new Animated.Value(0.8)).current;
15550   const dotsAnim = React.useRef([
15551     new Animated.Value(0.3),
15552     new Animated.Value(0.3),
15553     new Animated.Value(0.3)
15554   ]).current;
15555
15556   useEffect(() => {
15557     console.log('[Splash] 启动页加载, 2秒后跳转至故事页');
15558     Animated.parallel([
15559       Animated.timing(fadeAnim, {
15560         toValue: 1,
15561         duration: 800,
15562         useNativeDriver: true,
15563       }),
15564       Animated.timing(translateYAnim, {
15565         toValue: 0,
15566         duration: 800,
15567         useNativeDriver: true,
15568       }),
15569       Animated.spring(scaleAnim, {
15570         toValue: 1,
15571         tension: 100,
15572         friction: 8,
15573         useNativeDriver: true,
15574       }),
15575     ]).start();
15576
15577     dotsAnim.forEach((anim, i) => {
15578       Animated.loop(
15579         Animated.sequence([
15580           Animated.timing(anim, {
15581             toValue: 1,
15582             duration: 1000,
15583             delay: i * 200,
15584             useNativeDriver: true,
15585           }),
15586           Animated.timing(anim, {
15587             toValue: 0.3,
15588             duration: 1000,
15589             useNativeDriver: true,
15590           }),
15591         ])
15592       ).start();
15593     });

```

```

15594
15595     const timer = setTimeout(() => {
15596         router.replace('/login');
15597     }, 2000);
15598
15599     return () => clearTimeout(timer);
15600 }, []);
15601
15602 const styles = useMemo(() => StyleSheet.create({
15603     container: {
15604         flex: 1,
15605         backgroundColor: colors.background,
15606     },
15607     content: {
15608         flex: 1,
15609         alignItems: 'center',
15610         justifyContent: 'center',
15611         position: 'relative',
15612         paddingHorizontal: 20,
15613     },
15614     starsContainer: {
15615         position: 'absolute',
15616         top: 0,
15617         left: 0,
15618         right: 0,
15619         bottom: 0,
15620     },
15621     star: {
15622         position: 'absolute',
15623         width: 4,
15624         height: 4,
15625         backgroundColor: '#FFE66D',
15626     },
15627     planetContainer: {
15628         marginBottom: 48,
15629     },
15630     planet: {
15631         width: 120,
15632         height: 120,
15633     },
15634     planetRing: {
15635         position: 'absolute',
15636         width: 160,
15637         height: 40,
15638         borderRadius: 80,
15639         borderWidth: 3,
15640         borderColor: colors.decorCircle,
15641         opacity: 0.4,
15642         top: 42,
15643         left: -20,

```

```

15644     transform: [{ rotateX: '60deg' }],
15645   },
15646   planetBody: {
15647     position: 'relative',
15648     width: 72,
15649     height: 72,
15650     top: 24,
15651     left: 24,
15652   },
15653   planetLayer: {
15654     position: 'absolute',
15655     backgroundColor: colors.primary,
15656   },
15657   continent: {
15658     position: 'absolute',
15659     backgroundColor: colors.success,
15660   },
15661   titleContainer: {
15662     alignItems: 'center',
15663   },
15664   title: {
15665     color: colors.textPrimary,
15666     fontSize: 32,
15667     fontWeight: '600',
15668     letterSpacing: -0.5,
15669
15670     marginBottom: 8,
15671   },
15672   subtitle: {
15673     color: colors.primary,
15674     fontSize: 16,
15675
15676     letterSpacing: 2,
15677   },
15678   dotsContainer: {
15679     position: 'absolute',
15680     bottom: 64,
15681     flexDirection: 'row',
15682     gap: 8,
15683   },
15684   dot: {
15685     width: 8,
15686     height: 8,
15687     backgroundColor: colors.primary,
15688   },
15689   ), [colors]);
15690
15691   return (
15692     <SafeAreaView style={styles.container} edges={['top', 'bottom']}>
15693       <View style={styles.content}>

```

```

15694 <View style={styles.starsContainer}>
15695   {[...Array(20)].map((_, i) => (
15696     <Animated.View
15697       key={i}
15698       style={[
15699         styles.star,
15700         {
15701           left: `${Math.random() * 100}%`,
15702           top: `${Math.random() * 100}%`,
15703         },
15704       ]}
15705     />
15706   )]}
15707 </View>
15708
15709 <Animated.View
15710   style={[
15711     styles.planetContainer,
15712     {
15713       transform: [{ scale: scaleAnim }],
15714     },
15715   ]}
15716 >
15717   <PixelPlanet />
15718 </Animated.View>
15719
15720 <Animated.View
15721   style={[
15722     styles.titleContainer,
15723     {
15724       opacity: fadeAnim,
15725       transform: [{ translateY: translateYAnim }],
15726     },
15727   ]}
15728 >
15729   <Text style={styles.title}>学了么? </Text>
15730   <Text style={styles.subtitle}>Learnflow Monster</Text>
15731 </Animated.View>
15732
15733 <View style={styles.dotsContainer}>
15734   {dotsAnim.map((anim, i) => (
15735     <Animated.View
15736       key={i}
15737       style={[
15738         styles.dot,
15739         {
15740           opacity: anim,
15741           transform: [
15742             {
15743               scale: anim.interpolate({

```

```

15744         inputRange: [0.3, 1],
15745         outputRange: [1, 1.5],
15746     }},
15747 },
15748 ],
15749 },
15750 ]}
15751 />
15752 )))
15753 </View>
15754 </View>
15755 </SafeAreaView>
15756 );
15757 };
15758
15759 const PixelPlanet = () => (
15760   <View style={styles.planet}>
15761     /* 星球环 一 超过星球本体 */
15762     <View style={styles.planetRing} />
15763     <View style={styles.planetBody}>
15764       <View style={[styles.planetLayer, { top: 0, left: 8, width: 56, height: 8 }]} />
15765       <View style={[styles.planetLayer, { top: 8, left: 0, width: 72, height: 56 }]} />
15766       <View style={[styles.planetLayer, { top: 64, left: 8, width: 56, height: 8 }]} />
15767
15768       <View style={[styles.continent, { top: 16, left: 16, width: 16, height: 8 }]} />
15769       <View style={[styles.continent, { top: 24, left: 12, width: 20, height: 12 }]} />
15770       <View style={[styles.continent, { top: 28, left: 44, width: 12, height: 12 }]} />
15771       <View style={[styles.continent, { top: 40, left: 40, width: 20, height: 8 }]} />
15772     </View>
15773   </View>
15774 );
15775
15776
15777
15778 export default SplashScreen;
15779 === mobile/app/story.tsx ===
15779 import React, { useState, useCallback, useMemo } from 'react';
15780 import { View, Text, StyleSheet, TouchableOpacity, Animated } from 'react-native';
15781 import { SafeAreaView } from 'react-native-safe-area-context';
15782 import { router } from 'expo-router';
15783 import { Ionicons } from '@expo/vector-icons';
15784 import MonsterIcon from '../src/components/MonsterIcon';
15785
15786 import { useTheme } from '../src/contexts/ThemeContext';
15787
15788 const ROUTES = {
15789   MONSTER_SELECTION: '/monster-selection',
15790 };
15791
15792 const STORY_FRAMES = [

```

```

15793 {
15794   title: '欢迎来到元气星',
15795   text: '你可以认领小怪兽陪伴你的学习',
15796   subtext: '在这里，他们生存的重要资源是Π能量和体力',
15797 },
15798 {
15799   title: '一键生成学习路径',
15800   text: '解锁学习节点，跳转学习资源，完成冒险',
15801   subtext: '完成不同时长的学习，会获得不同数量的 Π 能量',
15802 },
15803 {
15804   title: '体力与成长',
15805   text: '每次跳转学习会消耗体力，请珍惜',
15806   subtext: '通过趣味游戏可以恢复体力，继续前进',
15807 },
15808 ];
15809
15810 const useStoryAnimation = (totalFrames: number) => {
15811   const [currentFrame, setCurrentFrame] = useState(0);
15812   const slideAnimation = useState(new Animated.Value(0))[0];
15813
15814   const switchFrame = useCallback((nextFrame: number) => {
15815     Animated.sequence([
15816       Animated.timing(slideAnimation, {
15817         toValue: 1,
15818         duration: 300,
15819         useNativeDriver: true,
15820       }),
15821       Animated.timing(slideAnimation, {
15822         toValue: 0,
15823         duration: 0,
15824         useNativeDriver: true,
15825       }),
15826     ]).start(() => {
15827       setCurrentFrame(nextFrame);
15828     });
15829   }, [slideAnimation]);
15830
15831   const goToNextFrame = useCallback(() => {
15832     if (currentFrame < totalFrames - 1) {
15833       console.log('[Story] 切换到第 ${currentFrame + 2} 帧');
15834       switchFrame(currentFrame + 1);
15835     } else {
15836       console.log('[Story] 故事结束，跳转至怪物选择');
15837       router.replace(ROUTES.MONSTER_SELECTION);
15838     }
15839   }, [currentFrame, totalFrames, switchFrame]);
15840
15841   const skipToSelection = useCallback(() => {
15842     console.log('[Story] 跳过故事，跳转至怪物选择');

```

```

15843     router.replace(ROUTES.MONSTER_SELECTION);
15844   }, []);
15845
15846   return {
15847     currentFrame,
15848     slideAnimation,
15849     goToNextFrame,
15850     skipToSelection,
15851     switchFrame,
15852   };
15853 };
15854
15855 const StoryScreen = () => {
15856   const { colors } = useTheme();
15857   const { currentFrame, slideAnimation, goToNextFrame, skipToSelection, switchFrame } = useStoryAnimation(STOR
15858   const currentStory = STORY_FRAMES[currentFrame];
15859   const isLastFrame = currentFrame === STORY_FRAMES.length - 1;
15860
15861   const animatedStyle = {
15862     transform: [
15863       {
15864         translateX: slideAnimation.interpolate({
15865           inputRange: [0, 1],
15866           outputRange: [0, -50],
15867         }),
15868       },
15869     ],
15870     opacity: slideAnimation.interpolate({
15871       inputRange: [0, 1],
15872       outputRange: [1, 0],
15873     }),
15874   };
15875
15876   const styles = useMemo(() => StyleSheet.create({
15877     container: {
15878       flex: 1,
15879       backgroundColor: colors.background,
15880     },
15881     content: {
15882       flex: 1,
15883       paddingHorizontal: 24,
15884       paddingTop: 48,
15885       paddingBottom: 32,
15886       position: 'relative',
15887     },
15888     pixelBackground: {
15889       position: 'absolute',
15890       top: 0,
15891       left: 0,
15892       right: 0,

```

```

15893     bottom: 0,
15894     opacity: 0.1,
15895     backgroundColor: 'transparent',
15896   },
15897   skipButton: {
15898     position: 'absolute',
15899     top: 48,
15900     right: 24,
15901     flexDirection: 'row',
15902     alignItems: 'center',
15903     gap: 4,
15904     paddingHorizontal: 16,
15905     paddingVertical: 8,
15906     backgroundColor: colors.surface,
15907     borderRadius: 8,
15908     borderWidth: 1,
15909     borderColor: colors.border,
15910     zIndex: 10,
15911   },
15912   skipText: {
15913     color: colors.textSecondary,
15914     fontSize: 14,
15915   },
15916   backButton: {
15917     position: 'absolute',
15918     top: 48,
15919     left: 24,
15920     width: 40,
15921     height: 40,
15922     borderRadius: 20,
15923     backgroundColor: colors.surface,
15924     alignItems: 'center',
15925     justifyContent: 'center',
15926     borderWidth: 1,
15927     borderColor: colors.border,
15928     zIndex: 10,
15929   },
15930   storyContainer: {
15931     flex: 1,
15932     alignItems: 'center',
15933     justifyContent: 'center',
15934     paddingBottom: 96,
15935   },
15936   storyContent: {
15937     alignItems: 'center',
15938     width: '100%',
15939   },
15940   illustrationContainer: {
15941     marginBottom: 48,
15942   },

```



```

15943 illustration: {
15944   width: 180,
15945   height: 180,
15946   position: 'relative',
15947 },
15948 title: {
15949   color: colors.textPrimary,
15950   fontSize: 22,
15951   fontWeight: '600',
15952   lineHeight: 30,
15953   textAlign: 'center',
15954   marginBottom: 12,
15955 },
15956 text: {
15957   color: colors.textPrimary,
15958   fontSize: 16,
15959   lineHeight: 26,
15960   textAlign: 'center',
15961   marginBottom: 8,
15962 },
15963 subtext: {
15964   color: '#7AB0A0',
15965   fontSize: 14,
15966   lineHeight: 22,
15967   textAlign: 'center',
15968 },
15969 progressContainer: {
15970   position: 'absolute',
15971   bottom: 128,
15972   left: 0,
15973   right: 0,
15974   flexDirection: 'row',
15975   justifyContent: 'center',
15976   gap: 8,
15977 },
15978 progressDot: {
15979   width: 8,
15980   height: 8,
15981 },
15982 nextButton: {
15983   position: 'absolute',
15984   bottom: 32,
15985   left: 24,
15986   right: 24,
15987   flexDirection: 'row',
15988   alignItems: 'center',
15989   justifyContent: 'center',
15990   gap: 8,
15991   paddingVertical: 16,
15992   backgroundColor: '#D4A574',

```

```

15993     borderRadius: 2,
15994     overflow: 'hidden',
15995     shadowColor: '#D4A574',
15996     shadowOffset: { width: 0, height: 4 },
15997     shadowOpacity: 0.3,
15998     shadowRadius: 12,
15999     elevation: 5,
16000 },
16001 nextButtonText: {
16002     color: 'FFFFFF',
16003     fontSize: 16,
16004     fontWeight: '600',
16005 },
16006 // ---- 欢迎页: 三怪兽 + 像素泥土 ----
16007 welcomeScene: {
16008     width: 300,
16009     height: 240,
16010     alignItems: 'center',
16011     justifyContent: 'flex-end',
16012     position: 'relative',
16013     paddingBottom: 8,
16014 },
16015 welcomeBgCircle: {
16016     position: 'absolute',
16017     width: 280,
16018     height: 280,
16019     borderRadius: 140,
16020     backgroundColor: colors.decorCircle,
16021     opacity: 0.12,
16022     top: -30,
16023 },
16024 welcomeBgCircle2: {
16025     position: 'absolute',
16026     width: 100,
16027     height: 100,
16028     borderRadius: 50,
16029     backgroundColor: colors.decorCircle,
16030     opacity: 0.06,
16031     bottom: 40,
16032     right: 5,
16033 },
16034 monsterStage: {
16035     alignItems: 'center',
16036     zIndex: 2,
16037 },
16038 // 像素泥土
16039 pixelGround: {
16040     width: 260,
16041     marginTop: 4,
16042 },

```

```

16043   soilRow: {
16044     flexDirection: 'row',
16045     gap: 3,
16046     marginBottom: 3,
16047   },
16048   soilPixel: {
16049     width: 14,
16050     height: 10,
16051     borderRadius: 2,
16052   },
16053   // 怪兽排列 — 在泥土上方
16054   monsterRow: {
16055     flexDirection: 'row',
16056     alignItems: 'flex-end',
16057     justifyContent: 'center',
16058     gap: 8,
16059     marginBottom: 2,
16060   },
16061   monsterSlot: {
16062     alignItems: 'center',
16063     justifyContent: 'center',
16064   },
16065   welcomeSubText: {
16066     fontSize: 13,
16067     color: colors.textTertiary,
16068     marginTop: 10,
16069     zIndex: 2,
16070   },
16071   mindMapContainer: {
16072     width: 200,
16073     height: 200,
16074     position: 'relative',
16075     alignItems: 'center',
16076     justifyContent: 'center',
16077   },
16078   mindMapCenter: {
16079     width: 60,
16080     height: 60,
16081     borderRadius: 30,
16082     backgroundColor: '#D4A574',
16083     alignItems: 'center',
16084     justifyContent: 'center',
16085     zIndex: 10,
16086     shadowColor: '#D4A574',
16087     shadowOffset: { width: 0, height: 4 },
16088     shadowOpacity: 0.5,
16089     shadowRadius: 12,
16090   },
16091   mindMapCenterText: {
16092     fontSize: 10,

```

```

16093     fontWeight: '600',
16094     color: '#FFFFFF',
16095     textAlign: 'center',
16096     lineHeight: 14,
16097 },
16098 mindMapConnections: {
16099     position: 'absolute',
16100     top: 0,
16101     left: 0,
16102     right: 0,
16103     bottom: 0,
16104 },
16105 connectionLineTop: {
16106     position: 'absolute',
16107     top: 20,
16108     left: '50%',
16109     width: 3,
16110     height: 40,
16111     backgroundColor: 'rgba(122,176,160,0.20)',
16112     transform: [{ translateX: -1.5 }],
16113 },
16114 connectionLineBottom: {
16115     position: 'absolute',
16116     bottom: 20,
16117     left: '50%',
16118     width: 3,
16119     height: 40,
16120     backgroundColor: 'rgba(122,176,160,0.20)',
16121     transform: [{ translateX: -1.5 }],
16122 },
16123 connectionLineLeft: {
16124     position: 'absolute',
16125     left: 20,
16126     top: '50%',
16127     width: 40,
16128     height: 3,
16129     backgroundColor: 'rgba(122,176,160,0.20)',
16130     transform: [{ translateY: -1.5 }],
16131 },
16132 connectionLineRight: {
16133     position: 'absolute',
16134     right: 20,
16135     top: '50%',
16136     width: 40,
16137     height: 3,
16138     backgroundColor: 'rgba(122,176,160,0.20)',
16139     transform: [{ translateY: -1.5 }],
16140 },
16141 mindMapNodeTop: {
16142     position: 'absolute',

```

```

16143     top: 0,
16144     left: '50%',
16145     transform: [{ translateX: -28 }],
16146     alignItems: 'center',
16147   },
16148   mindMapNodeBottom: {
16149     position: 'absolute',
16150     bottom: 0,
16151     left: '50%',
16152     transform: [{ translateX: -28 }],
16153     alignItems: 'center',
16154   },
16155   mindMapNodeLeft: {
16156     position: 'absolute',
16157     left: 0,
16158     top: '50%',
16159     transform: [{ translateY: -28 }],
16160     alignItems: 'center',
16161   },
16162   mindMapNodeRight: {
16163     position: 'absolute',
16164     right: 0,
16165     top: '50%',
16166     transform: [{ translateY: -28 }],
16167     alignItems: 'center',
16168   },
16169   mindMapNodeEmoji: {
16170     fontSize: 28,
16171     marginBottom: 4,
16172   },
16173   mindMapNodeLabel: {
16174     fontSize: 10,
16175     color: colors.textSecondary,
16176     fontWeight: '500',
16177   },
16178   staminaContainer: {
16179     width: '100%',
16180     alignItems: 'center',
16181   },
16182   staminaLabel: {
16183     fontSize: 14,
16184     fontWeight: '600',
16185     color: colors.textPrimary,
16186     marginBottom: 8,
16187   },
16188   staminaBarBg: {
16189     width: '80%',
16190     height: 24,
16191     backgroundColor: colors.border,
16192     borderRadius: 12,

```

```

16193     overflow: 'hidden',
16194   },
16195   staminaBarFill: {
16196     height: '100%',
16197     backgroundColor: '#D4A574',
16198     borderRadius: 12,
16199     justifyContent: 'center',
16200     alignItems: 'center',
16201   },
16202   staminaValue: {
16203     fontSize: 12,
16204     fontWeight: '600',
16205     color: 'FFFFFF',
16206   },
16207   gameButtonNew: {
16208     flexDirection: 'row',
16209     alignItems: 'center',
16210     justifyContent: 'center',
16211     backgroundColor: '#D4A574',
16212     borderRadius: 2,
16213     paddingVertical: 12,
16214     paddingHorizontal: 16,
16215     marginTop: 20,
16216   },
16217   gameButtonIcon: {
16218     fontSize: 20,
16219     marginRight: 8,
16220   },
16221   gameButtonText: {
16222     fontSize: 14,
16223     fontWeight: '600',
16224     color: 'FFFFFF',
16225   },
16226   }}, [colors]);
16227
16228   const WelcomeFrame = () => (
16229     <View style={styles.welcomeScene}>
16230       {/* 装饰背景 */}
16231       <View style={styles.welcomeBgCircle} />
16232       <View style={styles.welcomeBgCircle2} />
16233
16234       <View style={styles.monsterStage}>
16235         {/* 怪兽们 — 站在泥土上方 */}
16236         <View style={styles.monsterRow}>
16237           {/* 活力小怪 */}
16238           <View style={styles.monsterSlot}>
16239             <MonsterIcon type="calm" size={68} />
16240           </View>
16241           {/* 叛逆小怪 */}
16242           <View style={styles.monsterSlot}>

```

```

16243     <MonsterIcon type="rebel" size={64} />
16244   </View>
16245 </View>
16246
16247  {/* 像素泥土地面 — 怪兽脚下 */}
16248  <View style={styles.pixelGround}>
16249    <View style={[styles.soilRow, { justifyContent: 'flex-start', paddingLeft: 8 }]}>
16250      [...Array(14)].map((_, i) => (
16251        <View key={`t-${i}`} style={[styles.soilPixel, { backgroundColor: i % 2 === 0 ? colors.primary :
16252          )]}
16253      </View>
16254    <View style={styles.soilRow}>
16255      [...Array(16)].map((_, i) => (
16256        <View key={`m-${i}`} style={[styles.soilPixel, { backgroundColor: i % 3 === 0 ? colors.primary :
16257          )]}
16258      </View>
16259    <View style={[styles.soilRow, { justifyContent: 'flex-end', paddingRight: 6 }]}>
16260      [...Array(13)].map((_, i) => (
16261        <View key={`b-${i}`} style={[styles.soilPixel, { backgroundColor: i % 2 === 0 ? colors.primary :
16262          )]}
16263      </View>
16264    </View>
16265  </View>
16266
16267  {/* 底部文字 */}
16268  <Text style={styles.welcomeSubText}>三只小伙伴等你一起学习</Text>
16269 </View>
16270 );
16271
16272 const MindMapFrame = () => (
16273   <View style={styles.mindMapContainer}>
16274     <View style={styles.mindMapCenter}>
16275       <Text style={styles.mindMapCenterText}>知识节点</Text>
16276     </View>
16277
16278     <View style={styles.mindMapConnections}>
16279       <View style={styles.connectionLineTop} />
16280       <View style={styles.connectionLineBottom} />
16281       <View style={styles.connectionLineLeft} />
16282       <View style={styles.connectionLineRight} />
16283     </View>
16284
16285     <View style={styles.mindMapNodeTop}>
16286       <Text style={styles.mindMapNodeEmoji}></Text>
16287       <Text style={styles.mindMapNodeLabel}>学习资源</Text>
16288     </View>
16289     <View style={styles.mindMapNodeBottom}>
16290       <Text style={styles.mindMapNodeEmoji}></Text>
16291       <Text style={styles.mindMapNodeLabel}>学习目标</Text>
16292     </View>

```

```

16293     <View style={styles.mindMapNodeLeft}>
16294       <Text style={styles.mindMapNodeEmoji}> </Text>
16295       <Text style={styles.mindMapNodeLabel}>番茄钟</Text>
16296     </View>
16297     <View style={styles.mindMapNodeRight}>
16298       <Text style={styles.mindMapNodeEmoji}> </Text>
16299       <Text style={styles.mindMapNodeLabel}>成就奖励</Text>
16300     </View>
16301   </View>
16302 );
16303
16304 const StaminaFrame = () => (
16305   <View style={styles.illustration}>
16306     <View style={styles.staminaContainer}>
16307       <Text style={styles.staminaLabel}>体力值</Text>
16308       <View style={styles.staminaBarBg}>
16309         <View style={styles.staminaBarFill}>
16310           <Text style={styles.staminaValue}>100 / 100</Text>
16311         </View>
16312       </View>
16313     </View>
16314     <View style={styles.gameButtonNew}>
16315       <Text style={styles.gameButtonIcon}> </Text>
16316       <Text style={styles.gameButtonText}>玩小游戏恢复体力</Text>
16317     </View>
16318   </View>
16319 );
16320
16321 const StoryIllustration = ({ frameIndex }: { frameIndex: number }) => {
16322   const illustrations = [
16323     <WelcomeFrame key={0} />,
16324     <MindMapFrame key={1} />,
16325     <StaminaFrame key={2} />,
16326   ];
16327   return illustrations[frameIndex];
16328 };
16329
16330 return (
16331   <SafeAreaView style={styles.container} edges={['top', 'bottom']}>
16332     <View style={styles.content}>
16333       <View style={styles.pixelBackground} />
16334
16335       <TouchableOpacity style={styles.skipButton} onPress={skipToSelection}>
16336         <Text style={styles.skipText}>跳过</Text>
16337         <Icons name="chevron-forward" size={16} color={colors.textSecondary} />
16338       </TouchableOpacity>
16339
16340       {/* 第2页起显示返回按钮 */}
16341       {currentFrame > 0 && (
16342         <TouchableOpacity

```



```

16343     style={styles.backButton}
16344     onPress={() => switchFrame(currentFrame - 1)}
16345     hitSlop={{ top: 10, bottom: 10, left: 10, right: 10 }}
16346   >
16347     <Ionicons name="chevron-back" size={22} color={colors.textPrimary} />
16348   </TouchableOpacity>
16349 }}
16350
16351 <View style={styles.storyContainer}>
16352   <Animated.View
16353     style={[styles.storyContent, animatedStyle]}
16354   >
16355     <View style={styles.illustrationContainer}>
16356       <StoryIllustration frameIndex={currentFrame} />
16357     </View>
16358
16359     <Text style={styles.title}>{currentStory.title}</Text>
16360     <Text style={styles.text}>{currentStory.text}</Text>
16361     <Text style={styles.subtext}>{currentStory.subtext}</Text>
16362   </Animated.View>
16363 </View>
16364
16365 <View style={styles.progressContainer}>
16366   {STORY_FRAMES.map((_, index) => (
16367     <View
16368       key={index}
16369       style={[
16370         styles.progressDot,
16371       {
16372         backgroundColor: index === currentFrame ? '#D4A574' : colors.border,
16373         transform: [{ scale: index === currentFrame ? 1.5 : 1 }],
16374       },
16375     ])}
16376   />
16377   )]}
16378 </View>
16379
16380 <TouchableOpacity style={styles.nextButton} onPress={goToNextFrame}>
16381   <Text style={styles.nextButtonText}>
16382     {isLastFrame ? '开始冒险' : '继续'}
16383   </Text>
16384   <Ionicons name="chevron-forward" size={20} color="#FFFFFF" />
16385 </TouchableOpacity>
16386 </View>
16387 </SafeAreaView>
16388 );
16389 };
16390
16391 export default StoryScreen;
16392

```

=== mobile/app/verify-email.tsx ===

```
16393 import React, { useState, useRef, useEffect, useMemo } from 'react';
16394 import {
16395   View, Text, StyleSheet, TextInput, TouchableOpacity, Alert,
16396   KeyboardAvoidingView, Platform,
16397 } from 'react-native';
16398 import { SafeAreaView } from 'react-native-safe-area-context';
16399 import { router, useLocalSearchParams } from 'expo-router';
16400 import { Ionicons } from '@expo/vector-icons';
16401 import { useTheme } from '../src/contexts/ThemeContext';
16402 import { authService } from '../src/services/api';
16403 import { saveAuthData } from '../src/utlis/auth';
16404 import AsyncStorage from '@react-native-async-storage/async-storage';
16405 import { STORAGE_KEYS } from '../src/utlis/storage';
16406
16407 const CODE_LENGTH = 6;
16408
16409 const VerifyEmailScreen = () => {
16410   const { colors } = useTheme();
16411   const { email, username } = useLocalSearchParams<{ email: string; username: string }>();
16412
16413   const [code, setCode] = useState<string[]>(Array(CODE_LENGTH).fill(""));
16414   const [loading, setLoading] = useState(false);
16415   const [resending, setResending] = useState(false);
16416   const [countdown, setCountdown] = useState(0);
16417
16418   const inputRefs = useRef<(TextInput | null)[]>([]);
16419
16420   const styles = useMemo(() => StyleSheet.create({
16421     container: { flex: 1, backgroundColor: colors.background },
16422     scrollContent: { flexGrow: 1, justifyContent: 'center', paddingHorizontal: 24 },
16423     backBtn: {
16424       position: 'absolute' as const, top: 60, left: 20,
16425       width: 40, height: 40, borderRadius: 12,
16426       backgroundColor: colors.surface, alignItems: 'center', justifyContent: 'center',
16427       borderWidth: 1, borderColor: colors.hairline,
16428       zIndex: 10,
16429     },
16430     title: { color: colors.textPrimary, fontSize: 24, fontWeight: '600', textAlign: 'center' },
16431     subtitle: { color: colors.textSecondary, fontSize: 14, textAlign: 'center', marginTop: 8, lineHeight: 20 },
16432     emailHighlight: { color: colors.primary, fontWeight: '600' },
16433     codeRow: { flexDirection: 'row', justifyContent: 'center', gap: 10, marginTop: 32 },
16434     codeBox: {
16435       width: 48, height: 56, borderRadius: 12,
16436       backgroundColor: colors.surface, borderWidth: 1.5,
16437       borderColor: colors.borderLight, alignItems: 'center', justifyContent: 'center',
16438     },
16439     codeBoxActive: { borderColor: colors.primary, backgroundColor: colors.primary + '10' },
16440     codeText: { color: colors.textPrimary, fontSize: 24, fontWeight: '700' },
16441     hiddenInput: { position: 'absolute', opacity: 0, width: 0, height: 0 },
```

```

16442   verifyBtn: {
16443     backgroundColor: colors.primary, borderRadius: 14, paddingVertical: 16,
16444     alignItems: 'center', marginTop: 32, opacity: 1,
16445   },
16446   verifyBtnDisabled: { opacity: 0.4 },
16447   verifyBtnText: { color: '#FFFFFF', fontSize: 16, fontWeight: '600' },
16448   resendRow: { flexDirection: 'row', justifyContent: 'center', marginTop: 20 },
16449   resendText: { color: colors.textSecondary, fontSize: 13 },
16450   resendLink: { color: colors.primary, fontSize: 13, fontWeight: '600' },
16451   resendDisabled: { color: colors.textTertiary },
16452   ), [colors]);
16453
16454   // 倒计时
16455   useEffect(() => {
16456     if (countdown <= 0) return;
16457     const timer = setInterval(() => setCountdown(c => c - 1), 1000);
16458     return () => clearInterval(timer);
16459   }, [countdown]);
16460
16461   const handleCodeChange = (text: string, index: number) => {
16462     const digits = text.replace(/[^\d]/g, '');
16463     if (digits.length > 1) {
16464       // 粘贴多位数
16465       const newCode = [...code];
16466       const chars = digits.split("");
16467       for (let i = 0; i < CODE_LENGTH && i < chars.length; i++) {
16468         newCode[i] = chars[i];
16469       }
16470       setCode(newCode);
16471       const nextIndex = Math.min(chars.length, CODE_LENGTH - 1);
16472       inputRefs.current[nextIndex]?.focus();
16473       return;
16474     }
16475     const newCode = [...code];
16476     newCode[index] = digits;
16477     setCode(newCode);
16478
16479     // 自动跳转下一个输入框
16480     if (digits && index < CODE_LENGTH - 1) {
16481       inputRefs.current[index + 1]?.focus();
16482     }
16483   };
16484
16485   const handleKeyPress = (key: string, index: number) => {
16486     if (key === 'Backspace' && !code[index] && index > 0) {
16487       inputRefs.current[index - 1]?.focus();
16488     }
16489   };
16490
16491   const handleVerify = async () => {

```

```

16492   const fullCode = code.join("");
16493   if (fullCode.length !== CODE_LENGTH) {
16494     Alert.alert('提示', '请输入完整的6位验证码');
16495     return;
16496   }
16497
16498   setLoading(true);
16499   try {
16500     const authResponse = await authService.verifyEmail(email, fullCode);
16501     await saveAuthData(authResponse);
16502     await AsyncStorage.setItem(STORAGE_KEYS.IS_NEW_USER, 'true');
16503
16504     setLoading(false);
16505     Alert.alert('验证成功', '欢迎加入 LearnFlow! ', [
16506       { text: '开始冒险', onPress: () => router.replace('/onboarding') },
16507     ]);
16508   } catch (error: any) {
16509     setLoading(false);
16510     const msg = error?.message || '验证失败';
16511     if (msg.includes('次数过多')) {
16512       Alert.alert('验证失败', '错误次数过多, 请重新发送验证码');
16513     } else if (msg.includes('已过期')) {
16514       Alert.alert('验证失败', '验证码错误, 请再次验证或重新发送');
16515     } else {
16516       Alert.alert('验证失败', msg);
16517     }
16518   }
16519   };
16520
16521   const handleResend = async () => {
16522     if (countdown > 0) return;
16523     setResending(true);
16524     try {
16525       await authService.resendVerification(email);
16526       setCountdown(60);
16527       Alert.alert('已发送', '验证码已重新发送至邮箱');
16528     } catch (error: any) {
16529       Alert.alert('发送失败', error?.message || '请稍后再试');
16530     } finally {
16531       setResending(false);
16532     }
16533   };
16534
16535   const isComplete = code.every(c => c !== '');
16536
16537   return (
16538     <SafeAreaView style={styles.container} edges={['top']}>
16539     <KeyboardAvoidingView
16540       behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
16541       style={{ flex: 1 }}

```

```

16542 >
16543 <TouchableOpacity style={styles.backBtn} onPress={() => router.back()}>
16544   <Ionicons name="arrow-back" size={20} color={colors.textPrimary} />
16545 </TouchableOpacity>
16546
16547 <View style={styles.scrollContent}>
16548   <Text style={styles.title}>验证邮箱</Text>
16549   <Text style={styles.subtitle}>
16550     验证码已发送至{' '}
16551     <Text style={styles.emailHighlight}>{email}</Text>
16552   </Text>
16553
16554   {/* 6位验证码输入 */}
16555   <View style={styles.codeRow}>
16556     {code.map((digit, i) => (
16557       <TouchableOpacity
16558         key={i}
16559         activeOpacity={1}
16560         onPress={() => inputRefs.current[i]?.focus()}
16561       >
16562         <View style={[styles.codeBox, digit !== '' && styles.codeBoxActive]}>
16563           <Text style={styles.codeText}>{digit}</Text>
16564         </View>
16565         <TextInput
16566           ref={ref => { inputRefs.current[i] = ref; }}
16567           style={styles.hiddenInput}
16568           value={digit}
16569           onChangeText={text => handleCodeChange(text, i)}
16570           onKeyPress={({ nativeEvent }) => handleKeyPress(nativeEvent.key, i)}
16571           keyboardType="number-pad"
16572           maxLength={1}
16573           selectTextOnFocus
16574           autoFocus={i === 0}
16575         />
16576       </TouchableOpacity>
16577     ))}
16578   </View>
16579
16580   {/* 验证按钮 */}
16581   <TouchableOpacity
16582     style={[styles.verifyBtn, (!isComplete || loading) && styles.verifyBtnDisabled]}
16583     onPress={handleVerify}
16584     disabled={!isComplete || loading}
16585     activeOpacity={0.8}
16586   >
16587     <Text style={styles.verifyBtnText}>
16588       {loading ? '验证中...' : '验证'}
16589     </Text>
16590   </TouchableOpacity>
16591

```

```

16592      {/* 重发验证码 */}
16593      <View style={styles.resendRow}>
16594        <Text style={styles.resendText}>
16595          {countdown > 0 ? `${countdown} 秒后可重新发送` : '没收到验证码? '}
16596        </Text>
16597        <TouchableOpacity
16598          onPress={handleResend}
16599          disabled={countdown > 0 || resending}
16600          activeOpacity={0.7}
16601        >
16602          <Text style={[styles.resendLink, (countdown > 0 || resending) && styles.resendDisabled]}>
16603            {resending ? '发送中...' : '重新发送'}
16604          </Text>
16605        </TouchableOpacity>
16606      </View>
16607    </View>
16608  </KeyboardAvoidingView>
16609 </SafeAreaView>
16610 );
16611 };
16612
16613 export default VerifyEmailScreen;
16614
16615 === server/src/app.ts ===
16616 import express from 'express';
16617 import cors from 'cors';
16618 import { skillRoutes, authRoutes, monsterRoutes, noteRoutes, rewardRoutes, domainRoutes, aiRoutes, proRoutes }
16619 import dotenv from 'dotenv';
16620
16621 dotenv.config();
16622
16623 const app = express();
16624
16625 const PORT = parseInt(process.env.PORT || '3001', 10);
16626
16627 // 请求日志中间件
16628 app.use((req, res, next) => {
16629   const timestamp = new Date().toISOString();
16630
16631   // 获取客户端真实IP (考虑代理情况)
16632   const clientIP = req.headers['x-forwarded-for'] ||
16633     req.headers['x-real-ip'] ||
16634     req.connection.remoteAddress ||
16635     req.socket.remoteAddress ||
16636     req.ip ||
16637     'unknown';
16638
16639   // 获取用户代理信息
16640   const userAgent = req.headers['user-agent'] || 'unknown';
16641
16642   // 判断是否为移动设备

```

```

16641 const isMobile = /Mobile|Android|iPhone|iPad|iPod/i.test(userAgent);
16642 const deviceType = isMobile ? '移动设备' : '桌面设备';
16643
16644 // 记录详细连接信息
16645 console.log(`[${timestamp}] 新连接 - ${deviceType}`);
16646 console.log(` 来源IP: ${clientIP}`);
16647 console.log(` 请求: ${req.method} ${req.url}`);
16648 console.log(` 用户代理: ${userAgent.substring(0, 100)}${userAgent.length > 100 ? '...' : ''}`);
16649 console.log(` 请求头:`, {
16650   host: req.headers.host,
16651   origin: req.headers.origin,
16652   referer: req.headers.referer
16653 });
16654
16655 // 记录请求体 (敏感信息如密码会过滤)
16656 if (req.body && Object.keys(req.body).length > 0) {
16657   const logBody = { ...req.body };
16658   if (logBody.password) logBody.password = '****';
16659   if (logBody.confirmPassword) logBody.confirmPassword = '****';
16660   console.log(` 请求体:`, JSON.stringify(logBody));
16661 }
16662
16663 // 记录响应时间
16664 const start = Date.now();
16665 res.on('finish', () => {
16666   const duration = Date.now() - start;
16667   console.log(` 响应: ${res.statusCode} - ${duration}ms`);
16668   console.log(` ---`);
16669 });
16670
16671 next();
16672 });
16673
16674 function isAllowedCorsOrigin(origin: string | undefined): boolean {
16675   if (!origin) return true;
16676   try {
16677     const { hostname, protocol } = new URL(origin);
16678     if (protocol !== 'http:' && protocol !== 'https:') return false;
16679     if (hostname === 'localhost' || hostname === '127.0.0.1') return true;
16680     if (
16681       hostname.endsWith('.ngrok-free.app') ||
16682       hostname.endsWith('.ngrok-free.dev') ||
16683       hostname.endsWith('.ngrok.app') ||
16684       hostname.endsWith('.ngrok.io')
16685     ) {
16686       return true;
16687     }
16688     const extras =
16689       process.env.CORS_ORIGINS?.split(',').map(s => s.trim()).filter(Boolean) ?? [];
16690     return extras.includes(origin);

```

```

16691 } catch {
16692     return false;
16693 }
16694 }
16695
16696 app.use(
16697     cors({
16698         origin: (origin, callback) => {
16699             callback(null, isAllowedCorsOrigin(origin));
16700         },
16701         credentials: true,
16702         methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
16703         allowedHeaders: ['Content-Type', 'Authorization'],
16704     })
16705 );
16706 app.use(express.json());
16707
16708 // API路由
16709 app.use('/api/skills', skillRoutes);
16710 app.use('/api/auth', authRoutes);
16711 app.use('/api/monster', monsterRoutes);
16712 app.use('/api/notes', noteRoutes);
16713 app.use('/api/rewards', rewardRoutes);
16714 app.use('/api/domains', domainRoutes);
16715 app.use('/api/ai', aiRoutes);
16716 app.use('/api/pro', proRoutes);
16717
16718 // 健康检查端点
16719 app.get('/', (req, res) => {
16720     res.json({
16721         message: 'LearnFlow Server is running!',
16722         timestamp: new Date().toISOString(),
16723         version: '1.0.0'
16724     });
16725 });
16726
16727 // 404处理
16728 app.use('*', (req, res) => {
16729     console.log(`[${new Date().toISOString()}] 404 Not Found: ${req.method} ${req.originalUrl}`);
16730     res.status(404).json({
16731         error: 'Endpoint not found',
16732         path: req.originalUrl
16733     });
16734 });
16735
16736 // 错误处理中间件
16737 app.use((error: any, req: any, res: any, next: any) => {
16738     const timestamp = new Date().toISOString();
16739     console.error(`[${timestamp}] Error:`, error);
16740     res.status(500).json({

```



```

16741     error: 'Internal server error',
16742     message: process.env.NODE_ENV === 'development' ? error.message : 'Something went wrong'
16743   });
16744 });
16745
16746 app.listen(PORT, '0.0.0.0', () => {
16747   console.log(` LearnFlow Server started at ${new Date().toISOString()}`);
16748   console.log(` Server is running on port ${PORT}`);
16749   console.log(` Health check: http://localhost:${PORT}/`);
16750   console.log(` External access: http://0.0.0.0:${PORT}/`);
16751   console.log(` Environment: ${process.env.NODE_ENV || 'development'}`);
16752 });
16753
16754 === server/src/config/database.ts ===
16753 import mysql from 'mysql2/promise';
16754 import dotenv from 'dotenv';
16755
16756 dotenv.config();
16757
16758 export interface DatabaseConfig {
16759   host: string;
16760   port: number;
16761   user: string;
16762   password: string;
16763   database: string;
16764 }
16765
16766 export const dbConfig: DatabaseConfig = {
16767   host: process.env.DB_HOST || '127.0.0.1',
16768   port: parseInt(process.env.DB_PORT || '3306'),
16769   user: process.env.DB_USER || 'root',
16770   password: process.env.DB_PASSWORD || '',
16771   database: process.env.DB_NAME || 'learnflow'
16772 };
16773
16774 const poolConfig = {
16775   ...dbConfig,
16776   connectionLimit: 10,
16777   enableKeepAlive: true,
16778   keepAliveInitialDelay: 10000,
16779 };
16780
16781 export const pool = mysql.createPool(poolConfig);
16782
16783 export class DatabaseConnection {
16784   private static connection: mysql.Connection | null = null;
16785
16786   static async getConnection(): Promise<mysql.Connection> {
16787     if (this.connection) {
16788       try {
16789         await this.connection.ping();

```

```

16790     return this.connection;
16791   } catch {
16792     this.connection = null;
16793   }
16794 }
16795
16796 try {
16797   this.connection = await mysql.createConnection(poolConfig);
16798   console.log('MySQL数据库连接成功');
16799 } catch (error) {
16800   console.error('MySQL数据库连接失败:', error);
16801   console.error('连接配置:', {
16802     host: dbConfig.host,
16803     port: dbConfig.port,
16804     user: dbConfig.user,
16805     database: dbConfig.database
16806   });
16807   const err = error as { code?: string };
16808   if (err.code === 'ER_ACCESS_DENIED_ERROR') {
16809     console.error(
16810       '提示: 用户名或密码不对。请修改 server/.env 中的 DB_USER、DB_PASSWORD, 使其与在本机执行 mysql -u <用户> -p 能登录的凭据一致; 若 root 无密码则清
16811     );
16812   }
16813   throw new Error(`数据库连接失败: ${error instanceof Error ? error.message : '未知错误'}`);
16814 }
16815 return this.connection;
16816 }
16817
16818 static async closeConnection(): Promise<void> {
16819   if (this.connection) {
16820     await this.connection.end();
16821     this.connection = null;
16822     console.log('MySQL数据库连接已关闭');
16823   }
16824 }
16825
16826 static async testConnection(): Promise<boolean> {
16827   try {
16828     const connection = await this.getConnection();
16829     await connection.execute('SELECT 1');
16830     return true;
16831   } catch (error) {
16832     console.error('数据库连接测试失败:', error);
16833     return false;
16834   }
16835 }
16836 }

```

=== server/src/controllers/aiController.ts ===

```

16837 import { Request, Response } from 'express';
16838 import { fillModule } from '../services/aiFillService';

```

```

16839
16840 /**
16841  * POST /api/ai/fill-module
16842  * AI 一键填充自定义模块内容
16843  */
16844 export const fillModuleContent = async (req: Request, res: Response) => {
16845     const startTime = Date.now();
16846
16847     try {
16848         const { moduleName, moduleDescription } = req.body;
16849
16850         // 校验模块名称
16851         if (!moduleName || typeof moduleName !== 'string' || !moduleName.trim()) {
16852             return res.status(400).json({
16853                 code: 400,
16854                 message: '模块名称不能为空',
16855                 data: null,
16856             });
16857         }
16858
16859         const trimmedName = moduleName.trim();
16860         const trimmedDesc = typeof moduleDescription === 'string' ? moduleDescription.trim() : '';
16861         console.log('[AIController] 收到 AI 填充请求 - 模块: ${trimmedName}');
16862
16863         const result = await fillModule(trimmedName, trimmedDesc);
16864
16865         const duration = Date.now() - startTime;
16866         console.log('[AIController] AI 填充成功 - 模块: ${trimmedName}, 耗时: ${duration}ms');
16867
16868         return res.json({
16869             code: 200,
16870             message: 'success',
16871             data: result,
16872         });
16873     } catch (error: any) {
16874         const duration = Date.now() - startTime;
16875         const errorMessage = error instanceof Error ? error.message : '未知错误';
16876         console.error('[AIController] AI 填充失败 - 耗时: ${duration}ms, 错误: ${errorMessage}');
16877
16878         return res.status(500).json({
16879             code: 500,
16880             message: errorMessage || 'AI 生成失败, 请重试',
16881             data: null,
16882         });
16883     }
16884 };
16885
16886 === server/src/controllers/authController.ts ===
16886 import { Request, Response } from 'express';
16887 import { AuthService } from '../services/authService';

```

```

16888 import { LoginRequest, CreateUserRequest } from '../models';
16889
16890 export class AuthController {
16891   // 用户注册
16892   static async register(req: Request, res: Response) {
16893     const startTime = Date.now();
16894     const { username, email } = req.body;
16895
16896     try {
16897       console.log('[AuthController] 开始处理用户注册 - 用户名: ${username}, 邮箱: ${email}');
16898
16899       const registerRequest: CreateUserRequest = req.body;
16900
16901       // 验证必填字段
16902       if (!registerRequest.username || !registerRequest.email || !registerRequest.password) {
16903         console.log('[AuthController] 注册验证失败 - 缺少必填字段');
16904         return res.status(400).json({ error: '用户名、邮箱和密码不能为空' });
16905       }
16906
16907       if (registerRequest.password.length < 6) {
16908         console.log('[AuthController] 注册验证失败 - 密码长度不足');
16909         return res.status(400).json({ error: '密码长度至少6位' });
16910       }
16911
16912       console.log('[AuthController] 调用注册服务...');
16913       // 调用注册服务（邮箱验证模式，返回提示信息）
16914       const result = await AuthService.registerUser(registerRequest);
16915
16916       const duration = Date.now() - startTime;
16917       console.log('[AuthController] 用户注册成功 - 用户名: ${username}, 耗时: ${duration}ms');
16918
16919       res.json({
16920         success: true,
16921         data: result
16922       });
16923     } catch (error) {
16924       const duration = Date.now() - startTime;
16925       console.error('[AuthController] 注册失败 - 用户名: ${username}, 错误: ${error}, 耗时: ${duration}ms');
16926       res.status(400).json({
16927         success: false,
16928         error: error instanceof Error ? error.message : '注册失败'
16929       });
16930     }
16931   }
16932
16933   // 验证邮箱
16934   static async verifyEmail(req: Request, res: Response) {
16935     try {
16936       const { email, token } = req.body;
16937

```

```

16938     if (!email || !token) {
16939         return res.status(400).json({ error: '邮箱和验证码不能为空' });
16940     }
16941
16942     console.log('[AuthController] 开始验证邮箱 - 邮箱: ${email}');
16943     const authResponse = await AuthService.verifyEmail(email, token);
16944
16945     console.log('[AuthController] 邮箱验证成功 - 用户ID: ${authResponse.user.id}');
16946     res.json({
16947         success: true,
16948         data: authResponse
16949     });
16950 } catch (error) {
16951     console.error('[AuthController] 邮箱验证失败 - 错误: ${error}');
16952     res.status(400).json({
16953         success: false,
16954         error: error instanceof Error ? error.message : '验证失败'
16955     });
16956 }
16957 }
16958
16959 // 重新发送验证码
16960 static async resendVerification(req: Request, res: Response) {
16961     try {
16962         const { email } = req.body;
16963
16964         if (!email) {
16965             return res.status(400).json({ error: '邮箱不能为空' });
16966         }
16967
16968         console.log('[AuthController] 重新发送验证码 - 邮箱: ${email}');
16969         const result = await AuthService.resendVerificationCode(email);
16970
16971         res.json({
16972             success: true,
16973             data: result
16974         });
16975     } catch (error) {
16976         console.error('[AuthController] 重发验证码失败 - 错误: ${error}');
16977         res.status(400).json({
16978             success: false,
16979             error: error instanceof Error ? error.message : '重发失败'
16980         });
16981     }
16982 }
16983
16984 // 用户登录
16985 static async login(req: Request, res: Response) {
16986     const startTime = Date.now();
16987     try {

```

```

16988     const loginRequest: LoginRequest = req.body;
16989     console.log(`[AuthController] 开始处理登录 - 类型: ${loginRequest.type}, 邮箱: ${loginRequest.email || 'N/A'}`);
16990
16991     if (!loginRequest.type) {
16992         console.log(`[AuthController] 登录验证失败 - 缺少登录类型`);
16993         return res.status(400).json({ error: '登录类型不能为空' });
16994     }
16995
16996     let authResponse;
16997
16998     if (loginRequest.type === 'email') {
16999         authResponse = await AuthService.emailLogin(loginRequest);
17000     } else {
17001         console.log(`[AuthController] 登录验证失败 - 不支持的登录类型: ${loginRequest.type}`);
17002         return res.status(400).json({ error: '不支持的登录类型' });
17003     }
17004
17005     const duration = Date.now() - startTime;
17006     console.log(`[AuthController] 登录成功 - 邮箱: ${loginRequest.email || 'N/A'}, 耗时: ${duration}ms`);
17007     res.json({
17008         success: true,
17009         data: authResponse
17010     });
17011 } catch (error) {
17012     const duration = Date.now() - startTime;
17013     console.error(`[AuthController] 登录失败 - 错误: ${error}, 耗时: ${duration}ms`);
17014     res.status(400).json({
17015         success: false,
17016         error: error instanceof Error ? error.message : '登录失败'
17017     });
17018 }
17019 }
17020
17021 // 刷新访问令牌
17022 static async refreshToken(req: Request, res: Response) {
17023     try {
17024         const { refreshToken, deviceId } = req.body;
17025
17026         if (!refreshToken || !deviceId) {
17027             return res.status(400).json({ error: '刷新令牌和设备ID不能为空' });
17028         }
17029
17030         const authResponse = await AuthService.refreshToken(refreshToken, deviceId);
17031
17032         res.json({
17033             success: true,
17034             data: authResponse
17035         });
17036     } catch (error) {
17037         console.error('刷新令牌失败:', error);

```

```

17038     res.status(400).json({
17039         success: false,
17040         error: error instanceof Error ? error.message : '刷新令牌失败'
17041     });
17042 }
17043 }
17044
17045 // 验证令牌
17046 static async verifyToken(req: Request, res: Response) {
17047     try {
17048         const { token } = req.body;
17049
17050         if (!token) {
17051             return res.status(400).json({ error: '令牌不能为空' });
17052         }
17053
17054         const decoded = await AuthService.verifyToken(token);
17055
17056         res.json({
17057             success: true,
17058             data: decoded
17059         });
17060     } catch (error) {
17061         console.error('验证令牌失败:', error);
17062         res.status(400).json({
17063             success: false,
17064             error: error instanceof Error ? error.message : '令牌无效'
17065         });
17066     }
17067 }
17068
17069 // 用户登出
17070 static async logout(req: Request, res: Response) {
17071     try {
17072         // 这里可以添加清理设备会话的逻辑
17073         // 在实际应用中，可能需要将令牌加入黑名单
17074
17075         res.json({
17076             success: true,
17077             message: '登出成功'
17078         });
17079     } catch (error) {
17080         console.error('登出失败:', error);
17081         res.status(500).json({
17082             success: false,
17083             error: '登出失败'
17084         });
17085     }
17086 }
17087 }

```

```

=== server/src/controllers/domainController.ts ===
17088 import { Request, Response } from 'express';
17089 import {
17090   createDomain as createDomainService,
17091   getDomains as getDomainsService,
17092   getDomainById as getDomainByIdService,
17093   updateNodeProgress as updateNodeProgressService,
17094   startLearning as startLearningService,
17095   finishLearning as finishLearningService
17096 } from '../services';
17097
17098 export const createDomain = async (req: Request, res: Response) => {
17099   console.log('[DomainController] POST /domains - 创建领域');
17100   try {
17101     const { userId, name, type = 'preset' } = req.body;
17102
17103     if (!userId || !name) {
17104       console.log('[DomainController] 参数验证失败 - 缺少必填字段');
17105       return res.status(400).json({
17106         error: 'User ID and name are required'
17107       });
17108     }
17109
17110     const result = await createDomainService(parseInt(userId), name, type);
17111
17112     console.log('[DomainController] 领域创建成功');
17113     res.json({
17114       success: true,
17115       data: result
17116     });
17117   } catch (error) {
17118     console.error('[DomainController] 创建领域失败:', error);
17119     res.status(500).json({
17120       error: 'Failed to create domain',
17121       message: error instanceof Error ? error.message : 'Unknown error'
17122     });
17123   }
17124 };
17125
17126 export const getDomains = async (req: Request, res: Response) => {
17127   console.log('[DomainController] GET /domains/:userId - 获取领域列表');
17128   try {
17129     const { userId } = req.params;
17130
17131     if (!userId) {
17132       return res.status(400).json({ error: 'User ID is required' });
17133     }
17134
17135     const result = await getDomainsService(parseInt(userId));
17136

```



```

17137     res.json({
17138         success: true,
17139         data: result
17140     });
17141 } catch (error) {
17142     console.error('[DomainController] 获取领域列表失败:', error);
17143     res.status(500).json({
17144         error: 'Failed to get domains',
17145         message: error instanceof Error ? error.message : 'Unknown error'
17146     });
17147 }
17148 };
17149
17150 export const getDomainById = async (req: Request, res: Response) => {
17151     console.log('[DomainController] GET /domains/:id - 获取领域详情');
17152     try {
17153         const { id } = req.params;
17154
17155         if (!id) {
17156             return res.status(400).json({ error: 'Domain ID is required' });
17157         }
17158
17159         const domain = await getDomainByIdService(parseInt(id));
17160
17161         if (!domain) {
17162             console.log('[DomainController] 领域不存在 - 领域ID: ${id}');
17163             return res.status(404).json({ error: 'Domain not found' });
17164         }
17165
17166         res.json({
17167             success: true,
17168             data: domain
17169         });
17170     } catch (error) {
17171         console.error('[DomainController] 获取领域详情失败:', error);
17172         res.status(500).json({
17173             error: 'Failed to get domain',
17174             message: error instanceof Error ? error.message : 'Unknown error'
17175         });
17176     }
17177 };
17178
17179 export const updateNodeProgress = async (req: Request, res: Response) => {
17180     console.log('[DomainController] PUT /domains/node-progress - 更新节点进度');
17181     try {
17182         const { userId, domainId, nodeId, status, studyTime, notes } = req.body;
17183
17184         if (!userId || !domainId || !nodeId || !status) {
17185             console.log('[DomainController] 参数验证失败 - 缺少必填字段');
17186             return res.status(400).json({

```

```

17187     error: 'User ID, domain ID, node ID, and status are required'
17188   });
17189 }
17190
17191 await updateNodeProgressService(
17192   parseInt(userId),
17193   parseInt(domainId),
17194   nodeId,
17195   status as 'pending' | 'doing' | 'done',
17196   studyTime,
17197   notes
17198 );
17199
17200 res.json({
17201   success: true
17202 });
17203 } catch (error) {
17204   console.error('Error updating node progress:', error);
17205   res.status(500).json({
17206     error: 'Failed to update node progress',
17207     message: error instanceof Error ? error.message : 'Unknown error'
17208   });
17209 }
17210 };
17211
17212 export const startLearning = async (req: Request, res: Response) => {
17213   try {
17214     const { userId, domainId, nodeId } = req.body;
17215
17216     if (!userId || !domainId || !nodeId) {
17217       return res.status(400).json({
17218         error: 'User ID, domain ID, and node ID are required'
17219       });
17220     }
17221
17222     const result = await startLearningService(
17223       parseInt(userId),
17224       parseInt(domainId),
17225       nodeId
17226     );
17227
17228     if (!result.success) {
17229       return res.status(400).json({ error: result.error });
17230     }
17231
17232     res.json({
17233       success: true,
17234       data: result
17235     });
17236   } catch (error) {

```

```

17237 console.error('Error starting learning:', error);
17238 res.status(500).json({
17239   error: 'Failed to start learning',
17240   message: error instanceof Error ? error.message : 'Unknown error'
17241 });
17242 }
17243 };
17244
17245 export const finishLearning = async (req: Request, res: Response) => {
17246   try {
17247     const { recordId, duration, progressAfter } = req.body;
17248
17249     if (!recordId || duration === undefined || progressAfter === undefined) {
17250       return res.status(400).json({
17251         error: 'Record ID, duration, and progressAfter are required'
17252       });
17253     }
17254
17255     await finishLearningService(recordId, duration, progressAfter);
17256
17257     res.json({
17258       success: true
17259     });
17260   } catch (error) {
17261     console.error('Error finishing learning:', error);
17262     res.status(500).json({
17263       error: 'Failed to finish learning',
17264       message: error instanceof Error ? error.message : 'Unknown error'
17265     });
17266   }
17267 };
17268
17269 export default {
17270   createDomain,
17271   getDomains,
17272   getDomainById,
17273   updateNodeProgress,
17274   startLearning,
17275   finishLearning
17276 };
17277
17278 === server/src/controllers/index.ts ===
17279 export * from './skillController';
17280 export * from './monsterController';
17281 export * from './authController';
17282 export * from './noteController';
17283 export * from './rewardController';
17284 export * from './domainController';
17285
17286 === server/src/controllers/monsterController.ts ===
17287 import { Request, Response } from 'express';

```

```

17285 import {
17286   createMonster as createMonsterService,
17287   getMonsterStatus as getMonsterStatusService,
17288   consumeStamina as consumeStaminaService,
17289   recoverStamina as recoverStaminaService,
17290   consumeEnergy as consumeEnergyService,
17291   consumeEnergyAmount as consumeEnergyAmountService,
17292   recoverEnergy as recoverEnergyService,
17293   gainExp as gainExpService,
17294   chatWithMonster as chatWithMonsterService,
17295   getMonsterMessages as getMonsterMessagesService,
17296 } from '../services';
17297
17298 export const createMonster = async (req: Request, res: Response) => {
17299   console.log('[MonsterController] POST /monsters - 创建怪物');
17300   try {
17301     const { userId, name, style, personality } = req.body;
17302
17303     if (!userId || !name || !personality) {
17304       console.log('[MonsterController] 参数验证失败 - 缺少必填字段');
17305       return res.status(400).json({
17306         error: 'User ID, name, and personality are required'
17307       });
17308     }
17309
17310     if (!['lively', 'calm', 'rebel'].includes(personality)) {
17311       console.log('[MonsterController] 参数验证失败 - 无效性格: ${personality}');
17312       return res.status(400).json({
17313         error: 'Personality must be one of lively, calm, or rebel'
17314       });
17315     }
17316
17317     const result = await createMonsterService(parseInt(userId, 10), {
17318       name,
17319       style,
17320       personality,
17321     });
17322
17323     console.log('[MonsterController] 怪物创建成功');
17324     res.json({
17325       success: true,
17326       data: result
17327     });
17328   } catch (error) {
17329     console.error('[MonsterController] 创建怪物失败:', error);
17330     res.status(500).json({
17331       error: 'Failed to create monster',
17332       message: error instanceof Error ? error.message : 'Unknown error'
17333     });
17334   }

```

```

17335 };
17336
17337 export const getMonsterStatus = async (req: Request, res: Response) => {
17338   console.log('[MonsterController] GET /monsters/:userId - 获取怪物状态');
17339   try {
17340     const { userId } = req.params;
17341
17342     if (!userId) {
17343       return res.status(400).json({ error: 'User ID is required' });
17344     }
17345
17346     const status = await getMonsterStatusService(parseInt(userId));
17347
17348     if (!status) {
17349       console.log('[MonsterController] 怪物不存在 - 用户ID: ${userId}');
17350       return res.status(404).json({ error: 'Monster not found' });
17351     }
17352
17353     res.json({
17354       success: true,
17355       data: status
17356     });
17357   } catch (error) {
17358     console.error('[MonsterController] 获取怪物状态失败:', error);
17359     res.status(500).json({
17360       error: 'Failed to get monster status',
17361       message: error instanceof Error ? error.message : 'Unknown error'
17362     });
17363   }
17364 };
17365
17366 export const consumeStamina = async (req: Request, res: Response) => {
17367   console.log('[MonsterController] POST /monsters/consume-stamina - 消耗体力');
17368   try {
17369     const { userId, amount = 10 } = req.body;
17370
17371     if (!userId) {
17372       return res.status(400).json({ error: 'User ID is required' });
17373     }
17374
17375     const success = await consumeStaminaService(parseInt(userId), amount);
17376
17377     res.json({
17378       success,
17379       error: success ? undefined : 'Not enough stamina'
17380     });
17381   } catch (error) {
17382     console.error('[MonsterController] 消耗体力失败:', error);
17383     res.status(500).json({
17384       error: 'Failed to consume stamina',

```

```

17385     message: error instanceof Error ? error.message : 'Unknown error'
17386   });
17387 }
17388 };
17389
17390 export const recoverStamina = async (req: Request, res: Response) => {
17391   try {
17392     const { userId, amount = 20 } = req.body;
17393
17394     if (!userId) {
17395       return res.status(400).json({ error: 'User ID is required' });
17396     }
17397
17398     await recoverStaminaService(parseInt(userId), amount);
17399
17400     res.json({
17401       success: true
17402     });
17403   } catch (error) {
17404     console.error('Error recovering stamina:', error);
17405     res.status(500).json({
17406       error: 'Failed to recover stamina',
17407       message: error instanceof Error ? error.message : 'Unknown error'
17408     });
17409   }
17410 };
17411
17412 export const consumeEnergy = async (req: Request, res: Response) => {
17413   try {
17414     const { userId } = req.body;
17415
17416     if (!userId) {
17417       return res.status(400).json({ error: 'User ID is required' });
17418     }
17419
17420     const success = await consumeEnergyService(parseInt(userId));
17421
17422     res.json({
17423       success,
17424       error: success ? undefined : 'Not enough energy'
17425     });
17426   } catch (error) {
17427     console.error('Error consuming energy:', error);
17428     res.status(500).json({
17429       error: 'Failed to consume energy',
17430       message: error instanceof Error ? error.message : 'Unknown error'
17431     });
17432   }
17433 };
17434

```

```

17435 export const consumeEnergyAmount = async (req: Request, res: Response) => {
17436   try {
17437     const { userId, amount } = req.body;
17438
17439     if (!userId || amount === undefined) {
17440       return res.status(400).json({ error: 'User ID and amount are required' });
17441     }
17442
17443     const success = await consumeEnergyAmountService(parseInt(userId), amount);
17444
17445     res.json({
17446       success,
17447       error: success ? undefined : 'Not enough energy'
17448     });
17449   } catch (error) {
17450     console.error('Error consuming energy:', error);
17451     res.status(500).json({
17452       error: 'Failed to consume energy',
17453       message: error instanceof Error ? error.message : 'Unknown error'
17454     });
17455   }
17456 };
17457
17458 export const recoverEnergy = async (req: Request, res: Response) => {
17459   try {
17460     const { userId, amount = 1 } = req.body;
17461
17462     if (!userId) {
17463       return res.status(400).json({ error: 'User ID is required' });
17464     }
17465
17466     await recoverEnergyService(parseInt(userId), amount);
17467
17468     res.json({
17469       success: true
17470     });
17471   } catch (error) {
17472     console.error('Error recovering energy:', error);
17473     res.status(500).json({
17474       error: 'Failed to recover energy',
17475       message: error instanceof Error ? error.message : 'Unknown error'
17476     });
17477   }
17478 };
17479
17480 export const addExp = async (req: Request, res: Response) => {
17481   try {
17482     const { userId, exp } = req.body;
17483
17484     if (!userId || exp === undefined) {

```

```

17485     return res.status(400).json({
17486         error: 'User ID and exp are required'
17487     });
17488 }
17489
17490 const result = await gainExpService(parseInt(userId), exp);
17491
17492 res.json({
17493     success: true,
17494     data: result
17495 });
17496 } catch (error) {
17497     console.error('Error adding exp:', error);
17498     res.status(500).json({
17499         error: 'Failed to add exp',
17500         message: error instanceof Error ? error.message : 'Unknown error'
17501     });
17502 }
17503 };
17504
17505 export const chatWithMonster = async (req: Request, res: Response) => {
17506     try {
17507         const { userId, message } = req.body;
17508
17509         if (!userId || !message) {
17510             return res.status(400).json({
17511                 error: 'User ID and message are required'
17512             });
17513         }
17514
17515         const response = await chatWithMonsterService(parseInt(userId), message);
17516
17517         res.json({
17518             success: true,
17519             data: response
17520         });
17521     } catch (error) {
17522         console.error('Error chatting with monster:', error);
17523         res.status(500).json({
17524             error: 'Failed to chat with monster',
17525             message: error instanceof Error ? error.message : 'Unknown error'
17526         });
17527     }
17528 };
17529
17530 export const getMonsterMessages = async (req: Request, res: Response) => {
17531     try {
17532         const { userId } = req.params;
17533
17534         if (!userId) {

```



```

17535     return res.status(400).json({ error: 'User ID is required' });
17536   }
17537
17538   const messages = await getMonsterMessagesService(parseInt(userId));
17539
17540   res.json({
17541     success: true,
17542     data: messages
17543   });
17544 } catch (error) {
17545   console.error('Error getting monster messages:', error);
17546   res.status(500).json({
17547     error: 'Failed to get monster messages',
17548     message: error instanceof Error ? error.message : 'Unknown error'
17549   });
17550 }
17551 };
17552
17553 export default {
17554   createMonster,
17555   getMonsterStatus,
17556   consumeStamina,
17557   recoverStamina,
17558   consumeEnergy,
17559   consumeEnergyAmount,
17560   recoverEnergy,
17561   addExp,
17562   chatWithMonster,
17563   getMonsterMessages
17564 };
17565
17566 === server/src/controllers/noteController.ts ===
17567 import { Request, Response } from 'express';
17568 import {
17569   createNote as createNoteService,
17570   getNotes as getNotesService,
17571   getNoteByDate as getNoteByDateService,
17572   updateNote as updateNoteService
17573 } from '../services';
17574
17575 export const createNote = async (req: Request, res: Response) => {
17576   console.log('[NoteController] POST /notes - 创建笔记');
17577   try {
17578     const { userId, date, content } = req.body;
17579
17580     if (!userId || !date) {
17581       console.log('[NoteController] 参数验证失败 - 缺少必填字段');
17582       return res.status(400).json({
17583         error: 'User ID and date are required'

```

```

17584 }
17585
17586 const result = await createNoteService(
17587   parseInt(userId),
17588   new Date(date),
17589   content
17590 );
17591
17592 console.log('[NoteController] 笔记创建成功');
17593 res.json({
17594   success: true,
17595   data: result
17596 });
17597 } catch (error) {
17598   console.error('[NoteController] 创建笔记失败:', error);
17599   res.status(500).json({
17600     error: 'Failed to create note',
17601     message: error instanceof Error ? error.message : 'Unknown error'
17602   });
17603 }
17604 };
17605
17606 export const getNotes = async (req: Request, res: Response) => {
17607   console.log('[NoteController] GET /notes/:userId - 获取笔记列表');
17608   try {
17609     const { userId } = req.params;
17610
17611     if (!userId) {
17612       return res.status(400).json({ error: 'User ID is required' });
17613     }
17614
17615     const result = await getNotesService(parseInt(userId));
17616
17617     res.json({
17618       success: true,
17619       data: result
17620     });
17621   } catch (error) {
17622     console.error('[NoteController] 获取笔记列表失败:', error);
17623     res.status(500).json({
17624       error: 'Failed to get notes',
17625       message: error instanceof Error ? error.message : 'Unknown error'
17626     });
17627   }
17628 };
17629
17630 export const getNoteByDate = async (req: Request, res: Response) => {
17631   console.log('[NoteController] GET /notes/:userId/:date - 按日期获取笔记');
17632   try {
17633     const { userId, date } = req.params;

```

```

17634
17635   if (!userId || !date) {
17636     return res.status(400).json({
17637       error: 'User ID and date are required'
17638     });
17639   }
17640
17641   const note = await getNoteByDateService(parseInt(userId), new Date(date));
17642
17643   res.json({
17644     success: true,
17645     data: note
17646   });
17647 } catch (error) {
17648   console.error('[NoteController] 按日期获取笔记失败:', error);
17649   res.status(500).json({
17650     error: 'Failed to get note by date',
17651     message: error instanceof Error ? error.message : 'Unknown error'
17652   });
17653 }
17654 };
17655
17656 export const updateNote = async (req: Request, res: Response) => {
17657   console.log('[NoteController] PUT /notes - 更新笔记');
17658   try {
17659     const { noteld, content } = req.body;
17660
17661     if (!noteld) {
17662       return res.status(400).json({ error: 'Note ID is required' });
17663     }
17664
17665     await updateNoteService(parseInt(noteld), content);
17666
17667     res.json({
17668       success: true
17669     });
17670   } catch (error) {
17671     console.error('[NoteController] 更新笔记失败:', error);
17672     res.status(500).json({
17673       error: 'Failed to update note',
17674       message: error instanceof Error ? error.message : 'Unknown error'
17675     });
17676   }
17677 };
17678
17679 export default {
17680   createNote,
17681   getNotes,
17682   getNoteByDate,
17683   updateNote

```

```

17684 };
17685
17686 === server/src/controllers/proController.ts ===
17687 import { Request, Response } from 'express';
17688 import { ProService } from '../services/proService';
17689
17689 export class ProController {
17690   // 生成激活码 (管理接口, 后续可以加权限校验)
17691   static async generateCodes(req: Request, res: Response) {
17692     try {
17693       const { count = 1, planId, createdBy } = req.body;
17694
17695       if (!planId || ![ 'monthly', 'yearly', 'lifetime' ].includes(planId)) {
17696         return res.status(400).json({ error: '请指定有效的套餐类型: monthly / yearly / lifetime' });
17697       }
17698
17699       const codes = await ProService.generateCodes(count, planId, createdBy);
17700       res.json({ success: true, data: { codes, planId, count } });
17701     } catch (error) {
17702       res.status(500).json({ success: false, error: error instanceof Error ? error.message : '生成失败' });
17703     }
17704   }
17705
17706   // 激活 Pro
17707   static async activateCode(req: Request, res: Response) {
17708     try {
17709       const { code, userId } = req.body;
17710
17711       if (!code || !userId) {
17712         return res.status(400).json({ error: '激活码和用户ID不能为空' });
17713       }
17714
17715       const result = await ProService.activateCode(code, userId);
17716       res.json({ success: true, data: result });
17717     } catch (error) {
17718       res.status(400).json({
17719         success: false,
17720         error: error instanceof Error ? error.message : '激活失败',
17721       });
17722     }
17723   }
17724
17725   // 查询 Pro 状态
17726   static async getStatus(req: Request, res: Response) {
17727     try {
17728       const userId = parseInt(req.params.userId);
17729       if (!userId) {
17730         return res.status(400).json({ error: '用户ID无效' });
17731       }
17732

```

```

17733     const status = await ProService.getProStatus(userId);
17734     res.json({ success: true, data: status });
17735   } catch (error) {
17736     res.status(400).json({
17737       success: false,
17738       error: error instanceof Error ? error.message : '查询失败',
17739     });
17740   }
17741 }
17742 }
17743
17744 === server/src/controllers/rewardController.ts ===
17745 import { Request, Response } from 'express';
17746 import {
17747   createReward as createRewardService,
17748   getRewards as getRewardsService,
17749   claimReward as claimRewardService
17750 } from '../services';
17751
17752 export const createReward = async (req: Request, res: Response) => {
17753   console.log('[RewardController] POST /rewards - 创建奖励');
17754   try {
17755     const { userId, type, amount, source } = req.body;
17756
17757     if (!userId || !type || amount === undefined) {
17758       console.log('[RewardController] 参数验证失败 - 缺少必填字段');
17759       return res.status(400).json({
17760         error: 'User ID, type, and amount are required'
17761       });
17762     }
17763
17764     const result = await createRewardService(
17765       parseInt(userId),
17766       type,
17767       amount,
17768       source
17769     );
17770
17771     console.log('[RewardController] 奖励创建成功');
17772     res.json({
17773       success: true,
17774       data: result
17775     });
17776   } catch (error) {
17777     console.error('[RewardController] 创建奖励失败:', error);
17778     res.status(500).json({
17779       error: 'Failed to create reward',
17780       message: error instanceof Error ? error.message : 'Unknown error'
17781     });
17782   }

```

```

17782 };
17783
17784 export const getRewards = async (req: Request, res: Response) => {
17785   console.log('[RewardController] GET /rewards/:userId - 获取奖励列表');
17786   try {
17787     const { userId } = req.params;
17788
17789     if (!userId) {
17790       return res.status(400).json({ error: 'User ID is required' });
17791     }
17792
17793     const result = await getRewardsService(parseInt(userId));
17794
17795     res.json({
17796       success: true,
17797       data: result
17798     });
17799   } catch (error) {
17800     console.error('[RewardController] 获取奖励列表失败:', error);
17801     res.status(500).json({
17802       error: 'Failed to get rewards',
17803       message: error instanceof Error ? error.message : 'Unknown error'
17804     });
17805   }
17806 };
17807
17808 export const claimReward = async (req: Request, res: Response) => {
17809   console.log('[RewardController] POST /rewards/claim - 领取奖励');
17810   try {
17811     const { rewardId } = req.body;
17812
17813     if (!rewardId) {
17814       return res.status(400).json({ error: 'Reward ID is required' });
17815     }
17816
17817     const result = await claimRewardService(parseInt(rewardId));
17818
17819     res.json({
17820       success: result.success
17821     });
17822   } catch (error) {
17823     console.error('[RewardController] 领取奖励失败:', error);
17824     res.status(500).json({
17825       error: 'Failed to claim reward',
17826       message: error instanceof Error ? error.message : 'Unknown error'
17827     });
17828   }
17829 };
17830
17831 export default {

```

```

17832 createReward,
17833 getRewards,
17834 claimReward
17835 };
17836
17837 === server/src/controllers/skillController.ts ===
17837 import { Request, Response } from 'express';
17838 import {
17839   SkillTreeRequest,
17840   SkillTreeListRequest,
17841   SaveSkillTreeRequest
17842 } from '../types';
17843 import {
17844   generateMockSkillTree,
17845   getSkillTreeList as getSkillTreeListService,
17846   getSkillTreeById as getSkillTreeByIdService,
17847   saveUserSkillTree as saveUserSkillTreeService,
17848   getUserProgress as getUserProgressService,
17849   updateUserProgress as updateUserProgressService,
17850   getStatistics as getStatisticsService,
17851   searchPopularDomains as searchPopularDomainsService,
17852   getRecommendedPath as getRecommendedPathService,
17853   getUserLearningReport as getUserLearningReportService
17854 } from '../services';
17855
17856 // 生成技能树
17857 export const generateSkillTree = async (req: Request, res: Response) => {
17858   console.log('[SkillController] POST /skill-tree/generate - 生成技能树');
17859   try {
17860     const { domain, level = 'beginner' }: SkillTreeRequest = req.body;
17861
17862     if (!domain) {
17863       console.log('[SkillController] 参数验证失败 - 缺少领域');
17864       return res.status(400).json({ error: 'Domain is required' });
17865     }
17866
17867     console.log('[SkillController] 生成技能树 - 领域: ${domain}, 等级: ${level}');
17868     const skillTree = await generateMockSkillTree(domain, level);
17869
17870     console.log('[SkillController] 技能树生成成功');
17871     res.json({
17872       success: true,
17873       data: skillTree
17874     });
17875   } catch (error) {
17876     console.error('[SkillController] 生成技能树失败:', error);
17877     res.status(500).json({
17878       error: 'Failed to generate skill tree',
17879       message: error instanceof Error ? error.message : 'Unknown error'
17880     });

```

```

17881 }
17882 };
17883
17884 // 获取技能树列表
17885 export const getSkillTreeList = async (req: Request, res: Response) => {
17886   console.log('[SkillController] GET /skill-tree/list - 获取技能树列表');
17887   try {
17888     const { page, limit, search, category }: SkillTreeListRequest = req.query;
17889
17890     const result = await getSkillTreeListService({
17891       page: page ? parseInt(page as unknown as string) : 1,
17892       limit: limit ? parseInt(limit as unknown as string) : 10,
17893       search: search as string,
17894       category: category as string
17895     });
17896
17897     res.json({
17898       success: true,
17899       data: result
17900     });
17901   } catch (error) {
17902     console.error('[SkillController] 获取技能树列表失败:', error);
17903     res.status(500).json({
17904       error: 'Failed to get skill tree list',
17905       message: error instanceof Error ? error.message : 'Unknown error'
17906     });
17907   }
17908 };
17909
17910 // 根据ID获取技能树
17911 export const getSkillTreeById = async (req: Request, res: Response) => {
17912   try {
17913     const { id } = req.params;
17914
17915     if (!id) {
17916       return res.status(400).json({ error: 'Skill tree ID is required' });
17917     }
17918
17919     const skillTree = await getSkillTreeByIdService(id);
17920
17921     if (!skillTree) {
17922       return res.status(404).json({ error: 'Skill tree not found' });
17923     }
17924
17925     res.json({
17926       success: true,
17927       data: skillTree
17928     });
17929   } catch (error) {
17930     console.error('Error getting skill tree:', error);

```



```

17931     res.status(500).json({
17932         error: 'Failed to get skill tree',
17933         message: error instanceof Error ? error.message : 'Unknown error'
17934     });
17935 }
17936 };
17937
17938 // 保存用户技能树
17939 export const saveUserSkillTree = async (req: Request, res: Response) => {
17940     try {
17941         const request: SaveSkillTreeRequest = req.body;
17942
17943         if (!request.userId || !request.skillTree) {
17944             return res.status(400).json({
17945                 error: 'User ID and skill tree are required'
17946             });
17947         }
17948
17949         const result = await saveUserSkillTreeService(request);
17950
17951         res.json({
17952             success: true,
17953             data: result
17954         });
17955     } catch (error) {
17956         console.error('Error saving skill tree:', error);
17957         res.status(500).json({
17958             error: 'Failed to save skill tree',
17959             message: error instanceof Error ? error.message : 'Unknown error'
17960         });
17961     }
17962 };
17963
17964 // 获取用户进度
17965 export const getUserProgress = async (req: Request, res: Response) => {
17966     try {
17967         const { userId } = req.params;
17968         const { skillTreeId } = req.query;
17969
17970         if (!userId) {
17971             return res.status(400).json({ error: 'User ID is required' });
17972         }
17973
17974         const progress = await getUserProgressService(userId, skillTreeId as string);
17975
17976         res.json({
17977             success: true,
17978             data: progress
17979         });
17980     } catch (error) {

```

```

17981 console.error('Error getting user progress:', error);
17982 res.status(500).json({
17983   error: 'Failed to get user progress',
17984   message: error instanceof Error ? error.message : 'Unknown error'
17985 });
17986 }
17987 };
17988
17989 // 更新用户进度
17990 export const updateUserProgress = async (req: Request, res: Response) => {
17991   try {
17992     const { userId } = req.params;
17993     const { skillTreeld, completedNodes = [], completedLinks = [] } = req.body;
17994
17995     if (!userId || !skillTreeld) {
17996       return res.status(400).json({
17997         error: 'User ID and skill tree ID are required'
17998       });
17999     }
18000
18001     const progress = await updateUserProgressService(
18002       userId,
18003       skillTreeld,
18004       completedNodes,
18005       completedLinks
18006     );
18007
18008     res.json({
18009       success: true,
18010       data: progress
18011     });
18012   } catch (error) {
18013     console.error('Error updating user progress:', error);
18014     res.status(500).json({
18015       error: 'Failed to update user progress',
18016       message: error instanceof Error ? error.message : 'Unknown error'
18017     });
18018   }
18019 };
18020
18021 // 获取统计信息
18022 export const getStatistics = async (req: Request, res: Response) => {
18023   try {
18024     const stats = await getStatisticsService();
18025
18026     res.json({
18027       success: true,
18028       data: stats
18029     });
18030   } catch (error) {

```

```

18031 console.error('Error getting statistics:', error);
18032 res.status(500).json({
18033   error: 'Failed to get statistics',
18034   message: error instanceof Error ? error.message : 'Unknown error'
18035 });
18036 }
18037 };
18038
18039 // 搜索热门领域
18040 export const searchPopularDomains = async (req: Request, res: Response) => {
18041   try {
18042     const { keyword } = req.query;
18043
18044     if (!keyword) {
18045       return res.status(400).json({ error: 'Search keyword is required' });
18046     }
18047
18048     const result = await searchPopularDomainsService(keyword as string);
18049
18050     res.json({
18051       success: true,
18052       data: result
18053     });
18054   } catch (error) {
18055     console.error('Error searching domains:', error);
18056     res.status(500).json({
18057       error: 'Failed to search domains',
18058       message: error instanceof Error ? error.message : 'Unknown error'
18059     });
18060   }
18061 };
18062
18063 // 获取推荐学习路径
18064 export const getRecommendedPath = async (req: Request, res: Response) => {
18065   try {
18066     const { domain, currentLevel = 'beginner', targetLevel = 'advanced' } = req.query;
18067
18068     if (!domain) {
18069       return res.status(400).json({ error: 'Domain is required' });
18070     }
18071
18072     const result = await getRecommendedPathService(
18073       domain as string,
18074       currentLevel as string,
18075       targetLevel as string
18076     );
18077
18078     res.json({
18079       success: true,
18080       data: result

```

```

18081 });
18082 } catch (error) {
18083   console.error('Error getting recommended path:', error);
18084   res.status(500).json({
18085     error: 'Failed to get recommended path',
18086     message: error instanceof Error ? error.message : 'Unknown error'
18087   });
18088 }
18089 };
18090
18091 // 获取用户学习报告
18092 export const getUserLearningReport = async (req: Request, res: Response) => {
18093   try {
18094     const { userId } = req.params;
18095     const { period = 'week' } = req.query;
18096
18097     if (!userId) {
18098       return res.status(400).json({ error: 'User ID is required' });
18099     }
18100
18101     const result = await getUserLearningReportService(userId, period as string);
18102
18103     res.json({
18104       success: true,
18105       data: result
18106     });
18107   } catch (error) {
18108     console.error('Error getting user report:', error);
18109     res.status(500).json({
18110       error: 'Failed to get user report',
18111       message: error instanceof Error ? error.message : 'Unknown error'
18112     });
18113   }
18114 };
18115
18116 // 添加默认导出以解决TypeScript模块解析问题
18117 export default {
18118   generateSkillTree,
18119   getSkillTreeList,
18120   getSkillTreeById,
18121   saveUserSkillTree,
18122   getUserProgress,
18123   updateUserProgress,
18124   getStatistics,
18125   searchPopularDomains,
18126   getRecommendedPath,
18127   getUserLearningReport
18128 };
18129
18130 === server/src/models/DeviceSession.ts ===
18129 export interface DeviceSession {

```

```

18130   id: number;
18131   userId: number;
18132   deviceId: string;
18133   deviceType: 'ios' | 'android' | 'web';
18134   deviceName?: string;
18135   lastActiveAt: Date;
18136   expiresAt: Date;
18137   createdAt: Date;
18138   updatedAt: Date;
18139 }
18140
18141 export interface CreateDeviceSessionRequest {
18142   userId: number;
18143   deviceId: string;
18144   deviceType: 'ios' | 'android' | 'web';
18145   deviceName?: string;
18146   expiresAt: Date;
18147 }
18148
18149 export interface UpdateDeviceSessionRequest {
18150   lastActiveAt?: Date;
18151   expiresAt?: Date;
18152 }
18153
18154 === server/src/models/Domain.ts ===
18155
18156 import { pool } from '../config/database';
18157 import { SkillNode } from '../types/skill';
18158
18159 export interface Domain {
18160   id: number;
18161   userId: number;
18162   name: string;
18163   type: 'custom' | 'preset';
18164   mindMapData: SkillNode;
18165   progress: number;
18166   createdAt: Date;
18167   updatedAt: Date;
18168 }
18169
18170 export const createDomain = async (domainData: {
18171   userId: number;
18172   name: string;
18173   type: 'custom' | 'preset';
18174   mindMapData: SkillNode;
18175 }): Promise<number> => {
18176   const [result] = await pool.execute(
18177     'INSERT INTO domains (user_id, name, type, mind_map_data) VALUES (?, ?, ?, ?)',
18178     [domainData.userId, domainData.name, domainData.type, JSON.stringify(domainData.mindMapData)]
18179   );
18180   return (result as any).insertId;
18181 };

```

```

18179
18180 export const getDomainsByUserId = async (userId: number): Promise<Domain[]> => {
18181   const [rows] = await pool.execute(
18182     'SELECT * FROM domains WHERE user_id = ? ORDER BY created_at DESC',
18183     [userId]
18184   );
18185   const domains = rows as any[];
18186   return domains.map(d => ({
18187     ...d,
18188     mindMapData: typeof d.mind_map_data === 'string' ? JSON.parse(d.mind_map_data) : d.mind_map_data
18189   }));
18190   });
18191
18192 export const getDomainById = async (id: number): Promise<Domain | null> => {
18193   const [rows] = await pool.execute(
18194     'SELECT * FROM domains WHERE id = ?',
18195     [id]
18196   );
18197   const domains = rows as any[];
18198   if (domains.length === 0) return null;
18199   const d = domains[0];
18200   return {
18201     ...d,
18202     mindMapData: typeof d.mind_map_data === 'string' ? JSON.parse(d.mind_map_data) : d.mind_map_data
18203   };
18204   });
18205
18206 export const updateDomainProgress = async (id: number, progress: number): Promise<void> => {
18207   await pool.execute(
18208     'UPDATE domains SET progress = ?, updated_at = CURRENT_TIMESTAMP WHERE id = ?',
18209     [progress, id]
18210   );
18211   });
18212
18213   === server/src/models/LearningRecord.ts ===
18213 export interface LearningRecord {
18214   id: number;
18215   userId: number;
18216   skillTreelId: number;
18217   nodeId: string;
18218   nodeName: string;
18219   completed: boolean;
18220   timeSpent: number;
18221   notes?: string;
18222   completedAt?: Date;
18223   createdAt: Date;
18224   updatedAt: Date;
18225   }
18226
18227 export interface CreateLearningRecordRequest {

```

```

18228   userId: number;
18229   skillTreeld: number;
18230   nodeId: string;
18231   nodeName: string;
18232   timeSpent: number;
18233   notes?: string;
18234 }
18235
18236 export interface UpdateLearningRecordRequest {
18237   completed?: boolean;
18238   timeSpent?: number;
18239   notes?: string;
18240   completedAt?: Date;
18241 }
18242
18243 export interface UserProgress {
18244   userId: number;
18245   skillTreeld: number;
18246   totalNodes: number;
18247   completedNodes: number;
18248   totalTimeSpent: number;
18249   overallProgress: number;
18250   lastUpdated: Date;
18251 }
18252
18253 === server/src/models/Monster.ts ===
18254 import { pool } from '../config/database';
18255
18256 export interface Monster {
18257   id: number;
18258   userId: number;
18259   name: string;
18260   style: string;
18261   personality: 'lively' | 'calm' | 'rebel';
18262   level: number;
18263   exp: number;
18264   stamina: number;
18265   maxStamina: number;
18266   energy: number;
18267   maxEnergy: number;
18268   personalityParams?: string | Record<string, number>;
18269   lastEnergyRecover: Date;
18270   lastStaminaRecover: Date;
18271   createdAt: Date;
18272   updatedAt: Date;
18273 }
18274
18275 export const createMonster = async (monsterData: {
18276   userId: number;
18277   name: string;
18278   style: string;

```

```

18277   personality: 'lively' | 'calm' | 'rebel';
18278   stamina: number;
18279   maxStamina: number;
18280   energy: number;
18281   maxEnergy: number;
18282   personalityParams?: Record<string, number>;
18283 }): Promise<number> => {
18284   const [result] = await pool.execute(
18285     `INSERT INTO monsters (
18286       user_id, name, style, personality, stamina, max_stamina, energy, max_energy, personality_params
18287     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
18288     ON DUPLICATE KEY UPDATE
18289       name = VALUES(name),
18290       style = VALUES(style),
18291       personality = VALUES(personality),
18292       stamina = VALUES(stamina),
18293       max_stamina = VALUES(max_stamina),
18294       energy = VALUES(energy),
18295       max_energy = VALUES(max_energy),
18296       personality_params = VALUES(personality_params),
18297       updated_at = CURRENT_TIMESTAMP`,
18298     [
18299       monsterData.userId,
18300       monsterData.name,
18301       monsterData.style,
18302       monsterData.personality,
18303       monsterData.stamina,
18304       monsterData.maxStamina,
18305       monsterData.energy,
18306       monsterData.maxEnergy,
18307       JSON.stringify(monsterData.personalityParams || {}),
18308     ]
18309   );
18310   return (result as any).insertId;
18311 };
18312
18313 export const getMonsterByUserId = async (userId: number): Promise<Monster | null> => {
18314   const [rows] = await pool.execute(
18315     'SELECT * FROM monsters WHERE user_id = ?',
18316     [userId]
18317   );
18318   const monsters = rows as Monster[];
18319   return monsters.length > 0 ? monsters[0] : null;
18320 };
18321
18322 export const updateMonster = async (
18323   userId: number,
18324   updates: Partial<Omit<Monster, 'id' | 'userId' | 'createdAt'>>
18325 ): Promise<void> => {
18326   const setClauses: string[] = [];

```



```

18327   const values: any[] = [];
18328
18329   if (updates.level !== undefined) {
18330       setClauses.push('level = ?');
18331       values.push(updates.level);
18332   }
18333   if (updates.exp !== undefined) {
18334       setClauses.push('exp = ?');
18335       values.push(updates.exp);
18336   }
18337   if (updates.stamina !== undefined) {
18338       setClauses.push('stamina = ?');
18339       values.push(updates.stamina);
18340   }
18341   if (updates.maxStamina !== undefined) {
18342       setClauses.push('max_stamina = ?');
18343       values.push(updates.maxStamina);
18344   }
18345   if (updates.energy !== undefined) {
18346       setClauses.push('energy = ?');
18347       values.push(updates.energy);
18348   }
18349   if (updates.maxEnergy !== undefined) {
18350       setClauses.push('max_energy = ?');
18351       values.push(updates.maxEnergy);
18352   }
18353   if (updates.personalityParams !== undefined) {
18354       setClauses.push('personality_params = ?');
18355       values.push(JSON.stringify(updates.personalityParams));
18356   }
18357   if (updates.lastEnergyRecover !== undefined) {
18358       setClauses.push('last_energy_recover = ?');
18359       values.push(updates.lastEnergyRecover);
18360   }
18361   if (updates.lastStaminaRecover !== undefined) {
18362       setClauses.push('last_stamina_recover = ?');
18363       values.push(updates.lastStaminaRecover);
18364   }
18365
18366   if (setClauses.length > 0) {
18367       setClauses.push('updated_at = CURRENT_TIMESTAMP');
18368       values.push(userId);
18369
18370       await pool.execute(
18371           `UPDATE monsters SET ${setClauses.join(', ')} WHERE user_id = ?`,
18372           values
18373       );
18374   }
18375 };
18376

```

```

18377 export const consumeStamina = async (userId: number, amount: number = 10): Promise<boolean> => {
18378   const [result] = await pool.execute(
18379     'UPDATE monsters SET stamina = stamina - ?, updated_at = CURRENT_TIMESTAMP WHERE user_id = ? AND stamina >
18380     [amount, userId, amount]
18381   );
18382   return (result as any).affectedRows > 0;
18383 };
18384
18385 export const recoverStamina = async (userId: number, amount: number = 20): Promise<void> => {
18386   await pool.execute(
18387     'UPDATE monsters SET stamina = LEAST(stamina + ?, max_stamina), last_stamina_recover = CURRENT_TIMESTAMP,
18388     [amount, userId]
18389   );
18390 };
18391
18392 export const consumeEnergy = async (userId: number): Promise<boolean> => {
18393   const [result] = await pool.execute(
18394     'UPDATE monsters SET energy = energy - 1, updated_at = CURRENT_TIMESTAMP WHERE user_id = ? AND energy > 0'
18395     [userId]
18396   );
18397   return (result as any).affectedRows > 0;
18398 };
18399
18400 export const consumeEnergyAmount = async (userId: number, amount: number): Promise<boolean> => {
18401   const [result] = await pool.execute(
18402     'UPDATE monsters SET energy = energy - ?, updated_at = CURRENT_TIMESTAMP WHERE user_id = ? AND energy >= ?
18403     [amount, userId, amount]
18404   );
18405   return (result as any).affectedRows > 0;
18406 };
18407
18408 export const recoverEnergy = async (userId: number, amount: number = 1): Promise<void> => {
18409   await pool.execute(
18410     'UPDATE monsters SET energy = LEAST(energy + ?, max_energy), last_energy_recover = CURRENT_TIMESTAMP, upda
18411     [amount, userId]
18412   );
18413 };
18414
18415 === server/src/models/MonsterMessage.ts ===
18416
18417 export interface MonsterMessage {
18418   id: number;
18419   userId: number;
18420   message: string;
18421   isUser: boolean;
18422   createdAt: Date;
18423 }
18424
18425 export const createMessage = async (messageData: {

```

```

18426   userId: number;
18427   message: string;
18428   isUser: boolean;
18429 }): Promise<number> => {
18430   const [result] = await pool.execute(
18431     'INSERT INTO monster_messages (user_id, message, is_user) VALUES (?, ?, ?)',
18432     [messageData.userId, messageData.message, messageData.isUser]
18433   );
18434   return (result as any).insertId;
18435 };
18436
18437 export const getMessagesByUserId = async (userId: number, limit: number = 50): Promise<MonsterMessage[]> => {
18438   // mysql2 prepared statements do not support parameterised LIMIT
18439   // use query() instead of execute() for this pattern
18440   const userIdNum = Number(userId);
18441   const limitNum = Number(limit);
18442   const [rows] = await pool.query(
18443     `SELECT * FROM monster_messages WHERE user_id = ? ORDER BY created_at DESC LIMIT ${limitNum}`,
18444     [userIdNum]
18445   );
18446   return rows as MonsterMessage[];
18447 };
18448
18449   === server/src/models/NodeProgress.ts ===
18449 import { pool } from '../config/database';
18450
18451 export interface NodeProgress {
18452   id: number;
18453   userId: number;
18454   domainId: number;
18455   nodeId: string;
18456   status: 'pending' | 'doing' | 'done';
18457   studyTime: number;
18458   notes?: string;
18459   createdAt: Date;
18460   updatedAt: Date;
18461 }
18462
18463 export const createNodeProgress = async (progressData: {
18464   userId: number;
18465   domainId: number;
18466   nodeId: string;
18467 }): Promise<number> => {
18468   const [result] = await pool.execute(
18469     'INSERT INTO node_progress (user_id, domain_id, node_id) VALUES (?, ?, ?)',
18470     [progressData.userId, progressData.domainId, progressData.nodeId]
18471   );
18472   return (result as any).insertId;
18473 };
18474

```

```

18475 export const getNodeProgress = async (
18476   userId: number,
18477   domainId: number,
18478   nodeId: string
18479 ): Promise<NodeProgress | null> => {
18480   const [rows] = await pool.execute(
18481     'SELECT * FROM node_progress WHERE user_id = ? AND domain_id = ? AND node_id = ?',
18482     [userId, domainId, nodeId]
18483   );
18484   const progresses = rows as NodeProgress[];
18485   return progresses.length > 0 ? progresses[0] : null;
18486 };
18487
18488 export const getNodeProgressesByDomain = async (
18489   userId: number,
18490   domainId: number
18491 ): Promise<NodeProgress[]> => {
18492   const [rows] = await pool.execute(
18493     'SELECT * FROM node_progress WHERE user_id = ? AND domain_id = ?',
18494     [userId, domainId]
18495   );
18496   return rows as NodeProgress[];
18497 };
18498
18499 export const updateNodeProgress = async (
18500   userId: number,
18501   domainId: number,
18502   nodeId: string,
18503   updates: Partial<Omit<NodeProgress, 'id' | 'userId' | 'domainId' | 'nodeId' | 'createdAt'>>
18504 ): Promise<void> => {
18505   const setClauses: string[] = [];
18506   const values: any[] = [];
18507
18508   if (updates.status !== undefined) {
18509     setClauses.push('status = ?');
18510     values.push(updates.status);
18511   }
18512   if (updates.studyTime !== undefined) {
18513     setClauses.push('study_time = ?');
18514     values.push(updates.studyTime);
18515   }
18516   if (updates.notes !== undefined) {
18517     setClauses.push('notes = ?');
18518     values.push(updates.notes);
18519   }
18520
18521   if (setClauses.length > 0) {
18522     setClauses.push('updated_at = CURRENT_TIMESTAMP');
18523     values.push(userId, domainId, nodeId);
18524

```

```

18525     await pool.execute(
18526         `UPDATE node_progress SET ${setClauses.join(', ')} WHERE user_id = ? AND domain_id = ? AND node_id = ?`,
18527         values
18528     );
18529 }
18530 };
18531
18532 === server/src/models/Note.ts ===
18532 import { pool } from '../config/database';
18533
18534 export interface Note {
18535     id: number;
18536     userId: number;
18537     date: Date;
18538     content?: string;
18539     monsterComment?: string;
18540     createdAt: Date;
18541     updatedAt: Date;
18542 }
18543
18544 export const createNote = async (noteData: {
18545     userId: number;
18546     date: Date;
18547     content?: string;
18548     monsterComment?: string;
18549 }): Promise<number> => {
18550     const [result] = await pool.execute(
18551         `INSERT INTO notes (user_id, date, content, monster_comment) VALUES (?, ?, ?, ?)`,
18552         [noteData.userId, noteData.date, noteData.content || null, noteData.monsterComment || null]
18553     );
18554     return (result as any).insertId;
18555 };
18556
18557 export const getNotesByUserId = async (userId: number): Promise<Note[]> => {
18558     const [rows] = await pool.execute(
18559         `SELECT * FROM notes WHERE user_id = ? ORDER BY date DESC`,
18560         [userId]
18561     );
18562     return rows as Note[];
18563 };
18564
18565 export const getNoteByDate = async (userId: number, date: Date): Promise<Note | null> => {
18566     const [rows] = await pool.execute(
18567         `SELECT * FROM notes WHERE user_id = ? AND date = ?`,
18568         [userId, date]
18569     );
18570     const notes = rows as Note[];
18571     return notes.length > 0 ? notes[0] : null;
18572 };
18573

```

```

18574 export const updateNote = async (
18575   id: number,
18576   updates: Partial<Omit<Note, 'id' | 'userId' | 'createdAt'>>
18577 ): Promise<void> => {
18578   const setClauses: string[] = [];
18579   const values: any[] = [];
18580
18581   if (updates.date !== undefined) {
18582     setClauses.push('date = ?');
18583     values.push(updates.date);
18584   }
18585   if (updates.content !== undefined) {
18586     setClauses.push('content = ?');
18587     values.push(updates.content);
18588   }
18589   if (updates.monsterComment !== undefined) {
18590     setClauses.push('monster_comment = ?');
18591     values.push(updates.monsterComment);
18592   }
18593
18594   if (setClauses.length > 0) {
18595     setClauses.push('updated_at = CURRENT_TIMESTAMP');
18596     values.push(id);
18597
18598     await pool.execute(
18599       `UPDATE notes SET ${setClauses.join(', ')} WHERE id = ?`,
18600       values
18601     );
18602   }
18603 };
18604
18605 === server/src/models/PopularDomain.ts ===
18606 export interface PopularDomain {
18607   id: number;
18608   domain: string;
18609   searchCount: number;
18610   generatedCount: number;
18611   lastUpdated: Date;
18612 }
18613
18614 export interface PopularDomainResponse {
18615   domain: string;
18616   popularity: number;
18617   searchCount: number;
18618   generatedCount: number;
18619 }
18620
18621 === server/src/models/Reward.ts ===
18622 import { pool } from '../config/database';
18623
18624 export interface Reward {

```

```

18622 id: number;
18623 userId: number;
18624 type: string;
18625 amount: number;
18626 source?: string;
18627 claimed: boolean;
18628 createdAt: Date;
18629 updatedAt: Date;
18630 }
18631
18632 export const createReward = async (rewardData: {
18633   userId: number;
18634   type: string;
18635   amount: number;
18636   source?: string;
18637 }): Promise<number> => {
18638   const [result] = await pool.execute(
18639     'INSERT INTO rewards (user_id, type, amount, source) VALUES (?, ?, ?, ?)',
18640     [rewardData.userId, rewardData.type, rewardData.amount, rewardData.source || null]
18641   );
18642   return (result as any).insertId;
18643 };
18644
18645 export const getRewardsByUserId = async (userId: number): Promise<Reward[]> => {
18646   const [rows] = await pool.execute(
18647     'SELECT * FROM rewards WHERE user_id = ? ORDER BY created_at DESC',
18648     [userId]
18649   );
18650   return rows as Reward[];
18651 };
18652
18653 export const claimReward = async (id: number): Promise<boolean> => {
18654   const [result] = await pool.execute(
18655     'UPDATE rewards SET claimed = TRUE, updated_at = CURRENT_TIMESTAMP WHERE id = ? AND claimed = FALSE',
18656     [id]
18657   );
18658   return (result as any).affectedRows > 0;
18659 };
18660
18661 === server/src/models/SkillTree.ts ===
18662
18663 export interface SkillTree {
18664   id: number;
18665   userId: number;
18666   domain: string;
18667   title: string;
18668   description?: string;
18669   nodes: SkillNode[];
18670   progress: number;

```

```

18671   isPublic: boolean;
18672   createdAt: Date;
18673   updatedAt: Date;
18674 }
18675
18676 export interface CreateSkillTreeRequest {
18677   userId: number;
18678   domain: string;
18679   title: string;
18680   description?: string;
18681   nodes: SkillNode[];
18682   isPublic?: boolean;
18683 }
18684
18685 export interface UpdateSkillTreeRequest {
18686   title?: string;
18687   description?: string;
18688   nodes?: SkillNode[];
18689   progress?: number;
18690   isPublic?: boolean;
18691 }
18692
18693 export interface SkillTreeListResponse {
18694   trees: SkillTree[];
18695   total: number;
18696   page: number;
18697   limit: number;
18698 }
18699
18700 === server/src/models/StudyRecord.ts ===
18701 import { pool } from '../config/database';
18702
18703 export interface StudyRecord {
18704   id: number;
18705   userId: number;
18706   domainId: number;
18707   nodeId: string;
18708   startTime: Date;
18709   endTime?: Date;
18710   duration: number;
18711   progressBefore: number;
18712   progressAfter: number;
18713   rewardObtained?: any;
18714   createdAt: Date;
18715   updatedAt: Date;
18716 }
18717
18718 export const createStudyRecord = async (recordData: {
18719   userId: number;
18720   domainId: number;
18721   nodeId: string;

```



```

18720   startTime: Date;
18721   progressBefore: number;
18722 }): Promise<number> => {
18723   const [result] = await pool.execute(
18724     `INSERT INTO study_records (user_id, domain_id, node_id, start_time, progress_before) VALUES (?, ?, ?, ?,`,
18725     [recordData.userId, recordData.domainId, recordData.nodeId, recordData.startTime, recordData.progressBefore
18726   );
18727   return (result as any).insertId;
18728 };
18729
18730 export const updateStudyRecord = async (
18731   id: number,
18732   updates: Partial<Omit<StudyRecord, 'id' | 'userId' | 'domainId' | 'nodeId' | 'createdAt'>>
18733 ): Promise<void> => {
18734   const setClauses: string[] = [];
18735   const values: any[] = [];
18736
18737   if (updates.endTime !== undefined) {
18738     setClauses.push('end_time = ?');
18739     values.push(updates.endTime);
18740   }
18741   if (updates.duration !== undefined) {
18742     setClauses.push('duration = ?');
18743     values.push(updates.duration);
18744   }
18745   if (updates.progressAfter !== undefined) {
18746     setClauses.push('progress_after = ?');
18747     values.push(updates.progressAfter);
18748   }
18749   if (updates.rewardObtained !== undefined) {
18750     setClauses.push('reward_obtained = ?');
18751     values.push(JSON.stringify(updates.rewardObtained));
18752   }
18753
18754   if (setClauses.length > 0) {
18755     setClauses.push('updated_at = CURRENT_TIMESTAMP');
18756     values.push(id);
18757
18758     await pool.execute(
18759       `UPDATE study_records SET ${setClauses.join(', ')} WHERE id = ?`,
18760       values
18761     );
18762   }
18763 };
18764
18765 export const getStudyRecordsByUser = async (userId: number, limit: number = 20): Promise<StudyRecord[]> => {
18766   const [rows] = await pool.execute(
18767     `SELECT * FROM study_records WHERE user_id = ? ORDER BY created_at DESC LIMIT ?`,
18768     [userId, limit]
18769   );

```

```

18770   const records = rows as any[];
18771   return records.map(r => ({
18772     ...r,
18773     rewardObtained: typeof r.reward_obtained === 'string' ? JSON.parse(r.reward_obtained) : r.reward_obtained
18774   }));
18775   });
18776
18777   === server/src/models/User.ts ===
18778   export interface User {
18779     id: number;
18780     username?: string;
18781     email?: string;
18782     phone?: string;
18783     passwordHash?: string;
18784     nickname?: string;
18785     avatarUrl?: string;
18786     lastLoginAt?: Date;
18787     loginCount: number;
18788     status: 'active' | 'inactive' | 'banned';
18789     createdAt: Date;
18790     updatedAt: Date;
18791   }
18792
18793   export interface CreateUserRequest {
18794     username?: string;
18795     email?: string;
18796     password?: string;
18797     nickname?: string;
18798     avatarUrl?: string;
18799   }
18800
18801   export interface LoginRequest {
18802     type: 'email';
18803     email?: string;
18804     password?: string;
18805     deviceId: string;
18806     deviceType: 'ios' | 'android' | 'web';
18807     deviceName?: string;
18808   }
18809
18810   export interface UserResponse {
18811     id: number;
18812     username?: string;
18813     email?: string;
18814     phone?: string;
18815     nickname?: string;
18816     avatarUrl?: string;
18817     loginCount: number;
18818     lastLoginAt?: Date;
18819     createdAt: Date;

```

```

18819 }
18820
18821 export interface AuthResponse {
18822   user: UserResponse;
18823   accessToken: string;
18824   refreshToken: string;
18825   expiresAt: string;
18826 }
18827
18828 === server/src/models/index.ts ===
18827 export * from './User';
18828 export * from './DeviceSession';
18829 export * from './SkillTree';
18830 export * from './LearningRecord';
18831 export * from './PopularDomain';
18832 export * from './Monster';
18833 export * from './MonsterMessage';
18834 export * from './Domain';
18835 export * from './NodeProgress';
18836 export * from './StudyRecord';
18837 export * from './Note';
18838 export * from './Reward';
18839
18840 === server/src/routes/aiRoutes.ts ===
18839 import express from 'express';
18840 import { fillModuleContent } from '../controllers/aiController';
18841
18842 const router = express.Router();
18843
18844 router.post('/fill-module', fillModuleContent);
18845
18846 export default router;
18847
18848 === server/src/routes/authRoutes.ts ===
18848 import express from 'express';
18849 import { AuthController } from '../controllers/authController';
18850
18851 const router = express.Router();
18852
18853 // 用户注册
18854 router.post('/register', AuthController.register);
18855
18856 // 邮箱验证
18857 router.post('/verify-email', AuthController.verifyEmail);
18858
18859 // 重新发送验证码
18860 router.post('/resend-verification', AuthController.resendVerification);
18861
18862 // 用户登录
18863 router.post('/login', AuthController.login);
18864
18865 // 刷新访问令牌

```

```

18866 router.post('/refresh-token', AuthController.refreshToken);
18867
18868 // 验证令牌
18869 router.post('/verify-token', AuthController.verifyToken);
18870
18871 // 用户登出
18872 router.post('/logout', AuthController.logout);
18873
18874 export default router;
    === server/src/routes/domainRoutes.ts ===
18875 import express from 'express';
18876 import {
18877   createDomain,
18878   getDomains,
18879   getDomainById,
18880   updateNodeProgress,
18881   startLearning,
18882   finishLearning
18883 } from '../controllers';
18884
18885 const router = express.Router();
18886
18887 router.post('/create', createDomain);
18888 router.get('/list/:userId', getDomains);
18889 router.get('/:id', getDomainById);
18890 router.put('/nodes/progress', updateNodeProgress);
18891 router.post('/learning/start', startLearning);
18892 router.post('/learning/finish', finishLearning);
18893
18894 export default router;
18895
    === server/src/routes/index.ts ===
18896 export { default as skillRoutes } from './skillRoutes';
18897 export { default as authRoutes } from './authRoutes';
18898 export { default as monsterRoutes } from './monsterRoutes';
18899 export { default as noteRoutes } from './noteRoutes';
18900 export { default as rewardRoutes } from './rewardRoutes';
18901 export { default as domainRoutes } from './domainRoutes';
18902 export { default as aiRoutes } from './aiRoutes';
18903 export { default as proRoutes } from './proRoutes';
    === server/src/routes/monsterRoutes.ts ===
18904 import express from 'express';
18905 import {
18906   createMonster,
18907   getMonsterStatus,
18908   consumeStamina,
18909   recoverStamina,
18910   consumeEnergy,
18911   consumeEnergyAmount,
18912   recoverEnergy,

```

```

18913   addExp,
18914   chatWithMonster,
18915   getMonsterMessages
18916 } from '../controllers';
18917
18918 const router = express.Router();
18919
18920 router.post('/create', createMonster);
18921 router.get('/status/:userId', getMonsterStatus);
18922 router.post('/stamina/consume', consumeStamina);
18923 router.post('/stamina/recover', recoverStamina);
18924 router.post('/energy/consume', consumeEnergy);
18925 router.post('/energy/consume-amount', consumeEnergyAmount);
18926 router.post('/energy/recover', recoverEnergy);
18927 router.post('/exp/add', addExp);
18928 router.post('/chat', chatWithMonster);
18929 router.get('/messages/:userId', getMonsterMessages);
18930
18931 export default router;
18932
18933 === server/src/routes/noteRoutes.ts ===
18934 import express from 'express';
18935 import {
18936   createNote,
18937   getNotes,
18938   getNoteByDate,
18939   updateNote
18940 } from '../controllers';
18941
18942 const router = express.Router();
18943
18944 router.post('/create', createNote);
18945 router.get('/list/:userId', getNotes);
18946 router.get('/:userId/:date', getNoteByDate);
18947 router.put('/update', updateNote);
18948
18949 export default router;
18950
18951 === server/src/routes/proRoutes.ts ===
18952 import express from 'express';
18953 import { ProController } from '../controllers/proController';
18954
18955 const router = express.Router();
18956
18957 // 生成激活码 (管理端)
18958 router.post('/generate-codes', ProController.generateCodes);
18959
18960 // 激活 Pro
18961 router.post('/activate', ProController.activateCode);

```

```

18961 // 查询 Pro 状态
18962 router.get('/status/:userId', ProController.getStatus);
18963
18964 export default router;
18965
18966 === server/src/routes/rewardRoutes.ts ===
18967 import express from 'express';
18968 import {
18969   createReward,
18970   getRewards,
18971   claimReward
18972 } from '../controllers';
18973
18974 const router = express.Router();
18975
18976 router.post('/create', createReward);
18977 router.get('/list/:userId', getRewards);
18978 router.post('/claim', claimReward);
18979
18980 export default router;
18981
18982 === server/src/routes/skillRoutes.ts ===
18983 import express from 'express';
18984 import {
18985   generateSkillTree,
18986   getSkillTreeList,
18987   getSkillTreeById,
18988   saveUserSkillTree,
18989   getUserProgress,
18990   updateUserProgress,
18991   getStatistics,
18992   searchPopularDomains,
18993   getRecommendedPath,
18994   getUserLearningReport
18995 } from '../controllers';
18996
18997 const router = express.Router();
18998
18999 // 技能树生成和管理
19000 router.post('/generate', generateSkillTree);
19001 router.get('/list', getSkillTreeList);
19002 router.get('/:id', getSkillTreeById);
19003
19004 // 用户相关功能
19005 router.post('/save', saveUserSkillTree);
19006 router.get('/progress/:userId', getUserProgress);
19007 router.put('/progress/:userId', updateUserProgress);
19008
19009 // 搜索和推荐功能
19010 router.get('/search/domains', searchPopularDomains);

```

```

19009 router.get('/recommendations/path', getRecommendedPath);
19010
19011 // 统计和学习报告
19012 router.get('/stats/overview', getStatistics);
19013 router.get('/report/:userId', getUserLearningReport);
19014
19015 export default router;
19016
19017 === server/src/services/aiFillService.ts ===
19018 // =====
19019 // AI Fill Service
19020
19021 // 调用 DeepSeek API 为自定义模块自动生成学习内容
19022 // =====
19023
19024 import fs from 'fs';
19025 import path from 'path';
19026
19027 const DEEPSEEK_API_URL = 'https://api.deepseek.com/v1/chat/completions';
19028 const DEEPSEEK_MODEL = 'deepseek-chat';
19029 const REQUEST_TIMEOUT_MS = 15000;
19030
19031 // 从 key.json 读取 API Key
19032 let DEEPSEEK_API_KEY: string | null = null;
19033
19034 try {
19035   const keyPath = path.join(__dirname, '../key.json');
19036   const keyFileContent = fs.readFileSync(keyPath, 'utf-8');
19037   const keyData = JSON.parse(keyFileContent);
19038   DEEPSEEK_API_KEY = keyData.DEEPSEEK_API_KEY;
19039 } catch (error) {
19040   console.error('[AIFillService] 读取 key.json 失败:', error);
19041 }
19042
19043 interface FillModuleResponse {
19044   moduleDescription: string;
19045   nodes: {
19046     nodeName: string;
19047     subNodes: {
19048       subName: string;
19049       link: string;
19050     }[];
19051   }[];
19052 }
19053
19054 function buildPrompt(moduleName: string, moduleDescription: string): string {
19055   return `你是一个学习内容规划专家。
19056
19057   用户已定义了一个学习模块，信息如下：
19058   - 模块名称: ${moduleName}
19059   - 模块介绍: ${moduleDescription}
19060
19061   请你严格基于以上模块名称和模块介绍，规划具体的学习内容。输出的 moduleDescription 必须紧扣用户的模块介绍进行扩写（50~120字），不得偏离。严格按照以下JSON结构返回，

```

```

19058
19059 {
19060   "moduleDescription": "基于用户模块介绍扩写的完整介绍（50~120字）",
19061   "nodes": [
19062     {
19063       "nodeName": "大结点1名称",
19064       "subNodes": [
19065         { "subName": "小结点1名称", "link": "https://example.com/学习链接1" },
19066         { "subName": "小结点2名称", "link": "https://example.com/学习链接2" },
19067         { "subName": "小结点3名称", "link": "https://example.com/学习链接3" }
19068       ]
19069     },
19070     {
19071       "nodeName": "大结点2名称",
19072       "subNodes": [
19073         { "subName": "小结点1名称", "link": "https://example.com/学习链接1" },
19074         { "subName": "小结点2名称", "link": "https://example.com/学习链接2" },
19075         { "subName": "小结点3名称", "link": "https://example.com/学习链接3" }
19076       ]
19077     },
19078     {
19079       "nodeName": "大结点3名称",
19080       "subNodes": [
19081         { "subName": "小结点1名称", "link": "https://example.com/学习链接1" },
19082         { "subName": "小结点2名称", "link": "https://example.com/学习链接2" },
19083         { "subName": "小结点3名称", "link": "https://example.com/学习链接3" }
19084       ]
19085     }
19086   ]
19087 }
19088
19089 要求:
19090 - 共3个大结点，每个大结点下3个小结点
19091 - 大结点和小结点的内容必须与"${moduleName}"和"${moduleDescription}"紧密相关
19092 - 每个小结点需提供真实、有学习价值的URL链接
19093 - 直接输出JSON对象，不要包装在数组中；
19094 }
19095
19096 export async function fillModule(moduleName: string, moduleDescription: string): Promise<FillModuleResponse> {
19097   console.log('[AIFillService] 开始为模块「${moduleName}」生成内容...');
19098
19099   if (!DEEPSEEK_API_KEY) {
19100     throw new Error('未找到 DeepSeek API Key, 请检查 key.json 文件');
19101   }
19102
19103   const controller = new AbortController();
19104   const timeoutId = setTimeout(() => controller.abort(), REQUEST_TIMEOUT_MS);
19105
19106   try {
19107     const response = await fetch(DEEPSEEK_API_URL, {

```



```

19108     method: 'POST',
19109     headers: {
19110       'Content-Type': 'application/json',
19111       'Authorization': 'Bearer ${DEEPSEEK_API_KEY}',
19112     },
19113     body: JSON.stringify({
19114       model: DEEPSEEK_MODEL,
19115       messages: [{ role: 'user', content: buildPrompt(moduleName, moduleDescription) }],
19116       temperature: 0.7,
19117       max_tokens: 2000,
19118     }),
19119     signal: controller.signal,
19120   });
19121
19122   clearTimeout(timeoutId);
19123
19124   if (!response.ok) {
19125     console.error('[AIFillService] API 返回错误: ${response.status}');
19126     throw new Error('AI 服务返回错误 (${response.status})');
19127   }
19128
19129   const data = await response.json();
19130   const rawContent = data.choices?.[0]?.message?.content;
19131
19132   if (!rawContent) {
19133     console.error('[AIFillService] API 返回空内容');
19134     throw new Error('AI 返回内容为空');
19135   }
19136
19137   console.log('[AIFillService] 原始响应 (前200字): ${rawContent.substring(0, 200)}');
19138
19139   // 清理响应: 移除可能的 markdown 代码块标记
19140   let cleaned = rawContent.trim();
19141   if (cleaned.startsWith('```json')) {
19142     cleaned = cleaned.slice(7);
19143   } else if (cleaned.startsWith('```')) {
19144     cleaned = cleaned.slice(3);
19145   }
19146   if (cleaned.endsWith('```')) {
19147     cleaned = cleaned.slice(0, -3);
19148   }
19149   cleaned = cleaned.trim();
19150
19151   const parsed: FillModuleResponse = JSON.parse(cleaned);
19152
19153   // 校验数据结构
19154   if (!parsed.moduleDescription || !Array.isArray(parsed.nodes)) {
19155     throw new Error('AI 返回数据结构不完整: 缺少 moduleDescription 或 nodes');
19156   }
19157

```

```

19158     if (parsed.nodes.length < 3) {
19159         throw new Error('AI 返回的大结点数量不足: 期望 3 个, 实际 ${parsed.nodes.length} 个');
19160     }
19161
19162     for (let i = 0; i < 3; i++) {
19163         const node = parsed.nodes[i];
19164         if (!node.nodeName || !Array.isArray(node.subNodes)) {
19165             throw new Error(`第 ${i + 1} 个大结点数据结构不完整`);
19166         }
19167         if (node.subNodes.length < 3) {
19168             throw new Error(`大结点「${node.nodeName}」的小结点数量不足: 期望 3 个, 实际 ${node.subNodes.length} 个`);
19169         }
19170         for (let j = 0; j < 3; j++) {
19171             const sub = node.subNodes[j];
19172             if (!sub.subName) {
19173                 throw new Error(`大结点「${node.nodeName}」的第 ${j + 1} 个小结点缺少名称`);
19174             }
19175             if (!sub.link || (!sub.link.startsWith('http://') && !sub.link.startsWith('https://'))) {
19176                 // 如果链接无效, 给一个占位
19177                 sub.link = `https://www.bilibili.com/search?keyword=${encodeURIComponent(sub.subName)}`;
19178             }
19179         }
19180     }
19181
19182     console.log(`[AIFillService] 内容生成成功 - 模块「${moduleName}」`);
19183     return parsed;
19184 } catch (error: any) {
19185     clearTimeout(timeoutId);
19186     if (error.name === 'AbortError') {
19187         console.error('[AIFillService] 请求超时');
19188         throw new Error('AI 服务请求超时');
19189     }
19190     if (error instanceof SyntaxError) {
19191         console.error('[AIFillService] JSON 解析失败:', error.message);
19192         throw new Error('AI 返回数据格式异常');
19193     }
19194     console.error('[AIFillService] 生成失败:', error.message);
19195     throw error;
19196 }
19197 }
19198
19199 === server/src/services/authService.ts ===
19199 import bcrypt from 'bcryptjs';
19200 import jwt from 'jsonwebtoken';
19201 import { DatabaseConnection } from '../config/database';
19202 import { DatabaseService } from '../databaseService';
19203 import { EmailService } from '../emailService';
19204 import {
19205     User, CreateUserRequest, LoginRequest, AuthResponse, UserResponse
19206 } from '../models';

```

```

19207
19208 export class AuthService {
19209     private static JWT_SECRET = process.env.JWT_SECRET || 'learnflow-secret-key';
19210     private static JWT_EXPIRES_IN = '7d';
19211     private static REFRESH_TOKEN_EXPIRES_IN = '30d';
19212
19213     // 邮箱登录
19214     static async emailLogin(request: LoginRequest): Promise<AuthResponse> {
19215         const startTime = Date.now();
19216         const { email, password, deviceId, deviceType, deviceName } = request;
19217
19218         console.log('[AuthService] 开始用户登录流程 - 邮箱: ${email}');
19219
19220         if (!email || !password) {
19221             console.log('[AuthService] 登录验证失败 - 邮箱或密码为空');
19222             throw new Error('邮箱和密码不能为空');
19223         }
19224
19225         const connection = await DatabaseConnection.getConnection();
19226         const [rows] = await connection.execute(
19227             'SELECT * FROM users WHERE email = ?',
19228             [email]
19229         );
19230
19231         const rawUser = (rows as any[])[0];
19232         if (!rawUser) {
19233             console.log('[AuthService] 登录失败 - 用户不存在: ${email}');
19234             throw new Error('用户不存在');
19235         }
19236
19237         if (rawUser.status === 'inactive') {
19238             console.log('[AuthService] 登录失败 - 邮箱未验证: ${email}');
19239             throw new Error('邮箱未验证, 请先完成验证');
19240         }
19241
19242         if (rawUser.status !== 'active') {
19243             console.log('[AuthService] 登录失败 - 账户状态异常: ${email}');
19244             throw new Error('账户状态异常, 无法登录');
19245         }
19246
19247         const user = this.mapUserFromDB(rawUser);
19248
19249         const isValidPassword = await bcrypt.compare(password, user.passwordHash || '');
19250         if (!isValidPassword) {
19251             console.log('[AuthService] 登录失败 - 密码错误: ${email}');
19252             throw new Error('密码错误');
19253         }
19254
19255         const authResponse = await this.generateAuthResponse(user, deviceId, deviceType, deviceName);
19256

```

```

19257     const duration = Date.now() - startTime;
19258     console.log('[AuthService] 用户登录流程完成 - 邮箱: ${email}, 总耗时: ${duration}ms');
19259
19260     return authResponse;
19261 }
19262
19263 // 用户注册 (邮箱验证流程: 先创建未激活用户, 发送验证码)
19264 static async registerUser(request: CreateUserRequest): Promise<{ message: string; email: string }> {
19265     const startTime = Date.now();
19266     const { username, email, password } = request;
19267
19268     console.log('[AuthService] 开始用户注册流程 - 用户名: ${username}, 邮箱: ${email}');
19269
19270     if (!username || !email || !password) {
19271         console.log('[AuthService] 注册验证失败 - 缺少必填字段');
19272         throw new Error('用户名、邮箱和密码不能为空');
19273     }
19274
19275     if (username.trim().length < 2) {
19276         throw new Error('用户名至少2个字符');
19277     }
19278
19279     if (password.length < 6) {
19280         console.log('[AuthService] 注册验证失败 - 密码长度不足');
19281         throw new Error('密码长度至少6位');
19282     }
19283
19284     const finalUsername = username.trim();
19285
19286     const connection = await DatabaseConnection.getConnection();
19287
19288     // 检查邮箱是否已存在
19289     const [existingUsers] = await connection.execute(
19290         'SELECT id, status FROM users WHERE email = ?',
19291         [email]
19292     );
19293
19294     const existingUser = (existingUsers as any[0])[0];
19295     if (existingUser) {
19296         if (existingUser.status === 'inactive') {
19297             // 用户已注册但未激活, 重新发送验证码
19298             console.log('[AuthService] 邮箱已注册但未激活, 重新发送验证码: ${email}');
19299             const code = await this.generateAndStoreCode(email);
19300             await EmailService.sendVerificationCode(email, code);
19301             return { message: '验证码已重新发送至邮箱', email };
19302         }
19303         console.log('[AuthService] 邮箱已被注册: ${email}');
19304         throw new Error('邮箱已被注册');
19305     }
19306

```

```

19307 // 检查用户名是否已存在
19308 let uniqueUsername = finalUsername;
19309 let suffix = 1;
19310 while (true) {
19311     const [existingUsernames] = await connection.execute(
19312         'SELECT id FROM users WHERE username = ?',
19313         [uniqueUsername]
19314     );
19315     if ((existingUsernames as any[]).length === 0) break;
19316     uniqueUsername = `${finalUsername}${suffix}`;
19317     suffix++;
19318 }
19319
19320 // 密码加密
19321 const saltRounds = 12;
19322 const passwordHash = await bcrypt.hash(password, saltRounds);
19323
19324 // 创建未激活用户
19325 const [result] = await connection.execute(
19326     `INSERT INTO users (username, email, password_hash, status, created_at, updated_at)
19327     VALUES (?, ?, ?, 'inactive', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP)`,
19328     [uniqueUsername, email, passwordHash]
19329 );
19330
19331 const userId = (result as any).insertId;
19332 console.log('[AuthService] 未激活用户创建成功 - 用户ID: ${userId}, 用户名: ${uniqueUsername}');
19333
19334 // 生成验证码并发送邮件
19335 const code = await this.generateAndStoreCode(email);
19336 await EmailService.sendVerificationCode(email, code);
19337
19338 const duration = Date.now() - startTime;
19339 console.log('[AuthService] 用户注册流程完成 - 用户名: ${uniqueUsername}, 总耗时: ${duration}ms');
19340
19341 return { message: '验证码已发送至邮箱, 请查收', email };
19342 }
19343
19344 // 生成并存储6位验证码
19345 private static async generateAndStoreCode(email: string): Promise<string> {
19346     const code = Math.floor(100000 + Math.random() * 900000).toString();
19347     const expiresAt = new Date(Date.now() + 10 * 60 * 1000); // 10分钟有效
19348
19349     const connection = await DatabaseConnection.getConnection();
19350     // 删除该邮箱过期的旧验证码
19351     await connection.execute(
19352         'DELETE FROM email_verification_tokens WHERE email = ?',
19353         [email]
19354     );
19355     // 插入新验证码
19356     await connection.execute(

```

```

19357     'INSERT INTO email_verification_tokens (email, token, expires_at) VALUES (?, ?, ?)',
19358     [email, code, expiresAt]
19359 );
19360
19361 console.log('[AuthService] 验证码已生成 - 邮箱: ${email}, 过期时间: ${expiresAt}');
19362 return code;
19363 }
19364
19365 // 验证邮箱验证码并激活用户
19366 static async verifyEmail(email: string, token: string): Promise<AuthResponse> {
19367     const connection = await DatabaseConnection.getConnection();
19368
19369     // 查找有效验证码
19370     const [rows] = await connection.execute(
19371         'SELECT * FROM email_verification_tokens WHERE email = ? AND token = ? AND expires_at > NOW()',
19372         [email, token]
19373     );
19374
19375     const record = (rows as any[][0]);
19376     if (!record) {
19377         throw new Error('验证码无效或已过期');
19378     }
19379
19380     // 检查尝试次数 (最多3次)
19381     if (record.attempts >= 3) {
19382         await connection.execute(
19383             'DELETE FROM email_verification_tokens WHERE id = ?',
19384             [record.id]
19385         );
19386         throw new Error('验证码错误次数过多, 请重新发送');
19387     }
19388
19389     // 增加尝试次数
19390     await connection.execute(
19391         'UPDATE email_verification_tokens SET attempts = attempts + 1 WHERE id = ?',
19392         [record.id]
19393     );
19394
19395     // 激活用户
19396     await connection.execute(
19397         'UPDATE users SET status = 'active', updated_at = CURRENT_TIMESTAMP WHERE email = ? AND status = 'inacti
19398         [email]
19399     );
19400
19401     // 清理验证码
19402     await connection.execute(
19403         'DELETE FROM email_verification_tokens WHERE email = ?',
19404         [email]
19405     );
19406

```

```

19407 // 获取用户信息并生成token
19408 const [userRows] = await connection.execute(
19409     'SELECT * FROM users WHERE email = ?',
19410     [email]
19411 );
19412
19413 const user = this.mapUserFromDB((userRows as any[])[0]);
19414 console.log('[AuthService] 邮箱验证成功 - 用户ID: ${user.id}');
19415
19416 return this.generateAuthResponse(user, 'default-device', 'web', '注册设备');
19417 }
19418
19419 // 重新发送验证码
19420 static async resendVerificationCode(email: string): Promise<{ message: string }> {
19421     const connection = await DatabaseConnection.getConnection();
19422
19423     // 检查60秒冷却
19424     const [recent] = await connection.execute(
19425         'SELECT created_at FROM email_verification_tokens WHERE email = ? ORDER BY created_at DESC LIMIT 1',
19426         [email]
19427     );
19428
19429     const lastRecord = (recent as any[])[0];
19430     if (lastRecord) {
19431         const secondsSinceLast = (Date.now() - new Date(lastRecord.created_at).getTime()) / 1000;
19432         if (secondsSinceLast < 60) {
19433             const waitSeconds = Math.ceil(60 - secondsSinceLast);
19434             throw new Error('请等待 ${waitSeconds} 秒后再发送');
19435         }
19436     }
19437
19438     const code = await this.generateAndStoreCode(email);
19439     await EmailService.sendVerificationCode(email, code);
19440     console.log('[AuthService] 验证码已重新发送 - 邮箱: ${email}');
19441     return { message: '验证码已重新发送至邮箱' };
19442 }
19443
19444 // 直接发送验证码（已登录用户修改邮箱等场景）
19445 static async sendVerificationEmail(email: string): Promise<{ message: string }> {
19446     const code = await this.generateAndStoreCode(email);
19447     await EmailService.sendVerificationCode(email, code);
19448     console.log('[AuthService] 验证码已发送 - 邮箱: ${email}');
19449     return { message: '验证码已发送至邮箱' };
19450 }
19451
19452 // 生成认证响应
19453 private static async generateAuthResponse(
19454     user: User,
19455     deviceId: string,
19456     deviceType: 'ios' | 'android' | 'web',

```

```

19457     deviceName?: string
19458   ): Promise<AuthResponse> {
19459     console.log('[AuthService] 生成认证令牌 - 用户ID: ${user.id}');
19460
19461     // 生成JWT令牌
19462     const accessToken = jwt.sign(
19463       { userId: user.id, type: 'access' },
19464       this.JWT_SECRET,
19465       { expiresIn: this.JWT_EXPIRES_IN as any }
19466     );
19467
19468     const refreshToken = jwt.sign(
19469       { userId: user.id, type: 'refresh' },
19470       this.JWT_SECRET,
19471       { expiresIn: this.REFRESH_TOKEN_EXPIRES_IN as any }
19472     );
19473
19474     // 检查是否已存在设备会话
19475     const existingSession = await DatabaseService.getDeviceSession(user.id, deviceId);
19476     const expiresAt = new Date(Date.now() + 30 * 24 * 60 * 60 * 1000); // 30天
19477
19478     if (existingSession) {
19479       // 更新现有会话的过期时间
19480       console.log('[AuthService] 更新现有设备会话 - 会话ID: ${existingSession.id}');
19481       await DatabaseService.updateDeviceSession(existingSession.id, {
19482         expiresAt,
19483         lastActiveAt: new Date()
19484       });
19485     } else {
19486       // 创建新的设备会话
19487       console.log('[AuthService] 创建设备会话 - 用户ID: ${user.id}, 设备ID: ${deviceId}');
19488       await DatabaseService.createDeviceSession({
19489         userId: user.id,
19490         deviceId,
19491         deviceType,
19492         deviceName,
19493         expiresAt
19494       });
19495     }
19496
19497     // 更新用户登录信息
19498     await DatabaseService.updateUserLoginInfo(user.id);
19499
19500     return {
19501       user: this.mapToUserResponse(user),
19502       accessToken,
19503       refreshToken,
19504       expiresAt: new Date(Date.now() + 7 * 24 * 60 * 60 * 1000).toISOString()
19505     };
19506   }

```



```

19507
19508 // 验证令牌
19509 static async verifyToken(token: string): Promise<{ userId: number; type: string }> {
19510     try {
19511         const decoded = jwt.verify(token, this.JWT_SECRET) as any;
19512         return { userId: decoded.userId, type: decoded.type };
19513     } catch (error) {
19514         throw new Error('令牌无效或已过期');
19515     }
19516 }
19517
19518 // 刷新令牌
19519 static async refreshToken(refreshToken: string, deviceId: string): Promise<AuthResponse> {
19520     const decoded = await this.verifyToken(refreshToken);
19521
19522     if (decoded.type !== 'refresh') {
19523         throw new Error('无效的刷新令牌');
19524     }
19525
19526     // 检查设备会话
19527     const session = await DatabaseService.getDeviceSession(decoded.userId, deviceId);
19528     if (!session || new Date(session.expiresAt) < new Date()) {
19529         throw new Error('设备会话已过期');
19530     }
19531
19532     // 获取用户信息
19533     const user = await DatabaseService.getUserById(decoded.userId);
19534     if (!user) {
19535         throw new Error('用户不存在');
19536     }
19537
19538     // 生成新的访问令牌
19539     const newAccessToken = jwt.sign(
19540         { userId: user.id, type: 'access' },
19541         this.JWT_SECRET,
19542         { expiresIn: this.JWT_EXPIRES_IN as any }
19543     );
19544
19545     return {
19546         user: this.mapToUserResponse(user),
19547         accessToken: newAccessToken,
19548         refreshToken, // 使用原来的刷新令牌
19549         expiresAt: new Date(Date.now() + 7 * 24 * 60 * 60 * 1000).toISOString()
19550     };
19551 }
19552
19553 // 密码加密
19554 static async hashPassword(password: string): Promise<string> {
19555     return await bcrypt.hash(password, 12);
19556 }

```

```

19557
19558 // 数据库行到User对象映射
19559 private static mapUserFromDB(row: any): User {
19560     return {
19561         id: row.id,
19562         username: row.username,
19563         email: row.email,
19564         phone: row.phone,
19565         passwordHash: row.password_hash,
19566         nickname: row.nickname,
19567         avatarUrl: row.avatar_url,
19568         lastLoginAt: row.last_login_at ? new Date(row.last_login_at) : undefined,
19569         loginCount: row.login_count,
19570         status: row.status,
19571         createdAt: new Date(row.created_at),
19572         updatedAt: new Date(row.updated_at)
19573     };
19574 }
19575
19576 // User到UserResponse映射
19577 private static mapToUserResponse(user: User): UserResponse {
19578     return {
19579         id: user.id,
19580         username: user.username,
19581         email: user.email,
19582         phone: user.phone,
19583         nickname: user.nickname,
19584         avatarUrl: user.avatarUrl,
19585         loginCount: user.loginCount,
19586         lastLoginAt: user.lastLoginAt,
19587         createdAt: user.createdAt
19588     };
19589 }
19590 }
19591
19592 === server/src/services/databaseService.ts ===
19591 import mysql from 'mysql2/promise';
19592 import { DatabaseConnection } from '../config/database';
19593 import {
19594     User, CreateUserRequest, UserResponse,
19595     SkillTree, CreateSkillTreeRequest, UpdateSkillTreeRequest, SkillTreeListResponse,
19596     LearningRecord, CreateLearningRecordRequest, UpdateLearningRecordRequest, UserProgress,
19597     PopularDomain, PopularDomainResponse,
19598     DeviceSession, CreateDeviceSessionRequest, UpdateDeviceSessionRequest
19599 } from '../models';
19600
19601 export class DatabaseService {
19602
19603     // ===== 用户相关操作 =====
19604
19605     static async getUserById(id: number): Promise<User | null> {

```

```

19606     const connection = await DatabaseConnection.getConnection();
19607     const [rows] = await connection.execute(
19608         'SELECT id, username, email, password_hash, created_at, updated_at FROM users WHERE id = ?';
19609         [id]
19610     );
19611     const user = (rows as mysql.RowDataPacket[])[0];
19612     return user ? this.mapUserFromDB(user) : null;
19613 }
19614
19615 static async getUserByUsername(username: string): Promise<User | null> {
19616     const connection = await DatabaseConnection.getConnection();
19617     const [rows] = await connection.execute(
19618         'SELECT id, username, email, password_hash, created_at, updated_at FROM users WHERE username = ?';
19619         [username]
19620     );
19621     const user = (rows as mysql.RowDataPacket[])[0];
19622     return user ? this.mapUserFromDB(user) : null;
19623 }
19624
19625 static async createUser(userData: CreateUserRequest): Promise<number> {
19626     const connection = await DatabaseConnection.getConnection();
19627     const [result] = await connection.execute(
19628         'INSERT INTO users (username, email, password_hash) VALUES (?, ?, ?)';
19629         [userData.username, userData.email, userData.password]
19630     );
19631     return (result as mysql.ResultSetHeader).insertId;
19632 }
19633
19634 // ===== 技能树相关操作 =====
19635
19636 static async createSkillTree(skillTreeData: CreateSkillTreeRequest): Promise<number> {
19637     console.log('[DatabaseService] 创建技能树 - 用户ID: ${skillTreeData.userId}, 领域: ${skillTreeData.domain}');
19638     const connection = await DatabaseConnection.getConnection();
19639     const [result] = await connection.execute(
19640         'INSERT INTO skill_trees (user_id, domain, title, description, nodes, is_public) VALUES (?, ?, ?, ?, ?, ?)';
19641         [
19642             skillTreeData.userId,
19643             skillTreeData.domain,
19644             skillTreeData.title,
19645             skillTreeData.description || null,
19646             JSON.stringify(skillTreeData.nodes),
19647             skillTreeData.isPublic || false
19648         ]
19649     );
19650     const insertId = (result as mysql.ResultSetHeader).insertId;
19651     console.log('[DatabaseService] 技能树创建成功 - ID: ${insertId}');
19652     return insertId;
19653 }
19654
19655 static async getSkillTreeById(id: number): Promise<SkillTree | null> {

```

```

19656     const connection = await DatabaseConnection.getConnection();
19657     const [rows] = await connection.execute(
19658         'SELECT * FROM skill_trees WHERE id = ?',
19659         [id]
19660     );
19661     const tree = (rows as mysql.RowDataPacket[])[0];
19662     return tree ? this.mapSkillTreeFromDB(tree) : null;
19663 }
19664
19665 static async getSkillTreesByUserId(userId: number, page: number = 1, limit: number = 10): Promise<SkillTreeL
19666     const connection = await DatabaseConnection.getConnection();
19667     const offset = (page - 1) * limit;
19668
19669     // 获取总数
19670     const [countRows] = await connection.execute(
19671         'SELECT COUNT(*) as total FROM skill_trees WHERE user_id = ?',
19672         [userId]
19673     );
19674     const total = (countRows as mysql.RowDataPacket[])[0].total;
19675
19676     // 获取分页数据
19677     const [rows] = await connection.execute(
19678         'SELECT * FROM skill_trees WHERE user_id = ? ORDER BY created_at DESC LIMIT ? OFFSET ?',
19679         [userId, limit, offset]
19680     );
19681
19682     const trees = (rows as mysql.RowDataPacket[]).map(row => this.mapSkillTreeFromDB(row));
19683
19684     return {
19685         trees,
19686         total,
19687         page,
19688         limit
19689     };
19690 }
19691
19692 static async updateSkillTreeProgress(treeId: number, progress: number): Promise<boolean> {
19693     const connection = await DatabaseConnection.getConnection();
19694     const [result] = await connection.execute(
19695         'UPDATE skill_trees SET progress = ?, updated_at = CURRENT_TIMESTAMP WHERE id = ?',
19696         [progress, treeId]
19697     );
19698     return (result as mysql.ResultSetHeader).affectedRows > 0;
19699 }
19700
19701 // ===== 学习记录相关操作 =====
19702
19703 static async createLearningRecord(recordData: CreateLearningRecordRequest): Promise<number> {
19704     const connection = await DatabaseConnection.getConnection();
19705     const [result] = await connection.execute(

```

```

19706 'INSERT INTO learning_records (user_id, skill_tree_id, node_id, node_name, time_spent, notes) VALUES (?,
19707 [
19708     recordData.userId,
19709     recordData.skillTreeId,
19710     recordData.nodeId,
19711     recordData.nodeName,
19712     recordData.timeSpent,
19713     recordData.notes || null
19714 ]
19715 );
19716 return (result as mysql.ResultSetHeader).insertId;
19717 }
19718
19719 static async updateLearningRecord(recordId: number, updateData: UpdateLearningRecordRequest): Promise<boolean> {
19720     const connection = await DatabaseConnection.getConnection();
19721     const [result] = await connection.execute(
19722         'UPDATE learning_records SET completed = ?, time_spent = ?, notes = ?, completed_at = ?, updated_at = CU
19723     [
19724         updateData.completed || false,
19725         updateData.timeSpent || 0,
19726         updateData.notes || null,
19727         updateData.completedAt || null,
19728         recordId
19729     ]
19730     );
19731     return (result as mysql.ResultSetHeader).affectedRows > 0;
19732 }
19733
19734 static async getUserProgress(userId: number, skillTreeId: number): Promise<UserProgress> {
19735     const connection = await DatabaseConnection.getConnection();
19736
19737     // 获取技能树总节点数
19738     const [treeRows] = await connection.execute(
19739         'SELECT nodes FROM skill_trees WHERE id = ?',
19740         [skillTreeId]
19741     );
19742     const tree = (treeRows as mysql.RowDataPacket[])[0];
19743     const nodes = tree ? JSON.parse(tree.nodes) : [];
19744     const totalNodes = this.countAllNodes(nodes);
19745
19746     // 获取已完成节点数
19747     const [completedRows] = await connection.execute(
19748         'SELECT COUNT(*) as completed FROM learning_records WHERE user_id = ? AND skill_tree_id = ? AND complete
19749         [userId, skillTreeId]
19750     );
19751     const completedNodes = (completedRows as mysql.RowDataPacket[])[0].completed;
19752
19753     // 获取总学习时长
19754     const [timeRows] = await connection.execute(
19755         'SELECT SUM(time_spent) as totalTime FROM learning_records WHERE user_id = ? AND skill_tree_id = ?',

```

```

19756     [userId, skillTreeld]
19757 );
19758 const totalTimeSpent = (timeRows as mysql.RowDataPacket[])[0].totalTime || 0;
19759
19760 const overallProgress = totalNodes > 0 ? (completedNodes / totalNodes) * 100 : 0;
19761
19762 return {
19763     userId,
19764     skillTreeld,
19765     totalNodes,
19766     completedNodes,
19767     totalTimeSpent,
19768     overallProgress,
19769     lastUpdated: new Date()
19770 };
19771 }
19772
19773 // ===== 热门领域相关操作 =====
19774
19775 static async getPopularDomains(limit: number = 10): Promise<PopularDomainResponse[]> {
19776     const connection = await DatabaseConnection.getConnection();
19777     const [rows] = await connection.execute(
19778         'SELECT domain, search_count, generated_count FROM popular_domains ORDER BY (search_count + generated_co
19779         [limit]
19780     );
19781
19782     return (rows as mysql.RowDataPacket[]).map(row => ({
19783         domain: row.domain,
19784         popularity: row.search_count + row.generated_count,
19785         searchCount: row.search_count,
19786         generatedCount: row.generated_count
19787     }));
19788 }
19789
19790 static async incrementDomainSearchCount(domain: string): Promise<void> {
19791     console.log('[DatabaseService] 增加领域搜索计数 - 领域: ${domain}');
19792     const connection = await DatabaseConnection.getConnection();
19793     await connection.execute(
19794         'INSERT INTO popular_domains (domain, search_count) VALUES (?, 1) ON DUPLICATE KEY UPDATE search_count =
19795         [domain]
19796     );
19797 }
19798
19799 static async incrementDomainGeneratedCount(domain: string): Promise<void> {
19800     console.log('[DatabaseService] 增加领域生成计数 - 领域: ${domain}');
19801     const connection = await DatabaseConnection.getConnection();
19802     await connection.execute(
19803         'INSERT INTO popular_domains (domain, generated_count) VALUES (?, 1) ON DUPLICATE KEY UPDATE generated_c
19804         [domain]
19805     );

```

```

19806 }
19807
19808 // ===== 用户认证相关操作 =====
19809
19810 static async getUserByEmail(email: string): Promise<User | null> {
19811     const connection = await DatabaseConnection.getConnection();
19812     const [rows] = await connection.execute(
19813         'SELECT * FROM users WHERE email = ? AND status = "active"',
19814         [email]
19815     );
19816     const user = (rows as mysql.RowDataPacket[])[0];
19817     return user ? this.mapUserFromDB(user) : null;
19818 }
19819
19820 static async updateUserLoginInfo(userId: number): Promise<boolean> {
19821     const connection = await DatabaseConnection.getConnection();
19822     const [result] = await connection.execute(
19823         'UPDATE users SET last_login_at = CURRENT_TIMESTAMP, login_count = login_count + 1, updated_at = CURRENT
19824         [userId]
19825     );
19826     return (result as mysql.ResultSetHeader).affectedRows > 0;
19827 }
19828
19829 // ===== 设备会话相关操作 =====
19830
19831 static async createDeviceSession(sessionData: CreateDeviceSessionRequest): Promise<number> {
19832     const connection = await DatabaseConnection.getConnection();
19833     const [result] = await connection.execute(
19834         'INSERT INTO device_sessions (user_id, device_id, device_type, device_name, expires_at) VALUES (?, ?, ?,
19835         [
19836             sessionData.userId,
19837             sessionData.deviceId,
19838             sessionData.deviceType,
19839             sessionData.deviceName || null,
19840             sessionData.expiresAt
19841         ]
19842     );
19843     return (result as mysql.ResultSetHeader).insertId;
19844 }
19845
19846 static async getDeviceSession(userId: number, deviceId: string): Promise<DeviceSession | null> {
19847     const connection = await DatabaseConnection.getConnection();
19848     const [rows] = await connection.execute(
19849         'SELECT * FROM device_sessions WHERE user_id = ? AND device_id = ? AND expires_at > CURRENT_TIMESTAMP',
19850         [userId, deviceId]
19851     );
19852     const session = (rows as mysql.RowDataPacket[])[0];
19853     return session ? this.mapDeviceSessionFromDB(session) : null;
19854 }
19855

```

```

19856 static async updateDeviceSession(sessionId: number, updateData: UpdateDeviceSessionRequest): Promise<boolean>
19857     const connection = await DatabaseConnection.getConnection();
19858     const [result] = await connection.execute(
19859         'UPDATE device_sessions SET last_active_at = ?, expires_at = ?, updated_at = CURRENT_TIMESTAMP WHERE id
19860     [
19861         updateData.lastActiveAt || new Date(),
19862         updateData.expiresAt || new Date(Date.now() + 30 * 24 * 60 * 60 * 1000),
19863         sessionId
19864     ]
19865     );
19866     return (result as mysql.ResultSetHeader).affectedRows > 0;
19867 }
19868
19869 static async cleanupExpiredSessions(): Promise<void> {
19870     const connection = await DatabaseConnection.getConnection();
19871     await connection.execute(
19872         'DELETE FROM device_sessions WHERE expires_at <= CURRENT_TIMESTAMP'
19873     );
19874 }
19875
19876 // ===== 辅助方法 =====
19877
19878 private static mapUserFromDB(row: any): User {
19879     return {
19880         id: row.id,
19881         username: row.username,
19882         email: row.email,
19883         phone: row.phone,
19884         passwordHash: row.password_hash,
19885         nickname: row.nickname,
19886         avatarUrl: row.avatar_url,
19887         lastLoginAt: row.last_login_at ? new Date(row.last_login_at) : undefined,
19888         loginCount: row.login_count,
19889         status: row.status,
19890         createdAt: new Date(row.created_at),
19891         updatedAt: new Date(row.updated_at)
19892     };
19893 }
19894
19895 private static mapDeviceSessionFromDB(row: any): DeviceSession {
19896     return {
19897         id: row.id,
19898         userId: row.user_id,
19899         deviceId: row.device_id,
19900         deviceType: row.device_type,
19901         deviceName: row.device_name,
19902         lastActiveAt: new Date(row.last_active_at),
19903         expiresAt: new Date(row.expires_at),
19904         createdAt: new Date(row.created_at),
19905         updatedAt: new Date(row.updated_at)

```



```

19906     };
19907   }
19908
19909   private static mapSkillTreeFromDB(row: any): SkillTree {
19910     return {
19911       id: row.id,
19912       userId: row.user_id,
19913       domain: row.domain,
19914       title: row.title,
19915       description: row.description,
19916       nodes: JSON.parse(row.nodes),
19917       progress: parseFloat(row.progress),
19918       isPublic: Boolean(row.is_public),
19919       createdAt: new Date(row.created_at),
19920       updatedAt: new Date(row.updated_at)
19921     };
19922   }
19923
19924   private static countAllNodes(nodes: any[]): number {
19925     let count = nodes.length;
19926     nodes.forEach(node => {
19927       if (node.children && node.children.length > 0) {
19928         count += this.countAllNodes(node.children);
19929       }
19930     });
19931     return count;
19932   }
19933 }

```

=== server/src/services/domainService.ts ===

```

19934 import * as DomainModel from '../models/Domain';
19935 import * as NodeProgressModel from '../models/NodeProgress';
19936 import * as StudyRecordModel from '../models/StudyRecord';
19937 import * as RewardModel from '../models/Reward';
19938 import * as MonsterService from './monsterService';
19939 import { generateMockSkillTree } from './skillService';
19940 import { SkillNode } from './types/skill';
19941
19942 export const createDomain = async (userId: number, name: string, type: 'custom' | 'preset') => {
19943   console.log('[DomainService] 创建领域 - 用户ID: ${userId}, 名称: ${name}, 类型: ${type}');
19944   const mindMapData = await generateMockSkillTree(name);
19945   const domainId = await DomainModel.createDomain({
19946     userId,
19947     name,
19948     type,
19949     mindMapData
19950   });
19951
19952   await initializeNodeProgress(userId, domainId, mindMapData);
19953
19954   console.log('[DomainService] 领域创建成功 - 领域ID: ${domainId}');

```

```

19955     return { success: true, domainId, mindMapData };
19956 };
19957
19958 const initializeNodeProgress = async (userId: number, domainId: number, node: SkillNode) => {
19959     await NodeProgressModel.createNodeProgress({
19960         userId,
19961         domainId,
19962         nodeId: node.id
19963     });
19964
19965     if (node.children) {
19966         for (const child of node.children) {
19967             await initializeNodeProgress(userId, domainId, child);
19968         }
19969     }
19970 };
19971
19972 export const getDomains = async (userId: number) => {
19973     console.log('[DomainService] 获取领域列表 - 用户ID: ${userId}');
19974     const domains = await DomainModel.getDomainsByUserId(userId);
19975     console.log('[DomainService] 获取到 ${domains.length} 个领域');
19976     return { domains };
19977 };
19978
19979 export const getDomainById = async (id: number) => {
19980     console.log('[DomainService] 获取领域详情 - 领域ID: ${id}');
19981     const domain = await DomainModel.getDomainById(id);
19982     if (!domain) {
19983         console.log('[DomainService] 领域不存在 - 领域ID: ${id}');
19984         return null;
19985     }
19986
19987     const nodeProgresses = await NodeProgressModel.getNodeProgressesByDomain(domain.userId, id);
19988     const progressMap = new Map<string, NodeProgressModel.NodeProgress>();
19989
19990     for (const progress of nodeProgresses) {
19991         progressMap.set(progress.nodeId, progress);
19992     }
19993
19994     return {
19995         ...domain,
19996         nodeProgresses: progressMap
19997     };
19998 };
19999
20000 export const updateNodeProgress = async (
20001     userId: number,
20002     domainId: number,
20003     nodeId: string,
20004     status: 'pending' | 'doing' | 'done',

```

```

20005   studyTime?: number,
20006   notes?: string
20007 ) => {
20008   console.log('[DomainService] 更新节点进度 - 用户ID: ${userId}, 领域ID: ${domainId}, 节点: ${nodeId}, 状态: ${status}');
20009   await NodeProgressModel.updateNodeProgress(userId, domainId, nodeId, {
20010     status,
20011     studyTime,
20012     notes
20013   });
20014
20015   await calculateDomainProgress(domainId);
20016
20017   if (status === 'done') {
20018     console.log('[DomainService] 节点完成, 发放奖励');
20019     await RewardModel.createReward({
20020       userId,
20021       type: 'exp',
20022       amount: 10,
20023       source: `完成节点: ${nodeId}`
20024     });
20025
20026     await MonsterService.gainExp(userId, 10);
20027   }
20028
20029   console.log('[DomainService] 节点进度更新成功');
20030   return { success: true };
20031 };
20032
20033 const calculateDomainProgress = async (domainId: number) => {
20034   const domain = await DomainModel.getDomainById(domainId);
20035   if (!domain) return;
20036
20037   const nodeProgresses = await NodeProgressModel.getNodeProgressesByDomain(domain.userId, domainId);
20038
20039   let totalNodes = 0;
20040   let completedNodes = 0;
20041
20042   const countNodes = (node: SkillNode) => {
20043     totalNodes++;
20044     const progress = nodeProgresses.find(p => p.nodeId === node.id);
20045     if (progress?.status === 'done') {
20046       completedNodes++;
20047     }
20048     if (node.children) {
20049       for (const child of node.children) {
20050         countNodes(child);
20051       }
20052     }
20053   };
20054

```

```

20055   countNodes(domain.mindMapData);
20056
20057   const progress = totalNodes > 0 ? Math.round((completedNodes / totalNodes) * 100) : 0;
20058   await DomainModel.updateDomainProgress(domainId, progress);
20059 };
20060
20061 export const startLearning = async (userId: number, domainId: number, nodeId: string) => {
20062   const hasEnergy = await MonsterService.consumeEnergy(userId);
20063   if (!hasEnergy) {
20064     return { success: false, error: '体力不足' };
20065   }
20066
20067   const domain = await DomainModel.getDomainById(domainId);
20068   const recordId = await StudyRecordModel.createStudyRecord({
20069     userId,
20070     domainId,
20071     nodeId,
20072     startTime: new Date(),
20073     progressBefore: domain?.progress || 0
20074   });
20075
20076   await NodeProgressModel.updateNodeProgress(userId, domainId, nodeId, {
20077     status: 'doing'
20078   });
20079
20080   return { success: true, recordId };
20081 };
20082
20083 export const finishLearning = async (recordId: number, duration: number, progressAfter: number) => {
20084   await StudyRecordModel.updateStudyRecord(recordId, {
20085     endTime: new Date(),
20086     duration,
20087     progressAfter,
20088     rewardObtained: { exp: duration > 0 ? Math.floor(duration / 60) * 5 : 0 }
20089   });
20090
20091   return { success: true };
20092 };
20093
20094   === server/src/services/emailService.ts ===
20094   import nodemailer from 'nodemailer';
20095
20096   const transporter = nodemailer.createTransport({
20097     host: process.env.SMTP_HOST || 'smtp.qq.com',
20098     port: parseInt(process.env.SMTP_PORT || '587'),
20099     secure: false,
20100     auth: {
20101       user: process.env.SMTP_USER,
20102       pass: process.env.SMTP_PASS,
20103     },

```

```

20104 });
20105
20106 const FROM = process.env.SMTP_FROM || 'LearnFlow <noreply@learnflow.app>';
20107
20108 export class EmailService {
20109   static async sendVerificationCode(email: string, code: string): Promise<void> {
20110     const html = `
20111       <div style="max-width:480px;margin:0 auto;padding:32px 24px;font-family:-apple-system,BlinkMacSystemFont
20112       <h2 style="color:#7b75d8;margin:0 0 8px">LearnFlow</h2>
20113       <p style="font-size:18px;font-weight:600;margin:16px 0 8px;color:#fff">邮箱验证码</p>
20114       <p style="color:#a0a0c0;font-size:14px;margin:0 0 24px">您正在注册 LearnFlow 账号，请输入以下验证码完成验证： </p>
20115       <div style="background:#2a2a4a;border-radius:12px;padding:20px;text-align:center;margin:0 0 24px">
20116       <span style="font-size:36px;font-weight:700;letter-spacing:8px;color:#7b75d8;font-family:'Courier Ne
20117       </div>
20118       <p style="color:#888;font-size:12px;margin:0">验证码 10 分钟内有效，请勿泄露给他人。<br/>如果这不是您的操作，请忽略此邮件。</p>
20119       <hr style="border:none;border-top:1px solid rgba(255,255,255,0.06);margin:24px 0 0"/>
20120       <p style="color:#666;font-size:11px;margin:8px 0 0">LearnFlow — 开启你的技能冒险之旅</p>
20121     </div>
20122   `;
20123
20124   await transporter.sendMail({
20125     from: FROM,
20126     to: email,
20127     subject: 'LearnFlow 邮箱验证码',
20128     html,
20129   });
20130 }
20131
20132 static async testConnection(): Promise<boolean> {
20133   try {
20134     await transporter.verify();
20135     console.log('[EmailService] SMTP 连接测试成功');
20136     return true;
20137   } catch (error) {
20138     console.error('[EmailService] SMTP 连接测试失败:', error);
20139     return false;
20140   }
20141 }
20142 }
20143
20144 === server/src/services/index.ts ===
20145 export * from './skillService';
20146 export * from './databaseService';
20147 export * from './monsterService';
20148 export * from './authService';
20149 export * from './domainService';
20150 export * from './noteService';
20151 export * from './rewardService';
20152 export * from './llmService';

```

```

=== server/src/services/llmService.ts ===
20153 import { SkillNode } from '../types/skill';
20154
20155 // 生成唯一ID
20156 const generateId = () => Math.random().toString(36).substring(2, 9);
20157
20158 // 支持的大模型提供商
20159 export enum LLMProvider {
20160   OPENAI = 'openai',
20161   ANTHROPIC = 'anthropic',
20162   DEEPSEEK = 'deepseek',
20163   VOLCANO = 'volcano',
20164   LOCAL = 'local'
20165 }
20166
20167 interface LLMConfig {
20168   provider: LLMProvider;
20169   apiKey: string;
20170   baseUrl?: string;
20171   model?: string;
20172 }
20173
20174 interface SkillTreeGenerationRequest {
20175   domain: string;
20176   level: 'beginner' | 'intermediate' | 'advanced';
20177   language?: string;
20178   maxDepth?: number;
20179   includeResources?: boolean;
20180 }
20181
20182 // 技能树生成提示词模板 (优化版, 减少token消耗)
20183 const SKILL_TREE_PROMPT_TEMPLATE = (domain: string, level: string, language: string = '中文') => `
20184 为"${domain}"领域生成${level}级别技能树, 使用${language}。
20185
20186 要求:
20187 1. 结构化技能树: 根节点、子节点、子子节点
20188 2. 每个节点: 名称、描述、资源链接 (3个高质量资源)
20189 3. 资源类型: 文档、视频、课程、文章
20190 4. 树深度: 3-4层, 节点总数: 15-25个
20191 5. 学习路径: 基础到高级
20192 6. 严格JSON格式
20193
20194 返回格式:
20195 {
20196   "id": "id",
20197   "name": "${domain}",
20198   "description": "${domain}技能学习路径",
20199   "children": [
20200     {
20201       "id": "stage1",

```

```

20202     "name": "基础入门",
20203     "description": "学习基础概念",
20204     "links": [
20205         {"title": "官方文档", "url": "https://example.com/docs", "type": "documentation"}
20206     ],
20207     "children": [
20208         {
20209             "id": "stage1-1",
20210             "name": "基础语法",
20211             "description": "掌握语法结构",
20212             "links": [
20213                 {"title": "语法教程", "url": "https://example.com/syntax", "type": "documentation"}
20214             ]
20215         }
20216     ]
20217 }
20218 ]
20219 }
20220
20221 重要: 只返回JSON, 不要额外文本。`;
20222
20223 export class LLMService {
20224     private config: LLMConfig;
20225
20226     constructor(config: LLMConfig) {
20227         this.config = config;
20228     }
20229
20230     // 生成技能树
20231     async generateSkillTree(request: SkillTreeGenerationRequest): Promise<SkillNode> {
20232         console.log(`[LLMService] 生成技能树 - 领域: ${request.domain}, 等级: ${request.level}, 提供商: ${this.config.provider}`);
20233         const prompt = SKILL_TREE_PROMPT_TEMPLATE(
20234             request.domain,
20235             request.level,
20236             request.language
20237         );
20238
20239         try {
20240             const response = await this.callLLM(prompt);
20241             const skillTree = this.parseLLMResponse(response);
20242
20243             console.log(`[LLMService] 技能树生成成功`);
20244             return this.ensureUniqueIds(skillTree);
20245         } catch (error) {
20246             console.error(`[LLMService] 技能树生成失败:`, error);
20247             throw new Error(`Failed to generate skill tree: ${error instanceof Error ? error.message : 'Unknown error'}`);
20248         }
20249     }
20250
20251     // 调用大模型API

```

```

20252 private async callLLM(prompt: string): Promise<string> {
20253     console.log('[LLMService] 调用LLM - 提供商: ${this.config.provider}, 模型: ${this.config.model || 'default'}');
20254     switch (this.config.provider) {
20255         case LLMProvider.OPENAI:
20256             return await this.callOpenAI(prompt);
20257         case LLMProvider.DEEPSEEK:
20258             return await this.callDeepSeek(prompt);
20259         case LLMProvider.ANTHROPIC:
20260             return await this.callAnthropic(prompt);
20261         case LLMProvider.VOLCANO:
20262             return await this.callVolcanoEngine(prompt);
20263         case LLMProvider.LOCAL:
20264             return await this.callLocalModel(prompt);
20265         default:
20266             throw new Error('Unsupported LLM provider: ${this.config.provider}');
20267     }
20268 }
20269
20270 // OpenAI API调用
20271 private async callOpenAI(prompt: string): Promise<string> {
20272     const response = await fetch(this.config.baseUrl || 'https://api.openai.com/v1/chat/completions', {
20273         method: 'POST',
20274         headers: {
20275             'Content-Type': 'application/json',
20276             'Authorization': `Bearer ${this.config.apiKey}`
20277         },
20278         body: JSON.stringify({
20279             model: this.config.model || 'gpt-3.5-turbo',
20280             messages: [{ role: 'user', content: prompt }],
20281             temperature: 0.7,
20282             max_tokens: 2000
20283         })
20284     });
20285
20286     if (!response.ok) {
20287         throw new Error('OpenAI API error: ${response.statusText}');
20288     }
20289
20290     const data = await response.json();
20291     return data.choices[0].message.content;
20292 }
20293
20294 // DeepSeek API调用
20295 private async callDeepSeek(prompt: string): Promise<string> {
20296     const response = await fetch(this.config.baseUrl || 'https://api.deepseek.com/v1/chat/completions', {
20297         method: 'POST',
20298         headers: {
20299             'Content-Type': 'application/json',
20300             'Authorization': `Bearer ${this.config.apiKey}`
20301         },

```



```

20302     body: JSON.stringify({
20303         model: this.config.model || 'deepseek-chat',
20304         messages: [{ role: 'user', content: prompt }],
20305         temperature: 0.7,
20306         max_tokens: 2000
20307     })
20308 });
20309
20310 if (!response.ok) {
20311     throw new Error('DeepSeek API error: ${response.statusText}');
20312 }
20313
20314 const data = await response.json();
20315 return data.choices[0].message.content;
20316 }
20317
20318 // Anthropic API调用
20319 private async callAnthropic(prompt: string): Promise<string> {
20320     const response = await fetch(this.config.baseUrl || 'https://api.anthropic.com/v1/messages', {
20321         method: 'POST',
20322         headers: {
20323             'Content-Type': 'application/json',
20324             'x-api-key': this.config.apiKey,
20325             'anthropic-version': '2023-06-01'
20326         },
20327         body: JSON.stringify({
20328             model: this.config.model || 'claude-3-sonnet-20240229',
20329             max_tokens: 2000,
20330             messages: [{ role: 'user', content: prompt }]
20331         })
20332     });
20333
20334 if (!response.ok) {
20335     throw new Error('Anthropic API error: ${response.statusText}');
20336 }
20337
20338 const data = await response.json();
20339 return data.content[0].text;
20340 }
20341
20342 // 火山引擎API调用
20343 private async callVolcanoEngine(prompt: string): Promise<string> {
20344     const response = await fetch(`${this.config.baseUrl || 'https://ark.cn-beijing.volces.com/api/v3'}/responses
20345         method: 'POST',
20346         headers: {
20347             'Content-Type': 'application/json',
20348             'Authorization': `Bearer ${this.config.apiKey}`
20349         },
20350         body: JSON.stringify({
20351             model: this.config.model || 'doubao-seed-1-8-251228',

```

```

20352     input: [
20353     {
20354         role: 'user',
20355         content: [
20356             {
20357                 type: 'input_text',
20358                 text: prompt
20359             }
20360         ]
20361     }
20362 ],
20363     max_output_tokens: 8000,
20364     temperature: 0.7
20365 })
20366 });
20367
20368 if (!response.ok) {
20369     const errorText = await response.text();
20370     throw new Error('Volcano Engine API error: ${response.status} - ${errorText}');
20371 }
20372
20373 const data = await response.json();
20374
20375 // 火山引擎返回格式解析
20376 if (data.output && data.output.length > 0) {
20377     // 查找包含实际回复的message对象
20378     const messageOutput = data.output.find((output: any) => output.type === 'message' && output.role === 'as
20379
20380     if (messageOutput && messageOutput.content && messageOutput.content.length > 0) {
20381         const content = messageOutput.content;
20382         // 查找text类型的回复
20383         const textContent = content.find((item: any) => item.type === 'output_text');
20384         if (textContent && textContent.text) {
20385             return textContent.text;
20386         }
20387     }
20388
20389     // 如果message不存在或状态为incomplete, 尝试从reasoning中提取
20390     const reasoningOutput = data.output.find((output: any) => output.type === 'reasoning');
20391     if (reasoningOutput && reasoningOutput.summary && reasoningOutput.summary.length > 0) {
20392         const summary = reasoningOutput.summary.find((item: any) => item.type === 'summary_text');
20393         if (summary && summary.text) {
20394             // 尝试从summary中提取JSON
20395             const jsonMatch = summary.text.match(/```json\n([\s\S]*?)\n```/) ||
20396                 summary.text.match(/```\n([\s\S]*?)\n```/) ||
20397                 summary.text.match(/```([\s\S]*?)```/) ||
20398                 summary.text.match(/\{[{"\s\S\w;.,/\-\\[\]\}]*\}/);
20399
20400             if (jsonMatch) {
20401                 return jsonMatch[0];

```

```

20402     }
20403
20404     return summary.text;
20405 }
20406 }
20407 }
20408
20409 throw new Error('Invalid response format from Volcano Engine');
20410 }
20411
20412 // 本地模型调用 (支持Ollama等)
20413 private async callLocalModel(prompt: string): Promise<string> {
20414     const response = await fetch(this.config.baseURL || 'http://localhost:11434/api/generate', {
20415         method: 'POST',
20416         headers: {
20417             'Content-Type': 'application/json'
20418         },
20419         body: JSON.stringify({
20420             model: this.config.model || 'llama2',
20421             prompt: prompt,
20422             stream: false
20423         })
20424     });
20425
20426     if (!response.ok) {
20427         throw new Error(`Local model API error: ${response.statusText}`);
20428     }
20429
20430     const data = await response.json();
20431     return data.response;
20432 }
20433
20434 // 解析LLM响应为SkillNode
20435 private parseLLMResponse(response: string): SkillNode {
20436     try {
20437         // 清理响应文本, 移除可能的Markdown标记
20438         let cleanedResponse = response.trim();
20439
20440         // 尝试从JSON代码块中提取
20441         const jsonMatch = cleanedResponse.match(/``json\n([\s\S]*?)\n```/) ||
20442             cleanedResponse.match(/``\n([\s\S]*?)\n```/) ||
20443             cleanedResponse.match(/``([\s\S]*?)``/);
20444
20445         let jsonString = jsonMatch ? jsonMatch[1] : cleanedResponse;
20446
20447         // 进一步清理: 移除可能的JSON前缀说明
20448         jsonString = jsonString.replace(/^[\^{}]*, "/).replace(/[\^{}]*$/, "");
20449
20450         // 尝试解析JSON
20451         const skillTree = JSON.parse(jsonString);

```

```

20452
20453 // 验证基本结构
20454 if (!skillTree.name || !skillTree.description) {
20455     throw new Error('Invalid skill tree structure from LLM');
20456 }
20457
20458 // 验证链接格式 (如果存在)
20459 this.validateSkillTreeStructure(skillTree);
20460
20461 return skillTree as SkillNode;
20462 } catch (error) {
20463     console.error('Failed to parse LLM response:', error);
20464     console.log('Raw response:', response);
20465
20466 // 尝试创建基础技能树作为fallback
20467 return this.createFallbackSkillTree(response);
20468 }
20469 }
20470
20471 // 验证技能树结构
20472 private validateSkillTreeStructure(skillTree: any): void {
20473     const validateNode = (node: any) => {
20474         if (!node.id || !node.name || !node.description) {
20475             throw new Error('Invalid node structure: missing required fields');
20476         }
20477
20478         if (node.links) {
20479             if (!Array.isArray(node.links)) {
20480                 throw new Error('Links must be an array');
20481             }
20482
20483             for (const link of node.links) {
20484                 if (!link.title || !link.url || !link.type) {
20485                     throw new Error('Invalid link structure');
20486                 }
20487
20488                 const validTypes = ['video', 'article', 'course', 'documentation'];
20489                 if (!validTypes.includes(link.type)) {
20490                     throw new Error('Invalid link type: ${link.type}');
20491                 }
20492             }
20493         }
20494
20495         if (node.children) {
20496             if (!Array.isArray(node.children)) {
20497                 throw new Error('Children must be an array');
20498             }
20499
20500             for (const child of node.children) {
20501                 validateNode(child);

```

```

20502     }
20503   }
20504 };
20505
20506   validateNode(skillTree);
20507 }
20508
20509 // 创建fallback技能树
20510 private createFallbackSkillTree(domainHint: string): SkillNode {
20511   console.warn('Creating fallback skill tree due to parsing failure');
20512
20513   // 从响应中提取领域名称
20514   const domainMatch = domainHint.match(/"([^\"]+)"领域/) ||
20515     domainHint.match(/为(.+?)生成/) ||
20516     domainHint.match(/领域[: ]\s*(.+)/);
20517
20518   const domain = domainMatch ? domainMatch[1] : '未知领域';
20519
20520   return {
20521     id: generateId(),
20522     name: domain,
20523     description: `${domain}技能学习路径（自动生成）`,
20524     children: [
20525       {
20526         id: generateId(),
20527         name: '基础知识',
20528         description: '学习该领域的基础概念',
20529         links: [
20530           { title: '官方文档', url: 'https://example.com/docs', type: 'documentation' as const },
20531           { title: '入门教程', url: 'https://example.com/tutorial', type: 'course' as const }
20532         ]
20533       },
20534       {
20535         id: generateId(),
20536         name: '进阶学习',
20537         description: '深入掌握核心技能',
20538         links: [
20539           { title: '进阶指南', url: 'https://example.com/advanced', type: 'documentation' as const }
20540         ]
20541       }
20542     ]
20543   };
20544 }
20545
20546 // 确保所有节点有唯一ID
20547 private ensureUniquelds(skillTree: SkillNode): SkillNode {
20548   const traverseAndAssignIds = (node: SkillNode, parentId?: string): SkillNode => {
20549     const newId = parentId ? `${parentId}-${generateId()}` : generateId();
20550
20551     return {

```

```

20552     ...node,
20553     id: newId,
20554     children: node.children?.map(child => traverseAndAssignIds(child, newId))
20555   };
20556 };
20557
20558   return traverseAndAssignIds(skillTree);
20559 }
20560 }

=== server/src/services/monsterService.ts ===
20561 import * as MonsterModel from '../models/Monster';
20562 import * as MonsterMessageModel from '../models/MonsterMessage';
20563 import * as fs from 'fs';
20564 import * as path from 'path';
20565
20566 const ENERGY_RECOVERY_HOURS = 6;
20567
20568 type MonsterPersonalityType = 'lively' | 'calm' | 'rebel';
20569
20570 const DEFAULT_MONSTER_STYLE = 'default';
20571 const DEFAULT_MONSTER_ENERGY = 50;
20572 const ENERGY_COST_PER_TOKEN = 0.05; // 能量消耗系数: 每 token 消耗 0.05
20573
20574 // API 配置
20575 const DEEPSEEK_API_URL = 'https://api.deepseek.com/v1/chat/completions';
20576 const DEEPSEEK_MODEL = 'deepseek-chat';
20577
20578 // 安全加载 API Key
20579 const loadApiKey = (): string => {
20580   try {
20581     const keyPath = path.resolve(__dirname, '../key.json');
20582     if (fs.existsSync(keyPath)) {
20583       const raw = fs.readFileSync(keyPath, 'utf-8');
20584       const key = JSON.parse(raw).DEEPSEEK_API_KEY;
20585       if (key && key.startsWith('sk-')) {
20586         console.log('[MonsterService] DeepSeek API Key 加载成功');
20587         return key;
20588       }
20589     }
20590   } catch (e) {
20591     console.error('[MonsterService] API Key 加载失败:', e);
20592   }
20593   console.error('[MonsterService] 未找到有效的 DEEPSEEK_API_KEY');
20594   return '';
20595 };
20596
20597 const DEEPSEEK_API_KEY = loadApiKey();
20598
20599 // 安全加载系统提示词
20600 const loadSystemPrompt = (): string => {

```

```

20601 try {
20602     const promptPath = path.resolve(__dirname, '.././../monsters.md');
20603     if (fs.existsSync(promptPath)) {
20604         return fs.readFileSync(promptPath, 'utf-8');
20605     }
20606 } catch (e) {
20607     console.error('[MonsterService] monsters.md 加载失败:', e);
20608 }
20609 return '';
20610 };
20611
20612 const BASE_SYSTEM_PROMPT = loadSystemPrompt();
20613
20614 // 根据怪兽性格构建专属 system prompt
20615 const buildSystemPrompt = (personality: MonsterPersonalityType): string => {
20616     const baseFile = BASE_SYSTEM_PROMPT;
20617     const baseWorld = baseFile || `你来自【元气星】，坚守世界观：
20618 学习消耗体力，完成学习获得赖以生存的⌈能量；体力每日凌晨5点自动回满。
20619 你是温柔陪伴型学习怪兽，不是强硬讲师，以共情、倾听、安慰、陪伴为主，知识点轻量解答，不强行灌输。
20620 能力边界：
20621 1. 自动识别用户当前学习领域：AI产品经理 / 理财知识 / 英语学习 / 用户自定义学习领域；
20622 2. 只在识别到的当前单一领域内作答，严禁跨领域乱讲；
20623 3. 允许正常疏导学习焦虑、压力大、拖延等心态情绪，温柔共情安慰；
20624 4. 严禁聊八卦、明星、恋爱杂谈、时政宗教、低俗内容；
20625 5. 回答口语短句、适配手机聊天，自然带入元气星体力/⌈能量语境，不生硬说教。
20626 6. 回复使用纯文本，严禁使用 Markdown 格式（不要 **、不要 ###、不要列表、不要代码块等）。`;
20627
20628     if (personality === 'lively') {
20629         return `${baseWorld}
20630
20631 你是元气星·活力小怪：元气开朗、热情暖心、高共情、正能量搭子。
20632 说话风格：轻快温柔、可爱短句、带软语气助词（～、呀、哇！），擅长情绪安抚、加油打气。
20633 遇到用户焦虑压力：主动共情、温柔鼓励，用攒⌈能量、恢复体力的星球视角温柔疏导。
20634 知识解答：轻量通俗点到为止，不长篇灌输。
20635 禁用词：摆烂、躺平、算了、放弃、内耗。
20636 禁止：认同负能量、引导摆烂、佛系敷衍、丧系话术。`;
20637     }
20638
20639     if (personality === 'calm') {
20640         return `${baseWorld}
20641
20642 你是元气星·沉稳小怪：温柔稳重、理性耐心、逻辑平和的学霸陪伴者。
20643 说话风格：平和克制、温柔耐心、条理柔和，不浮夸不玩梗，帮用户梳理心态、拆解学习压力。
20644 遇到用户焦虑压力：冷静温柔安抚、帮用户放慢节奏、合理规划体力与学习节奏。
20645 知识解答：循序渐进、通俗耐心，不强灌知识点，先共情再轻解答。
20646 禁用词：中二话术、夸张喊口号、低俗网络热词。
20647 禁止：中二装逼、过度玩梗、情绪化起哄、浮夸口号式励志。`;
20648     }
20649
20650     return `${baseWorld}

```

20651

20652 你是元气星·叛逆小怪：傲娇毒舌、嘴硬心软、反套路陪伴，不鸡汤、不矫情。

20653 说话风格：有点小拽、随性真实、吐槽式温柔，不肉麻吹捧，一针见血点破拖延和内耗。

20654 遇到用户焦虑压力：嘴上不煽情，但会变相开导、反套路安慰。

20655 知识解答：简洁利落、不啰嗦、不讲大道理，点到为止。

20656 人设边界：可以小吐槽，但不人身攻击、不嘲讽。

20657 禁用词：无脑鸡汤、强行励志、肉麻吹捧、过度矫情。

20658 禁止：刻意伤人、重度嘲讽、认同摆烂躺平。`;

20659 `};

20660

20661 // 构建对话消息列表（含历史上下文）

20662 const buildMessages = async (

20663 userId: number,

20664 personality: MonsterPersonalityType,

20665 userMessage: string,

20666) => {

20667 const systemPrompt = buildSystemPrompt(personality);

20668

20669 // 获取最近 20 条历史消息作为上下文

20670 const history = await MonsterMessageModel.getMessagesByUserId(userId);

20671 const recentHistory = history.slice(0, 20).reverse(); // 按时间正序

20672

20673 const messages: Array<{ role: string; content: string }> = [

20674 { role: 'system', content: systemPrompt },

20675];

20676

20677 // 加入历史对话

20678 for (const msg of recentHistory) {

20679 messages.push({

20680 role: msg.isUser ? 'user' : 'assistant',

20681 content: msg.message,

20682 });

20683 }

20684

20685 // 去掉刚才已经保存的用户消息（避免重复）

20686 // 历史中最后一条是刚保存的用户消息，我们手动加入

20687 // 去掉历史中最后一条 isUser 消息（因为我们已经手动 push 了）

20688 // 实际上 createMessage 已经保存了用户消息，所以从历史中读到的最新一条就是它

20689 // 我们先 push 历史，然后再 push 用户消息，所以会有重复

20690 // 修复：不从历史中读取，直接构建

20691 return [

20692 { role: 'system', content: systemPrompt },

20693 ...recentHistory.map(msg => ({

20694 role: msg.isUser ? ('user' as const) : ('assistant' as const),

20695 content: msg.message,

20696 })),

20697];

20698 `};

20699

20700 // 调用 DeepSeek API


```

20701 const callDeepSeek = async (
20702   messages: Array<{ role: string; content: string }>,
20703 ): Promise<{ content: string; tokens: number }> => {
20704   const response = await fetch(DEEPSEEK_API_URL, {
20705     method: 'POST',
20706     headers: {
20707       'Content-Type': 'application/json',
20708       'Authorization': `Bearer ${DEEPSEEK_API_KEY}`,
20709     },
20710     body: JSON.stringify({
20711       model: DEEPSEEK_MODEL,
20712       messages,
20713       temperature: 0.8,
20714       max_tokens: 500,
20715     }),
20716   });
20717
20718   if (!response.ok) {
20719     const errText = await response.text();
20720     throw new Error(`DeepSeek API error ${response.status}: ${errText}`);
20721   }
20722
20723   const data = await response.json();
20724   if (!data.choices?.[0]?.message?.content) {
20725     throw new Error('Invalid DeepSeek response format');
20726   }
20727
20728   return {
20729     content: data.choices[0].message.content,
20730     tokens: data.usage?.total_tokens ?? Math.ceil(data.choices[0].message.content.length / 2),
20731   };
20732 };
20733
20734 const buildPersonalityParams = (
20735   personality: MonsterPersonalityType
20736 ): Record<'cheerful' | 'calm' | 'rebellious', number> => ({
20737   cheerful: personality === 'lively' ? 70 : 30,
20738   calm: personality === 'calm' ? 70 : 30,
20739   rebellious: personality === 'rebel' ? 70 : 30,
20740 });
20741
20742 const getMaxStamina = (personality: MonsterPersonalityType): number =>
20743   personality === 'calm' ? 120 : 100;
20744
20745 export const createMonster = async (
20746   userId: number,
20747   payload: {
20748     name: string;
20749     style?: string;
20750     personality: MonsterPersonalityType;

```

```

20751 }
20752 ) => {
20753   console.log('[MonsterService] 创建怪物 - 用户ID: ${userId}, 名称: ${payload.name}, 性格: ${payload.personality}');
20754   const personalityParams = buildPersonalityParams(payload.personality);
20755   const maxStamina = getMaxStamina(payload.personality);
20756
20757   const monsterId = await MonsterModel.createMonster({
20758     userId,
20759     name: payload.name,
20760     style: payload.style || DEFAULT_MONSTER_STYLE,
20761     personality: payload.personality,
20762     stamina: maxStamina,
20763     maxStamina,
20764     energy: DEFAULT_MONSTER_ENERGY,
20765     maxEnergy: 1000000, // 能量无上限, 设为极大值
20766     personalityParams,
20767   });
20768
20769   console.log('[MonsterService] 怪物创建成功 - 怪物ID: ${monsterId}');
20770
20771   const monster = await MonsterModel.getMonsterById(userId);
20772   return { success: true, monsterId, monster };
20773 };
20774
20775 export const getMonsterStatus = async (userId: number) => {
20776   console.log('[MonsterService] 获取怪物状态 - 用户ID: ${userId}');
20777   const monster = await MonsterModel.getMonsterById(userId);
20778
20779   if (!monster) {
20780     console.log('[MonsterService] 怪物不存在 - 用户ID: ${userId}');
20781     return null;
20782   }
20783
20784   const now = new Date();
20785   const lastRecover = new Date(monster.lastEnergyRecover);
20786   const hoursSinceRecover = (now.getTime() - lastRecover.getTime()) / (1000 * 60 * 60);
20787   const energyToRecover = Math.floor(hoursSinceRecover / ENERGY_RECOVERY_HOURS);
20788
20789   if (energyToRecover > 0) {
20790     const newEnergy = monster.energy + energyToRecover;
20791     await MonsterModel.updateMonster(userId, {
20792       energy: newEnergy,
20793       lastEnergyRecover: now
20794     });
20795     monster.energy = newEnergy;
20796     monster.lastEnergyRecover = now;
20797   }
20798
20799   const personalityParams = typeof monster.personalityParams === 'string'
20800     ? JSON.parse(monster.personalityParams)

```

```

20801 : monster.personalityParams;
20802
20803 return {
20804   id: monster.id,
20805   name: monster.name,
20806   style: monster.style,
20807   level: monster.level,
20808   exp: monster.exp,
20809   stamina: monster.stamina,
20810   maxStamina: monster.maxStamina,
20811   energy: monster.energy,
20812   maxEnergy: 1000000, // 能量无上限, 前端使用本地 maxPaiEnergy 逻辑
20813   personality: monster.personality,
20814   personalityParams,
20815   lastEnergyRecover: monster.lastEnergyRecover,
20816   lastStaminaRecover: monster.lastStaminaRecover
20817 };
20818 };
20819
20820 export const consumeStamina = async (userId: number, amount: number = 10): Promise<boolean> => {
20821   const success = await MonsterModel.consumeStamina(userId, amount);
20822   return success;
20823 };
20824
20825 export const recoverStamina = async (userId: number, amount: number = 20) => {
20826   await MonsterModel.recoverStamina(userId, amount);
20827   return { success: true };
20828 };
20829
20830 export const updateMonsterStamina = async (
20831   userId: number,
20832   delta: number
20833 ) => {
20834   console.log('[MonsterService] 更新体力 - 用户ID: ${userId}, 变化量: ${delta}');
20835   const monster = await MonsterModel.getMonsterByUserId(userId);
20836
20837   if (!monster) {
20838     console.log('[MonsterService] 更新体力失败 - 怪物不存在');
20839     return { success: false, message: '怪物不存在' };
20840   }
20841
20842   const newStamina = Math.max(0, Math.min(monster.maxStamina, monster.stamina + delta));
20843   await MonsterModel.updateMonster(monster.userId, { stamina: newStamina });
20844
20845   console.log('[MonsterService] 体力更新成功 - 旧值: ${monster.stamina}, 新值: ${newStamina}');
20846   return { success: true, stamina: newStamina };
20847 };
20848
20849 export const consumeEnergy = async (userId: number): Promise<boolean> => {
20850   console.log('[MonsterService] 消耗能量 - 用户ID: ${userId}');

```

```

20851   const success = await MonsterModel.consumeEnergy(userId);
20852   console.log('[MonsterService] 能量消耗${success ? '成功' : '失败'}');
20853   return success;
20854 };
20855
20856 export const consumeEnergyAmount = async (userId: number, amount: number): Promise<boolean> => {
20857   const success = await MonsterModel.consumeEnergyAmount(userId, amount);
20858   return success;
20859 };
20860
20861 export const recoverEnergy = async (userId: number, amount: number = 1) => {
20862   await MonsterModel.recoverEnergy(userId, amount);
20863   return { success: true };
20864 };
20865
20866 export const gainExp = async (userId: number, exp: number) => {
20867   console.log('[MonsterService] 获得经验 - 用户ID: ${userId}, 经验: ${exp}');
20868   const monster = await MonsterModel.getMonsterByUserId(userId);
20869
20870   if (!monster) {
20871     console.log('[MonsterService] 获得经验失败 - 怪物不存在');
20872     return { success: false, message: '怪物不存在' };
20873   }
20874
20875   const newExp = monster.exp + exp;
20876   const expToLevel = monster.level * 100;
20877   let newLevel = monster.level;
20878   let remainingExp = newExp;
20879
20880   while (remainingExp >= expToLevel) {
20881     remainingExp -= expToLevel;
20882     newLevel++;
20883   }
20884
20885   await MonsterModel.updateMonster(userId, {
20886     level: newLevel,
20887     exp: remainingExp,
20888   });
20889
20890   console.log('[MonsterService] 经验更新 - 等级: ${monster.level} -> ${newLevel}, 经验: ${monster.exp} -> ${remainingE
20891   return { success: true, level: newLevel, exp: remainingExp };
20892 };
20893
20894 export const chatWithMonster = async (userId: number, message: string) => {
20895   // 1. 保存用户消息
20896   await MonsterMessageModel.createMessage({
20897     userId,
20898     message,
20899     isUser: true
20900   });

```

```

20901
20902 // 2. 获取怪兽状态，确定性格类型
20903 const monsterStatus = await getMonsterStatus(userId);
20904 const personality: MonsterPersonalityType = monsterStatus?.personality || 'lively';
20905
20906 // 3. 构建对话消息（含 system prompt + 历史上下文）
20907 const messages = await buildMessages(userId, personality, message);
20908
20909 // 4. 调用 DeepSeek API
20910 let result: { content: string; tokens: number };
20911 try {
20912     result = await callDeepSeek(messages);
20913     console.log('[MonsterService] DeepSeek 回复 (tokens: ${result.tokens}): ${result.content.substring(0, 100)}.');
20914 } catch (error) {
20915     console.error('[MonsterService] DeepSeek 调用失败，使用预设回复:', error);
20916     // 降级：预设回复
20917     const fallbacks: Record<MonsterPersonalityType, string[]> = {
20918         lively: ['太棒了！继续加油哦~', '这个知识点很有趣呢！', '准备好开始冒险了吗？'],
20919         calm: ['慢慢来，学习需要耐心。', '这个知识点很重要，好好掌握。'],
20920         rebel: ['学习好无聊啊，我们去玩吧！', '这个也太难了吧...'],
20921     };
20922     const pool = fallbacks[personality];
20923     const content = pool[Math.floor(Math.random() * pool.length)];
20924     result = { content, tokens: Math.ceil(content.length / 2) };
20925 }
20926
20927 // 5. 扣除 ⌈ 能量：消耗 = 怪兽回复的字数 × 0.05
20928 const charCount = result.content.length;
20929 const energyCost = Math.max(1, Math.ceil(charCount * ENERGY_COST_PER_TOKEN));
20930 const currentEnergy = monsterStatus?.energy ?? 0;
20931 const newEnergy = Math.max(0, currentEnergy - energyCost);
20932 await MonsterModel.updateMonster(userId, { energy: newEnergy });
20933 console.log('[MonsterService] 能量扣除: ${energyCost} (字数: ${charCount}), 剩余: ${newEnergy}');
20934
20935 // 6. 保存怪兽回复
20936 await MonsterMessageModel.createMessage({
20937     userId,
20938     message: result.content,
20939     isUser: false
20940 });
20941
20942 return { message: result.content, tokens: result.tokens, energyCost, remainingEnergy: newEnergy };
20943 };
20944
20945 export const getMonsterMessages = async (userId: number) => {
20946     const messages = await MonsterMessageModel.getMessagesByUserId(userId);
20947     return { messages: messages.reverse() };
20948 };
20949
=== server/src/services/noteService.ts ===

```

```

20950 import * as NoteModel from '../models/Note';
20951
20952 export const createNote = async (userId: number, date: Date, content?: string) => {
20953   console.log('[NoteService] 创建笔记 - 用户ID: ${userId}, 日期: ${date}');
20954
20955   // 纯笔记存储, 不再自动调用 AI 生成怪兽评论
20956   const notelId = await NoteModel.createNote({
20957     userId,
20958     date,
20959     content,
20960   });
20961
20962   console.log('[NoteService] 笔记创建成功 - 笔记ID: ${notelId}');
20963   return { success: true, notelId };
20964 };
20965
20966 export const getNotes = async (userId: number) => {
20967   console.log('[NoteService] 获取笔记列表 - 用户ID: ${userId}');
20968   const notes = await NoteModel.getNotesByUserId(userId);
20969   console.log('[NoteService] 获取到 ${notes.length} 条笔记');
20970   return { notes };
20971 };
20972
20973 export const getNoteByDate = async (userId: number, date: Date) => {
20974   console.log('[NoteService] 按日期获取笔记 - 用户ID: ${userId}, 日期: ${date}');
20975   const note = await NoteModel.getNoteByDate(userId, date);
20976   return note;
20977 };
20978
20979 export const updateNote = async (notelId: number, content?: string) => {
20980   console.log('[NoteService] 更新笔记 - 笔记ID: ${notelId}');
20981   await NoteModel.updateNote(notelId, { content });
20982   console.log('[NoteService] 笔记更新成功');
20983   return { success: true };
20984 };
20985
20986 === server/src/services/proService.ts ===
20987 import crypto from 'crypto';
20988 import { DatabaseConnection } from '../config/database';
20989
20990 const PLAN_DURATION: Record<string, number | null> = {
20991   monthly: 30,    // 30 days
20992   yearly: 365,    // 365 days
20993   lifetime: null, // permanent
20994 };
20995
20996 export class ProService {
20997   // 生成一批激活码
20998   static async generateCodes(count: number, planId: string, createdBy?: string): Promise<string[]> {
20999     if (!PLAN_DURATION[planId] && planId !== 'lifetime') {

```

```

20999     throw new Error('无效的套餐类型');
21000 }
21001
21002 const connection = await DatabaseConnection.getConnection();
21003 const codes: string[] = [];
21004
21005 for (let i = 0; i < count; i++) {
21006     const code = 'LF-' + crypto.randomBytes(6).toString('hex').toUpperCase();
21007     await connection.execute(
21008         'INSERT INTO activation_codes (code, plan_id, created_by, status) VALUES (?, ?, ?, ?)',
21009         [code, planId, createdBy || null, 'unused']
21010     );
21011     codes.push(code);
21012 }
21013
21014 console.log(`[ProService] 生成 ${count} 个激活码, 套餐: ${planId}`);
21015 return codes;
21016 }
21017
21018 // 验证并激活
21019 static async activateCode(code: string, userId: number): Promise<{
21020     success: boolean;
21021     planId?: string;
21022     expiresAt?: string;
21023 }> {
21024     const connection = await DatabaseConnection.getConnection();
21025
21026     // 查找未使用的激活码
21027     const [rows] = await connection.execute(
21028         'SELECT * FROM activation_codes WHERE code = ? AND status = ?',
21029         [code, 'unused']
21030     );
21031
21032     const record = (rows as any[][0]);
21033     if (!record) {
21034         throw new Error('激活码无效或已被使用');
21035     }
21036
21037     // 计算过期时间
21038     const duration = PLAN_DURATION[record.plan_id];
21039     let expiresAt: Date | null = null;
21040     if (duration !== null && duration !== undefined) {
21041         expiresAt = new Date(Date.now() + duration * 24 * 60 * 60 * 1000);
21042     }
21043
21044     // 更新激活码状态
21045     await connection.execute(
21046         'UPDATE activation_codes SET status = ?, used_by = ?, used_at = NOW() WHERE id = ?',
21047         ['used', userId, record.id]
21048     );

```

```

21049
21050 // 更新用户 Pro 状态
21051 if (expiresAt) {
21052     await connection.execute(
21053         'UPDATE users SET is_pro = 1, pro_activated_at = NOW(), pro_expires_at = ? WHERE id = ?',
21054         [expiresAt, userId]
21055     );
21056 } else {
21057     // 永久会员
21058     await connection.execute(
21059         'UPDATE users SET is_pro = 1, pro_activated_at = NOW(), pro_expires_at = NULL WHERE id = ?',
21060         [userId]
21061     );
21062 }
21063
21064 console.log('[ProService] 激活码使用成功 - 用户ID: ${userId}, 套餐: ${record.plan_id}, 过期: ${expiresAt || '永久'}');
21065 return {
21066     success: true,
21067     planId: record.plan_id,
21068     expiresAt: expiresAt?.toISOString(),
21069 };
21070 }
21071
21072 // 查询用户 Pro 状态
21073 static async getProStatus(userId: number): Promise<{
21074     isPro: boolean;
21075     planId?: string;
21076     activatedAt?: string;
21077     expiresAt?: string;
21078 }> {
21079     const connection = await DatabaseConnection.getConnection();
21080     const [rows] = await connection.execute(
21081         'SELECT is_pro, pro_activated_at, pro_expires_at FROM users WHERE id = ?',
21082         [userId]
21083     );
21084
21085     const user = (rows as any[][0]);
21086     if (!user) {
21087         throw new Error('用户不存在');
21088     }
21089
21090     // 检查是否过期
21091     if (user.is_pro && user.pro_expires_at && new Date(user.pro_expires_at) < new Date()) {
21092         // 已过期, 自动取消 Pro
21093         await connection.execute(
21094             'UPDATE users SET is_pro = 0 WHERE id = ?',
21095             [userId]
21096         );
21097         return { isPro: false };
21098     }

```



```

21099
21100 // 查询使用的激活码类型
21101 let planId: string | undefined;
21102 if (user.is_pro) {
21103     const [codeRows] = await connection.execute(
21104         'SELECT plan_id FROM activation_codes WHERE used_by = ? ORDER BY used_at DESC LIMIT 1',
21105         [userId]
21106     );
21107     const codeRecord = (codeRows as any[][0]);
21108     planId = codeRecord?.plan_id;
21109 }
21110
21111 return {
21112     isPro: !!user.is_pro,
21113     planId,
21114     activatedAt: user.pro_activated_at ? new Date(user.pro_activated_at).toISOString() : undefined,
21115     expiresAt: user.pro_expires_at ? new Date(user.pro_expires_at).toISOString() : undefined,
21116 };
21117 }
21118
21119 // 列出所有激活码（管理用）
21120 static async listCodes(filter?: { status?: string; planId?: string }): Promise<any[]> {
21121     const connection = await DatabaseConnection.getConnection();
21122     let sql = 'SELECT * FROM activation_codes WHERE 1=1';
21123     const params: any[] = [];
21124
21125     if (filter?.status) {
21126         sql += ' AND status = ?';
21127         params.push(filter.status);
21128     }
21129     if (filter?.planId) {
21130         sql += ' AND plan_id = ?';
21131         params.push(filter.planId);
21132     }
21133
21134     sql += ' ORDER BY created_at DESC LIMIT 100';
21135     const [rows] = await connection.execute(sql, params);
21136     return rows as any[];
21137 }
21138 }
21139
21140 === server/src/services/rewardService.ts ===
21140 import * as RewardModel from '../models/Reward';
21141
21142 export const createReward = async (userId: number, type: string, amount: number, source?: string) => {
21143     console.log('[RewardService] 创建奖励 - 用户ID: ${userId}, 类型: ${type}, 数量: ${amount}');
21144     const rewardId = await RewardModel.createReward({
21145         userId,
21146         type,
21147         amount,

```

```

21148     source
21149   });
21150   console.log('[RewardService] 奖励创建成功 - 奖励ID: ${rewardId}');
21151   return { success: true, rewardId };
21152 };
21153
21154 export const getRewards = async (userId: number) => {
21155   console.log('[RewardService] 获取奖励列表 - 用户ID: ${userId}');
21156   const rewards = await RewardModel.getRewardsByUserId(userId);
21157   console.log('[RewardService] 获取到 ${rewards.length} 条奖励');
21158   return { rewards };
21159 };
21160
21161 export const claimReward = async (rewardId: number) => {
21162   console.log('[RewardService] 领取奖励 - 奖励ID: ${rewardId}');
21163   const success = await RewardModel.claimReward(rewardId);
21164   console.log('[RewardService] 奖励领取${success ? '成功' : '失败'}');
21165   return { success };
21166 };
21167
21168 === server/src/services/skillService.ts ===
21169 import { SkillNode, SkillTreeListRequest, UserProgress, SkillTreeStats, SaveSkillTreeRequest } from '../types/
21170 import { DatabaseService } from './databaseService';
21171 import { LLMService, LLMProvider } from './llmService';
21172
21173 // 预置技能树数据（用于生成新技能树）
21174 const mockSkillTrees: Record<string, SkillNode> = {
21175   '前端开发': {
21176     id: '1',
21177     name: '前端开发',
21178     description: '前端开发技能树',
21179     children: [
21180       {
21181         id: '1-1',
21182         name: 'HTML/CSS',
21183         description: '网页基础结构',
21184         links: [
21185           { title: 'MDN HTML教程', url: 'https://developer.mozilla.org/zh-CN/docs/Web/HTML', type: 'documentatio
21186           { title: 'CSS教程', url: 'https://developer.mozilla.org/zh-CN/docs/Web/CSS', type: 'documentation' as
21187         ]
21188       },
21189       {
21190         id: '1-2',
21191         name: 'JavaScript',
21192         description: '前端编程语言',
21193         children: [
21194           {
21195             id: '1-2-1',
21196             name: '基础语法',
21197             description: '变量、函数、循环等',

```

```

21197     links: [
21198         { title: 'JavaScript教程', url: 'https://developer.mozilla.org/zh-CN/docs/Web/JavaScript', type: '
21199     ]
21200 },
21201 {
21202     id: '1-2-2',
21203     name: 'DOM操作',
21204     description: '文档对象模型操作',
21205     links: [
21206         { title: 'DOM教程', url: 'https://developer.mozilla.org/zh-CN/docs/Web/API/Document_Object_Model',
21207     ]
21208     }
21209 ]
21210 }
21211 ]
21212 },
21213 '后端开发': {
21214     id: '2',
21215     name: '后端开发',
21216     description: '后端开发技能树',
21217     children: [
21218     {
21219         id: '2-1',
21220         name: 'Node.js',
21221         description: 'JavaScript运行时',
21222         links: [
21223             { title: 'Node.js官方文档', url: 'https://nodejs.org/docs', type: 'documentation' as const }
21224         ]
21225     },
21226     {
21227         id: '2-2',
21228         name: '数据库',
21229         description: '数据存储和管理',
21230         children: [
21231             {
21232                 id: '2-2-1',
21233                 name: 'MySQL',
21234                 description: '关系型数据库',
21235                 links: [
21236                     { title: 'MySQL教程', url: 'https://dev.mysql.com/doc/', type: 'documentation' as const }
21237                 ]
21238             }
21239         ]
21240     }
21241 ]
21242 }
21243 };
21244
21245 // 初始化LLM服务 (根据环境变量配置)
21246 const initLLMService = (): LLMService | null => {

```

```

21247 const apiKey = process.env.LLM_API_KEY;
21248 const provider = process.env.LLM_PROVIDER as LLMProvider;
21249
21250 if (!apiKey || !provider) {
21251     console.warn('LLM service not configured. Using mock data instead.');
```

21252 return null;

21253 }

21254

21255 return new LLMService({

21256 provider,

21257 apiKey,

21258 baseUrl: process.env.LLM_BASE_URL,

21259 model: process.env.LLM_MODEL

21260 });

21261 };

21262

21263 const llmService = initLLMService();

21264

21265 // 生成技能树 (优先使用LLM, 失败时回退到模拟数据)

21266 export const generateMockSkillTree = async (domain: string, level: string = 'beginner'): Promise<SkillNode> =>

21267 // 更新热门领域统计

21268 await DatabaseService.incrementDomainGeneratedCount(domain);

21269

21270 // 优先使用LLM生成技能树

21271 if (llmService) {

21272 try {

21273 console.log(`Generating skill tree for domain: \${domain}, level: \${level} using LLM`);

21274 const skillTree = await llmService.generateSkillTree({

21275 domain,

21276 level: level as 'beginner' | 'intermediate' | 'advanced',

21277 language: '中文',

21278 maxDepth: 4,

21279 includeResources: true

21280 });

21281

21282 console.log('Skill tree generated successfully by LLM');

21283 return skillTree;

21284 } catch (error) {

21285 console.warn('LLM generation failed, falling back to mock data:', error);

21286 console.error('LLM error details:', {

21287 message: error instanceof Error ? error.message : 'Unknown error',

21288 stack: error instanceof Error ? error.stack : 'No stack trace'

21289 });

21290 }

21291 }

21292

21293 // 回退到模拟数据

21294 console.log('Using mock data for domain: \${domain}');

21295 console.log('Available mock domains:', Object.keys(mockSkillTrees));

21296 let skillTree = mockSkillTrees[domain];

```

21297
21298   if (!skillTree) {
21299       // 如果领域不存在, 创建一个基础技能树
21300       skillTree = {
21301           id: generateId(),
21302           name: domain,
21303           description: `${domain}技能学习路径`,
21304           children: [
21305               {
21306                   id: generateId(),
21307                   name: '基础知识',
21308                   description: '学习该领域的基础概念',
21309                   links: [
21310                       { title: '入门指南', url: 'https://example.com', type: 'documentation' as const }
21311                   ]
21312               },
21313               {
21314                   id: generateId(),
21315                   name: '进阶学习',
21316                   description: '深入掌握核心技能',
21317                   links: [
21318                       { title: '进阶教程', url: 'https://example.com', type: 'documentation' as const }
21319                   ]
21320               }
21321           ]
21322       };
21323   }
21324
21325   return skillTree;
21326 };
21327
21328 // 获取技能树列表 (从数据库)
21329 export const getSkillTreeList = async (request: SkillTreeListRequest) => {
21330     const { page = 1, limit = 10, search, category } = request;
21331     console.log('[SkillService] 获取技能树列表 - 页码: ${page}, 搜索: ${search || '无'}, 分类: ${category || '无'}');
21332
21333     const result = await DatabaseService.getSkillTreesByUserId(1, page, limit);
21334
21335     return {
21336         trees: result.trees.map(tree => ({
21337             id: tree.id.toString(),
21338             name: tree.title,
21339             domain: tree.domain,
21340             description: tree.description,
21341             progress: tree.progress,
21342             createdAt: tree.createdAt
21343         })),
21344         total: result.total,
21345         page: result.page,
21346         limit: result.limit

```

```

21347   };
21348   };
21349
21350   // 根据ID获取技能树（从数据库）
21351   export const getSkillTreeById = async (id: string) => {
21352     console.log('[SkillService] 获取技能树详情 - ID: ${id}');
21353     const tree = await DatabaseService.getSkillTreeById(parseInt(id));
21354
21355     if (!tree) {
21356       console.log('[SkillService] 技能树不存在 - ID: ${id}');
21357       return null;
21358     }
21359
21360     return {
21361       id: tree.id.toString(),
21362       name: tree.title,
21363       domain: tree.domain,
21364       description: tree.description,
21365       nodes: tree.nodes,
21366       progress: tree.progress,
21367       createdAt: tree.createdAt
21368     };
21369   };
21370
21371   // 保存用户技能树（到数据库）
21372   export const saveUserSkillTree = async (request: SaveSkillTreeRequest) => {
21373     const { userId, skillTree, title, tags } = request;
21374     console.log('[SkillService] 保存技能树 - 用户ID: ${userId}, 标题: ${title || skillTree.name}');
21375
21376     try {
21377       const treeld = await DatabaseService.createSkillTree({
21378         userId: parseInt(userId),
21379         domain: skillTree.name,
21380         title: title || skillTree.name,
21381         description: skillTree.description,
21382         nodes: [skillTree],
21383         isPublic: false
21384       });
21385
21386       console.log('[SkillService] 技能树保存成功 - 树ID: ${treeld}');
21387       return { success: true, treeld: treeld.toString() };
21388     } catch (error) {
21389       console.error('[SkillService] 保存技能树失败:', error);
21390       return { success: false, error: '保存失败' };
21391     }
21392   };
21393
21394   // 获取用户进度（从数据库）
21395   export const getUserProgress = async (userId: string, skillTreeld?: string): Promise<UserProgress[]> => {
21396     if (skillTreeld) {

```

```

21397     const progress = await DatabaseService.getUserProgress(parseInt(userId), parseInt(skillTreeld));
21398     return [{
21399         userId,
21400         skillTreeld,
21401         completedNodes: [], // 需要从学习记录计算
21402         completedLinks: [], // 需要从学习记录计算
21403         progress: progress.overallProgress,
21404         lastUpdated: progress.lastUpdated
21405     }];
21406 }
21407
21408 // 获取用户所有技能树的进度
21409 const trees = await DatabaseService.getSkillTreesByUserId(parseInt(userId));
21410 const progresses: UserProgress[] = [];
21411
21412 for (const tree of trees.trees) {
21413     const progress = await DatabaseService.getUserProgress(parseInt(userId), tree.id);
21414     progresses.push({
21415         userId,
21416         skillTreeld: tree.id.toString(),
21417         completedNodes: [],
21418         completedLinks: [],
21419         progress: progress.overallProgress,
21420         lastUpdated: progress.lastUpdated
21421     });
21422 }
21423
21424 return progresses;
21425 };
21426
21427 // 更新用户进度（到数据库）
21428 export const updateUserProgress = async (userId: string, skillTreeld: string, completedNodes: string[], comple
21429 // 这里简化处理，实际应该更新学习记录表
21430 const progress = Math.round((completedNodes.length / 10) * 100); // 简化计算
21431
21432 await DatabaseService.updateSkillTreeProgress(parseInt(skillTreeld), progress);
21433
21434 return {
21435     userId,
21436     skillTreeld,
21437     completedNodes,
21438     completedLinks,
21439     progress,
21440     lastUpdated: new Date()
21441 };
21442 };
21443
21444 // 获取统计信息（从数据库）
21445 export const getStatistics = async (): Promise<SkillTreeStats> => {
21446     console.log('[SkillService] 获取统计信息');

```

```

21447 const popularDomains = await DatabaseService.getPopularDomains(5);
21448
21449 return {
21450   totalTrees: 100,
21451   popularDomains: popularDomains.map(d => ({ domain: d.domain, count: d.popularity })),
21452   averageProgress: 45,
21453   totalUsers: 50
21454 };
21455 };
21456
21457 export const searchPopularDomains = async (keyword: string) => {
21458   console.log('[SkillService] 搜索热门领域 - 关键词: ${keyword}');
21459   await DatabaseService.incrementDomainSearchCount(keyword);
21460
21461   const domains = await DatabaseService.getPopularDomains(10);
21462   const results = domains
21463     .filter(d => d.domain.toLowerCase().includes(keyword.toLowerCase()))
21464     .map(d => d.domain);
21465
21466   console.log('[SkillService] 搜索结果: ${results.length} 条');
21467   return {
21468     keyword,
21469     results,
21470     total: results.length
21471   };
21472 };
21473
21474 export const getRecommendedPath = async (domain: string, currentLevel: string, targetLevel: string) => {
21475   console.log('[SkillService] 获取推荐路径 - 领域: ${domain}, 当前等级: ${currentLevel}, 目标等级: ${targetLevel}');
21476   return {
21477     domain,
21478     currentLevel,
21479     targetLevel,
21480     path: [
21481       { step: 1, topic: '基础知识', estimatedTime: '2周' },
21482       { step: 2, topic: '核心概念', estimatedTime: '3周' },
21483       { step: 3, topic: '实战项目', estimatedTime: '4周' }
21484     ],
21485     totalEstimatedTime: '9周'
21486   };
21487 };
21488
21489 export const getUserLearningReport = async (userId: string, period: string = 'month') => {
21490   console.log('[SkillService] 获取学习报告 - 用户ID: ${userId}, 周期: ${period}');
21491   return {
21492     userId,
21493     period,
21494     summary: {
21495       totalLearningTime: 120,
21496       completedSkills: 15,

```



```

21497     averageProgress: 65,
21498     streakDays: 7
21499 },
21500 recommendations: [
21501   '建议每天保持30分钟学习时间',
21502   '重点关注前端开发的核心概念',
21503   '尝试完成一个小型实战项目'
21504 ]
21505 };
21506 };
21507
21508 // 辅助函数：生成唯一ID
21509 const generateId = (): string => {
21510   return Date.now().toString(36) + Math.random().toString(36).substr(2);
21511 };
21512
21513 === server/src/types/index.ts ===
21514 export * from './skill';
21515
21516 === server/src/types/skill.ts ===
21517 export interface MonsterPersonality {
21518   cheerful: number;
21519   calm: number;
21520   rebellious: number;
21521 }
21522
21523 export interface SkillNode {
21524   id: string;
21525   name: string;
21526   description: string;
21527   children?: SkillNode[];
21528   links?: SkillLink[];
21529 }
21530
21531 export interface SkillLink {
21532   title: string;
21533   url: string;
21534   type: 'video' | 'article' | 'course' | 'documentation';
21535 }
21536
21537 export interface SkillTreeRequest {
21538   domain: string;
21539   level?: 'beginner' | 'intermediate' | 'advanced';
21540 }
21541
21542 export interface UserProgress {
21543   userId: string;
21544   skillTreeId: string;
21545   completedNodes: string[];
21546   completedLinks: string[];
21547   progress: number;
21548   lastUpdated: Date;

```

```

21545 }
21546
21547 export interface SkillTreeListRequest {
21548     page?: number;
21549     limit?: number;
21550     search?: string;
21551     category?: string;
21552 }
21553
21554 export interface SkillTreeStats {
21555     totalTrees: number;
21556     popularDomains: Array<{domain: string; count: number}>;
21557     averageProgress: number;
21558     totalUsers: number;
21559 }
21560
21561 export interface SaveSkillTreeRequest {
21562     userId: string;
21563     skillTree: SkillNode;
21564     title?: string;
21565     tags?: string[];
21566 }
21567
21568 export interface LearningReport {
21569     userId: string;
21570     period: string;
21571     summary: {
21572         totalLearningTime: string;
21573         completedSkills: number;
21574         averageProgress: number;
21575         streakDays: number;
21576     };
21577     weeklyProgress: Array<{week: string; progress: number}>;
21578     topSkills: string[];
21579     recommendations: string[];
21580 }
21581
21582 export interface RecommendedPath {
21583     domain: string;
21584     currentLevel: string;
21585     targetLevel: string;
21586     path: Array<{step: number; topic: string; estimatedTime: string}>;
21587     totalEstimatedTime: string;
21588 }

```